

Computer Programming for Structural Analysis

Lecture Notes

Dr. Bilal Tayfur

October 2, 2026

These are the lecture notes for the undergraduate **Computer Programming for Structural Analysis** course in the Department of Civil Engineering. They follow the course syllabus week by week: each numbered section of Part I corresponds to one week of teaching.

The subject of this course is not structural analysis. You have met the principles already: equilibrium, compatibility, the stiffness of a member, the difference between a determinate and an indeterminate structure. What you have probably not met is what happens when those principles have to be handed to a computer — when a frame becomes three hundred lines of Python, when a material becomes eight numbers whose meanings you have to know, and when the answer arrives with six confident decimal places and no indication whatsoever of whether it is right.

That last point is the one this course is really about. A finite element program does not refuse a nonsensical model. It accepts what you give it, multiplies it together, and returns an answer with the same air whether you have modelled a bridge or a typing mistake. Learning to build the model is a few weeks' work; learning to know when to believe it is the rest of your career, and this is where it starts.

We use **OpenSees**, through its Python interface **openseespy**. It is free, it is what a great deal of published earthquake engineering research is done with, and — unlike the packages you may meet in an office — it has no graphical interface to hide behind. Every node, every element and every analysis decision is something you write down and can be asked to justify. By week 13 you will be running a fully nonlinear time-history analysis of a laboratory specimen and comparing it against what the specimen actually did.

How the notes are organised.

- **Week 0** comes before the syllabus. It sets out the four methods every structural analysis program is built on — exact, approximate, matrix and finite element — and closes with the ground rules the rest of the notes assume. Read it before week 1, even if you have met some of it before.
- **Part I — Weeks 1–14** follows the syllabus exactly. Every week opens with its learning outcomes and the new notation it introduces, and closes with a summary table and a set of exercises with answers. Weeks 9, 15 and 16 are the midterm, the project presentations and the final examination, and have no chapter.
- **Part II — Supplementary chapters** cover material every week needs and none owns: a catalogue of OpenSees failures and what each one actually means (S1), a reference to the command subset used here (S2), a guide to choosing from the material library (S3), and a continuum model of a concrete cylinder crushed to failure (S4).

How to use them. Run the code. Every listing in these notes is taken from a script in the accompanying repository, and the line numbers printed beside it are that file's own, so you can open it and find the lines. Change a number and run it again — that is the experiment the printed page cannot do for you.

Where a belief is both common and wrong you will find it marked like this: **what most people expect** → **what the experiment shows**. Several of these come with the experiment that settles them, and you are invited to make your prediction before reading the result. Getting it wrong is the point; that is what the marking is for.

A note on the numbers. Every number printed in these notes is recomputed by a separate program before they will compile, and every script shown is executed by it. If your own run disagrees with a printed value, that is worth reporting: it means something has changed underneath both of us.

A note on completeness. These notes are new and being written as the course runs. They will contain errors. If you find one, please report it — doing so is closer to the subject of this course than it might appear.

Contents

Week 0: From Exact Methods to Finite Elements	1
0.1 Exact methods	2
0.2 Approximate methods	4
0.3 When an approximate method is exact	7
0.4 The matrix method	9
0.5 The finite element method	13
0.6 Why structural analysis is done with finite elements	17
0.7 Ground rules	19
Week 1: Python and the Working Environment	24
1.1 The environment	24
1.2 The Python you actually need	25
1.3 Writing a model as a function	27
1.4 The analysis, and the check	28
Week 2: The OpenSees Model	33
2.1 One domain, and only one	33
2.2 Six steps, in the only order that works	34
2.3 A truss you can check by hand	36
2.4 Running it	37
Week 3: The Analysis Space, Recorders and Verification	42
3.1 The map	43
3.2 Getting numbers out	44
3.3 Four checks	46
Week 4: Linear Static Analysis of 1D Elements	50
4.1 Why a beam needs a transformation	50
4.2 Building the beam	51
4.3 A beam you can check by hand	53
4.4 How many elements?	54
Week 5: Linear Static Analysis of 2D Frames	58
5.1 The tag formula	58
5.2 Load combinations	61
5.3 Measuring drift, and a number that is not real	62
Week 6: Material Models I: Elastic and Nonlinear Elastic	67
6.1 A material on the bench	67
6.2 Three materials	68
6.3 The path matters	69
Week 7: Material Models II: Inelastic and Hysteretic	74
7.1 A protocol, not a list	74
7.2 Two shapes	74
Week 8: Sections and Fiber Models	81
8.1 A section as a list of fibres	81
8.2 Driving a section	83
8.3 How many fibres?	85

Week 10: Nonlinear Solution Strategies	90
10.1 Three separable things	90
10.2 A step that refuses to give up	91
10.3 And now the bad news	92
10.4 Choosing a tolerance	93
Week 11: Nonlinear Static Analysis of 3D Structures	98
11.1 What three dimensions demand	98
11.2 Staging the analysis	100
11.3 The answer a capacity curve cannot give	102
11.4 Looking at it	103
11.5 How much of the curve is real	104
Week 12: Introduction to Dynamic Analysis	108
12.1 Mass, and the unit that has been waiting since week 0	108
12.2 Damping, and the modes you did not choose	111
12.3 A response, and measuring it	113
Week 13: Fully Nonlinear Dynamic Analysis	117
13.1 The specimen	117
13.2 A step that retries, in real time	119
13.3 Verifying an analysis with no answer	120
Week 14: Post-processing, Data Extraction and Visualization	125
14.1 Five numbers, and what each hides	125
14.2 When, not only how much	126
14.3 Reconstructing what you already had	127
14.4 Writing a file the next tool can trust	129
S1 — Debugging OpenSees	132
S1.1 Three kinds of failure	132
S1.2 When it stops: reading the return code	132
S1.3 When it lies: four failures with no message	133
S1.3.1 The reactions are all exactly zero	133
S1.3.2 Mass in kilograms	134
S1.3.3 A second model built on the first	134
S1.3.4 The tolerance that stopped the complaints	135
S1.4 Before you analyse: two cheap checks	138
S1.4.1 Auditing the domain	138
S1.4.2 Is it a mechanism?	139
S1.5 When none of that helps: cut the model down	140
S1.6 The message index	140
S1.7 Summary	141
S2 — OpenSeesPy Command Reference	143
S2.1 Order is the program	143
S2.2 Counting the reference instead of writing it	143
S2.3 Where each command appears	145
S2.4 Finding the arguments your build expects	145
S2.5 The smallest complete model	147

S3 — Choosing a Material Model	149
S3.1 Sort by capability, not by substance	149
S3.2 Ask the build, do not read the manual	151
S3.3 Recognised, usable, correct	152
S3.4 Choosing: a walk down the tree	153
S3.5 The nD side, and why it is harder	154
S3.6 Summary	154
 S4 — Continuum Elements: Crushing a Cylinder	 157
S4.1 Meshing a circle	157
S4.2 A material that will lose strength	159
S4.2.1 The parameters are not the strength	160
S4.3 Crushing it	160
S4.4 Does the mesh change the answer?	162
S4.5 Crushing it dynamically	173
S4.6 Looking at it: the ParaView route	175
S4.6.1 Making it a time series	177
S4.7 Summary	178

Week 0: From Exact Methods to Finite Elements

Learning outcomes

After this week you should be able to:

1. say what an exact method is exact *to*, and solve a cantilever by integrating its differential equation;
2. explain what makes a method approximate, and turn an assumed deflected shape into a small system $\mathbf{K}\vec{a} = \vec{F}$ by the Rayleigh–Ritz method;
3. predict, before computing anything, when an approximate method will return the exact answer — everywhere, or at one point only;
4. write the stiffness matrix of a bar and of a beam, and say what each of its columns means physically;
5. assemble a structure's stiffness matrix from its elements, apply the supports and solve it, by hand for two unknowns and in code for any number;
6. explain what the finite element method adds to the matrix method, and why it is the method structural analysis programs are built on;
7. state the unit system of this course, and what it makes the unit of mass.

Notation. $w(x)$ deflection θ slope M, V moment and shear q distributed load EI, EA bending and axial stiffness Π total potential energy ϕ_i assumed shape N_i shape function $\mathbf{k}, \mathbf{K}$ element and structure stiffness matrices $\vec{u}, \vec{F}$ nodal displacements and forces

Every analysis in this course is done by a program, and the program is a finite element program. From week 2 onward you will be writing its vocabulary — node, element, degree of freedom, stiffness, mesh, convergence — into scripts, and every one of those words belongs to one of four methods: the exact methods you used by hand in structural analysis, the approximate methods that take over when no exact solution can be found, the matrix method that turns a structure into a system of linear equations, and the finite element method, which combines the last two. A program will run whether or not you know which of the four it is using. Knowing is what lets you say when its answer can be believed. Figure 0.1 is the map of the week, and the sections follow its arrows: each method is easiest to understand as the answer to a shortcoming of the one before it.

One structure carries the whole chapter. It is the steel cantilever you will model in OpenSees in week 1 — an IPE 300, 3 m long — carrying a uniform load of 10 kN/m. It is solved by integration, by assumed shapes, by the matrix method and by finite elements, and each time its tip deflection either comes out as 6.06 mm or it does not. The reasons are the content of the week. The chapter closes with the ground rules — units, numbers and code — that every later week assumes without restating them.

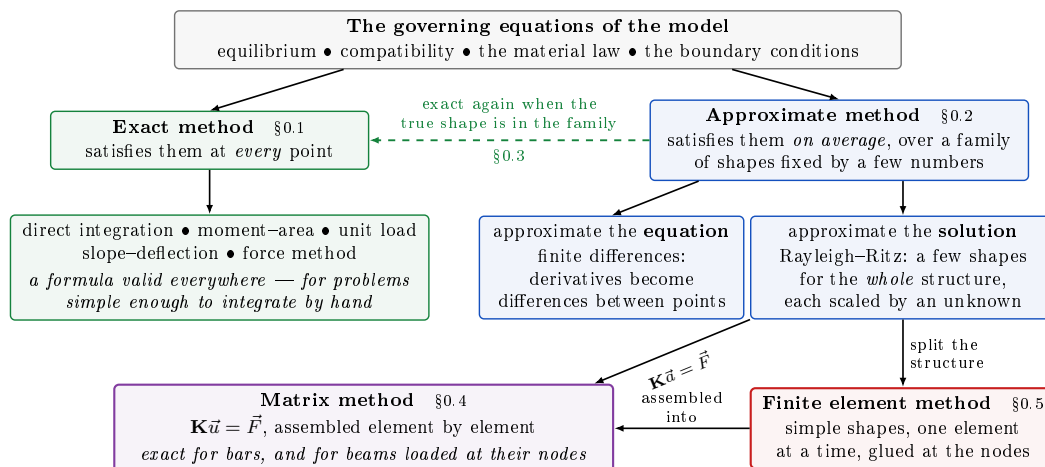
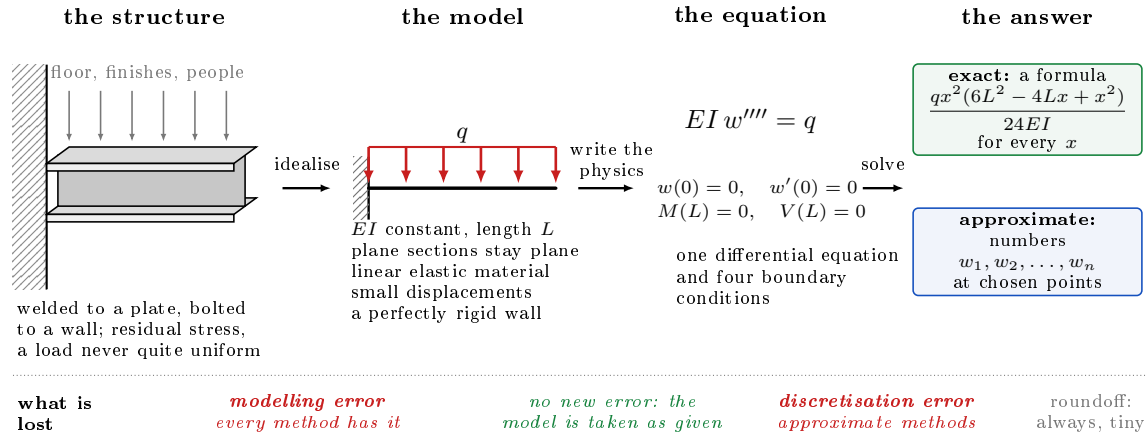


Figure 0.1: The four methods of this week and how each leads to the next. The dashed arrow is §0.3, when the approximate road reaches the exact answer.

0.1 Exact methods

A structural analysis never works on a structure. It works on a **model** (*TR: model*): a description of the structure in which some things are kept and others are deliberately thrown away. The method is then applied to the model. Figure 0.2 follows the cantilever through that chain, and marks where each kind of error gets in.



An exact method removes only the error at the third arrow. The difference between the steel beam and a line with an EI enters at the first arrow, and it is there whichever method is used afterwards: no amount of mathematics removes it. Checking a program against a closed form tests the third arrow; only a measurement tests the first.

Figure 0.2: From a steel beam to a number, and the error that enters at each arrow. “Exact” refers only to the third arrow.

Definition 0.1: Exact method

An **exact method** (*TR: kesin yöntem*) finds the displacement field that satisfies the governing equations of the model — equilibrium, compatibility and the material law — at *every* point, together with every boundary condition. Its answer is a formula valid at every position, and it carries no error beyond roundoff *with respect to the model*.

The last four words are the whole of the definition. The model in Fig. 0.2 assumes plane sections, a linear elastic material, small displacements and a perfectly rigid wall. None of these is exactly true of a steel beam bolted to a wall. An exact method returns the exact deflection of the idealised line, and says nothing about how well that line describes the beam.

For a straight beam of constant EI the governing equations reduce to one. Measure the deflection w and the load q downward, take the moment positive when it is hogging — the only sense a cantilever knows — and equilibrium of a short piece of beam together with the moment–curvature relation give

$$EI w'''' = q, \quad M = EI w'', \quad V = -EI w''', \quad (0.1)$$

with four boundary conditions for a cantilever fixed at $x = 0$: the wall allows neither deflection nor rotation, $w(0) = w'(0) = 0$, and the free end carries neither moment nor shear, $M(L) = V(L) = 0$.

Example 0.1 (The cantilever, by integration). The IPE 300 cantilever has $L = 3000$ mm, $E = 200\,000$ MPa and $I = 8.356 \times 10^7$ mm⁴, and carries $q = 10$ N/mm over its whole length. Find its deflected shape, its tip deflection and rotation, and the moment at the wall, exactly.

Solution

Model. Euler–Bernoulli beam, linear elastic, small displacements, rigid wall — Eq. (0.1) and its four boundary conditions.

Integration. Each integration of Eq. (0.1) produces one constant, and each constant is fixed by one boundary condition:

$$\begin{aligned}
 EI w''' &= qx + C_1 & V(L) &= 0 \Rightarrow C_1 = -qL, \\
 EI w'' &= \frac{1}{2}qx^2 - qLx + C_2 & M(L) &= 0 \Rightarrow C_2 = \frac{1}{2}qL^2, \\
 EI w' &= -\frac{1}{6}q(L-x)^3 + C_3 & w'(0) &= 0 \Rightarrow C_3 = \frac{1}{6}qL^3, \\
 EI w &= \frac{1}{24}q(L-x)^4 + \frac{1}{6}qL^3x + C_4 & w(0) &= 0 \Rightarrow C_4 = -\frac{1}{24}qL^4.
 \end{aligned}$$

Collecting the last line,

$$w(x) = \frac{qx^2(6L^2 - 4Lx + x^2)}{24EI}, \quad w(L) = \frac{qL^4}{8EI}, \quad \theta(L) = \frac{qL^3}{6EI}, \quad M(0) = \frac{qL^2}{2}. \quad (0.2)$$

Figure 0.3 draws the chain: four integrations, four constants, and exactly the four conditions a cantilever supplies.

Numbers. With $EI = (200000)(8.356 \times 10^7) = 1.671 \times 10^{13} \text{ N mm}^2$,

$$w(L) = \frac{(10)(3000)^4}{8(1.671 \times 10^{13})} = \frac{8.100 \times 10^{14}}{1.337 \times 10^{14}}, \quad \theta(L) = \frac{(10)(3000)^3}{6(1.671 \times 10^{13})} = \frac{2.700 \times 10^{11}}{1.003 \times 10^{14}},$$

$$M(0) = \frac{(10)(3000)^2}{2} = 4.500 \times 10^7 \text{ N mm}.$$

$$w(L) = 6.06 \text{ mm}, \quad \theta(L) = 2.69 \times 10^{-3} \text{ rad}, \quad M(0) = 45.0 \text{ kN m}$$

Check. A limiting case that needs no integration. Put the whole load, $qL = 30 \text{ kN}$, at the tip instead: the week 1 formula gives $PL^3/3EI = (30000)(3000)^3/[3(1.671 \times 10^{13})] = 16.16 \text{ mm}$. Spread along the span, the same load must deflect the tip less, because most of it acts close to the wall — and Eq. (0.2) says by how much: $qL^4/8EI = \frac{3}{8}(qL)L^3/EI$, which is $3/8$ of the tip-load value, $\frac{3}{8}(16.16) = 6.06 \text{ mm}$. The formula also satisfies all four boundary conditions, which can be checked by substitution in a minute.

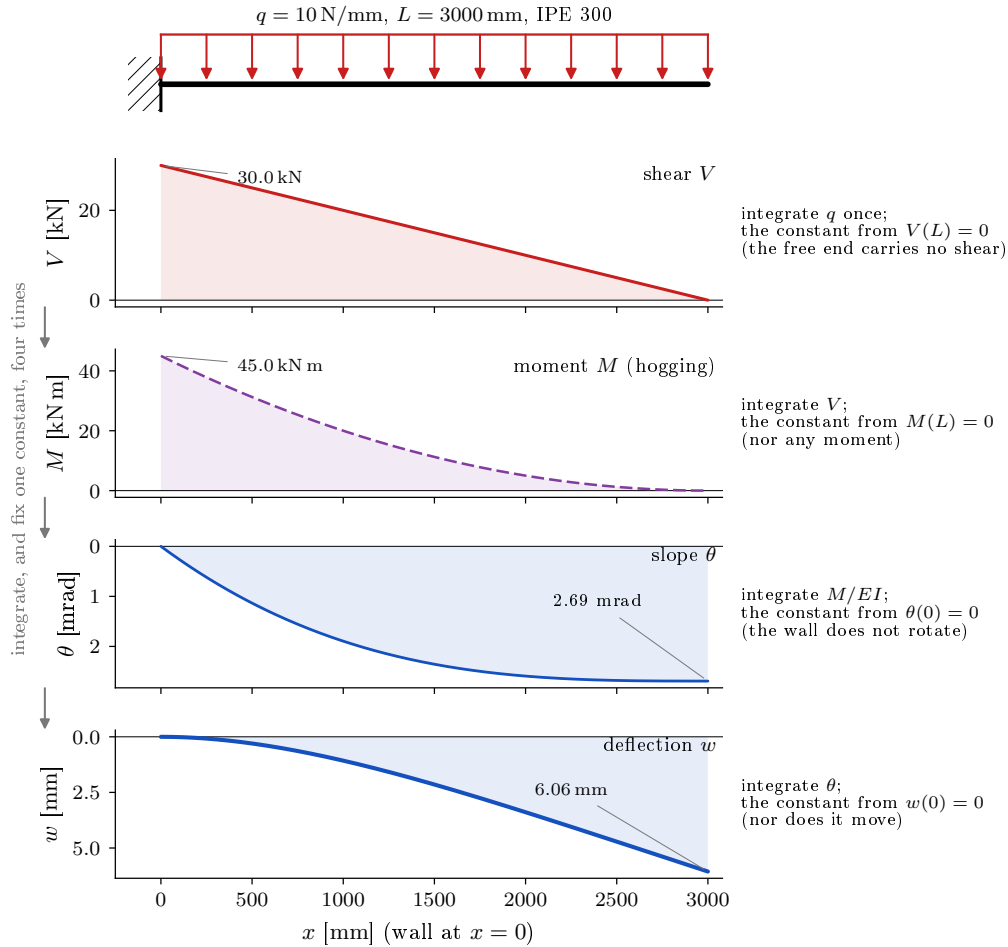


Figure 0.3: The exact method, drawn. Each integration of the beam equation produces the next quantity and one unknown constant; each constant is fixed by one boundary condition.

Direct integration, moment–area, the conjugate beam, the unit-load method, slope–deflection and the force method are all exact methods in the sense of [Definition 0.1](#), and moment distribution is an iteration that converges to one. What they share is that they satisfy the equations everywhere. What limits them is the same thing: every step is an integral, or a system of equations, that has to be done by hand. A tapered member, a frame with forty members, a wall with an opening, a section that has started to yield — for these there is no formula to find, not because nobody has looked, but because none exists in closed form.

Note

an exact method gives the true deflection of the structure → an exact method gives the exact deflection of the model

Figure 0.2 is the reason. Of the three errors in it, an exact method removes only the last. The modelling error enters before any method is chosen, and it is shared by every method that works on the same model — including the finite element program. Comparing a program with a closed form, as week 1 does, therefore checks the program and the input, not the idealisation. Only a measurement checks that.

0.2 Approximate methods

When the exact solution cannot be found, something has to be given up, and there are two candidates. [Figure 0.1](#) shows both roads.

Definition 0.2: Approximate method

An **approximate method** (TR: *yaklaşık yöntem*) gives up satisfying the governing equations at every point. Either it replaces the differential equation by a finite number of algebraic ones — **finite differences** (TR: *sonlu farklar*) replace each derivative by a difference between values at grid points — or it restricts the answer to a family of assumed shapes, each fixed by a finite number of unknowns, and picks the member of the family that satisfies the equations best on average. Its answer is a finite set of numbers, and it carries a **discretisation error** (TR: *ayrıklaştırma hatası*).

The second road is the one that leads to the finite element method, and it needs one idea from mechanics.

Definition 0.3: Total potential energy

The **total potential energy** (TR: *toplam potansiyel enerji*) of an elastic beam deflected into the shape $w(x)$ is its strain energy minus the work done by the load,

$$\Pi[w] = \frac{1}{2} \int_0^L EI (w'')^2 dx - \int_0^L q w dx. \quad (0.3)$$

Of all the shapes that respect the supports, the true deflected shape is the one that makes Π smallest.

The **Rayleigh–Ritz method** turns that principle into arithmetic. Assume the deflection is a combination of a few chosen shapes $\phi_i(x)$, each of which respects the supports, with unknown amounts a_i :

$$w(x) \approx \sum_{i=1}^n a_i \phi_i(x). \quad (0.4)$$

Substituted into Eq. (0.3), Π becomes an ordinary function of n numbers, and making it stationary with respect to each of them, $\partial\Pi/\partial a_i = 0$, gives n linear equations:

$$\sum_{j=1}^n K_{ij} a_j = F_i, \quad K_{ij} = \int_0^L EI \phi_i'' \phi_j'' dx, \quad F_i = \int_0^L q \phi_i dx. \quad (0.5)$$

That is a symmetric system $\mathbf{K}\vec{a} = \vec{F}$ — the same kind of system every finite element program solves, and with the same names. The rest of this chapter is, in one sense, about where $\mathbf{K}$ comes from.

The Rayleigh–Ritz method

1. Choose shapes ϕ_i that satisfy the *displacement* boundary conditions. They need not satisfy the force ones; the energy takes care of those, approximately.
2. Compute K_{ij} and F_i from Eq. (0.5).
3. Solve $\mathbf{K}\vec{a} = \vec{F}$.
4. Rebuild w from Eq. (0.4), and the slope, moment and shear from its derivatives.

Example 0.2 (The cantilever, with one assumed shape). Solve the cantilever of Example 0.1 with the single shape $\phi_1 = (x/L)^2$, by hand. Then use two and three shapes, $(x/L)^2$, $(x/L)^3$, $(x/L)^4$, by script, and compare each with the exact answer.

Solution

Model. As Example 0.1. The parabola has $w(0) = 0$ and $w'(0) = 0$, so it respects the wall. It does *not* have zero moment at the free end — $w'' = 2a/L^2$ is the same everywhere — and it does not have to: that is a force condition.

One shape, by hand. With $w = a(x/L)^2$, the curvature is $w'' = 2a/L^2$, and Eq. (0.3) becomes

$$\Pi(a) = \frac{1}{2}EI \left(\frac{2a}{L^2} \right)^2 L - q a \int_0^L \frac{x^2}{L^2} dx = \frac{2EI}{L^3} a^2 - \frac{qL}{3} a.$$

Setting $\frac{d\Pi}{da} = 0$ gives a one-by-one system, $(4EI/L^3)a = qL/3$, so

$$a = \frac{qL^4}{12EI} = \frac{2}{3} \cdot \frac{qL^4}{8EI},$$

two thirds of the exact tip deflection of Example 0.1, because $\phi_1(L) = 1$ makes a the tip deflection itself. The tip rotation is $w'(L) = 2a/L = qL^3/6EI$, and the moment is $M = EI w'' = 2EIa/L^2 = qL^2/6$, the same at every section.

$$w(L) \approx 4.04 \text{ mm (67\%)}, \quad \theta(L) = 2.69 \times 10^{-3} \text{ rad (exact)}, \quad M(0) \approx 15.0 \text{ kN m (33\%)}$$

More shapes, by script. Listing 0.1 builds and solves Eq. (0.5) for any number of shapes.

Listing 0.1: w00/ritz.py — The whole of the Rayleigh–Ritz method: a stiffness matrix, a load vector, and one call to a linear solver.

```

60 def ritz(p, n_terms, P_tip=0.0):
61     """Rayleigh-Ritz with the assumed shapes (x/L)^2, (x/L)^3, ...
62
63     Every one of these shapes has zero deflection and zero slope at x = 0, so
64     each already respects the wall. The method only has to decide HOW MUCH of
65     each shape to use: the amplitudes a_i, in mm. Making the total potential
66     energy stationary with respect to every a_i gives K a = F, with
67
68         K_ij = integral of EI phi_i'' phi_j'' dx      (stiffness)
69         F_i   = integral of q phi_i dx + P phi_i(L)  (load)
70
71     For powers of x both integrals are known in closed form, so they are
72     written out rather than computed numerically. Every shape equals one at
73     the tip, so a tip load P simply adds P to every F_i.
74     """
75     L, EI, q = p["L"], p["E"] * p["I"], p["q"]
76     powers = np.arange(2, n_terms + 2) # 2, 3, ..., n_terms + 1
77     K = np.zeros((n_terms, n_terms))
78     F = np.zeros(n_terms)
79     for a, i in enumerate(powers):
80         F[a] = q * L / (i + 1) + P_tip
81         for b, j in enumerate(powers):
82             K[a, b] = EI * i*(i-1) * j*(j-1) / ((i + j - 3) * L**3)
83     amplitudes = np.linalg.solve(K, F)
84     return powers, amplitudes

```

Cantilever, L = 3000 mm, q = 10 N/mm, w measured downward						
terms	tip deflection		tip rotation		wall moment	
	[mm]	/exact	[rad]	/exact	[kNm]	/exact
1	4.0390	0.6667	0.002693	1.0000	15.000	0.3333
2	6.0585	1.0000	0.002693	1.0000	37.500	0.8333
3	6.0585	1.0000	0.002693	1.0000	45.000	1.0000
exact	6.0585		0.002693		45.000	

Check. Two independent ones. The one-term row reproduces the hand calculation above to four figures. And the energy obeys the principle it came from: at its solution the one-term approximation has $\Pi_1 = -\frac{1}{2}F_1a = -q^2L^5/72EI$, while the exact solution has $\Pi = -\frac{1}{2}\int q w dx = -q^2L^5/40EI$. The approximation's energy is the higher of the two, as it must be when the true shape is the one that makes Π smallest.

Figure 0.4 puts the three approximations beside the exact answer. Read the left panel and they look good; read the right one and they do not.

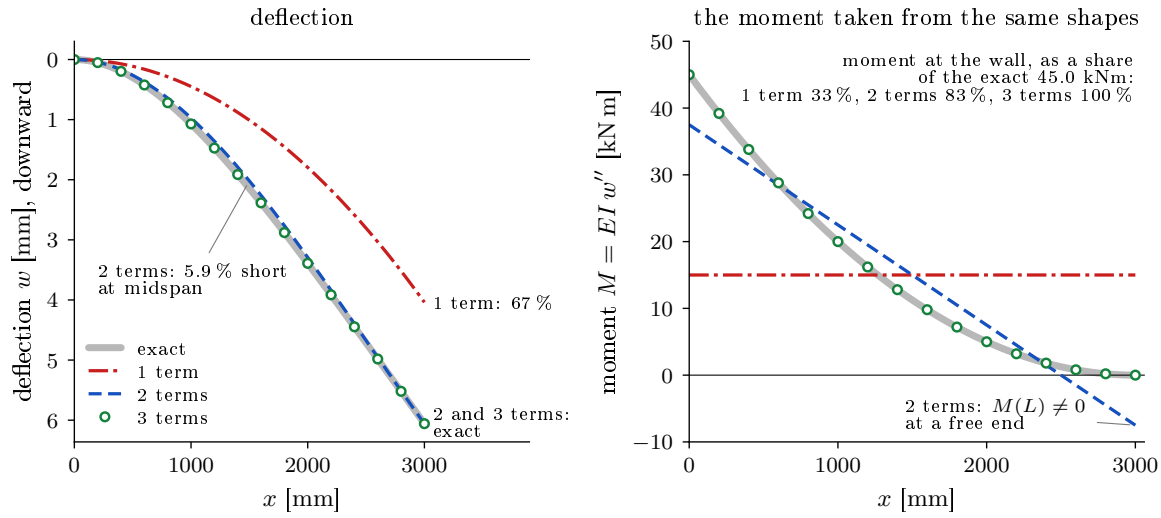


Figure 0.4: The same three approximations, as deflections and as moments. Three terms coincide with the exact solution; two get the tip deflection exactly and miss the middle; the moment is where all of them differ most.

Note

a good deflection means a good moment $\rightarrow$ the moment is the second derivative of the assumed shape, and every derivative of an approximation is worse than the approximation

Two terms have the tip deflection exactly right and the wall moment 17% low. This is the rule rather than an accident of this example: a displacement-based method is most accurate in displacement, less so in slope, less again in moment, and least in shear. Remember it whenever a program reports a stress.

0.3 When an approximate method is exact

Look at the one-term row once more. Its tip deflection is a third too small and its wall moment two thirds too small, yet its tip rotation is right to every digit printed. The intuitive expectation is that an approximation is wrong everywhere, by an amount that shrinks as it improves. The table does not support it, and working out why is the most useful thing in this chapter, because it is also the reason a finite element program is sometimes exact and sometimes not.

Rule 1 — the exact shape is in the family. The method returns the member of its family with the smallest Π . The exact solution has the smallest Π of all shapes whatsoever, so if it is a member of the family, it is the one returned: exactly, everywhere, moments and shears included. Equation (0.2) is a quartic, and three terms reach $(x/L)^4$, which is why the three-term row is exact. Under a tip load the exact shape is a cubic, and two terms suffice (Exercise 0.2).

Rule 2 — the unit-load shape is in the family. This is the rule the one-term row obeys. Making Π stationary over the family is the same as requiring

$$\int_0^L EI w_h'' v'' dx = \int_0^L q v dx \quad \text{for every shape } v \text{ in the family,} \quad (0.6)$$

where w_h is the approximate solution. Now let $g(x) = x^2/2EI$, the shape the cantilever takes under a unit moment at its tip. It is a parabola, so it is in the one-term family. By virtual work, the internal work of g 's moments through any admissible shape v equals the work of the unit moment through v 's tip rotation: $\int EI g'' v'' dx = v'(L)$. Use that twice:

$$w_h'(L) = \int_0^L EI g'' w_h'' dx \stackrel{(0.6)}{=} \int_0^L q g dx = \int_0^L EI g'' w'' dx = w'(L).$$

The third step holds because the exact solution satisfies Eq. (0.6) too, for every admissible shape. So the approximate tip rotation equals the exact one, even though the approximate shape is not the exact shape. The same argument with a unit *force* at the tip, whose shape $x^2(3L - x)/6EI$ is a cubic, is why two terms give the exact tip deflection.

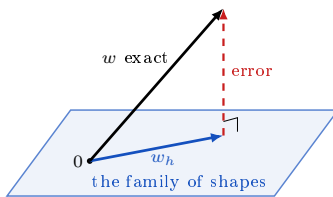
Definition 0.4: When an approximation is exact

An energy-based approximate method returns

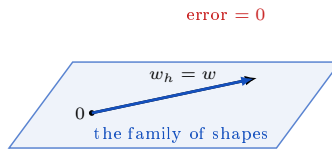
- (a) the exact solution everywhere, if the exact solution is one of the shapes it may choose;
- (b) one displacement exactly, if the shape produced by a unit load at that point, in that direction, is one of the shapes it may choose — whether or not the solution is.

Figure 0.5 shows both parts as geometry, and it is worth studying until the right angle in panel (c) looks obvious.

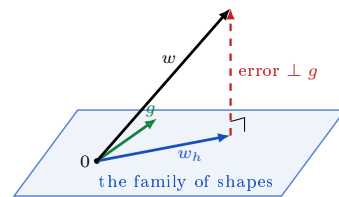
- (a) exact shape not in the family (b) exact shape in the family (c) only a unit-load shape in it



The method returns w_h , the member of the family *closest* to w , and the error stands at right angles to the whole family. One term, uniform load: the tip deflection is 67% of the exact value.



The member of the family closest to w is w itself, so the method cannot miss it. Three terms under a uniform load; two under a tip load; a beam element whenever its loads are at its nodes.



g is the shape a unit load at one point produces. It is in the family, so the error is at right angles to g as well, and the *one* displacement g measures is exact while the others are not. This is why a beam element is exact at its nodes.

“Closest” and “at right angles” are measured in strain energy: two shapes are at right angles when $\int EI w_1'' w_2'' dx = 0$.

Figure 0.5: Every shape as a point, and the family of allowed shapes as a sheet. The method returns the point of the sheet closest to the exact solution, so the error stands at right angles to the sheet.

Definition 0.4(b) has a consequence that week 4 finds by experiment. Between two nodes, a prismatic beam loaded only at its nodes deflects as a cubic, and a beam element uses cubics. So the unit-load shape of every nodal displacement is available to it, and every nodal displacement is exact — whatever the load between the nodes, provided that load is turned into nodal loads in the consistent way of §0.5. That is why refining the beam mesh in Example 4.2 changes nothing at the nodes.

Rule 3 — neither: convergence from the stiff side. Otherwise the answer is wrong, and the useful question is in which direction. Restricting the shapes can only stop the structure from deforming the way it wants to; it can never let it deform more easily. The approximation is *stiffer* than the model: its Π is never below the exact one, the work the load does is never larger, and — for a single load — the deflection under the load is never overestimated. Adding shapes can only move the answer towards the exact value, never past it. §0.5 meets this in a real mesh.

Finite differences obey the same logic in their own form. A central difference reproduces the second derivative of any polynomial up to a cubic exactly, so a problem whose solution is such a polynomial is solved exactly at the grid points.

Before trusting an approximate answer, ask

1. Can the family represent the exact solution? Then the answer is exact, and refining is a waste of time.

2. Can it represent the unit-load shape of the quantity I want? Then that quantity is exact, and the others need not be.
3. Neither? Then the model is too stiff. Refine, and watch the answer grow towards a limit.
4. In every case, trust displacements before moments, and moments before shears.

Note

an approximate method never gives the exact answer → it gives it whenever the exact answer — or the unit-load shape of the quantity you want — is among the shapes it may choose

And the converse is just as important: when neither is, refining the model is the only way to learn how wrong it is. Week 4 is a case of the first kind, week 8 (Example 8.2) and S4 are cases of the second.

0.4 The matrix method

The Rayleigh–Ritz method needs shapes that cover the whole structure, which is hopeless for anything more complicated than one member. The matrix method starts from the other end. It describes the structure by the displacements of a handful of points, the **nodes** (TR: *düğüm noktaları*), and asks how the forces at those points depend on them.

Definition 0.5: Degree of freedom and stiffness matrix

A **degree of freedom** (TR: *serbestlik derecesi*) is one independent displacement or rotation of one node. The **stiffness matrix** (TR: *rijitlik matrisi*) $\mathbf{K}$ relates the forces at the degrees of freedom to their displacements, $\vec{F} = \mathbf{K}\vec{u}$. Its entry K_{ij} is the force at degree of freedom i needed to hold the structure with a unit displacement at j and zero displacement at every other degree of freedom.

That definition is worth reading slowly, because it says how to find a stiffness matrix with no theory at all: impose one unit displacement, hold everything else still, and read off the forces that are needed. They are column j .

Structural analysis has two matrix methods. The **force method** (TR: *kuvvet yöntemi*) takes redundant forces as its unknowns and needs you to decide which ones. The **stiffness method** (TR: *rijitlik yöntemi*) takes the nodal displacements as its unknowns, and the nodes choose themselves. The second can therefore be carried out by a program that knows nothing about the structure except its list of nodes and elements, which is why essentially every structural analysis program uses it, OpenSees included. From here on, “the matrix method” means the stiffness method.

A bar. Along its own axis a bar is a spring of stiffness EA/L . Move one end by one unit with the other held, and the forces needed are EA/L at the moved end and $-EA/L$ at the held one:

$$\mathbf{k} = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (0.7)$$

A bar inclined in a plane has two degrees of freedom at each end. Projecting the axial spring onto the global axes with its direction cosines c and s gives the 4×4 matrix of Listing 0.2.

Listing 0.2: w00/stiffness.py — A bar in global axes. The whole of its physics is the one number EA/L ; the rest is geometry.

```

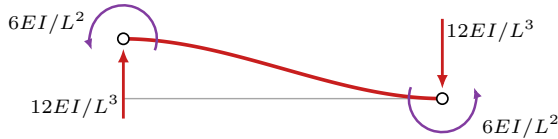
40 def truss_element(xi, yi, xj, yj, E, A):
41     """Stiffness of a pin-jointed bar, in GLOBAL axes: a 4x4 matrix acting on
42     (ux_i, uy_i, ux_j, uy_j).
43
44     Along its own axis a bar is a spring of stiffness EA/L. The cosines c and s
45     turn that one spring into the four global directions.
46     """
47     L = math.hypot(xj - xi, yj - yi)
48     c, s = (xj - xi) / L, (yj - yi) / L
49     t = np.array([[c*c, c*s], [c*s, s*s]])

```

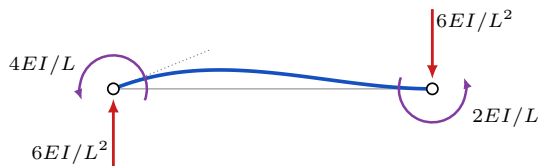
```
50 return (E * A / L) * np.block([[t, -t], [-t, t]])
```

A beam. A beam element has a deflection and a rotation at each end, so its matrix is 4×4 as well, and [Definition 0.5](#) builds it one column at a time. [Figure 0.6](#) does exactly that: four shapes, each with one end displacement set to one, and the end forces that hold it.

column 1: $v_i = 1$, others 0



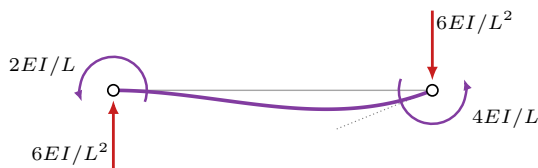
column 2: $\theta_i = 1$, others 0



column 3: $v_j = 1$, others 0



column 4: $\theta_j = 1$, others 0



$$\mathbf{k} = \frac{EI}{L^3} \begin{bmatrix} \text{1} & \text{2} & \text{3} & \text{4} \\ 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix} \begin{bmatrix} v_i \\ \theta_i \\ v_j \\ \theta_j \end{bmatrix}$$

Row: the force or moment at that end.

Column: the shape that produced it.

The matrix is symmetric: the force at 1 due to a unit movement at 2 equals the force at 2 due to a unit movement at 1. That is Maxwell's reciprocal theorem, written as $k_{12} = k_{21}$.

Figure 0.6: The four columns of a beam stiffness matrix, each drawn as the shape it holds. Every entry is a slope–deflection coefficient: $4EI/L$ to rotate one end, $2EI/L$ carried over to the other.

The entries are results you have met in slope–deflection: $4EI/L$ is the moment that rotates an end by one radian with the far end fixed, $2EI/L$ the moment carried over to the far end, and $6EI/L^2$ and $12EI/L^3$ the shears that keep each shape in equilibrium.

Listing 0.3: `w00/stiffness.py` — [Fig. 0.6](#) as code. The order of the degrees of freedom is part of the matrix: change it, and the matrix changes with it.

```
55 def beam_element(E, I, L):
56     """Stiffness of a prismatic beam lying along x: a 4x4 matrix acting on
57     (v_i, theta_i, v_j, theta_j).
58
59     Column k is the set of end forces and moments that holds the beam in the
60     shape where DOF k equals one and the other three are zero.
61     """
62     return (E * I / L**3) * np.array([
```

```

63     [ 12.0,    6*L,  -12.0,    6*L],
64     [  6*L,  4*L*L,   -6*L,   2*L*L],
65     [-12.0,   -6*L,   12.0,   -6*L],
66     [  6*L,  2*L*L,   -6*L,   4*L*L],
67 ])
```

A tip-loaded cantilever is one such element with node i at the wall. Striking out the two degrees of freedom the wall holds leaves a 2×2 system for the tip,

$$\frac{EI}{L^3} \begin{bmatrix} 12 & -6L \\ -6L & 4L^2 \end{bmatrix} \begin{bmatrix} v \\ \theta \end{bmatrix} = \begin{bmatrix} -P \\ 0 \end{bmatrix} \Rightarrow v = -\frac{PL^3}{3EI}, \quad \theta = -\frac{PL^2}{2EI}, \quad (0.8)$$

which is the closed form of week 1 exactly — by Definition 0.4(a), because the element's cubic *is* the deflected shape of a tip-loaded cantilever. With the week 1 numbers it gives -5.3854 mm, the value week 1 obtains from OpenSees.

Assembly. A structure is several elements sharing nodes. Where two elements meet, they are two springs holding the same joint, and their stiffnesses add. Figure 0.7 does it with the real numbers of the two-bar truss that week 2 models in OpenSees.

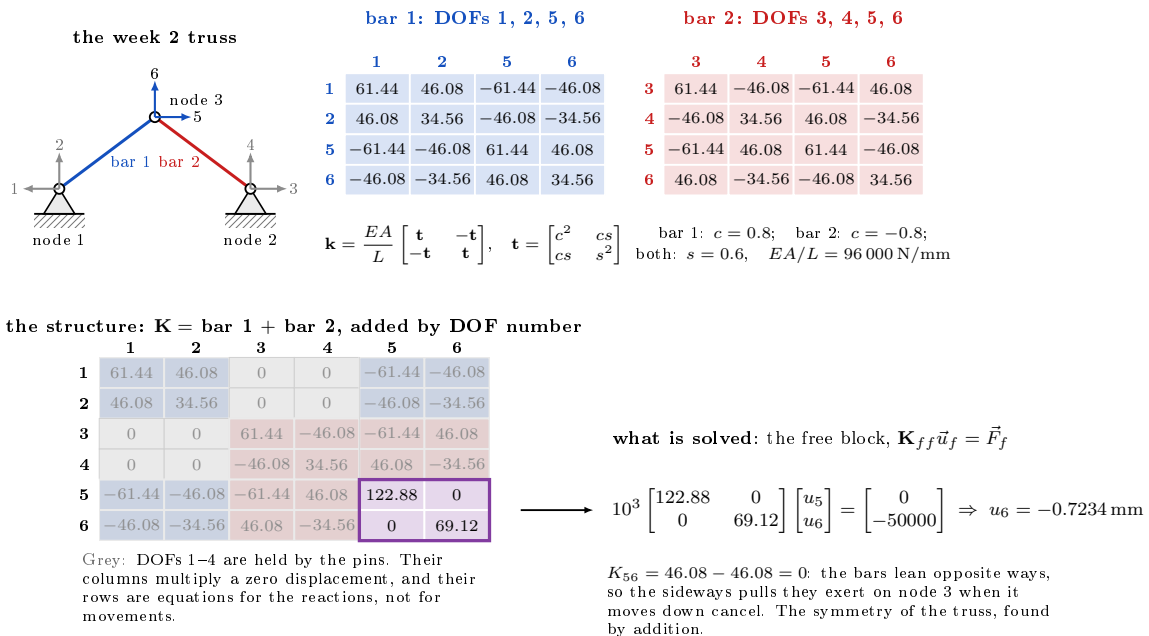


Figure 0.7: Assembly. Each bar's matrix is added into the structure's matrix at the rows and columns of its own degrees of freedom; the supports set aside DOFs 1–4, and a 2×2 system is what is solved.

The direct stiffness method

1. Number the degrees of freedom, node by node.
2. Compute each element's stiffness matrix in global axes.
3. Add each one into $\mathbf{K}$ at the rows and columns of its own degrees of freedom.
4. Build $\vec{F}$ from the nodal loads (and, in §0.5, from loads between nodes).
5. Set aside the rows and columns of the supported degrees of freedom and solve the rest, $\mathbf{K}_{ff} \vec{u}_f = \vec{F}_f$.
6. Recover the reactions, $\vec{R} = \mathbf{K} \vec{u} - \vec{F}$, and each member's forces from the displacements of its own nodes.

Example 0.3 (The week 2 truss, by the matrix method). Two steel bars, $E = 200\,000$ MPa and $A = 1200\text{ mm}^2$, run from pinned supports at $(0,0)$ and $(4000,0)$ to an apex at $(2000,1500)$, which carries $P = 50\text{ kN}$ downward. Find the apex movement and the bar forces.

Solution

Model. Pin-jointed bars, linear elastic, small displacements. Nodes 1, 2 and 3 carry DOFs $(1,2)$, $(3,4)$ and $(5,6)$, x before y , as in Fig. 0.7.

Element matrices. Both bars are 2500 mm long, so $EA/L = (200000)(1200)/2500 = 96\,000\text{ N/mm}$. Bar 1 points from node 1 to node 3 with $c = 0.8$, $s = 0.6$; bar 2 from node 2 to node 3 with $c = -0.8$, $s = 0.6$.

Assembly and supports. DOFs 1–4 are pinned. Only the $(5,6)$ block survives, and it receives one contribution from each bar:

$$\mathbf{K}_{ff} = 96000 \begin{bmatrix} c_1^2 + c_2^2 & c_1 s_1 + c_2 s_2 \\ c_1 s_1 + c_2 s_2 & s_1^2 + s_2^2 \end{bmatrix} = \begin{bmatrix} 122880 & 0 \\ 0 & 69120 \end{bmatrix} \text{ N/mm}.$$

Solve. With $\vec{F}_f = (0, -50000)$, the two equations are uncoupled: $u_5 = 0$ and $u_6 = -50000/69120 = -0.7234\text{ mm}$.

Member forces. A bar's axial force is EA/L times its elongation, and its elongation is the relative displacement of its ends projected on its axis: $N_1 = 96000(0.6)(-0.7234) = -41\,670\text{ N}$, and N_2 is the same by symmetry.

$$u_6 = -0.723\text{ mm}, \quad N_1 = N_2 = -41.7\text{ kN}$$

The script does the same, in the six steps of the recipe:

Listing 0.4: `w00/stiffness.py` — Assembly in two lines. Note the `+=`: it is the whole of the idea that two members meeting at a joint add their stiffness.

```

90 def assemble(n_dof, elements):
91     """Add every element matrix into the structure's matrix.
92
93     `elements` is a list of (k, dofs): an element's matrix and the global DOF
94     numbers its rows and columns belong to. Where two elements share a node
95     they share DOFs, and their stiffnesses ADD -- two members meeting at a
96     joint are two springs holding it.
97     """
98     K = np.zeros((n_dof, n_dof))
99     for k, dofs in elements:
100         K[np.ix_(dofs, dofs)] += k
101     return K

```

Listing 0.5: `w00/stiffness.py` — Supports, solution and reactions. The reactions come back from the full matrix afterwards.

```

106 def solve(K, F, fixed):
107     """Apply the supports, solve, and recover the reactions.
108
109     A support says a displacement is zero, so its column multiplies nothing and
110     its row is an equation for a reaction rather than for a displacement. Both
111     are set aside, the remaining 'free' block is solved, and the reactions come
112     back from the full matrix afterwards.
113     """
114     n = len(F)
115     free = [d for d in range(n) if d not in fixed]
116     u = np.zeros(n)
117     u[free] = np.linalg.solve(K[np.ix_(free, free)], F[free])
118     R = K @ u - F                                # nonzero only where a support is
119     return u, R, free

```

Listing 0.6: w00/stiffness.py — The week 2 truss, built from the three functions above. Python counts from zero, so DOF 6 of Fig. 0.7 is F[5] here.

```

124 def truss(p=TRUSS):
125     """The week 2 truss. Node n owns DOFs 2n and 2n+1 (x then y), counting
126     nodes from 0 as Python does."""
127     xy = [(0.0, 0.0), (p["span"], 0.0), (p["span"] / 2, p["H"])]
128     bars = [(0, 2), (1, 2)] # node pairs
129     elements = []
130     for i, j in bars:
131         k = truss_element(*xy[i], *xy[j], p["E"], p["A"])
132         elements.append((k, [2*i, 2*i + 1, 2*j, 2*j + 1]))
133
134     K = assemble(2 * len(xy), elements)
135     F = np.zeros(2 * len(xy))
136     F[5] = -p["P"] # apex, downward
137     u, R, free = solve(K, F, fixed=[0, 1, 2, 3]) # both supports pinned
138
139     N = []
140     for i, j in bars: # axial force from elongation
141         (xi, yi), (xj, yj) = xy[i], xy[j]
142         L = math.hypot(xj - xi, yj - yi)
143         c, s = (xj - xi) / L, (yj - yi) / L
144         elong = c * (u[2*j] - u[2*i]) + s * (u[2*j + 1] - u[2*i + 1])
145         N.append(p["E"] * p["A"] / L * elong)
146     return {"K": K, "K_ff": K[np.ix_(free, free)], "u": u, "R": R, "N": N}

```

```

Two-bar truss of week 2, by the direct stiffness method
structure matrix 6 x 6, free block 2 x 2:
      122880.0      0.0  N/mm
      0.0      69120.0  N/mm
apex      ux =      0.0000 mm      uy =     -0.7234 mm
bar forces      N1 =    -41666.67 N      N2 =    -41666.67 N
reactions, node 1 ( 33333.33, 25000.00) N
reactions, node 2 (-33333.33, 25000.00) N

```

Check. Against a method that shares nothing with this one but the model: joint equilibrium at the apex. Each bar carries $P/2$ vertically, so $N = -(P/2)/\sin\theta = -25000/0.6 = -41670$ N; each bar shortens by $NL/EA = -0.4340$ mm, and the apex drops by $-0.4340/0.6 = -0.7234$ mm. Both agree. So does equilibrium of the whole: the vertical reactions sum to 2×25000 N, which is P .

Note

a program that has never seen a truss needs to be told how a truss works → it needs element matrices and a list of which degrees of freedom each one touches, and nothing else

`assemble()` does not know what a bar is, nor what a node is. It adds matrices into a larger matrix at the positions it is given. Hand it a beam element, a plate element or a spring and it does the same thing, unchanged. That indifference is the property the rest of this chapter turns on.

0.5 The finite element method

The matrix method is exact for bars, and for beams loaded at their nodes, because for those two the shape between the nodes is known exactly: straight for a bar, a cubic for a beam. The finite element method is what the matrix method becomes when it is not.

Definition 0.6: Finite element method

The **finite element method** (*TR: sonlu elemanlar yöntemi*) divides a structure into **elements** (*TR: elemanlar*) joined at nodes, assumes inside each element a displacement field fixed entirely by the displacements of that element's own nodes, computes each element's stiffness matrix from that field, and then assembles and solves exactly as the matrix method does. It is the Rayleigh–Ritz method with shapes that are simple, local, and generated automatically.

The local shapes are called **shape functions** (*TR: şekil fonksiyonları*). Each belongs to one degree of freedom: it equals one there and zero at every other node, so that

$$u(x) \approx \sum_i N_i(x) u_i. \quad (0.9)$$

This is Eq. (0.4) with the amounts a_i given a physical meaning — u_i is now the displacement of node i — and Fig. 0.8 shows both kinds used in this course.

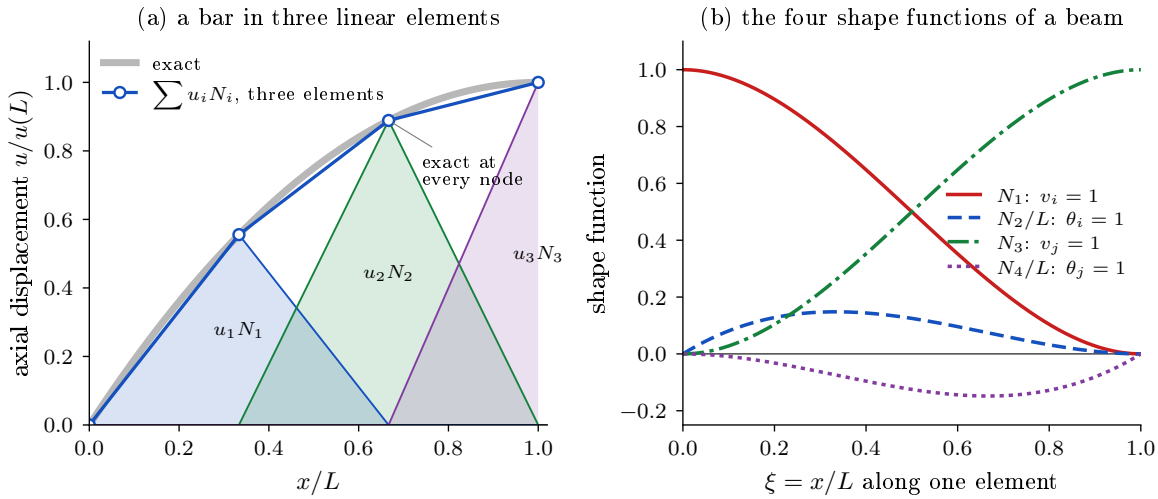


Figure 0.8: Left, a bar in three linear elements: the answer is the sum of the hats, straight between nodes and exact at them. Right, the four shape functions of a beam element, the shapes of Fig. 0.6.

Element stiffness from energy. For a bar element, Eq. (0.9) with linear hats makes the strain constant, $\varepsilon = u' = (u_j - u_i)/L$, and the strain energy $\frac{1}{2} \int EA \varepsilon^2 dx = \frac{1}{2} (EA/L)(u_j - u_i)^2$. Written as $\frac{1}{2} \vec{u}^T \mathbf{k} \vec{u}$, that is exactly Eq. (0.7). In general

$$k_{ij} = \int_0^L EA N'_i N'_j dx \text{ (bar)}, \quad k_{ij} = \int_0^L EI N''_i N''_j dx \text{ (beam)}, \quad (0.10)$$

which is Eq. (0.5) again, one element at a time. Put the four cubics of Fig. 0.8 into the second and out comes the beam matrix of Fig. 0.6, entry by entry. For bars and beams, the matrix method and the finite element method are the same method reached from two directions.

Loads between nodes. $\mathbf{K} \vec{u} = \vec{F}$ can only carry forces at nodes. A load spread along an element is replaced by the nodal forces that do the same work through every allowed shape, $F_i = \int q N_i dx$ — the **equivalent nodal loads** (*TR: eşdeğer düğüm yükleri*). For a uniform load on a beam element they are $qL/2$ at each end and a pair of moments $\pm qL^2/12$: the fixed-end forces of slope–deflection, reversed. OpenSees computes them whenever you use `eleLoad` (week 4).

Example 0.4 (The cantilever as finite elements). Model the cantilever of [Example 0.1](#) with one, two and four beam elements, and compare the tip deflection, tip rotation and wall moment with the exact values.

Solution

Model. Prismatic beam elements, the uniform load replaced by its equivalent nodal loads, the wall's two degrees of freedom set aside.

One element, by hand. The system is [Eq. \(0.8\)](#) with the tip load replaced by the equivalent nodal loads at the tip, $-qL/2$ and $+qL^2/12$:

$$\frac{EI}{L^3} \begin{bmatrix} 12 & -6L \\ -6L & 4L^2 \end{bmatrix} \begin{bmatrix} v \\ \theta \end{bmatrix} = \begin{bmatrix} -qL/2 \\ qL^2/12 \end{bmatrix} \implies v = -\frac{qL^4}{8EI}, \quad \theta = -\frac{qL^3}{6EI}.$$

Both are exact, from a single element. With its wall fixed, the element's shape is a combination of x^2 and x^3 — so the one-element model *is* the two-term Ritz approximation of [Example 0.2](#), and it inherits both its exact tip and its 5.9% error at midspan.

By script.

Listing 0.7: `w00/stiffness.py` — The same beam matrix and the same `assemble()` and `solve()` as the truss. Only the element and the loads are new.

```

151 def cantilever(p=BEAM, n_elements=1, P_tip=0.0):
152     """The week 1 cantilever, fixed at x = 0, in n equal beam elements.
153
154     A load BETWEEN nodes has to be turned into nodal forces before K u = F can
155     carry it. The uniform load q becomes, on each element, the 'equivalent
156     nodal loads' qL/2 at each end and a pair of moments qL^2/12.
157     """
158     n_dof = 2 * (n_elements + 1)          # v and theta at every node
159     Le = p["L"] / n_elements
160     elements, F = [], np.zeros(n_dof)
161     for e in range(n_elements):
162         dofs = [2*e, 2*e + 1, 2*e + 2, 2*e + 3]
163         elements.append((beam_element(p["E"], p["I"], Le), dofs))
164         q = -p["q"]                        # downward
165         F[dofs] += [q*Le/2, q*Le**2/12, q*Le/2, -q*Le**2/12]
166         F[-2] += -P_tip                    # optional point load at the tip
167
168     K = assemble(n_dof, elements)
169     u, R, free = solve(K, F, fixed=[0, 1]) # wall: v = 0, theta = 0
170     return {"v_tip": u[-2], "theta_tip": u[-1], "R_wall": R[0], "M_wall": R[1],
171            "v": u[0::2], "theta": u[1::2], "K_ff": K[np.ix_(free, free)]}
```

The same cantilever under q = 10 N/mm, n elements			
elements	tip deflection [mm]	tip rotation [rad]	wall moment [kNm]
1	-6.0585	-0.002693	45.000
2	-6.0585	-0.002693	45.000
4	-6.0585	-0.002693	45.000

every mesh: $v = -6.06 \text{ mm}$, $\theta = -2.69 \times 10^{-3} \text{ rad}$, $M(0) = 45.0 \text{ kN m}$

Check. Against [Eq. \(0.2\)](#): all three agree to every digit printed, and the signs are those of global axes with y upward. Two things deserve notice. Refining the mesh changes nothing, as [Definition 0.4\(b\)](#) predicts. And the wall moment is exact even with one element, although the same element, differentiated twice, would give the two-term Ritz value of 37.5 kN m: the end forces come from $k\bar{u}$ and the equivalent loads — from equilibrium — not from the second derivative of the shape. OpenSees's `elasticBeamColumn` recovers its end forces the same way.

Where the elements are genuinely approximate. In a wall, a slab or a solid, no simple polynomial is the exact displacement field, and with that the exactness of the last example disappears: [Definition 0.4](#) no longer applies, and Rule 3 takes over.

Example 0.5 (A wall in quadrilaterals). A concrete wall 3000 mm long, 600 mm deep and 200 mm thick ($E = 30\,000$ MPa, $\nu = 0.2$) is built in at one end and carries 100 kN downward, spread over the other. Model it as a two-dimensional solid in four-node quadrilateral elements on five meshes, from 5×1 to 80×16 , and compare the tip deflection with beam theory.

Solution

Model. Plane stress, linear elastic, every node at the built-in end fixed. Inside each element the displacement varies bilinearly — and a bilinear field cannot bend without also shearing, which a real beam does not need to do. The analyses run in OpenSees; the script is in the repository, and it will read easily once weeks 2–5 are behind you.



Python: w00/continuum.py

The wall, meshed, analysed on five meshes, and compared with beam theory.

Cantilever wall 3000 x 600 x 200 mm, E = 30000 MPa, P = 100 kN			
elements	tip deflection [mm]	/ finest mesh	
5	5.8571	0.6858	
20	7.6376	0.8942	
80	8.3017	0.9720	
320	8.4911	0.9942	
1280	8.5411	1.0000	
Euler-Bernoulli beam	8.3333		
Timoshenko beam	8.5733		

tip deflection: 5.86 mm (5 elements) → 8.54 mm (1280 elements)

Check. Beam theory is independent of the mesh. With $I = th^3/12 = 3.600 \times 10^9 \text{ mm}^4$, bending alone gives $PL^3/3EI = 8.333 \text{ mm}$; a wall this deep also shears, and Timoshenko's correction adds $PL/(\frac{5}{6}GA) = 0.2400 \text{ mm}$ with $G = E/2(1 + \nu) = 12\,500 \text{ MPa}$, for 8.573 mm. The finest mesh is 0.4 % below that, and it has no way of knowing that beam theory exists. Every mesh is below the one after it: the approach is from the stiff side, as Rule 3 requires.

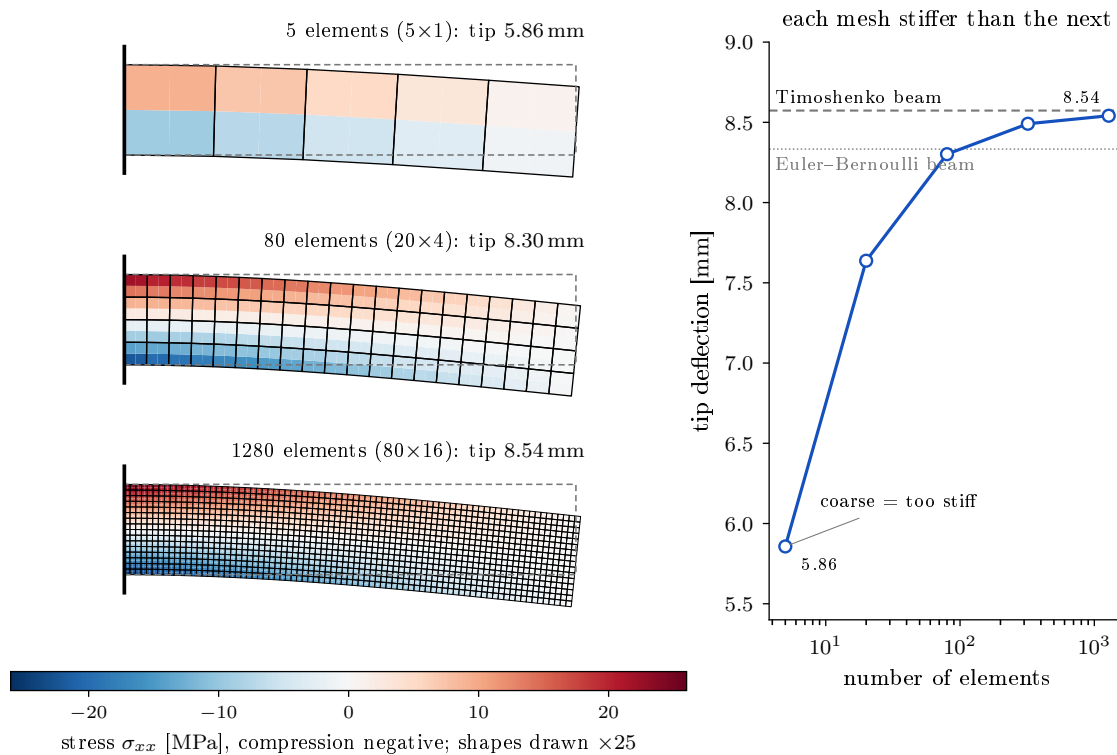


Figure 0.9: What a finite element model of a continuum looks like, and what refining it does. The coarsest mesh, one element through the depth, can hardly bend at all.

Note

a coarse model is a conservative model $\rightarrow$ a coarse displacement-based model is too stiff, and underestimates deflection

Five elements report 5.86 mm, less than even Euler-Bernoulli theory, which ignores shear altogether. Checked against a deflection limit, that model would pass a wall the finer ones fail. Figure 0.9 shows the mechanism: a bilinear element forced to bend resists by shearing instead, and its stresses come out too small as well.

0.6 Why structural analysis is done with finite elements

Every reason is something this chapter has already shown.

It asks for nothing the structure does not have. Any geometry, any supports, any loads, any material: an irregular building is a list of nodes and elements like any other. Bars, beams, walls, slabs, solids and springs sit in the same model, each described by its own element.

One algorithm serves every structure. `assemble()` never looked inside an element. The program is written once; new kinds of structure are handled by adding new elements, not new programs. OpenSees is organised exactly this way — libraries of elements, sections and materials, plugged into one domain that assembles and solves them.

It needs no judgement to set up the unknowns. The force method needs redundants chosen by an engineer; the stiffness method takes its unknowns from the node list. Statical indeterminacy costs nothing: add a third bar to the truss and the same script solves it (Exercise 0.4).

It is cheap enough to use on real structures. Each degree of freedom is coupled only to those of the nodes it shares an element with, so almost all of $\mathbf{K}$ is zero. Figure 0.10 shows the matrix of a ten-storey, three-bay frame: 120 degrees of freedom, 14 400 entries, and only 828 of them nonzero, 5.75%. The count per row stays near seven however large the frame grows — for a 40-storey, ten-bay frame it is 7.4 per row, 0.56% of the matrix — so storage and work grow roughly with the size of the model rather than with its square. Stored

in full, a matrix with 60 000 rows would occupy 28.8 GB; its nonzero values alone, at this frame's 7.4 per row, occupy 3.6 MB. Where the nonzeros sit depends only on the order the degrees of freedom are numbered in, and choosing that order is the job of `ops.numberer` (week 3).

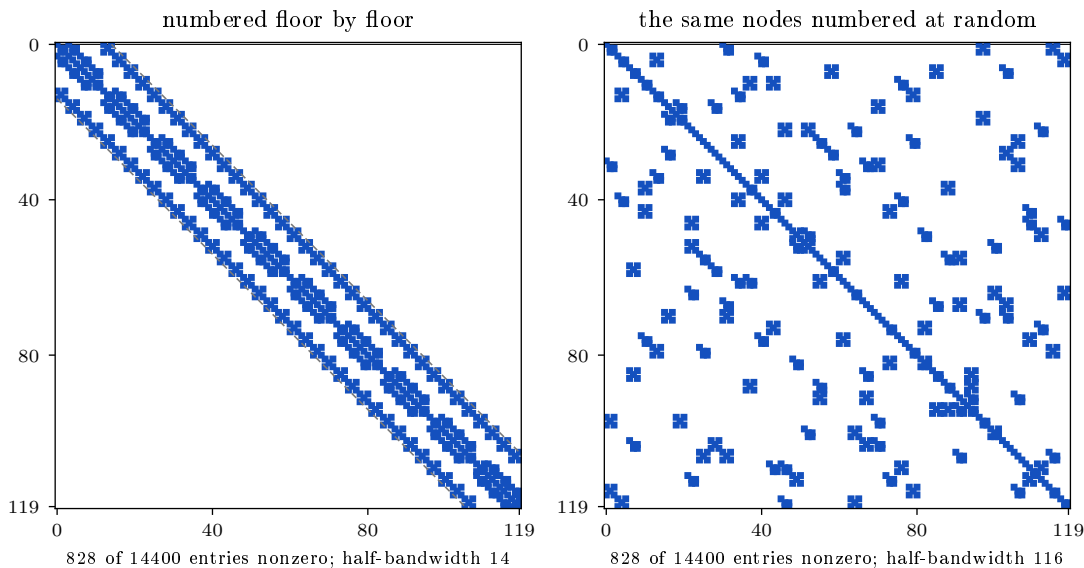


Figure 0.10: The stiffness matrix of a ten-storey, three-bay frame. Numbered floor by floor, its 828 nonzero entries lie within 14 places of the diagonal; numbered at random, the same 828 are scattered over the whole matrix.



Python: `w00/bandwidth.py`

The frame assembled with the same `frame_element()` and `assemble()` as §0.4, then examined rather than solved, under three numberings.

Its accuracy can be controlled and checked. Refine the mesh and the answer converges, from the stiff side; the rules of §0.3 say in advance when refining is pointless. An error that can be measured is an error that can be managed.

It extends without changing shape. When a material yields, $\mathbf{K}$ depends on $\vec{u}$ and the load is applied in steps, each iterated to equilibrium (weeks 6–11). When inertia matters, the same $\mathbf{K}$ is joined by a mass and a damping matrix, $\mathbf{M}\ddot{\vec{u}} + \mathbf{C}\dot{\vec{u}} + \mathbf{K}\vec{u} = \vec{p}(t)$ (weeks 12–13). Nodes, elements and assembly stay as they are.

What the finite element method does not do is tell you whether the model is right. It accepts a wall meshed with five elements as readily as one meshed with 1280, a modulus in the wrong units as readily as one in the right ones, and returns each answer with the same six confident digits. Figure 0.11 sets the steps of the method against the OpenSees commands that perform them. Seven of the nine are commands you will write in a script. The first and the last are not commands at all, and they are what this course is really about.

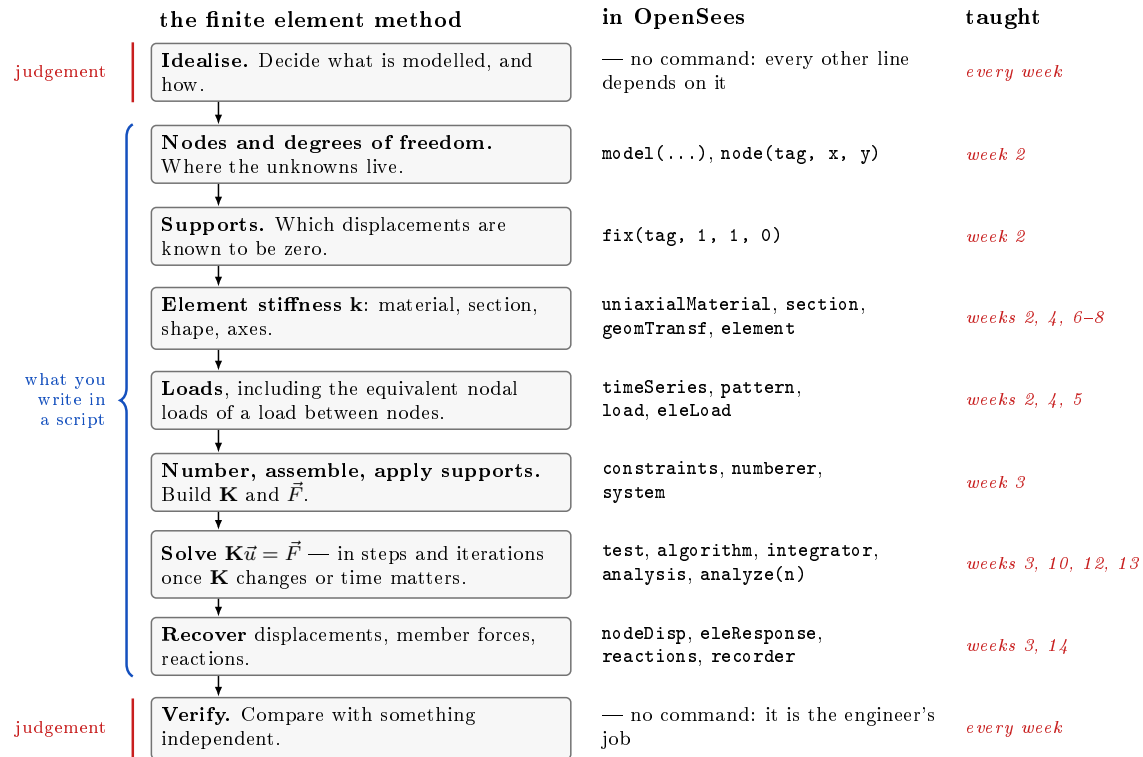


Figure 0.11: Each step of the finite element method, the OpenSees command that does it, and the week that teaches it.

0.7 Ground rules

A few conventions, used without further comment throughout these notes. You are expected to follow them in homework, in the project and in examinations. The first of them is the one that will otherwise cost you the most marks in this course.

Units

OpenSees has no units. It does not know whether a length is a metre or a millimetre. It multiplies the numbers you hand it — into exactly the element matrices of §0.4 — and returns numbers in whatever system those numbers implied. A model whose lengths are in metres and whose modulus is in megapascals is not rejected, does not warn, and produces an answer that is wrong by a factor of a billion while looking entirely ordinary.

This is the most common serious error made by people learning OpenSees, and the only defence is to choose one system and never deviate. This course uses **newton, millimetre, second**.

Quantity	Unit	As written	Note
length	mm	3000	a 3 m span
force	N	10000	a 10 kN load
time	s	0.01	
stress, modulus	MPa	200000	= N/mm ²
moment	N mm	3e7	
stiffness	N/mm	96000	an entry of K
mass	N s ² /mm	1.0	= 1000 kg. Not kilograms.
density	N s ² /mm ⁴	7.85e-9	steel
acceleration	mm/s ²	9810	<i>g</i>

The advantage of this choice is that stress comes out in megapascals, so material data can be written exactly as it appears in the standards: a modulus of 200000, a yield strength of 420, a concrete strength of 30. Nothing has to be converted, so nothing can be converted wrongly.

The price is the mass row, and it deserves a paragraph of its own. Once force and length are chosen, $F = ma$ fixes the unit of mass — it is not free. In this system that unit is N s²/mm, which happens to equal 1000 kg, one tonne. A mass entered in kilograms is therefore a thousand times too large, and since the natural period goes as $\sqrt{m}$, every period you compute comes out $\sqrt{1000} \approx 31.6$ times too long. That is far too large to be roundoff and far too orderly to be a modelling error, which is precisely why it is so often misdiagnosed. If a period looks absurd, check the mass units before you check anything else.

Numbers

The **decimal separator is a point**: 9.81, not 9,81. A space separates a number from its unit: 25 mm.

Intermediate values are carried to **four significant figures** and final answers quoted to **three**, with units. Rounding early is the commonest source of a wrong last digit.

A number that came out of a solver is quoted to four significant figures and compared with a **tolerance**, never tested for exact equality. Two correct computations of the same quantity routinely differ in the last bits, and a comparison that demands they agree exactly will fail for reasons that have nothing to do with structural engineering.

Code

Every script in these notes runs on its own. You can open it, run it, change a number and run it again, and that is the intended way to read them.

- The import is always `import openseespy.opensees as ops`.
- **Every model function begins with `ops.wipe()`**. OpenSees holds one model in memory; without this you silently extend the previous one.
- Parameters live in a dictionary at the top of the file. Below it, no bare numbers.
- Comments explain *why*, not what. The code already says what.

The listings printed in these notes are not transcriptions. They are pulled out of the real script files when the notes are compiled, so what you read here is exactly what is in the repository, and the two cannot drift apart. Each listing is headed by the path of the file it came from, and the line numbers are that file's own — so you can always open the script and find the lines.

Every number printed in these notes is recomputed by a separate program before they will build, and every script shown is executed by it. If you find a number here that your own run disagrees with, that is worth reporting: it means something has changed underneath both of us.

Common errors

Four that recur, listed here so they can be named later in one word:

- mixing unit systems — see above, and suspect it whenever an answer is out by a power of ten;
- forgetting `ops.wipe()`, so a model is built on top of the previous one;
- ignoring the return value of `ops.analyze()`, which reports failure by returning a negative number rather than by raising;
- entering a concrete compressive strength as a positive number where OpenSees expects it negative.

Summary

Item	Expression	Note
beam equation	$EI w'''' = q, M = EI w'', V = -EI w'''$	Eq. (0.1)
exact method	satisfies it at every point	exact to the model only
total potential energy	$\Pi = \frac{1}{2} \int EI (w'')^2 dx - \int q w dx$	smallest for the true shape
Rayleigh–Ritz	$K_{ij} = \int EI \phi_i'' \phi_j'' dx, F_i = \int q \phi_i dx$	Eq. (0.5)
exact again when	the solution, or the unit-load shape, is in the family	Definition 0.4
otherwise	too stiff; converges from below	Rule 3
stiffness coefficient	K_{ij} : force at i for unit displacement at j	Definition 0.5
bar	$\mathbf{k} = (EA/L) [1 \ -1; \ -1 \ 1]$	Eq. (0.7)
beam	$12EI/L^3, 6EI/L^2, 4EI/L, 2EI/L$	Fig. 0.6
assembly	$\mathbf{K}[\text{np.ix_}(\text{dofs}, \text{dofs})] += \mathbf{k}$	stiffnesses add
supports	solve $\mathbf{K}_{ff} \vec{u}_f = \vec{F}_f; \vec{R} = \mathbf{K} \vec{u} - \vec{F}$	
element stiffness	$k_{ij} = \int EI N_i'' N_j'' dx$	Eq. (0.10)
equivalent nodal loads	$F_i = \int q N_i dx: qL/2, qL^2/12$	reversed fixed-end forces
units	N, mm, s; mass in $\text{N s}^2/\text{mm}$	1 unit of mass = 1000 kg

Exercises

Exercise 0.1. ●●● Classify each of the following as exact or approximate, and say with respect to what: (a) the moment–area method applied to a prismatic beam; (b) finite differences with ten intervals along a beam; (c) OpenSees `elasticBeamColumn` elements for a frame loaded only at its nodes; (d) OpenSees `quad` elements for the wall of [Example 0.5](#).

Answer: (a) exact, to the model; (b) approximate; (c) exact at the nodes, to the frame model, because the cubic shape functions contain the exact shape; (d) approximate, and too stiff

Solution. (a) Moment–area integrates the curvature diagram exactly, so it is exact with respect to the Euler–Bernoulli model, and carries all of that model’s idealisation error.

(b) Finite differences replace derivatives by differences, so they are approximate — although by the end of §0.3 they would be exact at the grid points for a solution that is a cubic or less.

(c) This is the interesting one. Between nodes, a prismatic frame member loaded at its ends deflects as a cubic, and the element’s shape functions are cubics, so by [Definition 0.4](#)(a) the element reproduces the exact solution of the frame model. The program is an approximate method that happens, for this element and this loading, to be exact.

(d) A bilinear field cannot represent pure bending, so neither part of [Definition 0.4](#) applies and Rule 3 governs: the answer is too stiff and converges from below as the mesh is refined, exactly as

[Example 0.5](#) shows.

The point of the exercise is that “exact” and “approximate” describe a method *applied to a problem*, not a piece of software. The same program is exact in (c) and approximate in (d).

Exercise 0.2. ●●● Replace the uniform load of [Example 0.2](#) by the week 1 tip load, $P = 10$ kN, by calling `tip_values(dict(BEAM, q=0.0), n, P_tip=10000.0)` for one, two and three terms. Before running it, predict how many terms give the exact tip deflection. Then explain why the one-term answer is the same number as in [Example 0.2](#), although the load is entirely different.

Answer: 4.04 mm with one term (75 %); 5.39 mm, exact, with two and with three; the one-term approximation sees a load only through $F_1 = \int q\phi_1 dx + P\phi_1(L)$, which is 10 kN in both cases

Solution. The exact tip-load shape is the cubic $Px^2(3L - x)/6EI$, so two terms contain it and, by [Definition 0.4\(a\)](#), give it exactly: 5.385 mm everywhere, which is $PL^3/3EI$. A third term adds nothing, and its amplitude comes out zero to roundoff.

One term gives $a = PL^3/4EI = 4.039$ mm, three quarters of the exact value.

The coincidence with [Example 0.2](#) is not a coincidence. A one-term approximation knows the load only through the single number F_1 : for the uniform load that is $\int_0^L q(x/L)^2 dx = qL/3 = 10\,000$ N, and for the tip load it is $P\phi_1(L) = 10\,000$ N. Two different loads that do the same work through the only allowed shape are, to that approximation, the same load. This is the idea behind equivalent nodal loads in §0.5, met here from the other side.

Exercise 0.3. ●●● A student writes the loop inside `assemble()` as

```
for k, dofs in elements:
    K[np.ix_(dofs, dofs)] = k
```

The script runs without an error and reports an apex settlement of the order of 10^{15} mm. Find the mistake, and explain in terms of the structure why the answer is absurd rather than merely wrong.

Answer: = overwrites instead of adding, so node 3 is held by bar 2 alone; a single bar cannot hold a joint in two directions, the free block is singular to roundoff, and the solver returns noise

Solution. With = each element’s matrix replaces whatever is already at its degrees of freedom, so at

DOFs 5 and 6, where both bars meet, only bar 2 survives. Its block is $96000 \begin{bmatrix} 0.64 & -0.48 \\ -0.48 & 0.36 \end{bmatrix}$, whose

determinant is $0.64 \times 0.36 - 0.48^2 = 0$.

That zero is physical. One bar can resist movement of node 3 along its own axis and offers nothing across it, so the joint can slide freely at right angles to bar 2: a mechanism. The equations have no unique solution. Because the determinant is zero only up to roundoff, `np.linalg.solve` does not refuse — it pivots on a number that is roundoff rather than stiffness, and returns displacements of order 10^{15} mm.

The correction is +=. The lesson is the shape of the failure: no exception, an astronomically large displacement, and a cause several lines away. When an OpenSees analysis reports a singular system or an absurd displacement, the first question is the same one — what is holding this joint, and in which directions?

Exercise 0.4. ●●● Add a third bar to the truss of [Example 0.3](#): vertical, from the apex down to a new pinned support at (2000, 0), with the same section. Using `truss_element()`, `assemble()` and `solve()`, report the apex settlement and the force in each bar. Which method from structural analysis would have needed a redundant to be chosen here, and did the script need one?

Answer: $u_6 = -0.218$ mm; $N_1 = N_2 = -12.6$ kN, $N_3 = -34.9$ kN; the force method would need one redundant; the script needed nothing but a fourth node and a third element

Solution. The new bar is 1500 mm long with $c = 0$, $s = 1$, so it adds $EA/L = (200000)(1200)/1500 =$

160 000 N/mm to K_{66} and nothing to K_{55} or K_{56} . The free block becomes $\text{diag}(122880, 229120)$, and $u_6 = -50000/229120 = -0.2182$ mm.

The inclined bars shorten by 0.6×0.2182 and carry $N_1 = N_2 = 96000(0.6)(-0.2182) = -12570$ N; the vertical bar shortens by the full settlement and carries $N_3 = 160000(-0.2182) = -34920$ N. Equilibrium checks it: $2(12570)(0.6) + 34920 \approx 50000$ N, to the figures carried.

The truss is now statically indeterminate to the first degree. The force method would have needed one redundant released and a compatibility equation written for it. The stiffness method needed a fourth node, a third element and two more entries in the list of fixed DOFs, and the load shared itself between the bars in proportion to their stiffness without being asked.

Exercise 0.5. ●●● Run the cantilever of [Example 0.4](#) with two elements and report the deflection at midspan. Compare it with the exact value from [Eq. \(0.2\)](#) and with the one-element model's value at the same point, and explain the difference.

Answer: two elements: -2.15 mm, exact; one element, interpolated at midspan: -2.02 mm, 5.9 % short; midspan is a node in the first model and an interior point in the second

Solution. Equation (0.2) at $x = L/2$ gives $17qL^4/384EI = 2.146$ mm, and the two-element model reports -2.146 mm at its middle node — exact, by [Definition 0.4\(b\)](#), because midspan is now a node.

The one-element model has no node there. Its value at midspan is the element's cubic, interpolated from the tip values, which is the two-term Ritz shape: 2.02 mm, the 5.9 % shortfall of [Fig. 0.4](#).

So the value at a point is exact if the point is a node and approximate if it is not. The practical rule of week 4 follows directly: put a node wherever you want an answer.

Exercise 0.6. ●●● A student enters the modulus in `stiffness.py` as `E = 200`, meaning gigapascals, and leaves everything else in newtons and millimetres. What does the truss of [Example 0.3](#) report for the apex settlement, and how would you recognise the mistake without looking at the code?

Answer: -723 mm, a thousand times too large; an error of exactly a power of ten is a unit error

Solution. Every entry of every element matrix is proportional to E , so $\mathbf{K}$ is a thousand times too small and the displacements a thousand times too large: $-0.7234 \times 1000 = -723.4$ mm. The bar forces are unchanged — the truss is statically determinate, so equilibrium alone fixes them, whatever the stiffness.

Two things give it away at once. A settlement of 723 mm under a 4 m span is absurd for a steel truss. And its ratio to the correct value is exactly 1000: modelling mistakes almost never produce a clean power of ten, and unit mistakes almost always do.

Exercise 0.7. ●●● A classmate argues that the five-element model of [Example 0.5](#) is “conservative, because it is simpler”. Explain what is wrong with the argument, what the coarse model would do to a serviceability check, and what you would ask for before accepting any finite element deflection of a wall or a slab.

Answer: a displacement-based model is too stiff, so a coarse mesh underestimates deflection (5.86 mm against 8.54 mm) and would pass a wall that fails; ask for a mesh study showing the answer has stopped growing

Solution. Simplicity is not the same as conservatism, and here it points the other way. Restricting the shapes an element may take can only make the model stiffer (Rule 3), so the coarsest mesh reports the smallest deflection of all: 5.86 mm, 31 % below the converged 8.54 mm, and below even Euler–Bernoulli theory. Against a deflection limit between those two values, the coarse model passes a wall that fails.

What would make the answer trustworthy is evidence that it has converged: the same quantity on at least three meshes, each finer than the last, with the change between the last two small compared with the margin to the limit. For this wall that study is [Example 0.5](#) itself. Beam theory is a useful independent check, but only because the wall happens to be beam-like; for a slab with an opening there is no such formula, and the mesh study is the only check available.

Week 1: Python and the Working Environment

Learning outcomes

After this week you should be able to:

1. create an isolated Python environment and install a pinned set of packages into it;
2. explain why this course fixes package versions instead of taking the newest;
3. write a model as a function of a parameter dictionary, rather than as a list of numbers;
4. state the course unit system and say what it makes the unit of mass;
5. run a structural model and check its answer against an independent result.

Notation. L span E modulus of elasticity I second moment of area P point load δ deflection θ rotation

Every week of this course ends with a number that came out of a computer. The question that should worry you is not how to produce such a number — that turns out to be easy — but how to know whether to believe it. A finite element program does not refuse a nonsensical model. It accepts the numbers you give it, multiplies them together, and returns an answer with the same confident air whether the model is right or wrong.

So this week is about the two things that decide whether the rest of the course is possible: an environment that behaves the same today as it will in December, and the habit of never trusting a computed result until something independent agrees with it. Neither is glamorous. Both are the difference between engineering and typing.

1.1 The environment

Definition 1.1: Virtual environment

A **virtual environment** (*TR: sanal ortam*) is a private directory holding one Python interpreter and one set of installed packages, used by one project and invisible to every other. Installing into it cannot disturb anything else on the machine, and nothing else on the machine can disturb it.

The reason to insist on [Definition 1.1](#) is specific rather than hygienic. `openseespy` is not pure Python: it ships a compiled extension linked against one exact version of the Python runtime. Give it a different interpreter and the import fails with

```
ImportError: DLL load failed while importing opensees  
RuntimeError: Failed to import openseespy on Windows.
```

which names neither the real cause nor the version it wanted. This is the first error message of the course and a fair introduction to the rest of them: informative-looking, and almost entirely unhelpful. The cure is to decide the interpreter once and record it.

Setting up, once per machine

1. Install Python 3.12. Not the newest — the one `openseespy` is built for.
2. In the course folder, create the environment: `py -3.12 -m venv .venv`
3. Install the pinned packages: `.\.venv\Scripts\python.exe -m pip install -r requirements.txt`
4. Run the check script in [Listing 1.9](#). If the two columns agree, you are finished.
5. If they do not, stop here and fix it. Nothing later in the course will work around a broken installation.

Note

install the latest version of everything → install the versions in `requirements.txt`

Taking the newest release of each package is the habit of a user; pinning them is the habit of an engineer. Your results have to be reproducible by someone else — a marker, a colleague, yourself in six months —

and a result that only appears under one accidental combination of versions is not a result. This is why `requirements.txt` carries `==` and not `>=`.

1.2 The Python you actually need

This course assumes you can already program, so what follows is not an introduction. It is a short tour of the four constructs that make up almost every script in this course, shown doing the job they will be doing for the rest of the term. If you have written Python before, read it for the *shape* rather than the syntax; if your programming was in another language, this is the translation table.

The task is a real one. Four steel sections are offered for the 3 m cantilever, and we want to know which of them are stiff enough to satisfy a deflection limit of $L/250$. No OpenSees is needed — this is arithmetic — which is why it is a good place to look at the language on its own.

A dictionary names the numbers that belong together.

Listing 1.1: `w01/python_tour.py` — A **dict** groups the properties of one thing and gives each a name; a **list** holds several of those things.

```

24 # A dict groups the numbers that belong to ONE thing, and names them. A list
25 # holds several of those things. Almost every model in this course is described
26 # by a structure of exactly this shape.
27 SECTIONS = [
28     {"name": "IPE 200", "I": 1.943e7},      # mm^4, strong axis
29     {"name": "IPE 240", "I": 3.892e7},
30     {"name": "IPE 300", "I": 8.356e7},
31     {"name": "IPE 360", "I": 1.627e8},
32 ]
33
34 BEAM = {"L": 3000.0, "E": 200000.0, "P": 10000.0}
```

Reaching for `section["I"]` rather than `section[1]` is worth the extra characters: the second is correct until somebody inserts a column, and neither the computer nor the reader will notice when it stops being. Almost every model in this course is described by a structure of exactly this shape.

A function turns parameters into an answer.

Listing 1.2: `w01/python_tour.py` — **def** defines it, **return** hands the answer back, and arguments after the first carry default values.

```

39 def tip_deflection(I, L=3000.0, E=200000.0, P=10000.0):
40     """Tip deflection of a cantilever under a point load,  $PL^3/3EI$ .
41
42     Arguments after the first have DEFAULT values, so a caller who only wants
43     to vary the section writes tip_deflection(I) and a caller who wants a
44     different span writes tip_deflection(I, L=4500.0). The name in the call
45     makes it obvious which argument is which -- worth doing whenever a function
46     takes more than two or three.
47
48     The magnitude is returned, not the signed displacement: this function
49     answers "how far", and the direction is not in question.
50     """
51     return P * L ** 3 / (3.0 * E * I)
```

Defaults are the reason this function is pleasant to use. A caller varying only the section writes `tip_deflection(I)`; one who also wants a different span writes `tip_deflection(I, L=4500.0)`. Naming the argument in the call is a habit worth forming — in week 2 you will meet functions taking six numbers in a row, and `f(1, 3, 1200.0, 1)` tells the reader nothing.

An if statement turns an answer into a decision.

Listing 1.3: `w01/python_tour.py` — **if ... elif ... else**. The comparison is the engineering; the strings are only how it reports itself.

```

56 def verdict(delta, L=3000.0, limit_ratio=250.0):
57     """Turn a deflection into a decision: does it satisfy L/250?
58
59     An if statement is how a script stops being a calculator and starts being
60     an engineer. The comparison is the interesting line; the strings are just
61     how it reports itself.
62     """
63     allowed = L / limit_ratio
64     if delta <= allowed:
65         return "OK"
66     elif delta <= 1.5 * allowed:
67         return "marginal"
68     else:
69         return "too flexible"

```

Note that Python marks the body of every block by **indentation** rather than by braces or an **end**. The four spaces are not a matter of taste — they are the syntax, and a line indented wrongly is a different program rather than an untidy one.

A **for** loop does it for every candidate.

Listing 1.4: w01/python_tour.py — **for** takes each item of a list in turn. Write the calculation once; let the loop repeat it.

```

74 def survey(sections=SECTIONS, beam=BEAM):
75     """Apply the same two functions to every candidate section.
76
77     This is the loop that matters in this course. Writing the calculation once
78     and letting the loop repeat it is what makes a parameter study cost nothing
79     -- and from week 5 the thing being repeated is a whole finite element
80     model rather than one division.
81     """
82     results = []
83     for section in sections:
84         delta = tip_deflection(section["I"], L=beam["L"],
85                               E=beam["E"], P=beam["P"])
86         results.append({
87             "name": section["name"],
88             "delta": delta,
89             "verdict": verdict(delta, L=beam["L"]),
90         })
91     return results

```

This is the construct that matters most in this course, because it is what makes a parameter study cost nothing. Here it repeats one division. From week 5 the thing being repeated is a whole finite element model, and the loop is unchanged.

Listing 1.5: w01/python_tour.py — An f-string interpolates a value into text: a field written `{delta:8.2f}` means *this number, eight characters wide, two decimal places*.

```

96 def report(results, beam=BEAM):
97     """Print the survey as a table.
98
99     An f-string interpolates a value into text: {r['delta']:8.2f} means "this
100     number, in a field eight characters wide, to two decimal places". Lining
101     numbers up is not decoration -- a column of figures is read by eye, and a
102     ragged one hides the outlier you were looking for.
103     """
104     allowed = beam["L"] / 250.0
105     print(f"Cantilever, L = {beam['L']:.0f} mm, P = {beam['P'] / 1000:.0f} kN")
106     print(f"Deflection limit L/250 = {allowed:.2f} mm\n")
107     print(f" {'section':<10}{ 'delta [mm]':>12}{ 'verdict':>16}")
108     for r in results:
109         print(f"  {r['name']:<10}{r['delta']:12.2f}{r['verdict']:>16}")

```

```
Cantilever, L = 3000 mm, P = 10 kN
Deflection limit L/250 = 12.00 mm
```

section	delta [mm]	verdict
IPE 200	23.16	too flexible
IPE 240	11.56	OK
IPE 300	5.39	OK
IPE 360	2.77	OK

```
Acceptable sections: IPE 240, IPE 300, IPE 360
Lightest acceptable: IPE 240
```

Lining the numbers up is not decoration. A column of figures is read by eye, and a ragged one hides the outlier you were looking for.

Note

print the table and let the reader decide → answer the question that was asked

The last two lines of the script pick out the acceptable sections and name the lightest. That is the answer somebody wanted; the table is the working. A script that stops at the table has left the final step — the one requiring judgement about what the numbers mean — undone.

1.3 Writing a model as a function

The structure we will build is the one in Fig. 1.1: a steel cantilever with a load at its tip, chosen because its answer has been in textbooks for two centuries.

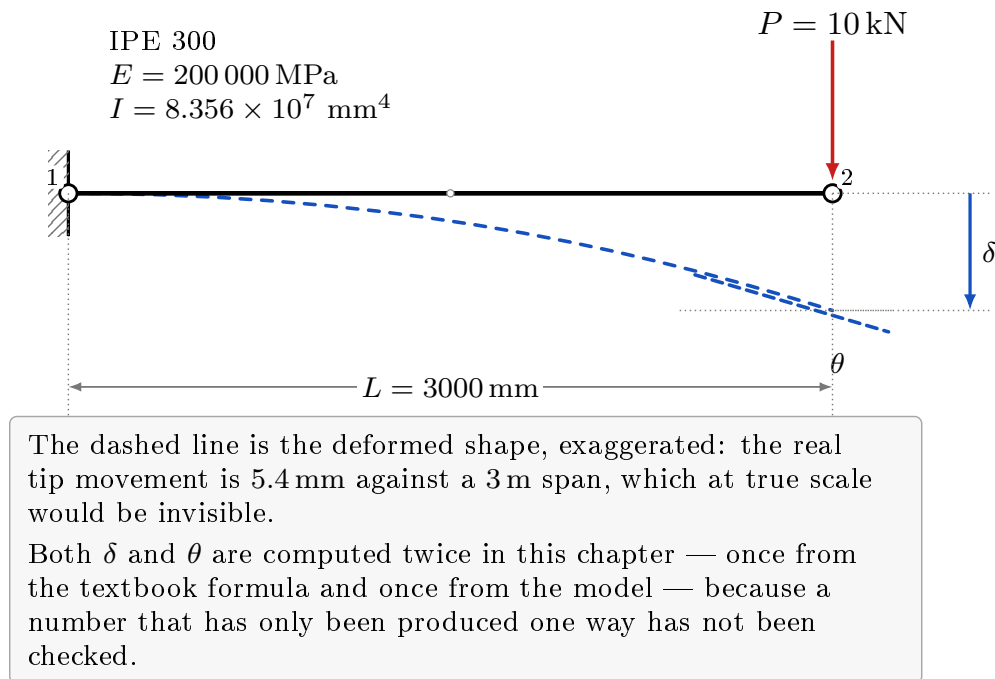


Figure 1.1: The week's structure. Both the tip deflection δ and the tip rotation θ are computed twice in this chapter, by two independent routes.

The single most useful habit in this course is to stop writing numbers into the body of a model and start writing them into a dictionary above it.

Listing 1.6: w01/environment_check.py — Every number the model needs, in one place, with its unit in a comment. Nothing below this block contains a bare number.

```

28 # Units: N, mm, s. Every number below obeys them and nothing else does.
29 CANTILEVER = {
30     "L": 3000.0,      # mm      span
31     "E": 200000.0,    # MPa     = N/mm2, structural steel
32     "I": 8.356e7,     # mm4    IPE 300, strong axis
33     "A": 5380.0,      # mm2    IPE 300
34     "P": 10000.0,     # N       = 10 kN, downward at the tip
35 }
```

It costs nothing on the first day and pays every day afterwards. A model written this way can be swept over a parameter, reused with a different section, or called twice with different loads without editing a single line of the model itself — which is exactly what [Example 1.1](#) and the exercises do. A model with the numbers buried in it can only be edited, and every edit is a chance to break something that was working.

Listing 1.7: w01/environment_check.py — The model as a function of its parameters. Note the first line.

```

40 def build_model(p):
41     """Build a 2D cantilever: fixed at node 1, loaded downward at node 2.
42
43     The first line is ops.wipe(). OpenSees keeps ONE global model in memory,
44     so without it you silently extend whatever was built last -- and the
45     symptom appears later, somewhere else, as a wrong answer rather than an
46     error. Start every model function this way, for the whole course.
47     """
48     ops.wipe()
49     ops.model("basic", "-ndm", 2, "-ndf", 3)
50
51     ops.node(1, 0.0, 0.0)
52     ops.node(2, p["L"], 0.0)
53     ops.fix(1, 1, 1, 1)          # fully fixed: ux, uy, rotation
54
55     ops.geomTransf("Linear", 1)
56     ops.element("elasticBeamColumn", 1, 1, 2, p["A"], p["E"], p["I"], 1)
57
58     ops.timeSeries("Linear", 1)
59     ops.pattern("Plain", 1, 1)
60     ops.load(2, 0.0, -p["P"], 0.0)    # negative = downward
```

That first line is not decoration. OpenSees keeps *one* model in memory for the whole process, so a second `ops.node(1, ...)` without a `ops.wipe()` does not start a new structure — it adds to the old one, or collides with it. The failure is silent, it appears later and somewhere else, and it will cost you an afternoon at least once this term. Begin every model function with `ops.wipe()` and it can never happen.

Note

one script, one model, so wiping is unnecessary → every model function starts with `ops.wipe()`

The argument that a short script only builds one model is true right up to the moment you call the function twice — in a parameter sweep, in an exercise, or in the verification harness that checks these notes. By then the habit is not there.

1.4 The analysis, and the check

Listing 1.8: w01/environment_check.py — The linear static recipe. Week 3 explains what each object is for; for now it is the shortest thing that works.

```

65 def run_analysis():
66     """Apply the load in one step and return the tip displacement and rotation.
67
68     This is the linear static recipe. Week 3 explains what each of these seven
```

```

69     objects is for; week 10 explains what to do when this recipe is not enough.
70     For now, take it as the shortest thing that works.
71     """
72     ops.system("BandGeneral")
73     ops.numberer("Plain")
74     ops.constraints("Plain")
75     ops.integrator("LoadControl", 1.0)
76     ops.algorithm("Linear")
77     ops.analysis("Static")
78
79     if ops.analyze(1) != 0:
80         raise RuntimeError("the analysis did not converge")
81
82     return ops.nodeDisp(2, 2), ops.nodeDisp(2, 3)

```

Notice what happens to the return value of `ops.analyze`. OpenSees signals failure by returning a negative number, not by raising an exception, so a script that ignores it carries on happily and reports whatever was in memory from the last step that did work. Checking the return value is one line, and it is the difference between an error and a wrong answer.

Example 1.1 (Tip deflection of a steel cantilever). An IPE 300 cantilever spans 3000 mm and carries a downward point load of 10 kN at its free end. Taking $E = 200\,000$ MPa and $I = 8.356 \times 10^7$ mm⁴, find the tip deflection — twice, by two independent routes, and compare.

Solution

Model. A single elastic beam element, fixed at one end, loaded at the other. Small displacements, linear elastic material, shear deformation neglected — which is what `elasticBeamColumn` assumes and what the textbook formula assumes, so the two are comparable.

Governing relation. For a cantilever under a tip point load,

$$\delta = \frac{PL^3}{3EI}, \quad \theta = \frac{PL^2}{2EI}. \quad (1.1)$$

Numbers. From Eq. (1.1), with the load acting downward,

$$\delta = -\frac{(10000)(3000)^3}{3(200000)(8.356 \times 10^7)} = -\frac{2.700 \times 10^{14}}{5.014 \times 10^{13}} = -5.385 \text{ mm}.$$

$$\delta = -5.39 \text{ mm}$$

Running the model gives the same quantity by an entirely different route:

Cantilever, units N-mm-s			
	OpenSees	closed form	rel. error
delta [mm]	-5.3854	-5.3854	1.65e-16
theta [rad]	-0.0027	-0.0027	4.83e-16

Check. The two agree to machine precision, and that is the expected result rather than a pleasant surprise: a single `elasticBeamColumn` element is the beam theory that produced Eq. (1.1), so there is no discretisation error to find. A relative difference of 10^{-3} here would not be roundoff — it would mean the model and the formula are not describing the same problem, and the model is the one to doubt.

Listing 1.9: `w01/environment_check.py` — Both routes, side by side. This is the shape of every verification in the course.

```

102 def check(p=CANTILEVER):
103     """Run both routes and return them side by side, with relative errors."""
104     build_model(p)
105     num_delta, num_theta = run_analysis()

```

```

106     exact_delta, exact_theta = closed_form(p)
107
108     return {
109         "delta": (num_delta, exact_delta, abs(num_delta - exact_delta) / abs(exact_delta)),
110         "theta": (num_theta, exact_theta, abs(num_theta - exact_theta) / abs(exact_theta)),
111     }

```

Summary

Item	Command or expression	Note
Create environment	<code>py -3.12 -m venv .venv</code>	once per machine
Install packages	<code>pip install -r requirements.txt</code>	pinned versions
Import	<code>import openseespy.opensees as ops</code>	one alias, always
Clear the domain	<code>ops.wipe()</code>	first line of every model
Start a model	<code>ops.model('basic', '-ndm', 2, '-ndf', 3)</code>	2D, 3 DOF per node
Run	<code>ops.analyze(1)</code>	returns 0 on success
Read a result	<code>ops.nodeDisp(node, dof)</code>	in model units
Cantilever deflection	$\delta = PL^3/3EI$	the check for Example 1.1

Exercises

Exercise 1.1. ●●● The span of the cantilever surveyed in §1.2 is increased from 3000 mm to 4000 mm, the load and the sections staying the same. Which sections still satisfy $L/250$? Change one number in the script and run it.

Answer: only IPE 300 and IPE 360; the limit rises to 16.0 mm but the deflections rise faster

Solution. Set `BEAM["L"] = 4000.0`. The allowable becomes $4000/250 = 16.0$ mm, and the deflections become 54.9 mm, 27.4 mm, 12.8 mm and 6.56 mm, so IPE 240 now fails as well.

The point is the direction of the two effects. The allowable grows in proportion to L , but the deflection grows as L^3 , so a longer span is harder to satisfy even though it is allowed to deflect more. Increasing the span by a third increases the deflection by $(4/3)^3 \approx 2.37$ against an allowance that only rose by $4/3$.

This is also the first exercise in which changing *one number in a dictionary* changed every row of the answer. That is what the dictionary was for.

Exercise 1.2. ●●● Halve the load in [Example 1.1](#) to 5 kN, leaving everything else unchanged, and report the tip deflection.

Answer: -2.69 mm

Solution. $\delta = -(5000)(3000)^3/[3(200000)(8.356 \times 10^7)] = -2.692$ mm. Because the model is linear elastic, the response scales exactly with the load — half the load, half the deflection, with no need to re-run anything. Hold on to that expectation: from week 10 onward it stops being true, and noticing *when* it stops is how you detect that a structure has yielded.

Exercise 1.3. ●●● The section is replaced by one of the same material and twice the depth, so that I increases by a factor of eight. Predict the new tip deflection before running anything, then run it.

Answer: -0.673 mm

Solution. $\delta \propto 1/I$, so eight times the second moment of area gives one eighth of the deflection: $-5.385/8 = -0.6732$ mm. The point of predicting first is that a prediction you have committed to is the cheapest verification there is, and the only one that costs nothing to run.

Exercise 1.4. ●●● The script below is meant to compare the deflections of two cantilevers of different span. It runs without error and reports that both spans give the same deflection. Find the mistake.

```
spans = [3000.0, 4000.0]
for L in spans:
    p = dict(CANTILEVER)
    p["L"] = L
    build_model(CANTILEVER)
    print(L, run_analysis()[0])
```

Answer: `build_model` is called with `CANTILEVER`, not with the modified copy `p`, so the same 3000 mm model is built and analysed twice

Solution. The loop does everything right except use its own work: `p` is built and then thrown away, and `build_model(CANTILEVER)` rebuilds the original each time. The corrected call is `build_model(p)`.

This is worth more than its two-word answer. The bug produces no error, no warning and no implausible number — two identical deflections for two different spans look like a result until you ask whether they should be identical. Eq. (1.1) says $\delta \propto L^3$, so the 4000 mm span must deflect $(4/3)^3 \approx 2.37$ times as much. Every find-the-bug exercise in this course is caught the same way: by knowing what the answer ought to look like before reading what it is.

Exercise 1.5. ●●● A classmate reports that their model of the cantilever in Example 1.1 gives a tip deflection of -5.385×10^{-3} mm, and cannot see what is wrong. Without seeing their script, say what they have most likely done, and explain how you reached that conclusion.

Answer: they have worked in newton-metre rather than newton-millimetre somewhere, most likely by entering E as 200×10^9 ; the answer is out by exactly 10^3 , and an error that is exactly a power of ten is a unit error rather than a modelling error

Solution. The number is right and the scale is wrong, by a factor of exactly 1000. Modelling mistakes — a wrong support, a wrong section, a wrong load position — almost never produce a clean power of ten; unit mistakes almost always do. That single observation localises the fault before any code is read.

An engineer's answer would go further and name the mechanism: $\delta = PL^3/3EI$ is linear in $1/E$, so an E entered in pascals instead of megapascals divides the deflection by 10^6 , while L in metres instead of millimetres multiplies it by 10^{-9} and I likewise. Only some combinations give exactly 10^3 , and reasoning about which is faster than reading the script.

This is the reason the course fixes one unit system in the ground rules and never varies it. OpenSees will not help you here — it has no idea what a metre is.

Exercise 1.6. ●●● Explain, in two or three sentences, why agreement between the model and Eq. (1.1) in Example 1.1 is weaker evidence than it appears — and name one check that would be stronger.

Answer: both routes assume the same beam theory, so the comparison tests the input data and the script but not the theory; a stronger check compares against something that does not share the assumption, such as a refined mesh of continuum elements, a hand calculation including shear deformation, or an experiment

Solution. A verification is only as strong as the independence of the two things being compared. Here `elasticBeamColumn` implements Euler-Bernoulli beam theory and Eq. (1.1) is the closed-form solution of that same theory, so exact agreement confirms that the span, modulus, second moment of area and load were entered correctly and that the script does what it says. That is genuinely worth confirming — most errors in practice are of exactly this kind — but it says nothing about whether beam theory

suits the problem.

A check with independent content would have to relax an assumption: refine the member into continuum elements and see whether the answer moves; add the shear term PL/GA_s by hand and see whether the difference is negligible for this span-to-depth ratio; or compare with a measured deflection. Knowing which of your checks are independent and which merely rehearse the same assumptions is a large part of what this course is teaching.

Week 2: The OpenSees Model

Learning outcomes

After this week you should be able to:

1. describe what the OpenSees domain is and why there is only ever one of it;
2. build a complete model — nodes, restraints, material, elements, loads — in the order the dependencies require;
3. choose `ndm` and `ndf` for a given structure and say what each one controls;
4. run a linear static analysis and read displacements and element forces back out;
5. explain why an OpenSees error is usually a warning on standard output rather than an exception, and what that means for how you write scripts.

Notation. N axial force A cross-sectional area E modulus L member length θ inclination u_x, u_y nodal displacements

Last week's cantilever was one element between two nodes, and it was chosen so that nothing about the model could be interesting. This week the model becomes the subject. A structure has to be described to OpenSees as a collection of numbered objects that refer to one another, and almost everything that goes wrong in the rest of the course goes wrong because two of those objects did not refer to each other the way you thought.

The good news is that there is very little to learn. Six commands build a model and seven more run it, and the same thirteen are still there in week 13 doing a nonlinear time-history analysis of a laboratory specimen. What changes between now and then is the arguments, not the vocabulary.

2.1 One domain, and only one

Definition 2.1: Domain

The **domain** (*TR: tanım bölgesi*) is the single collection of nodes, elements, materials, constraints and load patterns that OpenSees currently holds in memory. Every command you issue either adds something to it, changes it, or asks it a question.

The word *single* in Definition 2.1 is the whole of it. There is no model object to create, pass around and keep several of; there is one global domain, and `ops.node(1, 0.0, 0.0)` puts a node into it. Two consequences follow, and both bite.

The first is that building a second model does not replace the first — it adds to it. A script that defines node 1 twice does not get a fresh node 1; it gets a complaint on standard output and keeps the original. This is why `ops.wipe()` is the first line of every model function, and why week 1 made a rule of it before there was any way to see why.

The second is subtler and matters for the rest of the course: *results are read from the domain, not returned by the analysis*. `ops.analyze(1)` returns a status code, not displacements. The displacements are in the domain afterwards, and you go and ask for them.

2.2 Six steps, in the only order that works

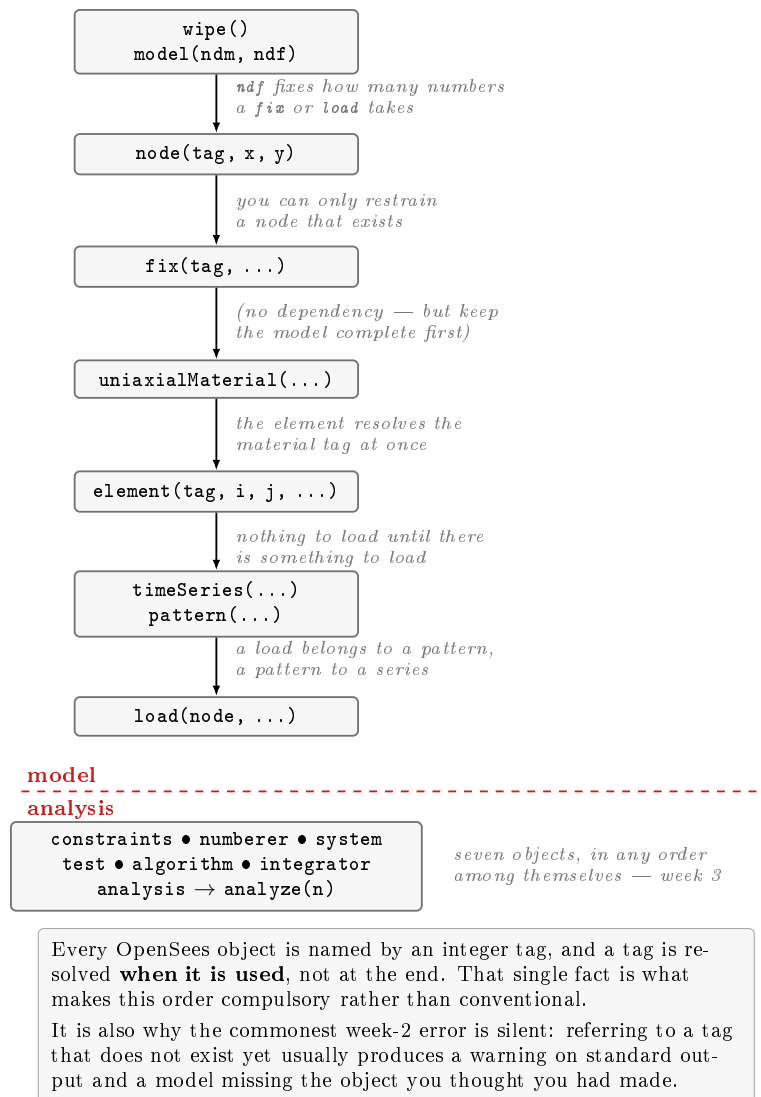


Figure 2.1: The order in which a model has to be built. Each arrow is a dependency, not a convention: the step below cannot be issued until the step above has created the thing it refers to.

Every object in OpenSees is named by an **integer tag**, and the tag is resolved *at the moment it is used*. When you write `ops.element("Truss", 1, 1, 3, A, 1)` the element looks up nodes 1 and 3 and material 1 immediately. If they do not exist yet, the element is not created. That single fact is what makes the order in Fig. 2.1 compulsory.

Listing 2.1: `w02/truss.py` — The six steps, with the dependency that fixes each one's position in a comment.

```

38 def build_model(p):
39     """Build the two-bar truss.
40
41     The six steps below are the whole of model building in OpenSees, and they
42     are in the only order that works. Each one names things by an integer TAG,

```

```

43 and those tags are how the later steps refer back to the earlier ones.
44 """
45 # 1. Start clean, and say what kind of model this is.
46 # ndm = spatial dimensions, ndf = degrees of freedom per node.
47 # A pin-jointed truss carries no moments, so 2 DOF per node is enough.
48 ops.wipe()
49 ops.model("basic", "-ndm", 2, "-ndf", 2)
50
51 # 2. Nodes: a tag and its coordinates. Nothing else.
52 ops.node(1, 0.0, 0.0) # left support
53 ops.node(2, p["span"], 0.0) # right support
54 ops.node(3, p["span"] / 2.0, p["H"]) # apex
55
56 # 3. Restraints: 1 means "held", 0 means "free", one flag per DOF.
57 ops.fix(1, 1, 1) # pinned
58 ops.fix(2, 1, 1) # pinned
59
60 # 4. Material, then elements that use it. The material must exist first --
61 # the element refers to it by tag, and OpenSees resolves that at once.
62 ops.uniaxialMaterial("Elastic", 1, p["E"])
63 ops.element("Truss", 1, 1, 3, p["A"], 1) # ele 1: node 1 to node 3
64 ops.element("Truss", 2, 2, 3, p["A"], 1) # ele 2: node 2 to node 3
65
66 # 5. How the load grows with pseudo-time, then the pattern that uses it.
67 ops.timeSeries("Linear", 1)
68 ops.pattern("Plain", 1, 1)
69
70 # 6. The load itself: one value per DOF of the node it acts on.
71 ops.load(3, 0.0, -p["P"])

```

Two arguments in step 1 deserve more than they usually get.

ndm is the number of spatial dimensions: how many coordinates a node needs. **ndf** is the number of degrees of freedom each node has: how many numbers a **fix** takes, how many a **load** takes, and how many displacements come back. They are independent. A 2D truss is **ndm=2**, **ndf=2**; a 2D frame is **ndm=2**, **ndf=3**, the third being rotation; a 3D frame is **ndm=3**, **ndf=6**.

Note

ndf is how many dimensions the structure has → **ndf** is how many displacements each node can undergo. Choosing **ndf=3** for a pin-jointed truss is not an error and OpenSees will not object. It gives every node a rotational degree of freedom that no element restrains, the stiffness matrix becomes singular, and the analysis fails with a message about the system being unsolvable — a long way from the line that caused it.

2.3 A truss you can check by hand

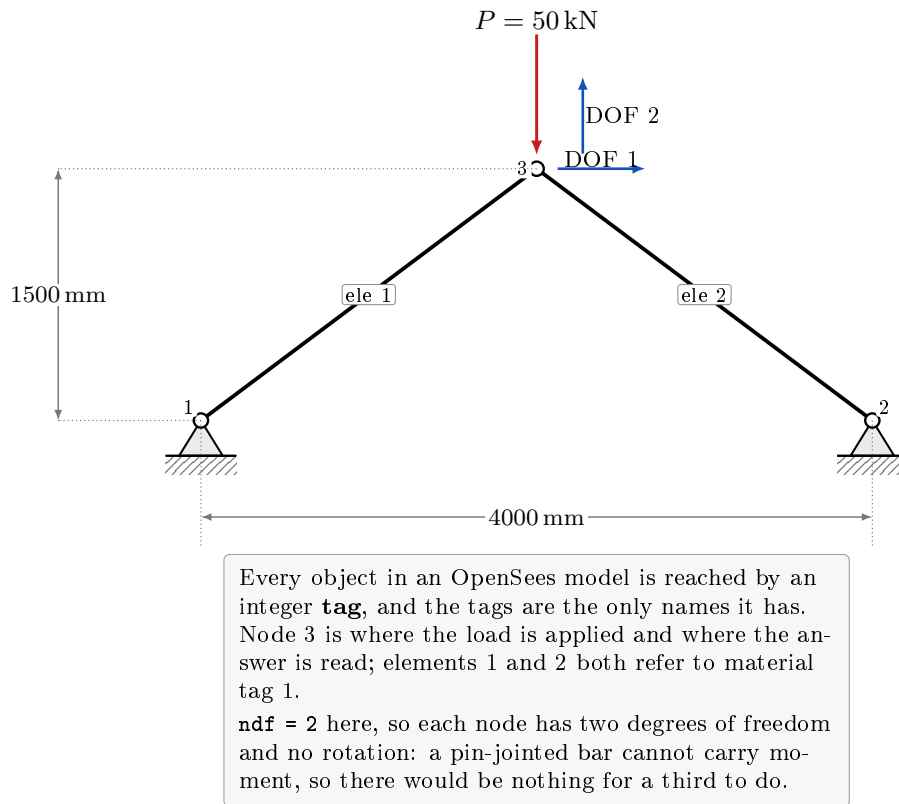


Figure 2.2: The week's model, labelled with the tags the script uses. The half-span and the rise are 2000 mm and 1500 mm, which makes a 3–4–5 triangle and a bar length of exactly 2500 mm.

The dimensions in Fig. 2.2 were not chosen to be realistic. They were chosen so that $\sin \theta = 0.6$ exactly, which means every number in the hand solution is clean and a discrepancy in the fourth digit is unambiguously the model's fault rather than the arithmetic's. Choosing verifiable numbers for a first model is a habit worth keeping.

Example 2.1 (Two-bar truss). The truss of Fig. 2.2 carries $P = 50$ kN downward at its apex. Both bars are steel, $E = 200\,000$ MPa, with area $A = 1200$ mm². Find the bar forces and the vertical movement of the apex, by hand and by model.

Solution

Model. Pin-jointed bars carrying axial force only; small displacements; supports pinned at both ends. The structure is statically determinate, so the bar forces follow from equilibrium alone and do not depend on A or E — a fact worth remembering, because it is the basis of one of the exercises.

Geometry. With half-span 2000 mm and rise 1500 mm,

$$L = \sqrt{2000^2 + 1500^2} = 2500 \text{ mm}, \quad \sin \theta = \frac{1500}{2500} = 0.6000. \quad (2.1)$$

Equilibrium at the apex. By symmetry each bar carries half the load vertically, and the vertical

component of a bar force N inclined at θ is $N \sin \theta$:

$$N = -\frac{P/2}{\sin \theta} = -\frac{25000}{0.6000} = -41\,670 \text{ N}, \quad (2.2)$$

negative because the bars are in compression.

Displacement. Each bar shortens by $\Delta L = NL/EA$, and the apex drops by $\Delta L/\sin \theta$:

$$\Delta L = \frac{(-41670)(2500)}{(200000)(1200)} = -0.4340 \text{ mm}, \quad u_y = \frac{-0.4340}{0.6000} = -0.7234 \text{ mm}.$$

$$N = -41.7 \text{ kN}, \quad u_y = -0.723 \text{ mm}$$

The model is asked the same two questions with two different commands:

Listing 2.2: `w02/truss.py` — `nodeDisp` asks a node what it did; `eleResponse` asks an element what it carries. Neither reads a file — the model is still in memory.

```

98 def results(p):
99     """Read the answers back out of the model.
100
101     Two different questions, two different commands. nodeDisp asks a node what
102     it did; eleResponse asks an element what it carries. Neither reads a file --
103     the model is still in memory and can simply be interrogated.
104     """
105     ux, uy = ops.nodeDisp(3, 1), ops.nodeDisp(3, 2)
106     n1 = ops.eleResponse(1, "axialForce")[0]
107     n2 = ops.eleResponse(2, "axialForce")[0]
108     return {"ux": ux, "uy": uy, "N1": n1, "N2": n2}

```

```

Two-bar truss, units N-mm-s
bar length          2500.00 mm
apex settlement    uy      -0.7234 mm (by hand -0.7234)
apex sway          ux      0.0000e+00 mm (symmetry: expect 0)
bar 1 axial        N1     -41666.67 N (by hand -41666.67)
bar 2 axial        N2     -41666.67 N

```

Check. Three things agree that did not have to. The bar forces match [Eq. \(2.2\)](#), which tests the geometry and the equilibrium. The apex settlement matches, which tests E , A and the compatibility. And the horizontal movement is zero to machine precision, which tests nothing about the arithmetic but everything about the model: a symmetric structure that sways has an unsymmetric support, an unsymmetric load, or a typo in a coordinate. That third check cost nothing and needed no textcourse.

2.4 Running it

Listing 2.3: `w02/truss.py` — The seven analysis objects. For a linear problem this combination always works; week 3 says what the alternatives are and week 10 says when you need them.

```

76 def run():
77     """Apply the load in a single step.
78
79     These seven objects are the analysis. Week 3 gives them names and says what
80     the alternatives are; for a linear problem this combination always works.
81     """
82     ops.constraints("Plain")
83     ops.numberer("RCM")
84     ops.system("BandSPD")
85     ops.test("NormDispIncr", 1.0e-8, 10)
86     ops.algorithm("Linear")

```

```
87 ops.integrator("LoadControl", 1.0)
88 ops.analysis("Static")
89
90 ok = ops.analyze(1)
91 if ok != 0:
92     raise RuntimeError(f"analysis failed, analyze() returned {ok}")
93 return ok
```

The line that matters most is the one that looks like housekeeping:

```
ok = ops.analyze(1)
if ok != 0:
    raise RuntimeError(f"analysis failed, analyze() returned {ok}")
```

Note

if something goes wrong, Python will raise an exception → OpenSees reports failure by returning a non-zero code and by printing to standard output

OpenSees is a C++ program with a Python interface, and it reports problems the way C++ programs written in 1998 reported problems: a message on standard output and a return code. Nothing is raised. A script that does not test the return value of `ops.analyze` will happily go on to print displacements — the ones left over from the last step that did converge. Test it every time, in one line, as above.

Reading the output of a model that did not work

1. Look at standard output first, not at the numbers. OpenSees warnings appear there and nowhere else.
2. `WARNING ... node not found` or `... material not found` means a tag was used before it was created: check the order in [Fig. 2.1](#).
3. A singular system usually means an unrestrained degree of freedom — most often too large an `ndf`, or a support you meant to fix and did not.
4. If it ran but the answer is wrong by a power of ten, it is units, not modelling.
5. Only then start reading the model.

Summary

Step	Command	Needs first
clear and start	<code>wipe(), model('basic', '-ndm', n, '-ndf', m)</code>	—
node	<code>node(tag, x, y)</code>	the model
restraint	<code>fix(tag, f1, f2, ...)</code>	the node
material	<code>uniaxialMaterial('Elastic', tag, E)</code>	—
element	<code>element('Truss', tag, i, j, A, matTag)</code>	nodes, material
load history	<code>timeSeries('Linear', tag)</code>	—
load pattern	<code>pattern('Plain', tag, tsTag)</code>	the series
load	<code>load(node, v1, v2, ...)</code>	the pattern
run	<code>analyze(1)</code>	all seven analysis objects
displacement	<code>nodeDisp(node, dof)</code>	a completed analysis
element force	<code>eleResponse(tag, 'axialForce')</code>	a completed analysis
bar length	$L = \sqrt{(\Delta x)^2 + (\Delta y)^2}$	Eq. (2.1)
bar force	$N = -(P/2)/\sin \theta$	Eq. (2.2)
bar extension	$\Delta L = NL/EA$	

Exercises

Exercise 2.1. ●●● Double the area of both bars in [Example 2.1](#) to 2400 mm² and report the apex settlement. Before running it, say what you expect the bar forces to do.

Answer: −0.362 mm; the bar forces do not change at all

Solution. Axial stiffness is EA/L , so doubling A halves the extension and halves the settlement: $-0.7234/2 = -0.3617$ mm.

The bar forces are unchanged, and that is the part worth understanding. The truss is statically determinate, so [Eq. \(2.2\)](#) came from equilibrium alone — no stiffness appears in it. Make one bar ten times stiffer and it still carries 41.7 kN. In a statically *indeterminate* structure this stops being true, and stiffness starts to decide how load is shared; that is one of the things week 5 is for.

Exercise 2.2. ●●● Halve the rise of the truss to 750 mm, leaving the span, the areas and the load alone. Report the bar force and the apex settlement, and explain why the settlement grows by more than a factor of two.

Answer: $N = -71.2$ kN, $u_y = -1.80$ mm — about 2.5 times the original

Solution. The bars are now 2136 mm long with $\sin \theta = 0.3511$, so $N = -25000/0.3511 = -71\,200$ N and $u_y = -1.8047$ mm.

The factor is 2.49, not 2, because halving the rise does two things at once. It raises the bar force, since a flatter bar needs more axial force to supply the same vertical component; and it then converts that larger extension into vertical movement less efficiently, dividing by a smaller $\sin \theta$ a second time. Both effects go the same way.

This is why very shallow trusses and nearly straight cable stays are so flexible, and it is the same mechanism that makes a tightly stretched washing line sag much more than its tension suggests.

Exercise 2.3. ●● The script below is meant to build the truss of [Example 2.1](#) and print the apex settlement. It prints nothing useful and OpenSees complains on standard output. Find the mistake — there is one line at fault.

```
ops.wipe()
ops.model('basic', '-ndm', 2, '-ndf', 2)
ops.node(1, 0.0, 0.0)
ops.node(2, 4000.0, 0.0)
ops.node(3, 2000.0, 1500.0)
ops.fix(1, 1, 1)
ops.fix(2, 1, 1)
ops.element('Truss', 1, 1, 3, 1200.0, 1)
ops.element('Truss', 2, 2, 3, 1200.0, 1)
ops.uniaxialMaterial('Elastic', 1, 200000.0)
```

Answer: the material is defined after the elements that use it; move the `uniaxialMaterial` line above the two `element` lines

Solution. The elements refer to material tag 1, and a tag is resolved when it is used, not at the end. At the moment `ops.element` runs there is no material 1, so neither element is created; the material is then defined and nothing ever uses it.

The reason this exercise exists is the shape of the failure rather than the fix. Nothing raises. The script runs to completion, and the only evidence is a warning on standard output and a domain with three nodes and no elements — so the next thing you see is a singular system, several lines away from the line that caused it. This is the single most common week-2 error, and [Fig. 2.1](#) is the map that prevents it.

Exercise 2.4. ●●● A classmate builds the same truss with `ndf=3` instead of `ndf=2`, adjusting the `fix` and `load` calls to take three numbers. The analysis fails. Explain what is wrong in terms of the structure, not the code.

Answer: each node now has a rotational degree of freedom, but a `Truss` element carries only axial force and contributes no rotational stiffness, so those degrees of freedom are unrestrained and the stiffness matrix is singular

Solution. A degree of freedom exists in the model if `ndf` says it does. It is *restrained* only if either a support fixes it or an element supplies stiffness to it. `Truss` elements supply axial stiffness and nothing else, so the rotation at node 3 has neither, and the equations have no unique solution.

The physical statement is that the question is ill-posed: asking how much the apex of a pin-jointed truss rotates has no answer, because nothing in the structure resists that rotation or defines it. The singular matrix is not the computer being fussy — it is the correct response to a question with infinitely many answers.

The general rule is worth carrying forward: `ndf` must match what the elements can actually carry. It is why a frame model needs `ndf=3` and a truss model does not, and it is the first thing to check whenever a model that looks right refuses to solve.

Exercise 2.5. ●●● [Example 2.1](#) checked the model against a hand solution. Suppose the truss had twenty bars instead of two, so that no hand solution was practical. Name two checks you could still make on the result, and say what each one would and would not catch.

Answer: equilibrium — the reactions must sum to the applied load — and symmetry, if the structure happens to be symmetric; the first catches lost or misapplied loads and unrestrained supports but not a wrong modulus, the second catches typos in coordinates and supports but passes any error that is itself symmetric

Solution. Both are **self-checks**: they compare the model against itself rather than against an external answer, which is what makes them available on structures that have no textbook solution.

Equilibrium is the stronger of the two and the one to reach for first. If the reactions do not sum

to the applied load, something is carrying force that should not be — a support you did not intend, a load applied to a node that does not exist, a restrained degree of freedom absorbing what should have gone into the structure. What it cannot catch is anything that preserves equilibrium: every bar given twice its correct area still balances perfectly, and so does a model in the wrong unit system.

Symmetry catches a different class: single-point mistakes that break the pattern, which is exactly what typos are. It is blind to any error you have made consistently on both sides — and since people build the second half of a symmetric model by copying the first, that is a real limitation rather than a theoretical one.

Neither proves the model right. That is not a defect in these particular checks; no test proves a model right. What they do is fail loudly on the mistakes that are actually common, which is all you can ask, and week 3 is about making that a routine rather than an afterthought.

Week 3: The Analysis Space, Recorders and Verification

Learning outcomes

After this week you should be able to:

1. place any analysis in the course on a map of two questions, and name the objects that change between the cells;
2. say what each of the seven analysis objects is responsible for;
3. choose between a direct query and a recorder, and set a recorder up at the right moment;
4. apply four checks — equilibrium, symmetry, scaling, closed form — and say what each one can and cannot catch;
5. explain why a result read from a recorder file is not bit-identical to the same result read from memory.

Notation. λ load factor $\vec{R}$ reactions $\vec{P}$ applied load u_y a displacement being tracked

You now know enough to build a model and run it, which means you know enough to get a confident, detailed and completely wrong answer. This week is the antidote, and it has three parts: a map of where the course is going, so the seven analysis objects stop looking arbitrary; the two ways of getting numbers out of a model, and the trap in the more important one; and four checks that cost almost nothing and catch most of what actually goes wrong.

Nothing here is difficult. It is the week that decides whether the remaining eleven produce engineering or output.

3.1 The map

	linear	nonlinear
static	<pre>algorithm('Linear') integrator('LoadControl')</pre> One step, one solve. The stiffness matrix is assembled once. <i>weeks 2, 4, 5</i>	<pre>algorithm('Newton') integrator('LoadControl') or ('DisplacementControl')</pre> Many steps, each iterating until test is satisfied. <i>weeks 6–8, 10, 11</i>
dynamic	<pre>algorithm('Linear') integrator('Newmark', 0.5, 0.25) analysis('Transient')</pre> Mass and damping now matter; the steps are real time. <i>week 12</i>	<pre>algorithm('Newton') integrator('Newmark', 0.5, 0.25) analysis('Transient')</pre> Iteration inside every time step. Where analyses fail. <i>week 13</i>

Unchanged in all four cells: constraints • numberer • system • test

These four are about *how the equations are stored and solved*, not about what kind of problem it is. Choosing them badly makes an analysis slow or fragile; it does not change what is being computed.

Only two questions decide the cell: **does the stiffness change as the load is applied?** (left or right), and **do inertia and damping matter?** (top or bottom). Everything else follows.

Figure 3.1: Every analysis in this course, and the two questions that place it. Only the algorithm and the integrator change between the cells; the other four objects are about how the equations are stored and solved.

Week 2 presented seven analysis objects as a recipe to be copied. [Figure 3.1](#) is the reason that was honest: for a linear static problem the combination really is fixed, and the choices only start to matter when you move out of the top-left cell.

Two questions place any analysis:

1. **Does the stiffness change as the load is applied?** If not, the structure can be solved in one step with `algorithm('Linear')`. If it does — because a material has yielded or the geometry has moved far enough to matter — the load has to be applied in increments and each increment iterated. That is weeks 6–8, 10 and 11.
2. **Do inertia and damping matter?** If not, `analysis('Static')` and pseudo-time. If they do, `analysis('Transient')`, a real time step, and an integrator that advances it. That is weeks 12 and 13.

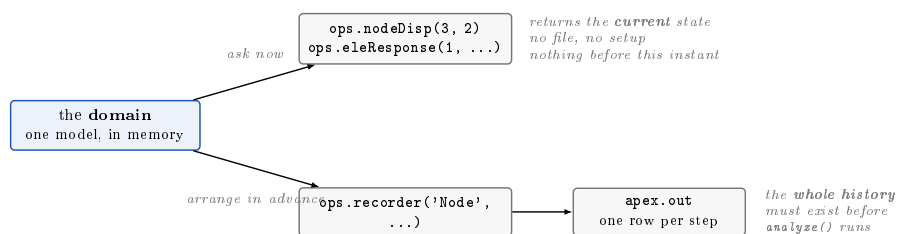
Definition 3.1: The seven analysis objects

constraints	how boundary conditions and ties are imposed on the equations.
numberer	the order in which degrees of freedom are numbered, which decides how wide the banded matrix is.
system	how the system of equations is stored and factorised.
test	when an iteration is close enough to be called converged.
algorithm	how each step iterates towards equilibrium.
integrator	how much load, or how much time, one step advances.
analysis	which of the two kinds this is: Static or Transient .

The first four of [Definition 3.1](#) are about arithmetic and the last three about mechanics. Getting the first four wrong makes an analysis slow or fragile; getting the last three wrong changes what is being computed. That is the distinction to hold on to, and it is why **test** sits with the arithmetic even though it is the one that will later report your failures.

3.2 Getting numbers out

There are two routes out of a model, and they are answers to different questions rather than alternatives — [Fig. 3.2](#) sets them side by side.



A recorder cannot be added afterwards. It only sees steps taken after it was created, so an analysis that finished without one has to be run again — and in week 13 that is an hour of your life, not a second. Set the recorders up with the model, not with the results.

The file is also *buffered*. Reading it while the model is still alive can give you a short file or an empty one; `ops.wipe()` closes the recorders and flushes what is still in memory.

Figure 3.2: The two routes out of a model, and the asymmetry between them. A query is available at any moment and describes only the present; a recorder must be arranged in advance and is the only thing that keeps the history.

`ops.nodeDisp` and `ops.eleResponse` ask the domain what it looks like now. That was sufficient in week 2 because the analysis had exactly one step, so “now” and “the whole history” were the same thing.

Listing 3.1: `w03/recorders.py` — A recorder is arranged before the analysis runs, and only ever sees the steps that come after it.

```

40 def setup_recorder(path, precision=6):
41     """Ask for the apex displacement to be written at every converged step.
42
43     Set this up AFTER the model exists -- the recorder refers to node 3 by tag,
44     so node 3 has to be there -- and BEFORE the analysis runs, because a
45     recorder only ever sees the steps that come after it.
46
47     '-precision' matters more than it looks. A recorder writes TEXT, and the
  
```

```

48     default is six significant figures, so what comes back out of the file is
49     not bit-identical to what was in memory: -0.723380 rather than
50     -0.7233796296296295. Six figures is plenty for a displacement you are going
51     to plot, and nowhere near enough if you are subtracting two nearly equal
52     recorded numbers -- a residual, a drift between two storeys, an increment
53     between two steps. Ask for more when you are going to do arithmetic on the
54     output.
55     """
56     path.parent.mkdir(parents=True, exist_ok=True)
57     return ops.recorder("Node", "-file", str(path),
58                         "-precision", precision,
59                         "-time",                               # first column = pseudo-time
60                         "-node", 3,
61                         "-dof", 1, 2, "disp")

```

Run the same truss in ten steps instead of one and the two routes stop being interchangeable.

Listing 3.2: `w03/recorders.py` — Ten steps of one tenth of the load. For a linear model the destination is identical; what is different is that there is now a path.

```

66 def run_stepped(n_steps):
67     """Apply the load in n_steps equal increments instead of all at once.
68
69     LoadControl's argument is the load factor added per step, so n steps of
70     1/n arrive at exactly the same place as one step of 1 -- for a linear
71     model. The reason to do it anyway is that it produces a PATH, and a path is
72     what you plot, check and compare against a test.
73     """
74     ops.constraints("Plain")
75     ops.numberer("RCM")
76     ops.system("BandSPD")
77     ops.test("NormDispIncr", 1.0e-8, 10)
78     ops.algorithm("Linear")
79     ops.integrator("LoadControl", 1.0 / n_steps)
80     ops.analysis("Static")
81
82     for step in range(n_steps):
83         if ops.analyze(1) != 0:
84             raise RuntimeError(f"failed at step {step + 1} of {n_steps}")

```

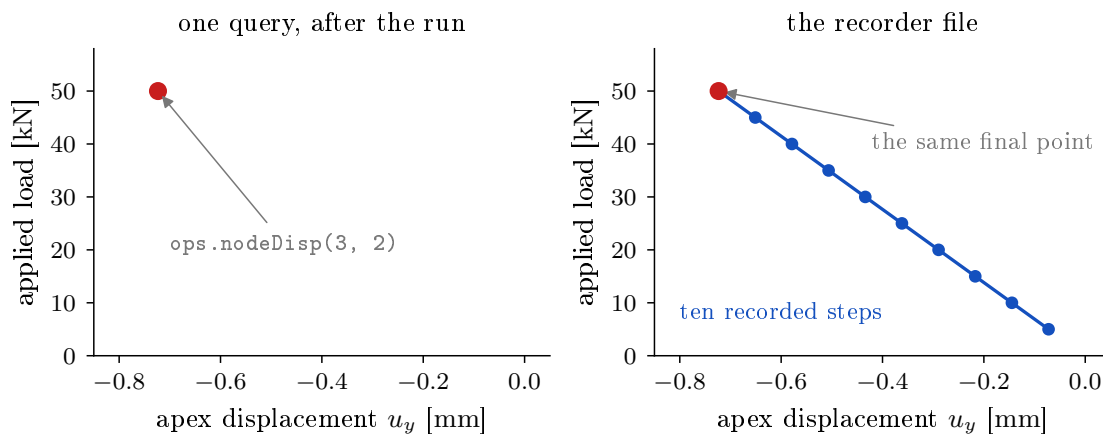


Figure 3.3: The same analysis, seen two ways. The query returns the red point; the recorder file contains the ten blue ones that led to it.

For this truss the path is a straight line and the nine extra points tell you nothing you could not have worked out. That is exactly why it is worth drawing now, while it is harmless. From week 10 the path stops

being straight, and the shape of it — where it softens, where it turns over — becomes the answer rather than a detail of how the answer was reached.

Note

run the analysis, then decide what to record → decide what to record, then run the analysis

A recorder only sees steps taken after it was created. An analysis that finished without one has to be run again. In week 13 that is an hour, not a second, and the fact that the run *succeeded* is what makes it so annoying.

Setting up output

1. Build the model. A recorder refers to nodes and elements by tag, so they must exist.
2. Create every recorder you might want. They are cheap; re-running is not.
3. Run the analysis.
4. Call `ops.wipe()`. This closes the recorders and flushes what is still buffered — reading the file before this can give you a short file or an empty one.
5. Only then read the files.

3.3 Four checks

Example 3.1 (Checking the week 2 truss four ways). Apply four independent checks to the two-bar truss, and say what each one establishes.

Solution

Model. Unchanged from [Example 2.1](#). What changes is what we ask of it.

1. Equilibrium. The reactions must sum to the applied load. Nothing else is being claimed, and no theory is needed, so this check works on any model however complicated.

Listing 3.3: `w03/checks.py` — Equilibrium needs no reference solution, which is what makes it the check to reach for first.

```

36 def check_equilibrium(p=TRUSS):
37     """Sum the reactions and compare them with what was applied.
38
39     ops.reactions() computes them; ops.nodeReaction() then reads them node by
40     node. Forgetting the first call is a classic: nodeReaction returns zeros,
41     which looks like a support that is not working rather than a query that was
42     never answered.
43
44     This check needs no theory and no reference solution, so it works on any
45     model, however complicated -- which is exactly what makes it the one to
46     reach for first.
47     """
48     build_model(p)
49     run()
50     ops.reactions()
51
52     rx = sum(ops.nodeReaction(n, 1) for n in (1, 2))
53     ry = sum(ops.nodeReaction(n, 2) for n in (1, 2))
54
55     # The only applied load is P downward at the apex.
56     return {"sum_rx": rx, "sum_ry": ry, "applied": -p["P"],
57            "residual": ry - p["P"]}

```

1. Equilibrium	
sum of vertical reactions	50000.0000 N
applied load	50000.0000 N
residual	7.2760e-12 N

The residual is 7×10^{-12} , which is roundoff. That distinction matters: a residual of 0.4 N would not be roundoff, it would be a real imbalance, and something in the model would be quietly carrying it.

2. Symmetry. A structure symmetric in geometry, supports *and* load must respond symmetrically. The apex sway is zero to machine precision and the two bar forces are identical.

3. Scaling. In a linear elastic model the response is exactly proportional to the load. Doubling P gives a ratio of 2.000 000, and *exactly* is the word to note — this is an identity, not an approximation.

4. Closed form. And finally the comparison with the hand solution, which agrees to 1.5×10^{-16} .

all four pass; the first three needed no external answer

Check. The check on the checks is to ask what each one would have missed. Equilibrium passes on a truss with every area wrong by the same factor. Symmetry passes on any error made consistently on both sides — and since people build the second half of a symmetric model by copying the first, that is a real gap. Scaling passes on any linear model whatsoever, correct or not. And the closed form, the strongest-looking of the four, tests the arithmetic of a pin-jointed idealisation against the same idealisation, so it says nothing about whether pin joints describe the real connection.

None of them proves the model right. Each of them fails loudly on a different common mistake, and using all four costs about six lines.

Note

a recorded number is the number that was computed → a recorded number has been through a text file
The four checks of [Example 3.1](#) all compared numbers held in memory. [Example 3.2](#) is about a discrepancy that appears once a number has been through a file. It is small, it is entirely benign here, and it is exactly the kind of thing that stops being benign when you start subtracting.

Example 3.2 (Why the file and the model disagree). The final row of the recorder file and the value returned by `ops.nodeDisp` are the same quantity. Are they the same number?

Solution

Model. The ten-step run of [Fig. 3.3](#), with the last recorded row compared against the query taken immediately before `ops.wipe()`.

```
in memory      -0.7233796296296295
in the file    -0.72338
difference      3.7037e-07
```

they agree to 6 significant figures, not to machine precision

Why. A recorder writes *text*, and the default is six significant figures. The value in memory is a double; the value in the file is a decimal string that was rounded on the way out.

Check. Is this ever large enough to matter? Six figures on a displacement you are going to plot is far more than the plot can show. But consider what you do with recorder output in practice: you subtract. Inter-storey drift is one storey's displacement minus another's. An increment is one step minus the previous one. Residual displacement is the end of a record minus its beginning. If the two numbers are close, the leading figures cancel and what is left is dominated by the digits that were thrown away. Two displacements of 12.3456 mm and 12.3457 mm have six good figures each and one good figure in their difference.

The fix is one argument, and it is in the listing above: **-precision**. Ask for more when the output is going to be differenced.

Summary

Item	Command or idea	Note
place the analysis	stiffness changing? inertia mattering?	Fig. 3.1
query the domain	<code>nodeDisp</code> , <code>eleResponse</code> , <code>nodeReaction</code>	the present only
reactions	<code>reactions()</code> then <code>nodeReaction(n, dof)</code>	the first call computes them
record a history	<code>recorder('Node', '-file', ...)</code>	before <code>analyze()</code>
write more digits	<code>'-precision', 12</code>	when output will be differenced
flush the files	<code>wipe()</code>	before reading them
equilibrium check	$\sum \vec{R} = \sum \vec{P}$	needs no theory; use always
symmetry check	symmetric input $\Rightarrow$ symmetric output	catches typos, not habits
scaling check	$2P \Rightarrow 2u$, exactly, while linear	becomes a yield detector
closed-form check	compare with theory	tests the arithmetic, not the idealisation

Exercises

Exercise 3.1. ●●● In which cell of [Fig. 3.1](#) does each of these belong: (a) finding the periods and mode shapes of an elastic frame; (b) pushing a reinforced concrete frame sideways until it fails; (c) the truss of [Example 2.1](#)?

Answer: (a) linear dynamic; (b) nonlinear static; (c) linear static

Solution. (a) is dynamic because it is entirely about mass and stiffness, and linear because a modal analysis is only defined for a linear system — week 12. (b) is static, because the pushing is slow enough that inertia is irrelevant, and nonlinear because the whole point is what happens after yielding — week 11. (c) is the top-left cell, which is where every model in the course has been so far.

The exercise is worth doing because (b) is the one people place wrongly. “Until it fails” sounds dynamic and violent; it is neither. What makes an analysis dynamic is whether inertia forces are large enough to matter, not whether the outcome is dramatic.

Exercise 3.2. ●●● The script below is meant to check equilibrium on the week 2 truss. It reports that the reactions sum to zero, which would mean the supports are carrying nothing. Find the mistake.

```
build_model(TRUSS)
run()
rx = sum(ops.nodeReaction(n, 1) for n in (1, 2))
ry = sum(ops.nodeReaction(n, 2) for n in (1, 2))
print(rx, ry)
```

Answer: `ops.reactions()` is never called, so the reactions were never computed and `nodeReaction` returns zeros

Solution. `ops.nodeReaction` reads a value that `ops.reactions()` has to compute first. Without that call it returns zero — not an error, not a warning, just zero.

The reason this one is worth a whole exercise is that the failure mimics a real structural condition. A support reporting zero reaction looks exactly like a support that is not engaged, so the natural response is to go and inspect the `fix` calls, the geometry and the load path — all of which are fine. The fault is one missing line in the *query*, not in the model.

The general lesson: when a result is suspiciously round — zero, or exactly the applied load, or exactly zero drift — suspect the question before you suspect the structure.

Exercise 3.3. ●●● Modify the recorder in [Example 3.2](#) to write twelve significant figures instead of six, re-run, and report the difference between the last recorded row and `ops.nodeDisp`. Then say, in one sentence, why the difference is not zero even then.

Answer: the difference falls to roughly 1×10^{-13} rather than 4×10^{-7} ; it is not zero because a decimal string of any finite length cannot represent a binary double exactly

Solution. Twelve figures brings the recorded value to within about 1×10^{-13} of the value in memory, which is the round-trip error of the decimal representation rather than anything to do with OpenSees.

It never becomes exactly zero, and it is worth being clear about why: 0.1 is not representable in binary, $-0.723\,379\,629\,629\,5$ mm is not representable in decimal at finite length, and every conversion between the two loses something. The practical rule is not “use more digits” but “know which of your numbers have been through a file, and do not subtract two nearly equal ones that have”.

Exercise 3.4. ●●● You are given a model of a twenty-storey frame built by someone else. It runs, and the roof displacement looks plausible. Describe, in order, the first three things you would check, and say what each one would rule out.

Answer: equilibrium first (rules out lost loads, unintended supports and restrained degrees of freedom absorbing load); then a units audit on one member (rules out the power-of-ten errors that look plausible); then a coarse hand estimate of the fundamental behaviour, such as a cantilever approximation of the whole frame (rules out order-of-magnitude modelling errors)

Solution. Equilibrium comes first because it is free, needs no understanding of the structure, and its failure is unambiguous. If $\sum \vec{R} \neq \sum \vec{P}$, nothing else is worth checking yet.

Units come second because they are the error most likely to survive a plausibility check. A roof displacement that “looks plausible” is exactly what a consistent unit error produces — the numbers stay in a believable range while being wrong by a factor of a thousand. Pick one member, take its E , A and I out of the model, and confirm they are the numbers you would write down for that section in N and mm.

Third, an independent estimate of the whole. Treat the frame as a cantilever, or use the rule of thumb that a moment frame’s fundamental period is roughly $0.1N$ seconds for N storeys, and see whether the model is in the same world. This will not catch a subtle error, and it is not meant to; it catches the one where a member was defined with its strong and weak axes exchanged, or a storey height entered in metres.

What all three have in common is that none requires the model to be correct in order to be applied, and none requires a reference solution. That is the property to look for when you are choosing checks for a structure nobody has solved before — which, after this course, is all of them.

Week 4: Linear Static Analysis of 1D Elements

Learning outcomes

After this week you should be able to:

1. build a beam model with `elasticBeamColumn`, and say why it needs a `geomTransf` when a truss did not;
2. apply a load along a member with `eleLoad`, in the right sign convention;
3. read reactions and internal moments out of a beam model;
4. choose how many elements to use, for the right reason;
5. explain why refining a beam mesh does not improve the answer at a node, and what it does improve.

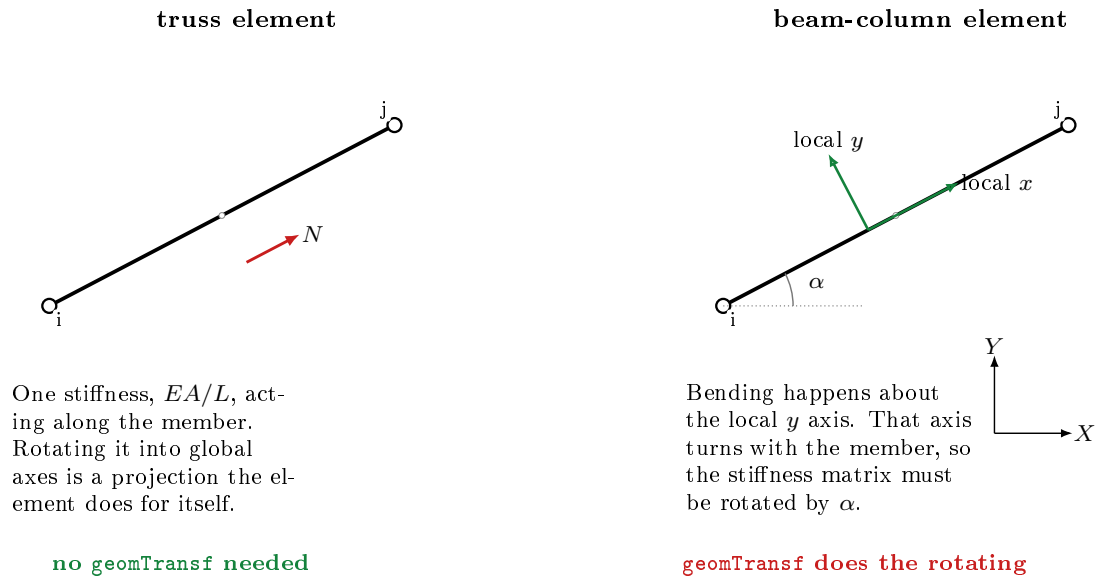
Notation. w distributed load L span I second moment of area M bending moment V shear force
 R reaction α member inclination

Everything so far has been pin-jointed. Bars carried axial force, nodes had two degrees of freedom, and nothing anywhere bent. That was deliberate: it let three weeks go by without a single modelling decision that could reasonably have gone another way.

This week ends that. A beam brings a third degree of freedom, a coordinate transformation and a load that acts along a member rather than at a point — and, more importantly, it brings the first question whose answer is genuinely yours: *how many elements?* The answer for beams is not the one most finite element courses give, and understanding why is worth more than the beam itself.

4.1 Why a beam needs a transformation

The difference between a bar and a beam is smaller than it looks and has one consequence that shows up immediately in the code. [Figure 4.1](#) sets the two side by side.



The transformation is a separate object with its own tag because it is a separate *decision*. **Linear** assumes the rotation α never changes, which is true while displacements are small. **PDelta** and **Corotational** recompute it as the structure moves — and that is the whole of geometric nonlinearity, waiting in week 10.

Figure 4.1: An element's stiffness is written in its own axes. For a bar those axes do not matter; for a beam they do, and something has to rotate between them.

A truss element has one stiffness, EA/L , acting along its own length. Whichever way the bar points, that is still what it does, and projecting an axial force onto the global axes is arithmetic the element can do for itself.

A beam is different. It resists bending about an axis perpendicular to itself, and that axis turns as the member turns. The element's stiffness matrix is written in local axes — along the member and across it — and must be rotated into the global axes the nodes live in. That rotation is what **geomTransf** is.

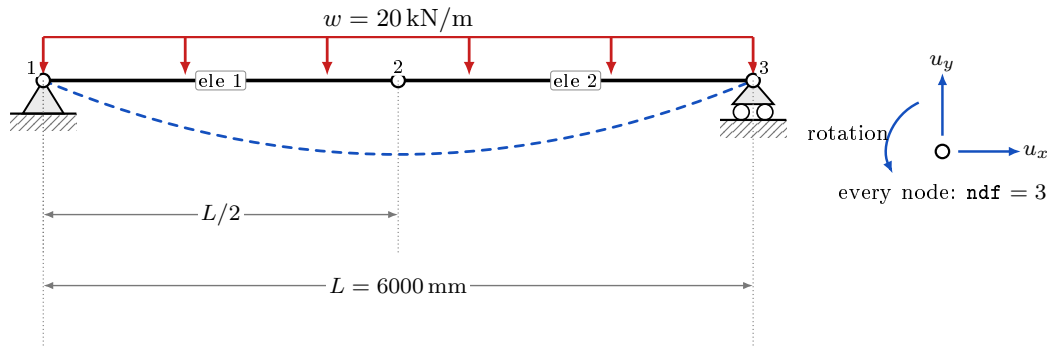
Definition 4.1: Geometric transformation

A **geometric transformation** (*TR: geometrik dönüşüm*) is the rule relating an element's local axes to the global axes of the model. **Linear** assumes the relationship is fixed — computed once from the undeformed geometry and never revisited.

That assumption is exactly what "small displacements" means, and it is the first place in this course where a *modelling* assumption is written as a separate object with its own tag. It is a separate object because it is a separate decision: **PDelta** and **Corotational** recompute the transformation as the structure moves, and swapping one word is the whole of geometric nonlinearity. Week 10 does exactly that.

4.2 Building the beam

The structure is the one in Fig. 4.2, and the reason it is drawn with two elements rather than one is the subject of §4.4.



A **pin and a roller**, not two pins. Fixing the horizontal movement at both ends would make the beam resist stretching as well as bending, which is a different structure and a stiffer one.

The mesh is two elements for one reason only: it puts a node at midspan, and a model can only report a displacement where it has a node.

Figure 4.2: The week's beam: 6 m simply supported, carrying 20 kN/m, meshed into two elements so that a node falls at midspan.

Listing 4.1: w04/beam.py — Three things are new, and all three follow from one fact: a beam carries bending and a bar does not.

```

38 def build_model(p, n_elements=2):
39     """Simply supported beam, divided into n_elements equal pieces.
40
41     Three things are new here and all three are consequences of one fact: a
42     beam carries bending, and a bar does not.
43
44     ndf is 3, not 2. A node now has a rotation as well as two translations,
45     because a beam end can rotate and the element resists that rotation.
46
47     A geomTransf appears. An element's stiffness is written in its own local
48     axes -- along the member and across it -- and something has to rotate that
49     into the global axes the nodes live in. A Truss needed no such thing
50     because axial stiffness looks the same from any direction.
51
52     The supports are no longer both pinned: fixing the rotation at both ends
53     would build a fixed-ended beam, which is a different structure.
54     """
55     ops.wipe()
56     ops.model("basic", "-ndm", 2, "-ndf", 3)
57
58     dx = p["L"] / n_elements
59     for i in range(n_elements + 1):
60         ops.node(i + 1, i * dx, 0.0)
61
62     ops.fix(1, 1, 1, 0)          # pin:   ux, uy held, rotation free
63     ops.fix(n_elements + 1, 0, 1, 0) # roller: uy held only
64
65     ops.geomTransf("Linear", 1)
66     for e in range(n_elements):
67         ops.element("elasticBeamColumn", e + 1, e + 1, e + 2,
68                     p["A"], p["E"], p["I"], 1)
69
70     ops.timeSeries("Linear", 1)
71     ops.pattern("Plain", 1, 1)
72

```

```

73 # A load along the member, not at a node. Wy is transverse in the element's
74 # LOCAL axes, so downward on a horizontal beam is negative. Every element
75 # needs its own share -- eleLoad applies to the element you name, and a
76 # forgotten one leaves a stretch of beam unloaded and nothing says so.
77 for e in range(n_elements):
78     ops.eleLoad("-ele", e + 1, "-type", "-beamUniform", -p["w"])

```

Three details in that listing repay attention.

ndf is 3. Each node now has a rotation as well as two translations, because a beam end can rotate and the element resists it. Week 2 warned that **ndf** must match what the elements can carry; this is the other side of that rule.

The supports are a pin and a roller, not two pins. Holding the horizontal movement at both ends would make the beam resist stretching as well as bending — a stiffer structure, and not the one the textbook formula describes.

And the load is applied with **eleLoad** rather than **load**, because it acts along the member and not at a node. Its argument **Wy** is transverse in the element's *local* axes, so downward on a horizontal beam is negative.

Note

one **eleLoad** loads the beam → every element needs its own **eleLoad**

eleLoad applies to the elements you name and to no others. Load two elements out of four and half the beam carries nothing — with no warning, and an answer that looks entirely reasonable. Exercise 4.3 is that mistake, and it is caught by exactly one of the checks from week 3.

4.3 A beam you can check by hand

Example 4.1 (Simply supported beam under a uniform load). A 6 m IPE 300 beam, simply supported, carries 20 kN/m over its whole span. Find the reactions, the midspan moment and the midspan deflection, and check the beam against a deflection limit of $L/250$.

Solution

Model. Two **elasticBeamColumn** elements, linear elastic, small displacements, shear deformation neglected. Pin at one end, roller at the other.

Governing relations. For a simply supported beam under a uniform load,

$$R = \frac{wL}{2}, \quad M_{\max} = \frac{wL^2}{8}, \quad \delta_{\max} = \frac{5wL^4}{384EI}. \quad (4.1)$$

Numbers. With $w = 20 \text{ N/mm}$ and $L = 6000 \text{ mm}$,

$$R = \frac{(20)(6000)}{2} = 60\,000 \text{ N}, \quad M_{\max} = \frac{(20)(6000)^2}{8} = 9.000 \times 10^7 \text{ N mm},$$

$$\delta_{\max} = -\frac{5(20)(6000)^4}{384(200000)(8.356 \times 10^7)} = -\frac{1.296 \times 10^{17}}{6.417 \times 10^{15}} = -20.20 \text{ mm}.$$

$$R = 60.0 \text{ kN}, \quad M_{\max} = 90.0 \text{ kN m}, \quad \delta_{\max} = -20.2 \text{ mm}$$

The model is asked the same three questions:

Listing 4.2: **w04/beam.py** — **eleResponse** returns six numbers for a 2D beam: axial, shear and moment at each end, in local axes.

```

101 def results(p, n_elements=2):
102     """Read back the deflected shape, the reactions and the internal moment.
103
104     eleResponse(tag, 'forces') returns six numbers for a 2D beam element:
105     axial, shear and moment at the i end, then the same three at the j end, all
106     in LOCAL axes. The moment at an interior node is therefore available twice
107     -- once from the element on each side -- and the two must agree. That is a

```

```

108     free equilibrium check, and it is used below.
109     """
110     ops.reactions()
111
112     n_nodes = n_elements + 1
113     dx = p["L"] / n_elements
114     shape = [(i * dx, ops.nodeDisp(i + 1, 2)) for i in range(n_nodes)]
115
116     reaction_left = ops.nodeReaction(1, 2)
117     reaction_right = ops.nodeReaction(n_nodes, 2)
118
119     # Internal moment at every interior node, taken from the element to its left.
120     moments = [ops.eleResponse(e + 1, "forces")[5] for e in range(n_elements - 1)]
121
122     return {
123         "shape": shape,
124         "R_left": reaction_left,
125         "R_right": reaction_right,
126         "node_moments": moments,
127     }

```

Simply supported beam, 6 m span, 20 kN/m, IPE 300

	model	hand
reaction, left [N]	60000.00	60000.00
reaction, right [N]	60000.00	60000.00
midspan moment [Nmm]	9e+07	9e+07
midspan deflection [mm]	-20.1951	-20.1951

Check. Three independent things agree. The reactions sum to $60\text{ kN} + 60\text{ kN} = 120\text{ kN}$, which is wL exactly — the equilibrium check from week 3, and the one that would catch a forgotten `eleLoad`. The moment and deflection match Eq. (4.1) to machine precision. And the serviceability question the example actually asked has an answer: $L/250 = 24.0\text{ mm}$, and 20.2 mm is inside it — but only by 16%, which is close enough that the next exercise is worth doing before choosing this section.

4.4 How many elements?

Here is the question the previous section quietly avoided. The beam was meshed into two elements. Why not one, or twenty?

Example 4.2 (What refining the mesh does, and does not, do). Solve the same beam with one, two, three, four and eight elements, and compare.

Solution

Model. Unchanged apart from `n_elements`.

Deflection reported, against number of elements:

1 element(s):	no node at midspan
2 element(s):	-20.1951 mm
3 element(s):	no node at midspan
4 element(s):	-20.1951 mm
8 element(s):	-20.1951 mm

two, four and eight elements give the *same* exact answer

Why. A prismatic Euler–Bernoulli beam element, given the consistent nodal load vector that `eleLoad` applies, reproduces the *exact* nodal displacements for any mesh. The approximation is in the shape

between the nodes, not at them. There is no convergence study to run here, because there is nothing to converge to that is not already reached. Figure 4.3 shows both halves of that statement.

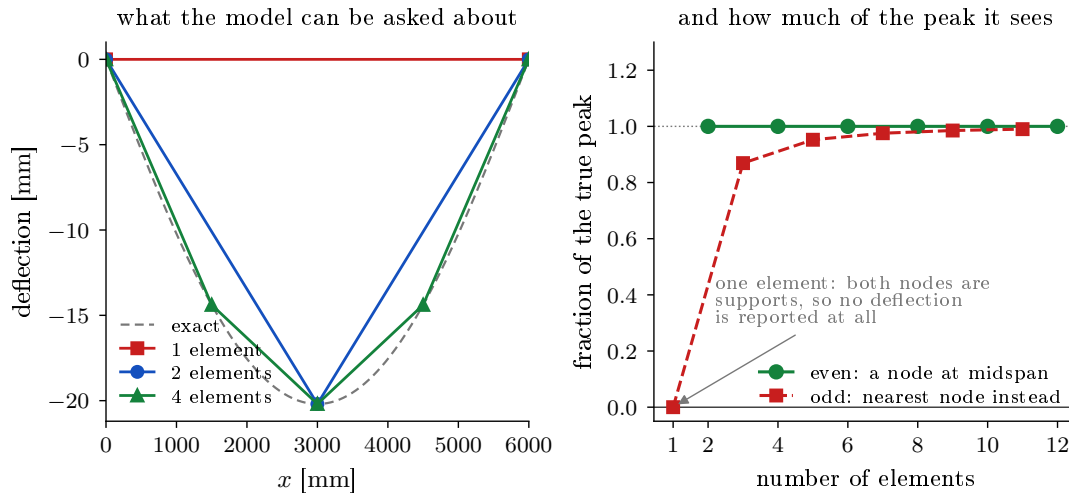


Figure 4.3: Every marker lies on the exact curve, at every mesh. What refinement buys is not accuracy but resolution — and with an odd number of elements the largest deflection in the span is not among the numbers available at all.

Check. The one-element case is the useful one. Both of its nodes are supports, so every displacement it can report is a restrained one, and the model dutifully reports that the beam does not deflect. Nothing is wrong with the analysis. The question was simply asked of a model that had nowhere to answer it from.

Note

refine the mesh until the answer stops changing → put nodes where you need answers

The convergence study is a real and necessary habit — for continuum elements, for fibre sections in week 8, and for anything where the element cannot represent the exact solution. For a prismatic elastic beam it is theatre: the answer never changes, so "it stopped changing" proves nothing.

What a beam mesh has to do instead is put a node at every point you intend to ask about, and enough in between to draw a smooth diagram. A node at every support, at every point load, at every change of section — and, if you want the midspan deflection, at midspan.

Choosing a beam mesh

1. A node wherever the structure changes: support, point load, change of section.
2. A node wherever you want to report a result.
3. Enough elements in between that a plotted diagram looks like a curve rather than a polygon — four to eight per span is usually plenty.
4. In week 8 and after, when elements stop being exact, add a genuine convergence check on top of all this.

Summary

Item	Command or expression	Note
degrees of freedom	<code>model('basic', '-ndm', 2, '-ndf', 3)</code>	rotation added
transformation	<code>geomTransf('Linear', tag)</code>	Definition 4.1
beam element	<code>element('elasticBeamColumn', tag,</code>	<code>i, j, A, E, I,</code> <code>transf</code>
pin support	<code>fix(tag, 1, 1, 0)</code>	rotation free
roller support	<code>fix(tag, 0, 1, 0)</code>	vertical only
load along a member	<code>eleLoad('-ele', t, '-type',</code> local axes; downward is negative; <i>every</i> element needs one	<code>'-beamUniform', Wy</code>
element forces	<code>eleResponse(tag, 'forces')</code>	N, V, M at each end
reaction	$R = wL/2$	Eq. (4.1)
maximum moment	$M = wL^2/8$	at midspan
maximum deflection	$\delta = 5wL^4/384EI$	at midspan
deflection limit	$L/250$	24 mm here

Exercises

Exercise 4.1. ●●● The span is increased to 7000 mm, everything else unchanged. Report the midspan deflection and say whether the beam still satisfies $L/250$.

Answer: -37.4 mm against an allowance of 28.0 mm — it fails

Solution. $\delta = 5(20)(7000)^4/[384(200000)(8.356 \times 10^7)] = -37.41$ mm, and $L/250 = 28.0$ mm.

A span increased by 17% turns a beam that passed into one that fails by a third, and the arithmetic says why: the deflection goes as L^4 while the allowance goes as L . The margin against a span-proportional limit therefore falls as L^3 . This is why deflection, not strength, governs the section of most long-span steel beams — and why the 16% margin in [Example 4.1](#) was worth noticing rather than being satisfied with.

Exercise 4.2. ●●● Report the bending moment at the quarter point, $x = L/4$, of the beam in [Example 4.1](#). State how many elements you need and why.

Answer: 67.5 kN m; the mesh must have a node at $L/4$, so the number of elements must be a multiple of four

Solution. $M(x) = wx(L - x)/2$, so $M(L/4) = (20)(1500)(4500)/2 = 6.75 \times 10^7$ N mm = 67.5 kN m.

With four elements the model reports node moments of 67.5, 90.0 and 67.5 kN m — the quarter point, midspan, and the symmetric quarter point. With two elements the value at $L/4$ simply does not exist in the output, for the same reason the midspan deflection did not exist with one element.

The general statement is the one worth carrying: an element reports forces *at its ends*. A question about a point inside an element has to be answered by putting a node there, or by integrating the loading by hand.

Exercise 4.3. ●●● A student meshes the beam of [Example 4.1](#) into two elements and writes

```
ops.eleLoad('-ele', 1, '-type', '-beamUniform', -w)
```

The analysis runs. The reported midspan deflection is -10.10 mm. Find the mistake, and name the check that catches it.

Answer: only element 1 was loaded, so half the beam carries nothing; the equilibrium check catches it, because the reactions sum to 60 kN instead of 120 kN

Solution. `eleLoad` applies to the elements named and no others. Element 2 carries no load, so half the span is unloaded, and the deflection comes out at almost exactly half the correct value.

That "almost exactly half" is what makes this worth an exercise. The answer is the right order of magnitude, the right sign, in the right units, and smaller than the limit — everything a plausibility check looks at, it passes. Comparing against [Eq. \(4.1\)](#) would catch it, but on a real structure there is no formula to compare against.

What does catch it, on any structure of any size, is equilibrium: $\sum R = 60$ kN against an applied $wL = 120$ kN. Week 3 called this the check to reach for first because it needs no theory; this is the week where that stops being an abstract claim.

Exercise 4.4. ● ● ● [Example 4.2](#) showed that refining a beam mesh does not change the nodal answer. Name a situation later in this course where refining *will* change the answer, and explain what is different about it.

Answer: a fibre section in week 8, where the number of fibres controls how well the stress distribution over the cross-section is integrated, and a nonlinear beam element whose deformation shape is no longer the cubic the exact solution requires

Solution. Nodal exactness in [Example 4.2](#) rests on a coincidence that will not survive the term: the cubic shape functions of an Euler–Bernoulli beam element happen to *be* the exact deflected shape of a prismatic elastic member under end forces. The element can represent the true answer, so it does.

Two things break that. Once the material yields, the curvature is no longer proportional to moment and the exact shape is no longer cubic — the element approximates, and more elements approximate better. And a fibre section integrates stress over the cross-section as a finite sum over fibres, so the number of fibres is a genuine discretisation with a genuine error.

The distinction to carry forward is between a discretisation that can represent the answer and one that cannot. In the first, refining is a waste of time; in the second, not refining is a wrong answer. Knowing which you are in is a judgement about the mechanics, not about the software — and it is not something the program will ever tell you.

Exercise 4.5. ● ● ● Explain in two or three sentences why the beam in [Example 4.1](#) was given a pin at one end and a roller at the other, rather than pins at both ends, and what would have changed had both been pinned.

Answer: two pins would restrain the horizontal movement at both ends, so the beam would resist stretching as well as bending; it would be stiffer and would carry axial force, which is a different structure from the one $5wL^4/384EI$ describes

Solution. A simply supported beam is defined by being free to change length. As it bends, the ends draw very slightly together; a roller lets that happen, a second pin does not.

For this beam the numerical difference is negligible — the deflections are small and the induced axial force is tiny. That is not the reason to get it right. The reason is that with two pins the model is no longer the structure [Eq. \(4.1\)](#) describes, so the comparison that verified it would no longer be a valid check, and a small unexplained discrepancy would appear with nothing to attribute it to.

The habit worth forming: the supports are part of the idealisation, not boilerplate. When a model and a formula disagree slightly, the boundary conditions are among the first places to look.

Week 5: Linear Static Analysis of 2D Frames

Learning outcomes

After this week you should be able to:

1. design a tag formula, and say why a readable tag beats a running counter;
2. generate a frame of any size from its dimensions with loops;
3. keep gravity and lateral load in separate patterns, and build load combinations from them;
4. compute inter-storey drift correctly, including which nodes it is measured on;
5. recognise when a reported quantity is an artefact of how it was measured rather than a property of the structure.

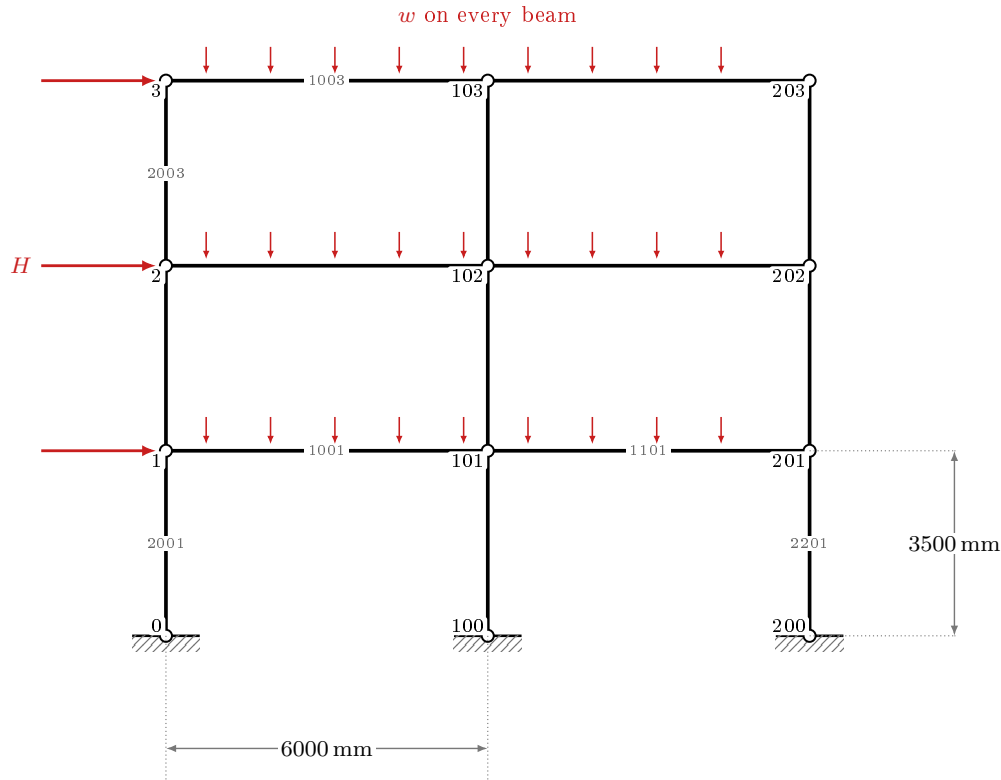
Notation. G gravity load case W lateral load case H storey lateral force h storey height θ inter-storey drift ratio

The structural content of this week is thin: it is the same linear static analysis as week 4, with more elements. That is deliberate. The physics is held flat so that the programming can rise, and the programming is where this week earns its place — because a three-storey frame has 12 nodes and 15 members, and typing those out by hand is not a skill, it is a liability.

The whole chapter comes down to one idea. Decide once how an object's tag follows from where it is, and the loops write themselves. Everything else — load combinations, drift, frames of any size — falls out of that decision.

5.1 The tag formula

Figure 5.1 is the week's frame, drawn with the tags the script actually generates.



Every tag here was computed, not typed. A node's tag is $100 \times \text{line} + \text{storey}$, so **203** is line 2, storey 3 and an error message naming it says where to look. Beams start at 1000 and columns at 2000, on the same formula.

A running counter would be just as unique and would tell you nothing. Changing `n_storeys` from 3 to 12 changes no other line of the script.

Figure 5.1: A two-bay, three-storey frame. Every tag in the picture was computed from a position, not typed.

Definition 5.1: Tag formula

A **tag formula** (*TR: etiket formülü*) is a rule turning an object's position in the structure into its integer tag. It must be unique, and it should be *invertible by eye*: given the tag, you should be able to say where the object is.

Listing 5.1: `w05/frame.py` — Three formulas, one idea. Node 203 is line 2, storey 3.

```

46 def node_tag(line, storey):
47     """The tag of the node on a given column line at a given floor.
48
49     line    0, 1, 2 ...   from the left
50     storey  0 at the base, 1 at the first floor, and so on
51
52     The formula is the design decision of this whole script. It has to be
53     unique, and it should be READABLE: node 203 is line 2, storey 3, and a
54     number in an error message tells you where to look. A running counter
55     would also be unique and would tell you nothing.
56
57     The 100 caps the frame at 99 storeys, which is a limit worth stating rather

```

```

58     than discovering.
59     """
60     return 100 * line + storey
61
62
63 def beam_tag(line, storey):
64     """Beam from `line` to `line + 1`, at `storey`. Tags start at 1000."""
65     return 1000 + 100 * line + storey
66
67
68 def column_tag(line, storey):
69     """Column from `storey - 1` up to `storey`. Tags start at 2000."""
70     return 2000 + 100 * line + storey

```

A running counter would be just as unique. The difference shows up the first time OpenSees prints **WARNING: node 203 not found:** with this scheme that message says *third column line, roof*, and with a counter it says nothing at all. You will read more of those messages than you expect.

The three families are separated by their leading digits — nodes below 1000, beams from 1000, columns from 2000 — so they cannot collide. The factor of 100 caps the frame at 99 storeys, which is a limit worth writing down rather than discovering.

Listing 5.2: `w05/frame.py` — Every loop has the same shape: walk over lines and storeys, and let the formulas turn a position into a name.

```

75 def build_model(p):
76     """Generate the whole frame from its dimensions.
77
78     Every loop below is the same shape: walk over the lines and storeys, and
79     let the tag formulas turn a position into a name. Nothing here changes when
80     the frame gets bigger.
81     """
82     ops.wipe()
83     ops.model("basic", "-ndm", 2, "-ndf", 3)
84
85     lines = range(p["n_bays"] + 1)
86     storeys = range(1, p["n_storeys"] + 1)
87
88     # Nodes, including the base.
89     for line in lines:
90         for storey in range(p["n_storeys"] + 1):
91             ops.node(node_tag(line, storey),
92                     line * p["bay"], storey * p["storey"])
93
94     # Fixed bases. A moment frame needs them: on pins the whole frame is a
95     # mechanism until the beams pick the sway up, and the drift is enormous.
96     for line in lines:
97         ops.fix(node_tag(line, 0), 1, 1, 1)
98
99     ops.geomTransf("Linear", 1)
100
101     # Columns: one per line, per storey.
102     for line in lines:
103         for storey in storeys:
104             ops.element("elasticBeamColumn", column_tag(line, storey),
105                         node_tag(line, storey - 1), node_tag(line, storey),
106                         p["A_col"], p["E"], p["I_col"], 1)
107
108     # Beams: one per bay, per floor.
109     for line in range(p["n_bays"]):
110         for storey in storeys:
111             ops.element("elasticBeamColumn", beam_tag(line, storey),
112                         node_tag(line, storey), node_tag(line + 1, storey),
113                         p["A_beam"], p["E"], p["I_beam"], 1)

```

Nothing in that listing mentions three storeys or two bays. Change `n_storeys` from 3 to 12 and the same twenty lines build a frame with 39 nodes and 60 elements.

5.2 Load combinations

Listing 5.3: `w05/frame.py` — Two patterns, not one — which is what makes a combination possible.

```

118 def apply_gravity(p, factor=1.0):
119     """Uniform load on every beam, in pattern 1."""
120     ops.timeSeries("Linear", 1)
121     ops.pattern("Plain", 1, 1)
122     for line in range(p["n_bays"]):
123         for storey in range(1, p["n_storeys"] + 1):
124             ops.eleLoad("-ele", beam_tag(line, storey),
125                        "-type", "-beamUniform", -factor * p["w"])
126
127
128 def apply_lateral(p, factor=1.0):
129     """A horizontal force at the left-hand node of every floor, in pattern 2.
130
131     Two patterns, not one. Keeping gravity and wind in separate patterns is
132     what makes a load COMBINATION possible: the factors below are applied when
133     the pattern is created, so 1.2G + 1.6W is two calls with two factors rather
134     than a rebuilt model.
135     """
136     ops.timeSeries("Linear", 2)
137     ops.pattern("Plain", 2, 2)
138     for storey in range(1, p["n_storeys"] + 1):
139         ops.load(node_tag(0, storey), factor * p["H"], 0.0, 0.0)

```

Keeping gravity and lateral load in separate patterns with separate tags is the small decision that makes load combinations cheap. Because the factor is applied when the pattern is created, $1.2G + 1.6W$ is two calls with two numbers rather than a model rebuilt by hand.

Example 5.1 (Four combinations on one frame). Report the roof sway and the base reactions of the frame under G , W , $G + W$ and $1.2G + 1.6W$.

Solution

Model. As generated above; fixed bases, **Linear** transformation, elastic throughout.

Frame: 2 bays x 3 storeys -> 12 nodes, 15 elements

Load combinations

combination	roof sway [mm]	base shear [N]
1.0 G	-0.0000	0.0
1.0 W	16.0127	-120000.0
1.0 G + 1.0 W	16.0127	-120000.0
1.2 G + 1.6 W	25.6204	-192000.0

under $1.2G + 1.6W$: $\delta_{\text{roof}} = 25.6 \text{ mm}$

Check. Four things, and none of them needed a textcourse.

Equilibrium in both directions: the base shear under W is -120 kN , which is exactly the three storey forces of 40 kN ; the base axial under G is 900 kN , which is exactly w times the total beam length.

Gravity produces no sway — the third row equals the second. And the factored combination is exactly 1.6 times the W row, because gravity contributes nothing to sway and the model is linear. Superposition holding *exactly* is a strong check while it lasts; from week 10 it stops holding, and noticing that it has stopped is how you detect yielding.

5.3 Measuring drift, and a number that is not real

Here is the part of the week that is not about programming at all, and is the reason the chapter exists. [Example 5.1](#) reported a roof sway for every combination without ever saying what that number was measured on, and the omission matters more than it looks.

Example 5.2 (Sway under gravity). The frame is symmetric and the gravity load is symmetric. What is the roof sway under gravity alone?

Solution

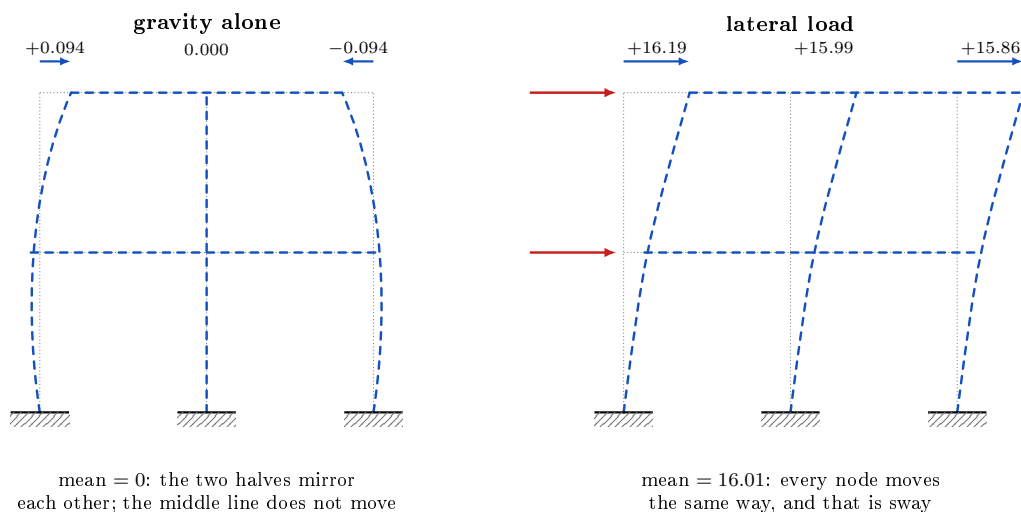
Model. Gravity only, no lateral load.

```
Gravity alone: the roof nodes, left to right
+0.0938 -0.0000 -0.0938 mm
mean = -0.0000 mm
```

$$\delta_{\text{sway}} = 0 \text{ exactly}$$

Why the question is a trap. The left roof node moves 0.094 mm. Ask that node what the sway is and it will tell you 0.094 mm of sway caused by vertical load — a small number, in the right units, of an entirely plausible size, and not a real effect at all.

[Figure 5.2](#) shows what is actually happening, and why one node cannot answer the question.



Sway is the movement of a floor *as a whole*, so it is the mean of its nodes. Take one node instead and the symmetric deformation on the left is reported as 0.094 mm of sway caused by vertical load — small, plausible, and not a real effect at all. Drift ratios inherit the error, because they are differences of these numbers.

Figure 5.2: Under gravity the two halves of a symmetric frame mirror each other and the middle line does not move: the mean is zero. Under lateral load every node moves the same way, and the mean is the sway.

Sway is the movement of a floor *as a whole*. The mean of its nodes is zero for any symmetric deformation and equals the translation when the floor really does translate, so the mean is the quantity to report.

Listing 5.4: w05/frame.py — Drift is a difference of floor sways, so it inherits whatever error the sway measurement has.

```

160 def floor_sway(p, storey):
161     """The sideways movement of a floor: the MEAN of its nodes, not one of them.
162
163     Reading one column line looks equivalent and is not. Under gravity alone
164     this frame spreads: the left roof node moves +0.094 mm, the right one
165     -0.094 mm, and the middle one not at all. Report the left node and you will
166     publish 0.094 mm of "sway" caused by vertical load, which is not sway at
167     all -- it is the two halves of a symmetric frame leaning apart.
168
169     The mean is zero for any symmetric spreading, and equals the sway when the
170     floor really does translate. That is the quantity drift is defined on.
171     """
172     lines = range(p["n_bays"] + 1)
173     ux = [ops.nodeDisp(node_tag(line, storey), 1) for line in lines]
174     return sum(ux) / len(ux)
175
176
177 def storey_drifts(p):
178     """Inter-storey drift ratio for every storey.
179
180     Drift ratio is the movement of a floor RELATIVE TO THE ONE BELOW, divided
181     by the storey height. Using the absolute displacement instead is the
182     commonest mistake in this calculation, and it flatters the upper storeys
183     badly: they move the most in total while often deforming the least.
184     """
185     return [(floor_sway(p, s) - floor_sway(p, s - 1)) / p["storey"]
186             for s in range(1, p["n_storeys"] + 1)]

```

Check. The mean is zero to machine precision, and the middle node does not move at all — both are consequences of symmetry that the model was never told about, so their appearing is evidence that the geometry, the supports and the loads were all entered symmetrically.

Note

read the corner node and call it the sway → take the mean of the floor's nodes

This is a measurement error, not a modelling error. The analysis was right; the question was asked of the wrong node. Nothing in the output would ever have told you — and because drift ratios are differences of these numbers, every drift in the building inherits it.

Example 5.3 (Drift profile). Report the inter-storey drift ratio of each storey under $G + W$, and say which storey governs.

Solution

Governing relation. The inter-storey drift ratio is

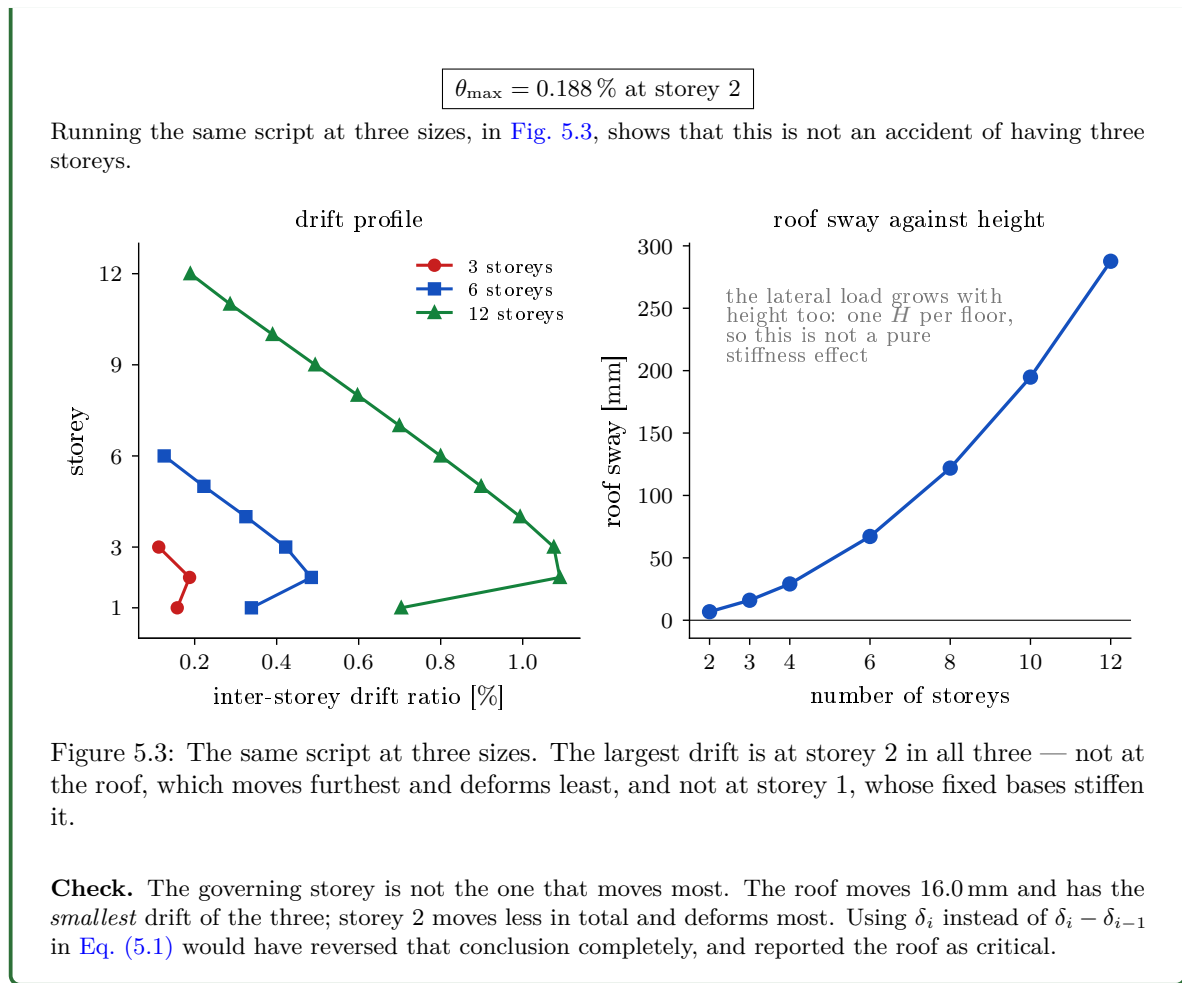
$$\theta_i = \frac{\delta_i - \delta_{i-1}}{h}, \quad (5.1)$$

the movement of a floor *relative to the one below*, divided by the storey height.

```

Inter-storey drift under 1.0 G + 1.0 W
storey 1: 0.1575 % (limit 2.00 %)
storey 2: 0.1876 % (limit 2.00 %)
storey 3: 0.1124 % (limit 2.00 %)

```



Summary

Item	Command or expression	Note
node tag	$100 \times \text{line} + \text{storey}$	Definition 5.1
beam tag	$1000 + 100 \times \text{line} + \text{storey}$	families cannot collide
column tag	$2000 + 100 \times \text{line} + \text{storey}$	
fixed base	<code>fix(tag, 1, 1, 1)</code>	a moment frame needs it
gravity	<code>pattern('Plain', 1, 1) + eleLoad</code>	one pattern
lateral	<code>pattern('Plain', 2, 2) + load</code>	a <i>separate</i> pattern
combination	factors passed when the pattern is built	$1.2G + 1.6W =$ two calls
floor sway	mean of the floor's nodal u_x	not one corner node
drift ratio	$\theta_i = (\delta_i - \delta_{i-1})/h$	Eq. (5.1)
equilibrium	$\sum V_{\text{base}} = \sum H$	and $\sum N_{\text{base}} = \sum wL$

Exercises

Exercise 5.1. ●●● Change `n_storeys` to 12, leaving everything else alone. Report the number of nodes and elements, and the roof sway under $G + W$.

Answer: 39 nodes, 60 elements, 288 mm

Solution. Nodes are $(n_{\text{bays}} + 1)(n_{\text{storeys}} + 1) = 3 \times 13 = 39$; elements are 3×12 columns plus 2×12 beams = 60. The sway is 287.7 mm.

Two things are worth noticing. The first is that no line of the model-building code changed — that is what the tag formula bought. The second is that the sway grew by a factor of 18 for four times the height, because a taller frame receives more total lateral load (one H per floor) applied further up. Sway is not a pure stiffness effect, and comparing buildings of different heights by their roof displacement is a mistake for exactly this reason. Drift ratio is the comparable quantity, and storey 2 still governs at twelve storeys.

Exercise 5.2. ●●● Write the combination $0.9G + 1.6W$ and report the base axial reaction. Explain why a designer would ever use a combination with a gravity factor below 1.0.

Answer: 810 kN; a reduced gravity factor is used when gravity helps, for example in checking overturning or uplift on a windward column

Solution. `solve(FRAME, gravity=0.9, lateral=1.6)` gives a base axial of $0.9 \times 900 \text{ kN} = 810 \text{ kN}$.

The reason such a combination exists is that load factors are about *uncertainty in the direction that hurts*, not about being conservative in general. For a windward column the lateral load causes tension and the gravity load relieves it, so assuming more gravity than you are sure of is unconservative. The factor is reduced to 0.9 precisely because gravity is the helpful load in that check.

This is why the model keeps the two cases in separate patterns rather than adding them once. A structure needs several combinations, some of which scale the same load case differently, and only a model that can be re-solved with new factors can produce them all.

Exercise 5.3. ●●● A student computes drift as δ_i/h — the absolute displacement of each floor divided by the storey height — instead of Eq. (5.1). Which storey do they conclude is critical, and why is that the opposite of the right answer?

Answer: they conclude the roof, because absolute displacement always increases with height; the correct answer is storey 2, and the roof in fact has the smallest drift of the three

Solution. Absolute displacement accumulates: every storey adds its own deformation to everything below it, so δ_i is monotonically increasing and the roof always wins. It would win on a perfectly rigid top storey.

The quantity that means anything is how much a storey deforms *itself*, which is the difference between its displacement and the one below. Here the roof moves 16.0 mm in total — the most — while contributing only 0.112% drift, the least of the three.

The error is worth dwelling on because it produces a confident, plausible, monotonic answer that is wrong in its conclusion rather than in its arithmetic. It is also the reason drift limits in codes are written on inter-storey drift and never on total displacement.

Exercise 5.4. ●●● The bases are fixed. Change them to pins — `ops.fix(node_tag(line, 0), 1, 1, 0)` — and predict, before running, what happens to the roof sway and to the drift profile. Then run it.

Answer: sway increases several-fold, and the drift profile changes shape: with the base rotation released, storey 1 becomes the critical storey instead of storey 2

Solution. Fixed bases contribute a large share of a moment frame's lateral stiffness, and releasing them removes it. The sway grows substantially.

The change in *shape* is the more instructive part. Storey 2 governed in Example 5.3 precisely because the fixed bases stiffened storey 1 below it. Release them and storey 1 has nothing resisting its

rotation but the beams above, so it becomes the softest storey and the drift maximum moves to the bottom.

This is a soft-storey mechanism in miniature, and it is worth seeing in an elastic model now, because in week 11 it will reappear as the thing that decides where a real frame fails. It also makes a practical point about modelling: a column base is rarely perfectly fixed or perfectly pinned, and the answer is sensitive enough to that choice that assuming one without thinking is a decision, not a default.

Exercise 5.5. ●●● **Example 5.2** showed a quantity that looked real and was not. Describe how you would detect the same class of error in a model whose structure is *not* symmetric, so that the mean-is-zero argument is unavailable.

Answer: apply the load case in both directions and compare: a genuine response reverses sign with the load, while a measurement artefact tied to the structure's own deformation does not

Solution. Symmetry gave a free answer in **Example 5.2** — we knew what the sway had to be. On an unsymmetric frame that check is gone, and something else has to take its place.

The most generally useful substitute is reversal. Run $+W$ and $-W$. Any genuine response to the lateral load changes sign exactly; anything that stays put is not a response to W at all. Taking half the difference of the two runs isolates the part that is genuinely due to the lateral load, and the part that survives the averaging is the gravity artefact.

The broader habit is to ask, of any number you are about to report, *what would make this quantity zero?* If you can name a case where it must vanish and the model produces it, the measurement is probably sound. If you cannot name such a case, you do not yet understand the number well enough to publish it — and that is true whether or not the model is right.

Week 6: Material Models I: Elastic and Nonlinear Elastic

Learning outcomes

After this week you should be able to:

1. test a material on its own, without a structure around it;
2. define **Elastic**, **ElasticPP** and **Hardening**, and say what each argument means;
3. explain why a monotonic pull cannot distinguish a nonlinear material from a linear one below yield, and what loading path can;
4. compute the permanent set left after loading and unloading;
5. recognise that a material's stress depends on the path taken to a strain, not only on the strain.

Notation. σ stress ε strain E_s modulus of elasticity f_y yield strength $\varepsilon_y = f_y/E_s$ yield strain b strain-hardening ratio

Every material so far has been **Elastic**: one number, a straight line, and no way to be interesting. That was fine while nothing was meant to yield. From this week the material is the part of the model that decides the answer, and it stops being possible to write one down without checking what it does.

So the first thing to learn is not a material at all. It is how to look at one.

6.1 A material on the bench

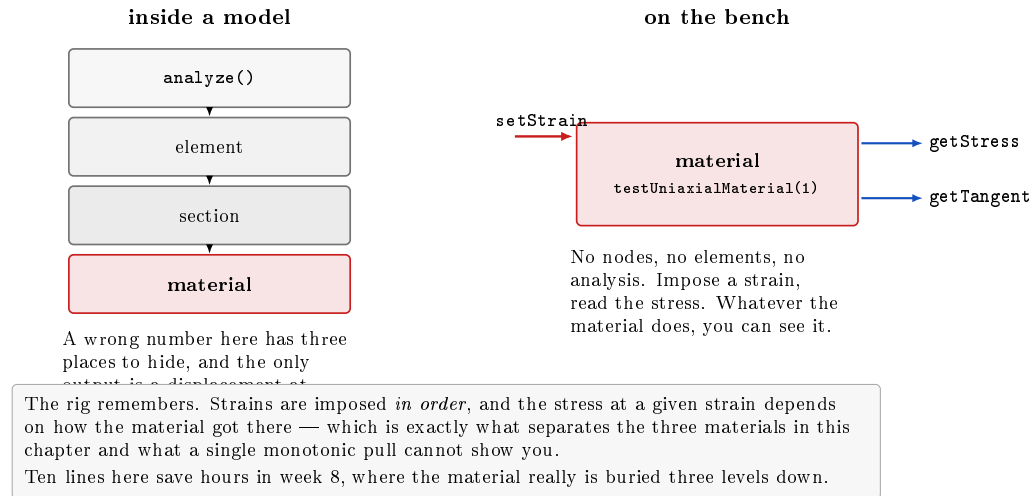


Figure 6.1: Left, the way a material is normally reached: buried inside a section inside an element, with a displacement at the far end as the only output. Right, the same material with a lever attached.

Definition 6.1: Material test rig

`ops.testUniaxialMaterial(tag)` detaches a material from the model. `ops.setStrain(eps)` imposes a strain on it, `ops.getStress()` reads the stress back and `ops.getTangent()` reads the current slope. There are no nodes, no elements and no analysis.

Listing 6.1: `w06/materials.py` — Ten lines. The material to test is passed in as a function, so the same rig tests every material in the chapter.

```

42 def probe(strains, define_material):
43     """Impose a strain path on one material and record what it does.
44
45     `define_material` is a function that creates the material with tag 1. It is
46     passed in rather than hard-coded so that the same rig tests every material
47     in this chapter -- which is the whole point of having a rig.
48
49     The strains are imposed IN ORDER and the material remembers. That is what
50     makes this a test of a material rather than of a formula: the stress at a
51     given strain depends on how the material got there.
52     """
53     ops.wipe()
54     define_material()
55     ops.testUniaxialMaterial(1)
56
57     stress, tangent = [], []
58     for eps in strains:
59         ops.setStrain(eps)
60         stress.append(ops.getStress())
61         tangent.append(ops.getTangent())
62     return stress, tangent

```

Two things about this deserve saying plainly.

The rig *remembers*. Strains are imposed in order, and what the material reports at a given strain depends on how it got there. That is the property being tested; a rig that reset between points would be testing a formula instead.

And it is worth the ten lines because of [Fig. 6.1](#). In week 8 the material will sit inside a fibre section inside a beam element, three levels down, and a wrong answer will have three places to hide. Knowing what the material does *by itself* is what lets you rule it out.

6.2 Three materials

Listing 6.2: w06/materials.py — Three definitions. Note what ElasticPP asks for.

```

67 def elastic(p=STEEL):
68     """A straight line, for ever. Never yields, never fails."""
69     ops.uniaxialMaterial("Elastic", 1, p["E"])
70
71
72 def elastic_pp(p=STEEL):
73     """Elastic-perfectly plastic: a straight line to yield, then flat.
74
75     ElasticPP is given the YIELD STRAIN, not the yield stress -- a place where
76     it is easy to hand OpenSees 420 where it wanted 0.0021 and get a material
77     a thousand times too strong, with no complaint.
78     """
79     ops.uniaxialMaterial("ElasticPP", 1, p["E"], p["eps_y"])
80
81
82 def hardening(p=STEEL):
83     """Bilinear with kinematic hardening: the second slope is b*E.
84
85     Arguments: E, sigmaY, H_iso, H_kin. Setting the isotropic modulus to zero
86     and the kinematic one to b*E/(1-b) gives a second branch of slope b*E,
87     which is how a hardening ratio is usually quoted.
88     """
89     h_kin = p["b"] * p["E"] / (1.0 - p["b"])
90     ops.uniaxialMaterial("Hardening", 1, p["E"], p["fy"], 0.0, h_kin)

```

Note

`ElasticPP(tag, E, fy) → ElasticPP(tag, E, eps_y)`

ElasticPP takes the yield *strain*, not the yield stress. Hand it 420 where it wanted 0.0021 and you get a material that yields at 420 strain — which is to say never, in any range a structure will ever see. Nothing warns you. Exercise 6.1 is that mistake, and the symptom is a model that behaves perfectly elastically no matter how hard it is pushed.

Example 6.1 (What the three materials do). Probe **Elastic**, **ElasticPP** and **Hardening** at four strains, using $E_s = 200\,000$ MPa, $f_y = 420$ MPa and $b = 0.02$.

Solution

Model. No model. The rig of [Definition 6.1](#), applied to each material in turn.

Governing relations. Below yield all three are $\sigma = E_s \varepsilon$. Above it,

$$\sigma = f_y + b E_s (\varepsilon - \varepsilon_y), \quad \varepsilon_y = \frac{f_y}{E_s} = \frac{420}{200000} = 0.002100. \quad (6.1)$$

Setting $b = 0$ in [Eq. \(6.1\)](#) gives **ElasticPP**; setting $b = 1$ recovers **Elastic**.

Stress at four strains, MPa				
material	0.00100	0.00210	0.01000	0.02000
Elastic	200.0	420.0	2000.0	4000.0
ElasticPP	200.0	420.0	420.0	420.0
Hardening	200.0	420.0	451.6	491.6

at $\varepsilon = 0.02$: $\sigma = 4000, 420, 491.6$ MPa

Check. The **Hardening** value follows from [Eq. \(6.1\)](#): $420 + 0.02(200000)(0.020 - 0.0021) = 420 + 71.6 = 491.6$ MPa, which is what the rig reports.

The **Elastic** column is the one to dwell on. 4000 MPa is about six times the tensile strength of any structural steel, and the model produces it without hesitation because nothing in **Elastic** knows that steel fails. The first column is equally instructive for the opposite reason: at 0.1% strain all three agree exactly. A monotonic pull that stops below yield cannot tell them apart, which means a model that never yields is not evidence that its material was chosen well.

6.3 The path matters

Example 6.2 (Loading, then releasing). Pull each material to 1% strain and release the load. What is left?

Solution

Model. The rig again, with the strain path of `load_unload`: up in steps to 0.010, then back down. [Figure 6.2](#) puts that beside the monotonic pull of [Example 6.1](#), and it is the second panel that does the work.

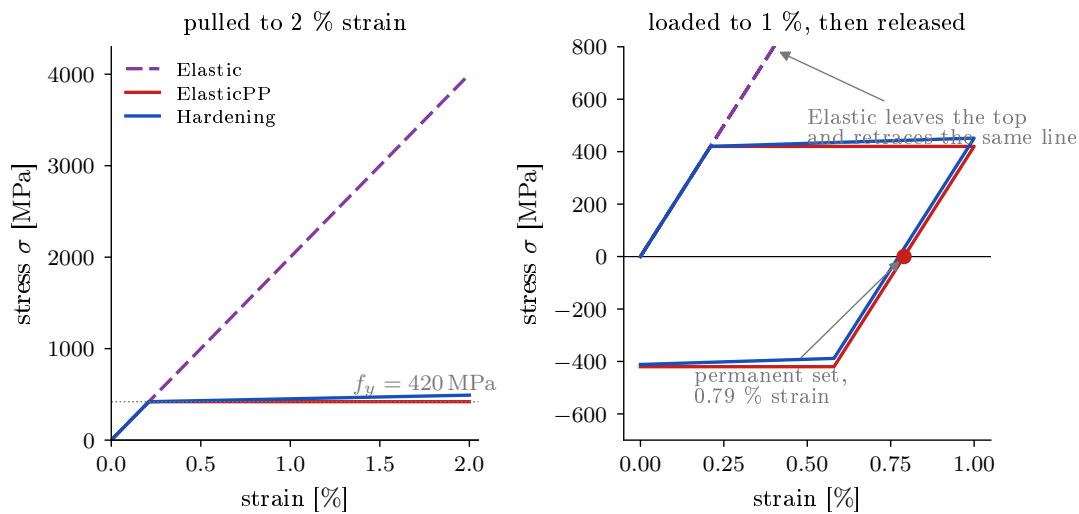


Figure 6.2: Left, a monotonic pull. Right, the same materials loaded to 1% and released — the panel that separates them.

Numbers. Unloading is elastic, with slope E_s , so the recovered strain is σ/E_s and what remains is

$$\varepsilon_{\text{res}} = \varepsilon_{\text{max}} - \frac{\sigma_{\text{max}}}{E_s} = 0.010 - \frac{420}{200000} = 0.007900. \quad (6.2)$$

After loading to 1 % strain and releasing the load		
material	residual strain	as a fraction
Elastic	0.000000	0.000
ElasticPP	0.007900	0.790
Hardening	0.007742	0.774

$$\varepsilon_{\text{res}} = 0.00790 \text{ for ElasticPP, } 0 \text{ for Elastic}$$

Check. Equation (6.2) agrees with the rig to four figures. The **Hardening** value is slightly *smaller*, and the reason is worth a moment: it unloads from 451.6 MPa rather than 420 MPa, so it recovers more elastic strain on the way down. A stronger material keeps less permanent set from the same imposed strain.

And **Elastic** returns to the origin exactly. A bar of it would spring back perfectly however far you stretched it, which is the one thing real steel certainly does not do.

Example 6.3 (The same strain, two different stresses). An **ElasticPP** bar is taken to $\varepsilon = 0.004$ and then unloaded to $\varepsilon = 0.002$. A second bar is loaded directly to $\varepsilon = 0.002$. Do they carry the same stress?

Solution

Model. Two strain paths ending at the same strain.

```
eps= 0.00000 sig= 0.000 MPa
eps= 0.00100 sig= 200.000 MPa
eps= 0.00210 sig= 420.000 MPa <- yields here
eps= 0.00400 sig= 420.000 MPa
eps= 0.00200 sig= 20.000 MPa <- unloaded to this strain
```

```
eps= 0.00000  sig= -380.000 MPa
```

20 MPa by one path, 400 MPa by the other

Why. Once the material has yielded it carries a plastic strain — here $0.004 - 0.0021 = 0.0019$ — that does not go away. Unloading is elastic from wherever it had reached, so at $\varepsilon = 0.002$ the elastic part is only $0.002 - 0.0019 = 0.0001$ and the stress is 20 MPa. The second bar has no plastic strain and reads $200000 \times 0.002 = 400$ MPa.

Check. The last row is the sharpest consequence. Returning the bar to *zero strain* requires squeezing out the 0.0019 of plastic strain elastically, and that costs $0.0019 \times 200000 = 380$ MPa of *compression*. A bar forced back to its original length after yielding in tension is left in compression — which is where residual stresses in welded and cold-formed sections come from.

Note also that unloading to zero *strain* and unloading to zero *stress* are two different questions with two different answers: -380 MPa and Eq. (6.2)'s 0.0079 respectively. Being clear which one you are asking is most of getting this right.

Summary

Item	Command or expression	Note
detach a material	<code>testUniaxialMaterial(tag)</code>	Definition 6.1
impose a strain	<code>setStrain(eps)</code>	the rig remembers
read back	<code>getStress()</code> , <code>getTangent()</code>	stress and slope
linear	<code>uniaxialMaterial('Elastic', t, E)</code>	never yields
elastic-plastic	<code>('ElasticPP', t, E, eps_y)</code>	strain , not stress
bilinear	<code>('Hardening', t, E, fy, 0, Hkin)</code>	$H_{\text{kin}} = bE/(1 - b)$
yield strain	$\varepsilon_y = f_y/E_s$	0.0021 here
hardening branch	$\sigma = f_y + bE_s(\varepsilon - \varepsilon_y)$	Eq. (6.1)
permanent set	$\varepsilon_{\text{res}} = \varepsilon_{\text{max}} - \sigma_{\text{max}}/E_s$	Eq. (6.2)

Exercises

Exercise 6.1. ●●● A student writes `ops.uniaxialMaterial("ElasticPP", 1, 200000.0, 420.0)` and reports that their column never yields however hard it is loaded. Explain, and give the stress their material returns at 1 % strain.

Answer: the fourth argument is the yield strain, not the yield stress, so yielding is set at a strain of 420; the material reads 2000 MPa at 1 % strain, exactly as Elastic would

Solution. ElasticPP is (tag, E, epsyP). Passing 420 sets the yield strain to 420, or 42 000 %, so the material is elastic across every strain any structure will ever reach. At $\varepsilon = 0.01$ it returns $200000 \times 0.01 = 2000$ MPa.

The symptom is the thing to remember: a model that stays perfectly elastic no matter how hard it is pushed. That is not a stiff structure, it is a material that has been told never to yield, and the test rig finds it in two lines.

Exercise 6.2. ●●● At what strain do the three materials of [Example 6.1](#) first give different answers, and what is the largest strain at which a monotonic pull would still fail to distinguish them?

Answer: they separate at the yield strain, $\varepsilon_y = 0.0021$; below it all three give $\sigma = E_s \varepsilon$ exactly

Solution. Below ε_y every one of them is on the same straight line, so at $\varepsilon = 0.0020$ all three read 400 MPa and at $\varepsilon = 0.0022$ they read 440, 420 and 420.4 MPa respectively.

The practical consequence is worth stating as a rule. *A model that never yields does not test its material.* If you run an analysis, find that nothing has gone plastic, and conclude that the material model was a good choice, you have learned nothing — you would have got the same answer from a straight line. The material only earns its place once something reaches ε_y , and until then the only honest verification is the test rig.

Exercise 6.3. ●●● The **Hardening** material in [Example 6.2](#) keeps a *smaller* permanent set than **ElasticPP** from the same imposed strain. Explain why, and say what would happen if b were increased to 0.10.

Answer: it unloads from a higher stress, 451.6 MPa rather than 420, so it recovers more elastic strain; at $b = 0.10$ the peak stress and hence the recovery are larger again, so the permanent set is smaller still

Solution. Permanent set is $\varepsilon_{\max} - \sigma_{\max}/E_s$, and the two materials share $\varepsilon_{\max}$. Only $\sigma_{\max}$ differs. **Hardening** reaches $420 + 0.02(200000)(0.010 - 0.0021) = 451.6$ MPa and recovers 0.002 258 of strain; **ElasticPP** reaches 420 and recovers 0.002 100.

At $b = 0.10$ the peak becomes $420 + 0.10(200000)(0.0079) = 578$ MPa, the recovery 0.002 89, and the permanent set 0.007 11.

The trend has a limit worth noticing: as $b \rightarrow 1$ the material becomes **Elastic** and the permanent set goes to zero. Hardening ratio is, in this sense, a dial between the two extremes of [Example 6.1](#).

Exercise 6.4. ●●● [Example 6.3](#) left a bar in compression after returning it to zero strain. Name a situation in real construction where exactly this happens, and say why it matters.

Answer: welding, and cold forming: material that has yielded locally and is then restrained to its original length by the surrounding steel is left with residual stresses, which add to the applied stresses and can cause a member to yield earlier than its nominal strength suggests

Solution. A weld cools and tries to contract. The surrounding plate does not let it, so the weld region is stretched — often past yield — and when the whole assembly settles at its original geometry the yielded material is left in tension and the surrounding material in compression. Cold-formed corners are the same story with plastic strain imposed mechanically rather than thermally.

Why it matters: these stresses are already present before any load is applied, so the material reaches yield at a lower *applied* stress than a coupon test would suggest. In compression members this is one reason real buckling curves sit below the theoretical Euler curve, and it is why codes distinguish between rolled and welded sections in their buckling formulas.

The modelling point is that none of this appears in an analysis that starts from a stress-free state. It is a limitation of the model, not of the material, and knowing which limitations you have accepted is the difference between a result and a number.

Exercise 6.5. ●●● Explain why the material test rig has no nodes and no elements, and why that is an advantage rather than a simplification.

Answer: a uniaxial material is a relationship between one strain and one stress with no geometry in it at all, so nothing is being left out; the advantage is that a discrepancy found on the rig can only be the material, whereas the same discrepancy in a full model could be the material, the section, the element or the analysis

Solution. A uniaxial material object answers exactly one question: given this strain history, what stress? Length, area and boundary conditions never enter. Attaching it to a one-element model with

unit length and unit area would give the identical numbers, with an analysis wrapped around them — so the rig is not a simplified version of that test, it is the same test with the wrapping removed.

The advantage is diagnostic and it is the reason this chapter opens with it rather than closing with it. In week 8 the same material sits inside a fibre section inside a `forceBeamColumn`, and a moment-curvature curve that looks wrong could be a mis-specified material, a badly discretised section, too few integration points, or an analysis that quietly stopped converging. The rig eliminates the first of those in ten lines, and eliminating possibilities cheaply is most of what debugging is.

Week 7: Material Models II: Inelastic and Hysteretic

Learning outcomes

After this week you should be able to:

1. generate a cyclic loading protocol rather than typing one;
2. define `Steel102` and `Concrete01`, and say what shape each produces;
3. explain the Bauschinger effect and what a square hysteresis loop overpredicts;
4. compute the energy dissipated by a material over a loading history;
5. test a warning you have read rather than repeating it.

Notation. σ stress ε strain f'_c concrete compressive strength ε_{c0} strain at peak stress ε_{cu} crushing strain W dissipated energy

Week 6 pulled materials in one direction and let go once. Under an earthquake a member is pushed and pulled dozens of times, and almost everything that matters — how much energy the structure absorbs, how fast it weakens, whether it survives — is invisible until you do that.

This week adds two materials and one idea. The materials are `Steel102`, which rounds the corner at yield because real steel does, and `Concrete01`, which is shaped nothing like steel at all. The idea is the *loading protocol*: a strain history generated from a rule rather than typed out, which is the week 5 tag formula applied to time instead of space.

7.1 A protocol, not a list

Listing 7.1: `w07/cyclic.py` — Amplitudes in, a strain history out. Each amplitude is repeated before the next one begins.

```

86 def cyclic_protocol(amplitudes, cycles_each=2, points_per_cycle=80):
87     """Build a symmetric strain history from a list of amplitudes.
88
89     Each amplitude is applied `cycles_each` times before moving to the next --
90     which is how a real loading protocol is written, because a material that
91     survives one cycle at an amplitude may not survive three.
92
93     Returned as a plain list of strains, so it can be fed straight to the rig.
94     """
95     path = [0.0]
96     for amp in amplitudes:
97         for _ in range(cycles_each):
98             quarter = points_per_cycle // 4
99             for i in range(1, quarter + 1):          # 0 -> +amp
100                 path.append(amp * i / quarter)
101             for i in range(1, 2 * quarter + 1):      # +amp -> -amp
102                 path.append(amp * (1.0 - 2.0 * i / (2 * quarter)))
103             for i in range(1, quarter + 1):          # -amp -> 0
104                 path.append(-amp * (1.0 - i / quarter))
105     return path

```

Repeating each amplitude is not decoration. A material that survives one cycle at a given amplitude may not survive three, and a protocol with one cycle per amplitude cannot show that. Real test protocols are written this way for exactly that reason.

7.2 Two shapes

Listing 7.2: `w07/cyclic.py` — The rounding parameters are a fitted shape, not physical constants.

```

46 def steel102(p=STEEL):

```

```

47  """Giuffre-Menegotto-Pinto steel: a rounded yield corner.
48
49  Arguments: fy, E0, b, then R0, cR1, cR2 which control how sharp the corner
50  is. R0 = 18 with 0.925 and 0.15 are the values almost everyone uses; they
51  are not physical constants so much as a shape that fits test data.
52
53  Rounding matters twice over. Real steel does not turn a corner - the
54  Bauschinger effect softens it as soon as the load reverses. And a sharp
55  corner is a discontinuity in the tangent, which is exactly what makes a
56  Newton solver overshoot and fail to converge in week 10.
57  """
58  ops.uniaxialMaterial("Steel02", 1, p["fy"], p["E"], p["b"],
59                      18.0, 0.925, 0.15)

```

Definition 7.1: Bauschinger effect

A metal that has yielded in one direction begins to yield in the opposite direction at a *lower* stress than its original yield strength. The stress-strain corner is therefore rounded on every reversal, not square.

Listing 7.3: w07/cyclic.py — Concrete, with no tensile strength at all.

```

64  def concrete01(p=CONCRETE):
65      """Kent-Scott-Park concrete with no tensile strength.
66
67      The arguments are written negative because that is the convention, and
68      every reference you will read insists on it. Test it, though, and you find
69      Concrete01 does not care: it takes the magnitudes and applies the
70      compression sign itself, so positive arguments give an identical material.
71      Exercise 7.1 is that test, and the point of it is that a warning repeated
72      everywhere is still worth checking once yourself.
73
74      What DOES bite is the sign of the strain you impose and the stress you read
75      back. Concrete01 has no tensile strength at all, so interrogate it at a
76      positive strain and it returns zero -- not an error, not a warning, just a
77      material that appears to have vanished.
78      """
79      ops.uniaxialMaterial("Concrete01", 1,
80                          -p["fc"], -p["eps_c0"],
81                          -p["fcu"], -p["eps_cu"])

```

Example 7.1 (The same history, two steels). Apply a protocol of two cycles each at 0.4%, 0.8% and 1.2% strain to ElasticPP and to Steel02. Compare the loops and the energy dissipated.

Solution

Model. The week 6 test rig, driven by `cyclic_protocol`.

Governing relation. The energy a material dissipates per unit volume is the area enclosed by its stress-strain path,

$$W = \oint \sigma \, d\varepsilon, \quad (7.1)$$

computed here as a trapezoidal sum along the imposed history.

Listing 7.4: w07/cyclic.py — Equation (7.1), as ten lines of trapezoid rule.

```

110  def dissipated_energy(strains, stresses):
111      """Area enclosed by the stress-strain loop, per unit volume.
112
113      The trapezoidal integral of stress with respect to strain. Over a closed

```

```

114 cycle this is the energy the material turned into heat and damage instead
115 of giving back -- which is the whole reason a ductile structure survives an
116 earthquake and a brittle one does not.
117
118 Units: MPa x (mm/mm) = N mm per mm^3.
119 """
120 total = 0.0
121 for i in range(1, len(strains)):
122     d_eps = strains[i] - strains[i - 1]
123     total += 0.5 * (stresses[i] + stresses[i - 1]) * d_eps
124 return total

```

Figure 7.1 puts the two loops side by side; the numbers below are read off them.

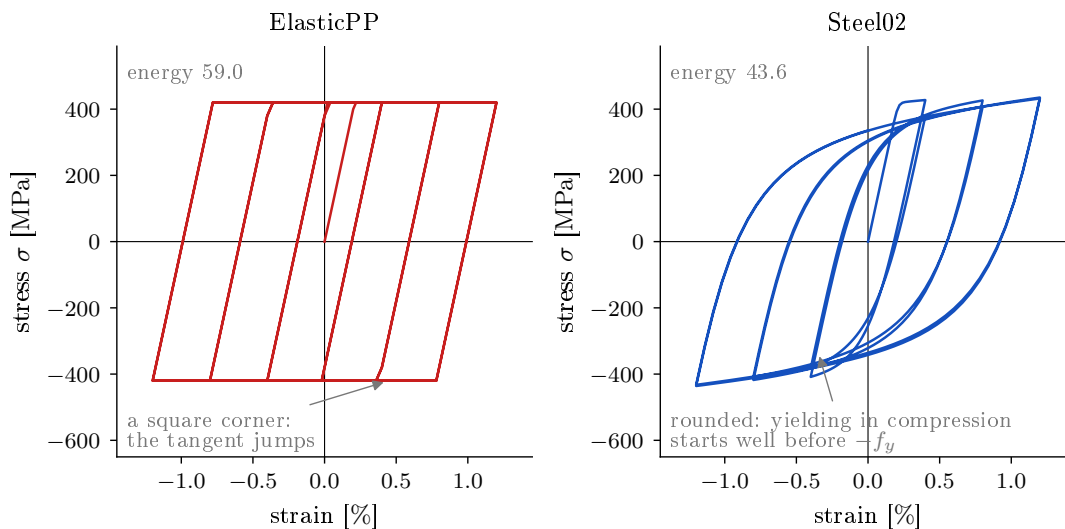


Figure 7.1: The same loading history, two steel models. The rounding is not a cosmetic difference: it takes about a quarter of the area out of every loop.

material	max tension	max compr.	energy
ElasticPP	420.0	-420.0	59.010
Hardening	459.6	-459.6	57.830
Steel02	435.1	-436.0	43.615

$$W_{\text{Steel02}}/W_{\text{ElasticPP}} = 0.739$$

Check. ElasticPP is exactly symmetric, +420 against -420, which is what a material with no hardening and no Bauschinger effect must give — a free check that the protocol really is symmetric.

The energy ratio is the result worth carrying. A square material dissipates about a third more energy than a rounded one under the identical history. Since dissipated energy is what limits how far a structure is driven in an earthquake, choosing ElasticPP for convenience is not a neutral simplification: it makes the structure look better damped than it is.

The rounding matters a second time, and week 10 will collect the debt. A square corner is a discontinuity in the tangent stiffness, and a Newton solver stepping across it overshoots and comes back. Rounded materials converge; square ones fight.

Example 7.2 (Concrete is not steel). Push Concrete01 into compression to 0.5% strain, release it, and then pull.

Solution

Model. The rig, with $f'_c = 30$ MPa at $\varepsilon_{c0} = 0.002$, crushing to 6 MPa at $\varepsilon_{cu} = 0.0035$.

Governing relation. Below the peak, Kent–Scott–Park is a parabola in $r = \varepsilon/\varepsilon_{c0}$:

$$\sigma = -f'_c (2r - r^2), \quad r = \frac{\varepsilon}{\varepsilon_{c0}}. \quad (7.2)$$

Figure 7.2 is the whole response, and it is worth comparing with Fig. 7.1 before reading the numbers: the two materials have almost nothing in common.

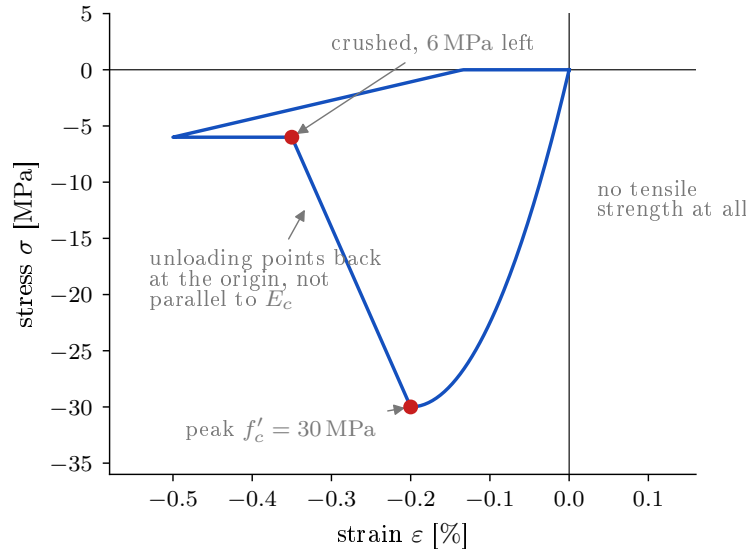


Figure 7.2: Three parts: a parabola to the peak, a straight descent to a residual crushing strength, then a floor. Above zero strain there is nothing.

strain	stress [MPa]
-0.0005	-13.12
-0.0010	-22.50
-0.0020	-30.00
-0.0035	-6.00
-0.0050	-6.00
+0.0020	0.00

$$\sigma_{\max} = -f'_c = -30.0 \text{ MPa at } \varepsilon = -0.002$$

Check. At $\varepsilon = -0.001$, $r = 0.5$ and Eq. (7.2) gives $-30(1.0 - 0.25) = -22.50$ MPa, which is the model's value exactly. The peak equals $-f'_c$ exactly, which is the check that the sign convention came out right.

Three features have consequences later. The material carries *nothing* in tension, so a concrete fibre on the tension side of a section contributes no stiffness at all — week 8 shows what that does to a moment–curvature curve. It retains a residual 6 MPa after crushing rather than dropping to zero, which is a numerical kindness as much as a physical claim: a material that loses all strength makes the stiffness matrix singular. And the unloading branch points back towards the origin rather than running parallel to the initial slope, so crushed concrete is permanently softer and a section that has seen one large cycle is not the section it started as.

Note

ask a concrete material what it does at positive strain → compress it with negative strain

Concrete01 returns zero at any positive strain — no error, no warning, just a material that appears to have vanished. If a concrete fibre seems to be doing nothing, check the sign of the strain before you check anything else.

Note that this is a statement about the *strain you impose*, not about the arguments you defined the material with. [Example 7.2](#) used negative arguments because that is the convention; exercise 7.1 asks you to find out whether the convention is enforced.

Summary

Item	Command or expression	Note
rounded steel	('Steel02', t, fy, E, b, 18, 0.925, 0.15)	Definition 7.1
concrete	('Concrete01', t, -fc, -e0, -fcu, -ecu)	signs are conventional only
protocol	amplitudes → strain history	repeat each amplitude
dissipated energy	$W = \oint \sigma \, d\varepsilon$	Eq. (7.1)
concrete parabola	$\sigma = -f'_c(2r - r^2)$, $r = \varepsilon/\varepsilon_{c0}$	Eq. (7.2)
peak	$-f'_c$ at ε_{c0}	-30 MPa at 0.002
tension	none, at any strain	returns exactly zero

Exercises

Exercise 7.1. ●●● Every reference on **Concrete01** insists its four arguments must be negative. Test whether it matters: define the material twice, once with negative arguments and once with positive, and probe both at $\varepsilon = -0.002$.

Answer: it makes no difference — both return -30 MPa; OpenSees takes the magnitudes and applies the compression sign itself

Solution. Both definitions give an identical material at every strain.

The expectation most people bring to this — and the claim you will find repeated in a great many places — is the opposite: that positive arguments produce a mirrored material carrying its strength in tension. Predict it, then run the two lines above, and the prediction fails.

The lesson is not that documentation is unreliable. It is that a two-line test settles a question that no amount of reading will, and you now have a rig that makes such tests two lines.

Note carefully what is *not* being claimed. The sign convention still matters everywhere else: you must compress concrete with a *negative strain*, and **Concrete02**'s tensile parameters are a separate question that this test says nothing about. Test that one too before relying on it.

Exercise 7.2. ●●● Repeat [Example 7.1](#) with hardening ratios $b = 0, 0.02, 0.10$ and 0.20 . Does more hardening dissipate more energy or less?

Answer: less, and monotonically: 59.0, 57.8, 53.1, 47.2

Solution. The peak stress rises with b — 420, 460, 618, 816 MPa — so the natural guess is that the loops enclose more area. They enclose less.

The reason is that hardening tilts the loop rather than widening it. The vertical width of a kinematic-hardening loop is fixed at $2f_y$; increasing b steepens the plastic branches, so more of each cycle's deformation is recovered elastically and less is dissipated. The limit makes it obvious: at $b = 1$

the material is **Elastic**, it traces a straight line, and it dissipates nothing at all.

This is worth getting the right way round, because the intuition that a stronger material is a better one does not survive contact with energy dissipation. A structure survives an earthquake by absorbing energy, and a material that hardens steeply gives less of that back.

Exercise 7.3. ●●● Apply one cycle at 1.2% strain to **Steel02**, then four cycles at the same amplitude. Is the total energy four times the single-cycle value?

Answer: no — 12.73 for one cycle and 49.90 for four, against 50.90 if they were equal; energy per cycle falls slightly after the first

Solution. The first excursion into the plastic range pays for the Bauschinger softening; the three repeats run on an already-softened loop and each encloses a little less area. Per cycle the values are 12.73, 12.55 and 12.47.

The difference is small here and the direction is what matters. Assuming that n cycles dissipate n times one cycle's energy is unconservative, and in materials with genuine cyclic degradation — concrete, and steel near fracture — the gap is far larger than 2%. It is also the reason a loading protocol repeats each amplitude: one cycle would have hidden the effect completely.

Exercise 7.4. ●●● **Example 7.1** found that **ElasticPP** dissipates about a third more energy than **Steel02**. A colleague argues this is unimportant because both materials have the same yield strength and the same strain history. Answer them.

*Answer: strength and strain history fix what the material can carry, not what it absorbs; dissipated energy is the area of the loop, and the rounded corners remove roughly a quarter of it, so a model built with **ElasticPP** will predict smaller displacements under a real earthquake than the structure will actually experience*

Solution. The colleague is right that the peak forces will be similar — 420 against 435 MPa — and that is precisely why the error is dangerous: a check on maximum force would not reveal it.

What differs is the area, and the area is what limits displacement. A structure driven by ground motion keeps moving until the energy put into it has been absorbed; a material that absorbs more per cycle stops it sooner. Overstate the dissipation and the model predicts a building that comes to rest earlier and less far than the real one — unconservative in exactly the quantity that governs collapse.

The general form of the argument is worth extracting. A simplification that preserves *strength* may still destroy *energy*, and which of those two your answer depends on is a question about the analysis you are doing, not about the material. In the static analyses of weeks 4 and 5 the difference would not have mattered at all.

Exercise 7.5. ●●● **Concrete01** keeps a residual strength of 6 MPa after crushing rather than falling to zero. Give one physical argument and one numerical argument for that choice, and say what would happen if the residual were set to zero.

Answer: physically, confined concrete retains some capacity through aggregate interlock and the confinement of the reinforcement; numerically, a fibre with zero stiffness and zero strength contributes nothing to the stiffness matrix, and enough such fibres make it singular so the analysis fails to converge

Solution. The physical case is real but partial. Confined concrete — the core inside a well-detailed column tie cage — does retain substantial strength after the cover has gone, and models such as **Concrete02** and **Concrete04** exist to represent that better. Unconfined cover concrete genuinely does approach zero, so the residual is more defensible for some fibres than others.

The numerical case is unavoidable. A material that has lost all strength has zero tangent stiffness, and a section made mostly of such fibres has almost none either. The stiffness matrix approaches singularity, the Newton iteration cannot find a direction to move in, and the analysis fails at the step where the answer was becoming interesting — which is the worst possible moment to lose it.

Setting the residual to zero therefore trades a convergence failure for a physical claim, and neither is free. This is the first appearance of a tension that runs through the rest of the course: the material model has to be a description of the material *and* something a solver can work with, and those two requirements do not always point the same way. Week 10 is largely about managing the gap.

Week 8: Sections and Fiber Models

Learning outcomes

After this week you should be able to:

1. build a fibre section from patches and layers, with more than one material in it;
2. explain what a fibre section assumes and what it therefore computes rather than being told;
3. drive a section with `DisplacementControl` and read its moment–curvature response;
4. run a convergence study on the fibre count, and say why week 4’s mesh needed none;
5. read the cover-spalling dip in a moment–curvature curve and say what it argues for.

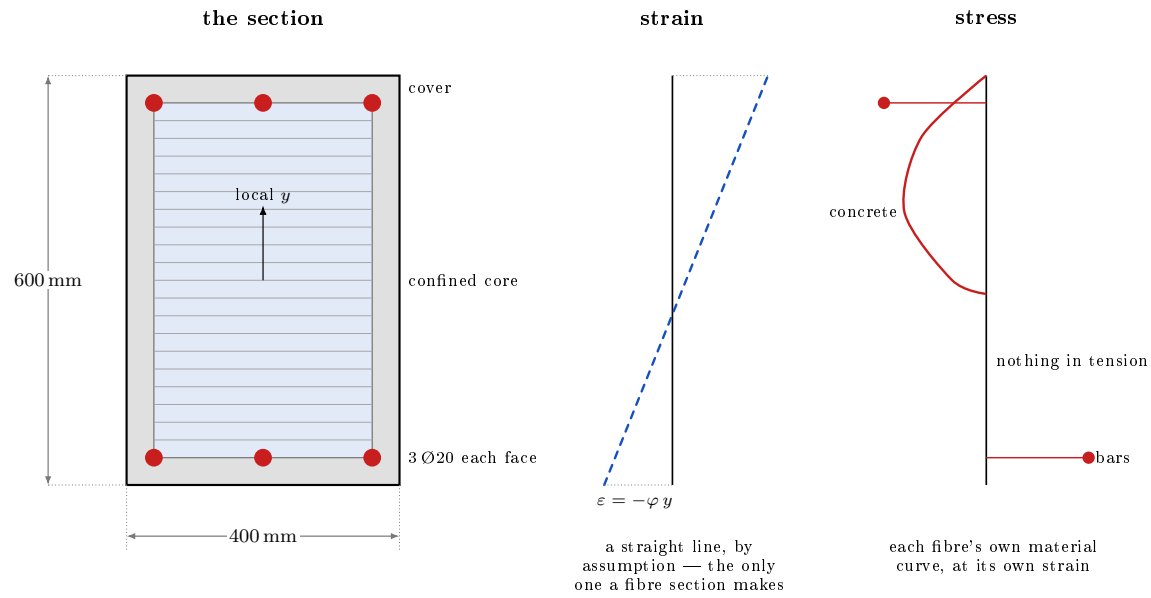
Notation. φ curvature M bending moment ε strain f'_c concrete strength N axial force y distance from the section centroid

This is the last chapter before the midterm, and it is where the previous four converge. Week 4 gave elements, week 5 gave the machinery to generate them, and weeks 6 and 7 gave materials that yield. A **fibre section** (*TR: lifli kesit*) is what happens when you stop describing a cross-section by its properties and start describing it by its contents.

The idea is almost embarrassingly simple. Cut the section into strips, give each one a material and a position, and insist they all obey the same straight-line strain distribution. Everything else — cracked stiffness, the interaction between axial load and moment, where the neutral axis is, what happens after the cover falls off — comes out as a *result* rather than going in as an assumption.

8.1 A section as a list of fibres

Figure 8.1 is the whole of it: a shape divided into fibres, one assumption about how they deform together, and every material left to speak for itself.



The section has no strength, no stiffness and no interaction diagram of its own. It has a list of fibres. Impose a curvature, get every fibre's strain from $\varepsilon = -\varphi y$, ask each material what stress that produces, and sum the moments about the centroid.

Which is why this is a real discretisation: the sum is an approximation to an integral, and with too few fibres it is simply wrong.

Figure 8.1: The section, the one assumption, and the consequence. Strain is a straight line because plane sections are assumed to remain plane; stress is whatever each fibre's own material makes of its own strain.

Definition 8.1: Fibre section

A **fibre section** is a list of fibres, each with an area, a position (y, z) and a uniaxial material. Given a curvature φ and an axial strain, every fibre's strain follows from $\varepsilon = -\varphi y$, its stress from its own material, and the section's axial force and moment from summing over all of them.

That is the whole model. Note what is *not* in it: no cracked second moment of area, no transformed section, no interaction diagram. Those are all consequences, and the section recomputes them at every step because it never knew them in the first place.

Listing 8.1: w08/section.py — Two concretes, one steel, three patches and two layers.

```
62 def define_section(p, n_fibres=20, gj=None):
63     """Build the fibre section: two concretes, one steel, three patches.
64
65     A `patch` fills a rectangle with a grid of fibres of one material. A
66     `layer` puts a row of discrete bars along a line. Coordinates are the
67     section's own local (y, z); for bending about the strong axis only y
68     matters, so the fibres are subdivided in y and left as one strip in z.
69
70     `n_fibres` is the number of concrete fibres across the depth, and it is the
71     parameter the convergence study varies.
72
73     `gj` is torsional stiffness, and it is None here because a 2D section has
74     nowhere to twist. In three dimensions it becomes compulsory -- a fibre
75     section models one axial strain and two curvatures and has no concept of
```

```

76     torsion at all, so OpenSees refuses to build one without being told.
77     Week 11 passes G*J.
78     """
79     ops.uniaxialMaterial("Concrete01", CORE,
80                          -p["fcc"], -p["eps_cc0"], -p["fccu"], -p["eps_ccu"])
81     ops.uniaxialMaterial("Concrete01", COVER,
82                          -p["fc"], -p["eps_c0"], -p["fcu"], -p["eps_cu"])
83     ops.uniaxialMaterial("Steel02", STEEL_MAT,
84                          p["fy"], p["Es"], p["b_steel"], 18.0, 0.925, 0.15)
85
86     y_out, z_out = p["h"] / 2.0, p["b"] / 2.0
87     y_in, z_in = y_out - p["cover"], z_out - p["cover"]
88
89     if gj is None:
90         ops.section("Fiber", SECTION)
91     else:
92         ops.section("Fiber", SECTION, "-GJ", gj)
93     # confined core
94     ops.patch("rect", CORE, n_fibres, 1, -y_in, -z_in, y_in, z_in)
95     # cover: the two faces that matter in bending, top and bottom
96     ops.patch("rect", COVER, 2, 1, y_in, -z_out, y_out, z_out)
97     ops.patch("rect", COVER, 2, 1, -y_out, -z_out, -y_in, z_out)
98     # reinforcement, one layer near each face
99     ops.layer("straight", STEEL_MAT, p["n_bars"], p["bar_area"],
100             y_in, z_in, y_in, -z_in)
101     ops.layer("straight", STEEL_MAT, p["n_bars"], p["bar_area"],
102             -y_in, z_in, -y_in, -z_in)

```

A `patch` fills a rectangle with a grid of fibres of one material; a `layer` puts a row of discrete bars along a line. The two concretes matter: the core inside the ties is **confined** and both stronger and far more ductile than the **cover** outside them, which spalls off and keeps nothing. Modelling them as one material would miss the most interesting thing this section does.

8.2 Driving a section

Listing 8.2: `w08/section.py` — Two nodes at the same point. The element has no length, so its deformations are the section's.

```

107 def build_rig(p, n_fibres=20):
108     """Two coincident nodes joined by a zeroLengthSection element.
109
110     The element has no length, so its deformations ARE the section's: the
111     relative rotation between the two nodes is the curvature, and the moment
112     the element carries is the section's moment. That is why this rig measures
113     a section property rather than a member response.
114     """
115     ops.wipe()
116     ops.model("basic", "-ndm", 2, "-ndf", 3)
117     ops.node(1, 0.0, 0.0)
118     ops.node(2, 0.0, 0.0)
119     ops.fix(1, 1, 1, 1)
120     ops.fix(2, 0, 1, 0)          # free to extend and to rotate
121
122     define_section(p, n_fibres)
123     ops.element("zeroLengthSection", 1, 1, 2, SECTION)

```

This is the week 6 material rig, one level up. There the loop was *impose a strain, read a stress*; here it is *impose a curvature, read a moment*. The rig measures a property of the section, not a response of a member, because a `zeroLengthSection` element has no length for member behaviour to happen in.

Listing 8.3: `w08/section.py` — `DisplacementControl` drives a degree of freedom to a value instead of applying a force.

```

154 def push_curvature(max_curv, n_steps=200):
155     """Impose curvature in steps and record the moment at each one.
156
157     DisplacementControl drives a chosen degree of freedom to a chosen value
158     instead of applying a chosen force. That is the only way to follow a
159     response past its peak: after the section starts to lose strength there is
160     no load that produces the next point, but there is always a curvature.
161
162     Week 10 generalises this to whole structures. Here it does exactly what
163     setStrain did for a material in week 6, one level up.
164     """
165     ops.timeSeries("Linear", 2)
166     ops.pattern("Plain", 2, 2)
167     ops.load(2, 0.0, 0.0, 1.0)          # a unit moment, to be scaled
168
169     d_curv = max_curv / n_steps
170     ops.integrator("DisplacementControl", 2, 3, d_curv)
171     ops.analysis("Static")
172
173     curvature, moment = [0.0], [0.0]
174     for _ in range(n_steps):
175         if ops.analyze(1) != 0:
176             break                    # the section has failed; stop cleanly
177         curvature.append(ops.nodeDisp(2, 3))
178         moment.append(-ops.eleResponse(1, "force")[2])
179     return curvature, moment

```

Definition 8.2: Displacement control

`integrator('DisplacementControl', node, dof, incr)` advances the analysis by imposing a displacement increment on one degree of freedom and solving for the load factor that produces it — the reverse of `LoadControl`.

This is the only way to follow a response past its peak. Once a section starts losing strength there is no load that produces the next point on the curve, but there is always a next curvature. Week 10 generalises this to whole structures; here it does exactly what `setStrain` did for a material.

Note

`timeSeries('Linear')` for the axial load → `timeSeries('Constant')`, then `loadConst`

The axial load must go on first and then stay put while the curvature increases. A `Linear` series would scale it along with everything else, so the section would be bent while its axial load grew — which is a perfectly well-defined analysis, and not the one anybody means by moment–curvature.

Example 8.1 (Moment–curvature of a reinforced concrete column). A 400 mm by 600 mm column, reinforced with three 20 mm bars each face, carries 1200 kN of compression. Find its moment–curvature response.

Solution

Model. The fibre section above, 32 concrete fibres across the depth, on a `zeroLengthSection` rig. Axial load applied and held; curvature imposed in 180 steps to $\varphi = 9 \times 10^{-5}/\text{mm}$.

Governing relations. The section's response is the pair of sums

$$N = \sum_i \sigma_i A_i, \quad M = - \sum_i \sigma_i A_i y_i, \quad \varepsilon_i = \varepsilon_0 - \varphi y_i, \quad (8.1)$$

with σ_i taken from fibre i 's own material at strain ε_i .

Context for the numbers. The axial load is 5.00 MPa over the gross area, or 0.167 of f'_c — a moderately loaded column, comfortably below the balance point. The reinforcement ratio is 0.785 %.

The result is Fig. 8.2; its left panel belongs to Example 8.2 and is discussed there.

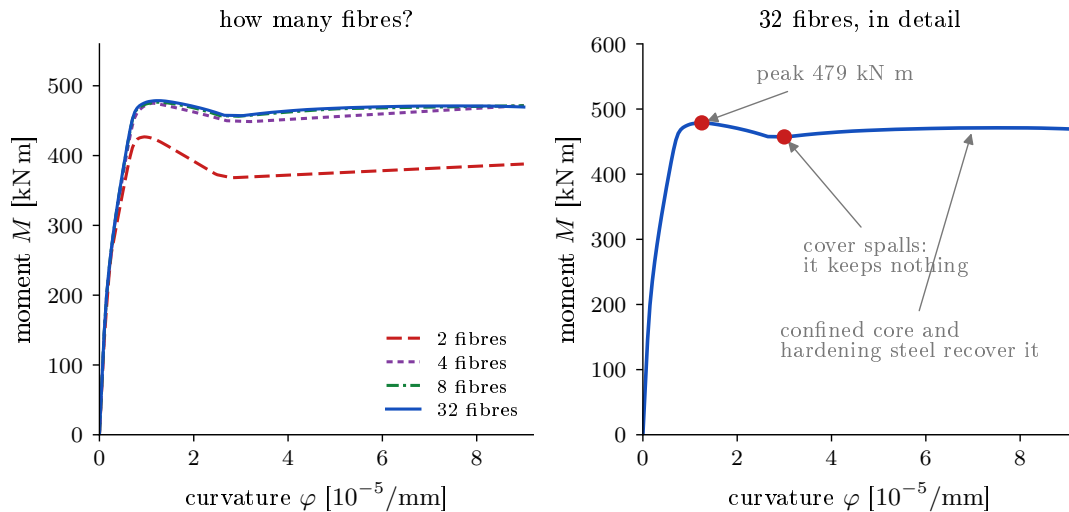


Figure 8.2: Left, the fibre-count study. Right, the converged response, which does something a linear model never would.

```
RC column 400 x 600 mm, axial -1200 kN
peak moment      478.8 kN m
at curvature     1.250e-05 1/mm
95 % curvature   7.500e-06 1/mm
steps completed  180 of 180
```

$$M_{\max} = 479 \text{ kN m at } \varphi = 1.25 \times 10^{-5} / \text{mm}$$

Check. The shape is the check, and it is worth reading carefully. The moment rises steeply while the section is essentially elastic, peaks at 479 kN m, then *falls* by 4.5% to 457 kN m, and then *recovers* to 470 kN m.

That dip is the unconfined cover reaching its crushing strain and losing everything — recall from week 7 that **Concrete01** was given $f_{cu} = 0$ for the cover. What follows it is the confined core and the strain-hardening steel taking up what the cover gave away. Neither the drop nor the recovery was put into the model; both fall out of Eq. (8.1) once the materials are right.

This is also the argument for close ties, in one picture. Without confinement the core would behave like the cover, and the curve would fall away after the peak instead of recovering.

8.3 How many fibres?

Week 4 asked this question about a beam mesh and the answer was that refinement bought nothing. Here the answer is different, and the difference is the point.

Example 8.2 (A convergence study that matters). Compute the peak moment with 2, 4, 8, 16, 32 and 64 fibres across the depth.

Solution

Model. Unchanged except for `n_fibres`.

Convergence with the number of concrete fibres		
fibres	peak moment [kN m]	change
2	426.82	
4	475.07	11.30%
8	476.47	0.29%
16	478.59	0.45%
32	478.76	0.04%
64	478.89	0.03%

2 fibres are 10.9 % low; 16 are within 0.1 %

Why this one is real. Equation (8.1) is a finite sum standing in for an integral over the cross-section. Two fibres cannot represent a stress block that varies from crushing at one face to nothing at the other, so they get the answer badly wrong, and no amount of care in the solver will recover it. Refining genuinely improves the answer, and it stops improving once the sum is fine enough to resolve the stress distribution.

Contrast week 4, where the beam element could reproduce the exact deflected shape and the nodal answer was therefore exact at any mesh. That is the distinction to carry: *a discretisation that can represent the answer needs no refinement; one that cannot needs a convergence study.*

Check. Notice the cost. Sixty-four fibres means sixty-four material evaluations for every iteration of every step, and in week 13 that inner loop will run inside a time-integration loop inside a nonlinear iteration. Sixteen to twenty fibres is the usual working choice, and this study is how you justify it rather than guessing.

Note

the analysis stopped, so the section failed → the analysis stopped, so find out why

Push this section beyond about $\varphi = 1 \times 10^{-4}/\text{mm}$ and the run stops converging. It is tempting to read that as a failure load. It is not, and cutting the step size does not help much either — a factor of eight buys about a quarter more curvature and then stops again.

It would be reasonable to conclude from that that the section has run out of tangent stiffness to iterate on. Reasonable, and wrong: week 10 pushes this same section, unchanged, to $5 \times 10^{-4}/\text{mm}$ by altering nothing but the solution strategy. What stopped here was the analysis, not the column.

That distinction — between an analysis that has finished and one that has given up — is what week 10 is about.

Summary

Item	Command or expression	Note
start a section	<code>section('Fiber', tag)</code>	Definition 8.1
fill a rectangle	<code>patch('rect', mat, nY, nZ, y1, z1, y2, z2)</code>	concrete
a row of bars	<code>layer('straight', mat, n, A, y1, z1, y2, z2)</code>	reinforcement
the section rig	<code>element('zeroLengthSection', t, i, j, sec)</code>	no length: pure section
hold a load	<code>timeSeries('Constant'), loadConst</code>	axial load stays put
impose curvature	<code>integrator('DisplacementControl', n, 3, dk)</code>	Definition 8.2
read the moment	<code>eleResponse(t, 'force')[2]</code>	sign: it is a reaction
fibre strain	$\varepsilon_i = \varepsilon_0 - \varphi y_i$	plane sections
section forces	$N = \sum \sigma_i A_i, M = - \sum \sigma_i A_i y_i$	Eq. (8.1)
fibre count	16–20 across the depth	justify it, do not guess

Exercises

Exercise 8.1. ●●● Recompute the peak moment with no axial load, and with 2400 kN. Does compression help or hurt?

Answer: 255 kN m with none, 479 kN m at 1200 kN, 578 kN m at 2400 kN — over this range compression nearly doubles the moment capacity

Solution. The first 1200 kN multiplies the capacity by 1.88; the second 1200 kN adds only another 21 %.

Compression helps because it closes the crack: with no axial load, everything below the neutral axis is cracked concrete carrying nothing, and only the bars resist. Adding compression pushes the neutral axis down, puts more concrete into service, and raises the lever arm the bars work on.

The diminishing return is the warning. This trend reverses above the balance point, where the concrete crushes before the bars yield and further compression *reduces* the moment capacity while making the failure brittle. The full picture is the N – M interaction diagram, and a fibre section will draw you one by repeating this exercise at a range of axial loads — which is a far better use of an afternoon than looking one up.

Exercise 8.2. ●●● [Example 8.2](#) refined until the peak moment stopped changing. Check the moment at the end of the run as well, at $\varphi = 9 \times 10^{-5}/\text{mm}$, and say whether the two quantities converge at the same rate.

Answer: for this section they do: at 2 fibres the errors are 10.9 % and 17.4 %, at 4 fibres both are under 1 %, and at 16 both are under 0.3 %

Solution. Both converge, monotonically, at broadly similar rates. Two fibres are badly wrong on everything; four are already within 1 % of both.

The natural prediction is that the post-peak branch converges much more slowly, since it depends on a few crushed fibres rather than on the whole section. Make that prediction before running the sweep, and then look at the table: it does not hold here.

The lesson is therefore narrower than the obvious one, and more useful. Convergence is a

property of a *quantity* on a *particular* model, not of a mesh in the abstract. Here two quantities happened to converge together; on a different section, or with a material that loses strength faster, they need not. The only way to know is to check both — which costs one extra column in the table you were printing anyway.

Exercise 8.3. ●● Model the cover with the confined material by mistake, so that the whole section is confined concrete. What happens to the dip after the peak, and why would this error be easy to miss?

Answer: the dip disappears and the curve rises smoothly, because nothing spalls; it is easy to miss because the peak moment barely changes and the response looks better rather than obviously wrong

Solution. The peak is governed by the section as a whole and is almost unaffected. What is lost is the drop at $\varphi \approx 3 \times 10^{-5}/\text{mm}$, because that drop was the cover reaching $\varepsilon_{cu} = 0.005$ and keeping nothing, and confined concrete does not reach its crushing strain until 0.020.

The error is dangerous precisely because the result is *prettier*. A monotonically rising curve with no awkward dip looks like a well-behaved analysis, and a reviewer checking the peak moment against a hand calculation would find nothing wrong. What has quietly disappeared is the loss of the cover, and with it the model's ability to tell you that a column with widely spaced ties behaves quite differently from one with close ties.

The general form: an error that removes a feature is much harder to spot than one that adds a wrong number, because there is nothing to check against.

Exercise 8.4. ●●● The rig uses a `zeroLengthSection` element. Explain why the moment–curvature curve it produces is a property of the *section* and not of any member, and say what extra information would be needed to predict how much a real column rotates.

Answer: the element has no length, so its deformation is the section's curvature directly with no integration along a member; to get a rotation you need the length over which that curvature acts, which means knowing the plastic hinge length

Solution. Curvature is a deformation per unit length. A real member's end rotation is $\int \varphi \, dx$ along it, so a moment–curvature curve on its own cannot give you one — it tells you what happens at a cross-section, not how much of the member is doing it.

The missing quantity is the plastic hinge length L_p : the distance over which the inelastic curvature is assumed to be spread. Once the section yields, curvature localises into a short region near the point of maximum moment, and the rotation is roughly $\varphi_p L_p$. Empirical expressions for L_p exist and they disagree with each other by a factor of two, which tells you how well this part is understood.

This matters immediately in week 11. A `forceBeamColumn` element integrates section response at a few points along its length, and the weight given to the end integration point acts as an implicit hinge length — so the number of integration points is not a numerical detail but a modelling decision about how far the plasticity spreads. A fibre section is exact about the cross-section and silent about the member, and that division of labour is worth being clear on before week 11 relies on it.

Exercise 8.5. ●●● The analysis in [Example 8.1](#) stops converging beyond $\varphi \approx 1 \times 10^{-4}/\text{mm}$, and smaller steps barely help. Give two changes that might get it further, and say what each would cost.

Answer: give the cover a small residual strength instead of zero, which trades a physical claim for numerical stability; or change the solution strategy — a different algorithm, or an adaptive step that retries on failure — which costs nothing physically but is week 10's material

Solution. The first change addresses the cause. A material with zero strength and zero tangent contributes nothing to the stiffness, and a section whose cover fibres have all reached that state is close to singular. Giving the cover a residual of even 0.5 MPa keeps the matrix invertible. The cost is honesty: unconfined cover concrete really does approach zero, and you have changed the model to suit the solver.

The second addresses the symptom, and is the better instinct. `Newton` is not the only algorithm;

ModifiedNewton and **KrylovNewton** fail differently, and a loop that catches a failed step, halves the increment and retries will walk past a difficulty that a fixed step cannot. Nothing about the model changes, so nothing about the answer is compromised.

Knowing which of the two you are doing is the whole point. Changing the model to make the analysis run is sometimes right and always a decision to be recorded; changing the solver is free. Reach for the second first, and week 10 is about how.

Week 10: Nonlinear Solution Strategies

Learning outcomes

After this week you should be able to:

1. separate the three things that can make a step fail — algorithm, convergence test, step size — and diagnose which one did;
2. write an analysis loop that falls back through algorithms and subdivides on failure;
3. choose a convergence tolerance, and say what is wrong with tolerances on either side of it;
4. explain why a completed analysis can be badly wrong and a failed one nearly right;
5. state the strain limits of a model and check a result against them.

Notation. φ curvature M moment ε strain ε_{cu} crushing strain Δu displacement increment in one iteration

Week 8 ended owing you an explanation. The moment–curvature analysis of the reinforced concrete column stopped converging at about $\varphi = 1 \times 10^{-4}/\text{mm}$, smaller steps barely helped, and the chapter suggested this was not the column failing. It was not.

This week collects that debt, and then takes something back. The first half shows how to make the analysis finish: about thirty lines that take the same model from 3 % of the target to 100 %. The second half shows why finishing was the easy part, and why the analysis that stopped was better company than the one that succeeded.

10.1 Three separable things



The ladder is cheap because it is only paid for when it is needed: on week 8's column the first rung carries 199 steps out of 200 and the second rescues the one that fails. It is also not a substitute for choosing the tolerance. A loose **test** does not fail — it converges, to the wrong answer, and no amount of falling back will notice.

Figure 10.1: A failed step is a failure of one of three things, and what to do about it depends on which. The ladder on the right is what one step does when it refuses.

Definition 10.1: Convergence test

The `test` decides when an iteration has got close enough to equilibrium to stop. `NormDispIncr(tol,  $\rightarrow$  iter)` stops when the norm of the displacement increment falls below `tol`; `NormUnbalance` tests the residual force instead, and `EnergyIncr` tests their product.

The three things in Fig. 10.1 fail differently and are fixed differently, so the one piece of advice worth more than any other is: *change one at a time*. An analysis that starts working after you have simultaneously switched algorithm, loosened the tolerance and halved the step has taught you nothing, and you will not know which change to keep.

10.2 A step that refuses to give up

Listing 10.1: `w10/robust.py` — Four algorithms, tried in order. The point is not that any one is better — it is that they fail differently.

```

35 # Tried in order. Newton is fastest when it works; the others fail differently,
36 # which is the only reason to have more than one.
37 ALGORITHMS = (
38     ("Newton", ("Newton",)),
39     ("ModifiedNewton", ("ModifiedNewton",)),
40     ("KrylovNewton", ("KrylovNewton",)),
41     ("NewtonLineSearch", ("NewtonLineSearch", "-type", "Bisection")),
42 )

```

Listing 10.2: `w10/robust.py` — One attempt. Note that everything is re-declared every time.

```

77 def try_step(node, dof, increment, algorithm, tol=1e-8, max_iter=15):
78     """One attempt at one step, with one algorithm. Returns True if it worked.
79
80     Note that everything is re-declared on every attempt. OpenSees keeps the
81     last algorithm and integrator you set, so a fallback that forgets to reset
82     them silently carries the previous choice into the next step.
83     """
84     ops.test("NormDispIncr", tol, max_iter)
85     ops.algorithm(*algorithm)
86     ops.integrator("DisplacementControl", node, dof, increment)
87     ops.analysis("Static")
88     return ops.analyze(1) == 0

```

Re-declaring the analysis objects on every attempt looks wasteful and is not. OpenSees remembers the last algorithm and integrator you set, so a fallback that changes the algorithm and then forgets to change it back will silently carry that choice into every subsequent step.

Listing 10.3: `w10/robust.py` — The ladder: every algorithm, then halve and recurse.

```

93 def robust_step(node, dof, increment, effort, tol=1e-8, depth=0, max_depth=5):
94     """Advance one step, whatever it takes.
95
96     The strategy, in order:
97     1. try each algorithm in turn at the requested increment;
98     2. if all of them fail, halve the increment and take two half-steps;
99     3. give up after max_depth halvings -- 1/32 of the original step.
100
101     Step (2) is recursive, so a single difficult step can quietly become
102     thirty-two tiny ones while the rest of the analysis marches on at full
103     size. That is exactly the behaviour you want, and exactly why the effort
104     counter exists.
105     """
106     for name, algorithm in ALGORITHMS:

```

```

107     ok = try_step(node, dof, increment, algorithm, tol=tol)
108     effort.record(name, ok)
109     if ok:
110         return True
111
112     if depth >= max_depth:
113         return False
114
115     effort.subdivisions += 1
116     half = increment / 2.0
117     return (robust_step(node, dof, half, effort, tol, depth + 1, max_depth)
118           and robust_step(node, dof, half, effort, tol, depth + 1, max_depth))

```

The recursion is what makes this practical. A single difficult step quietly becomes two, four or thirty-two smaller ones while the rest of the analysis marches on at full size, and the cost is paid only where it is needed.

Example 10.1 (The same model, two strategies). Push the week 8 column to $\varphi = 5 \times 10^{-4}/\text{mm}$, first with plain Newton and then with the fallback ladder. Change nothing else.

Solution

Model. Identical to [Example 8.1](#) — same section, same fibres, same axial load. Only the loop around `ops.analyze` differs.

strategy	reached	of target	effort
Newton only	1.500e-05	3%	7 attempts, 1 failed [Newton 6]
robust_push	5.000e-04	100%	202 attempts, 2 failed [Newton 199, KrylovNewton 1]

3% → 100% for two extra attempts in two hundred

Check. The effort counter is the check, and it is the reason to keep one. Newton carried 199 of the 200 steps; `KrylovNewton` rescued the one that failed. The ladder is almost free precisely because it is only paid for on the rungs it uses, and a script that silently subdivided without counting could have taken a hundred times longer without ever saying so.

Nothing about the column changed. The section, the materials, the axial load and the fibre count are all exactly as they were in week 8.

10.3 And now the bad news

That is a satisfying result, and taking it at face value would be the most expensive mistake in this course.

Example 10.2 (How much of that run is worth having?). The analysis reached $\varphi = 5 \times 10^{-4}/\text{mm}$. What are the material strains there?

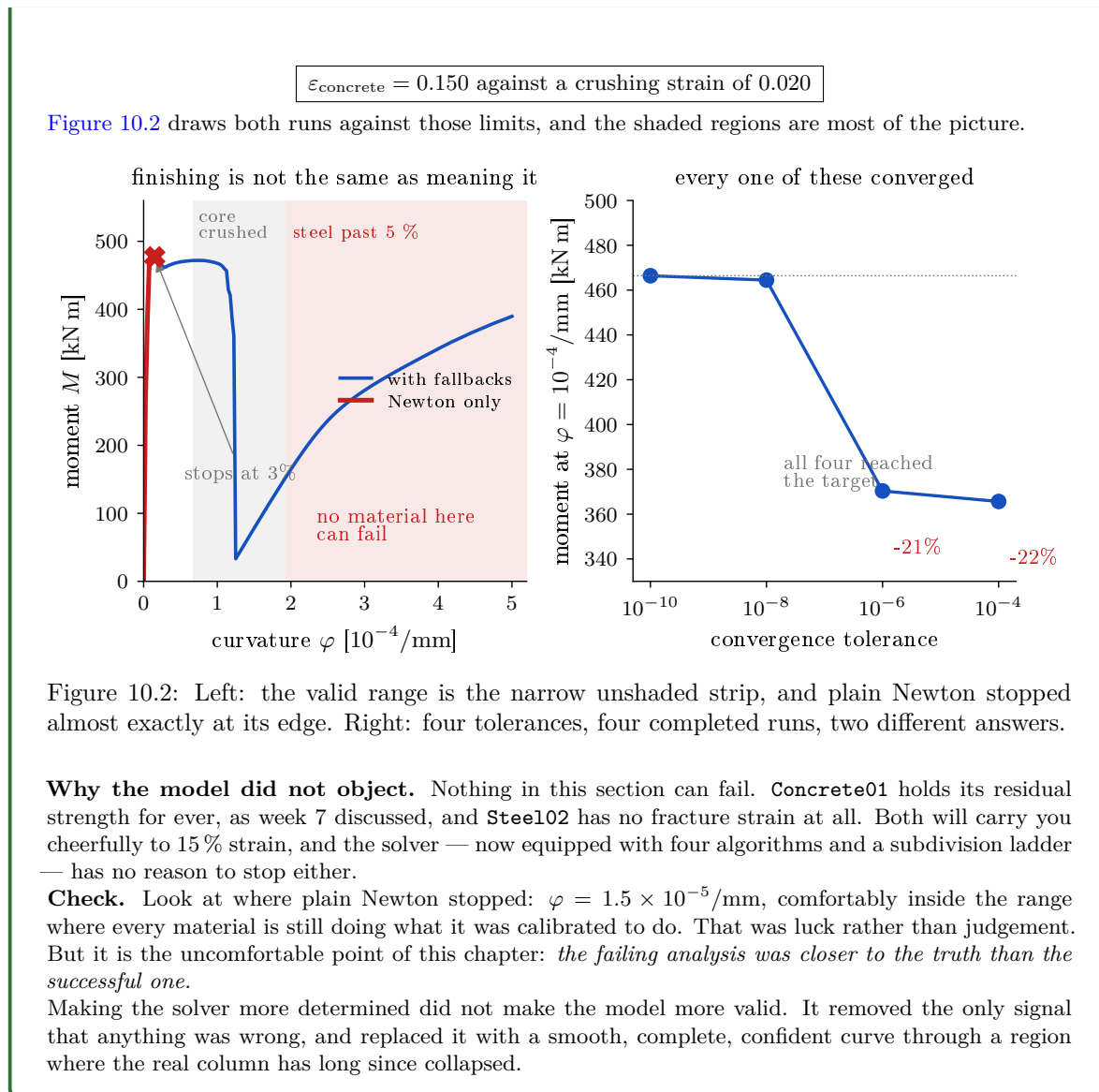
Solution

Model. No analysis needed. Plane sections make strain a distance times a curvature, so both extreme strains follow from [Eq. \(8.1\)](#) arithmetic alone:

$$\varepsilon_{\text{concrete}} = \varphi \frac{h}{2}, \quad \varepsilon_{\text{steel}} = \varphi \left(\frac{h}{2} - c \right). \quad (10.1)$$

But how much of that is worth having?

confined core crushed	6.667e-05	(13% of the target)
steel past 5 % strain	1.923e-04	(38% of the target)
at the end of the run	concrete 0.150, steel 0.130	



Note

the analysis finished, so the result is usable → the analysis finished, so now check the strains

A robust loop converts a visible failure into an invisible one. That is a good trade only if you supply the check that the solver used to supply by accident. Equation (10.1) costs two multiplications and needs no analysis — print it beside every nonlinear result you produce.

10.4 Choosing a tolerance

Example 10.3 (A tolerance that is tighter and worse). Repeat Example 10.1 with the convergence tolerance tightened from 10^{-8} to 10^{-9} .

Solution

The same comparison with the tolerance tightened to 1e-9

Newton only	3.625e-05	15%	30 attempts, 1 failed
robust_push	2.500e-04	100%	270 attempts, 70 failed

tighter tolerance, 15 % instead of 100 %

Why. A convergence test asks whether the iteration has stopped moving, not whether the answer is right. Tighten it far enough and you are asking the iteration to resolve differences smaller than the arithmetic can reliably produce, so steps that were converging perfectly well are refused. *Tighter is not safer.* It is a different failure mode.

Check. The ladder still finishes, but the effort tells the real story: 70 failures out of 270 attempts, against 2 out of 202 at the looser tolerance. Robustness is cheap when it is not needed and expensive when it is — which is the right way round, and worth measuring rather than assuming.

Example 10.4 (Four completed analyses, two answers). Run to $\varphi = 1 \times 10^{-4}/\text{mm}$ at tolerances of 10^{-4} , 10^{-6} , 10^{-8} and 10^{-10} . Compare the moments.

Solution

Convergence tolerance, plain Newton to 1e-4

tolerance	reached	moment [kN m]
1e-04	1.000e-04	365.65
1e-06	1.000e-04	370.35
1e-08	1.000e-04	464.47
1e-10	1.000e-04	466.41

365.7 kN m against 466.4 kN m : 22 %

Why this is the worst case in the chapter. Every one of these runs reached the target. Not one failed, complained, warned or stopped early. Each returned a smooth, complete, plausible moment-curvature curve, and two of them are wrong by more than a fifth.

A loose test does not fail to converge — it converges, to the solution of a slightly different problem, and that slightly wrong state becomes the starting point of the next step. The errors compound quietly along the whole analysis.

Check. The useful pattern is in the last two rows: tightening from 10^{-8} to 10^{-10} changes the answer by 0.4 %. That is how you find the sensible range — tighten until the answer stops moving, then stop, and note that this is a *convergence study on the tolerance*, exactly like the fibre-count study in week 8. The fallback ladder cannot help here, because nothing failed.

When an analysis stops

1. Read the message. **failed to converge** after n iterations is a different problem from a singular matrix.
2. Change **one** thing. Algorithm first — it is free and changes no physics.
3. Then the step size. If halving helps a lot, it was a step-size problem; if it buys a few percent, it was not.
4. Then the tolerance, in *both* directions. Tighter is not automatically safer.
5. If none of that works, the model may be right and the structure may have failed — check the strains against Eq. (10.1) before you decide.
6. And when it does finish: check the strains anyway.

Summary

Item	Command or expression	Note
algorithm	<code>algorithm('Newton')</code>	fastest when it works
	<code>('ModifiedNewton'), ('KrylovNewton')</code>	fail differently
	<code>('NewtonLineSearch', '-type', 'Bisection')</code>	last resort
convergence	<code>test('NormDispIncr', tol, iter)</code>	Definition 10.1
	<code>('NormUnbalance'), ('EnergyIncr')</code>	residual, or both
step control	<code>integrator('DisplacementControl', ...)</code>	halve and recurse on failure
return value	<code>analyze(1) == 0</code>	test it every time
working tolerance	10^{-8} for this problem	found, not assumed
strain check	$\varepsilon = \varphi h/2, \varphi(h/2 - c)$	Eq. (10.1)
effort	attempts, failures, subdivisions	count them, or robustness hides

Exercises

Exercise 10.1. ●●● [Example 10.1](#) cost 202 attempts for 200 steps; [Example 10.3](#) cost 270. What fraction of the effort was wasted in each, and what does the comparison tell you about when to reach for a robust loop?

Answer: 1 % against 26 %; the ladder costs almost nothing when the analysis was going to work anyway, so there is no reason not to have it — but a large wasted fraction is a signal that something else is wrong and should be investigated rather than absorbed

Solution. At $\text{tol} = 10^{-8}$: 2 failures in 202, or 1 %. At 10^{-9} : 70 in 270, or 26 %.

The first number is the argument for always using the ladder. It is not free, but at 1 % the cost is far below the cost of an analysis that stops overnight.

The second is the argument for watching the counter. A quarter of the work being thrown away is not a solver that is working hard, it is a diagnosis: something about the tolerance, the model or the step size is wrong, and the ladder is compensating for it rather than fixing it. Robustness that hides a problem is worse than a failure that exposes one — which is the same lesson as [Example 10.2](#), in a cheaper form.

Exercise 10.2. ●●● At what curvature does the outermost bar in the week 8 column reach 2 % strain? Use [Eq. \(10.1\)](#), not an analysis.

Answer: $\varphi = 7.69 \times 10^{-5}/\text{mm}$

Solution. The bar sits at $h/2 - c = 300 - 40 = 260$ mm from the centroid, so $\varphi = 0.02/260 = 7.69 \times 10^{-5}/\text{mm}$.

Two things are worth noticing. It took no analysis at all — plane sections make this arithmetic, and that is what makes it a check rather than a second opinion. And it is smaller than the $9 \times 10^{-5}/\text{mm}$ that week 8's chapter ran to, which means even that carefully limited analysis had its bars past 2 % strain at the end.

The habit to take away: work out the curvature at which each material leaves its calibrated range *before* running anything, and put those numbers on the plot.

Exercise 10.3. ●●● A colleague's nonlinear analysis fails at step 47 of 200. They immediately switch to `ModifiedNewton`, loosen the tolerance from 10^{-8} to 10^{-6} , and halve the step size. It now runs to completion and they report the result. Give two reasons to be unhappy.

Answer: they changed three things at once, so they do not know which mattered or whether the other two made things worse; and loosening the tolerance is the one change of the three that can silently alter the answer, as Example 10.4 shows

Solution. Changing three things simultaneously is a failure of method. If the analysis now works, they cannot say whether it was the algorithm — which costs nothing and changes no physics — or the tolerance, which changes the answer. Next time it fails they will have to start from scratch.

The second reason is specific and serious. Of the three changes, two are free: a different algorithm and a smaller step both converge on the same solution, only more slowly. Loosening the tolerance does not. Example 10.4 produced four complete, smooth, plausible runs that disagreed by 22%, and the loose ones gave no indication whatever that anything was wrong.

The right order is in the recipe: algorithm, then step size, then tolerance, one at a time, and tolerance last because it is the only one that can quietly change what you are computing.

Exercise 10.4. ●●● Example 10.2 argued that a failing analysis was better company than a successful one. Under what circumstances is that argument wrong, and what should you build instead of relying on it?

Answer: it is wrong whenever the analysis fails for a reason unrelated to the physics — a badly conditioned matrix, an unlucky step, a tolerance set too tight — which is most of the time; relying on failure as a validity signal means accepting a test that fires both when the model has stopped being valid and when it has not, so the thing to build is an explicit strain check that fires only for the first reason

Solution. The coincidence in Example 10.2 was exactly that: a coincidence. Example 10.3 shows the same model failing at 15% of the target purely because the tolerance was tightened — nothing physical changed, and the stopping point moved by a factor of seven. A signal that responds to the tolerance is not a signal about the structure.

So treating convergence failure as a failure criterion gives you a test with both kinds of error: it fires when nothing is wrong (Example 10.3) and stays silent when everything is (Example 10.1 beyond 13% of the run). No amount of care with the solver improves either, because the solver is not being asked about the material.

What to build instead is the two lines of Eq. (10.1), evaluated at every step and reported. It costs nothing, it fires for exactly one reason, and it keeps working when the analysis is behaving perfectly — which is when you most need it and least expect to. In week 13 the same check applies to a time-history analysis of a laboratory specimen, where 'it ran to the end' will be even more tempting to believe.

Exercise 10.5. ●●● The fallback ladder tries algorithms before subdividing. Argue for the opposite order, and then say which you would actually use.

Answer: subdividing first is more likely to succeed and never changes the answer, whereas a different algorithm may converge to a slightly different state; but subdividing is far more expensive, and since all four algorithms converge to the same equilibrium when they converge at all, trying them first is cheaper and equally safe

Solution. The case for subdividing first is that it is the more conservative change. A smaller step starts nearer the answer, so it is more likely to converge, and it cannot converge to anything the full step would not have. Algorithms differ in how they iterate, and a converged step is only as correct as the tolerance allows, so in principle two algorithms can leave the model in slightly different states.

The case for algorithms first is arithmetic. Halving a step doubles the work for that step, and the recursion in `robust_step` can multiply it by thirty-two; trying another algorithm costs one extra set of iterations. Since all four solve the same equations to the same tolerance, the difference between

their converged states is bounded by that tolerance — which is a quantity you have already had to choose carefully for other reasons.

I would use the order in the listing, and I would watch the counter. If the ladder starts reaching the subdivision rung regularly, the honest response is not to admire its persistence but to ask [Example 10.1](#)'s question: what fraction of the effort is being wasted, and what is that telling me?

Week 11: Nonlinear Static Analysis of 3D Structures

Learning outcomes

After this week you should be able to:

1. build a three-dimensional frame, including the two things 3D demands that 2D does not;
2. stage an analysis: gravity first, held constant, then lateral;
3. run a pushover and produce a capacity curve, including its descending branch;
4. read a mechanism from a drift profile, and say why the capacity curve cannot show you one;
5. report how much of a capacity curve lies inside the range its materials describe.

Notation. V_b base shear Δ_{roof} roof displacement θ storey drift ratio GJ torsional stiffness φ curvature

Everything arrives here at once. Week 5 generated a frame from its dimensions, week 8 built a fibre section and drove it with displacement control, and week 10 wrote a step that refuses to give up. A **pushover** (*TR: itme analizi*) is those three pointed at the same model: hold the gravity load constant, push the structure sideways further and further, and record what it takes.

Three dimensions add two obligations. Both of them fail loudly rather than quietly, which after week 10 you will recognise as a mercy.

11.1 What three dimensions demand

With `ndf=6` every node has three rotations as well as three translations. Two consequences follow immediately.

A `geomTransf` now needs a *vector*. In 2D there was only one way for a member to be oriented; in 3D the element must be told which way its local axes point, and the vector supplied must not be parallel to the member itself. Get it wrong and OpenSees says so at once.

Listing 11.1: `w11/pushover.py` — One extra argument, and the reason for it.

```

77 def define_column_section(p):
78     """The week 8 fibre section, made usable in three dimensions.
79
80     Exactly the same call as week 8, with one extra argument. A fibre section
81     models one axial strain and two curvatures; it has no concept of twist, so
82     its torsional stiffness is precisely zero. A 3D frame assembled from such
83     sections has a singular stiffness matrix.
84
85     OpenSees will not let you find that out the hard way. Omit -GJ in three
86     dimensions and the section is refused outright:
87
88         WARNING torsion not specified for FiberSection
89         Use either -GJ $GJ or -torsion $matTag
90
91     which is the most useful error message in this course, because it names
92     both the problem and the two ways out of it.
93
94     The elastic GJ is a patch rather than a model: the torsion stays linear
95     for ever and never interacts with the bending. For a building frame that
96     is entirely reasonable, and it is worth knowing that it is a choice.
97     """
98     define_section(COLUMN, p["n_fibres"], gj=p["G"] * p["J_col"])
```

Note

a fibre section works the same in 3D → a fibre section has *no* torsional stiffness

A fibre section models one axial strain and two curvatures. Twist is not among them, so its torsional stiffness is exactly zero and a 3D frame built from bare fibre sections has a singular stiffness matrix.

OpenSees refuses to build one rather than letting you discover this:

```
WARNING torsion not specified for FiberSection
Use either -GJ $GJ or -torsion $matTag
```

which is the most useful error message in this course, because it names both the problem and the two ways out. Supplying an elastic GJ is a patch rather than a model — the torsion stays linear for ever and never interacts with the bending — and for a building frame that is reasonable. It is still a choice.

Listing 11.2: `w11/pushover.py` — Sixteen nodes, twenty-four elements, generated from four numbers.

```
103 def build_model(p):
104     """A one-bay by one-bay, n-storey reinforced concrete frame.
105
106     The beams are elasticBeamColumn and stay elastic, so all the yielding
107     happens in the columns. That is not how a frame should be designed --
108     strong-column weak-beam is the whole point of capacity design -- and it is
109     exactly why this model is worth running: it shows what the mechanism looks
110     like when the rule is broken. Exercise 11.3 puts it right.
111     """
112     ops.wipe()
113     ops.model("basic", "-ndm", 3, "-ndf", 6)
114
115     lines = ((0, 0), (1, 0), (0, 1), (1, 1))
116     for storey in range(p["n_storeys"] + 1):
117         for ix, iy in lines:
118             ops.node(node_tag(ix, iy, storey),
119                     ix * p["bay_x"], iy * p["bay_y"], storey * p["storey"])
120
121     for ix, iy in lines:
122         ops.fix(node_tag(ix, iy, 0), 1, 1, 1, 1, 1, 1)
123
124     # A transformation needs a vector that is NOT parallel to the member.
125     # For a vertical column that means anything horizontal; for a horizontal
126     # beam, anything vertical. Getting this wrong is an immediate error rather
127     # than a wrong answer, which is the one kindness 3D offers.
128     ops.geomTransf("PDelta", COL_TRANSF, 1.0, 0.0, 0.0)
129     ops.geomTransf("Linear", BEAM_X_TRANSF, 0.0, 0.0, 1.0)
130     ops.geomTransf("Linear", BEAM_Y_TRANSF, 0.0, 0.0, 1.0)
131
132     define_column_section(p)
133
134     # How many points along the member the section response is sampled at, and
135     # where they sit. Five Lobatto points put one at each end, which is where
136     # the moment peaks and therefore where the plasticity will be -- and the
137     # weight given to that end point acts as an implicit plastic hinge length.
138     # This is a modelling decision disguised as a number, as week 8's last
139     # exercise warned.
140     ops.beamIntegration("Lobatto", COL_INTEG, FIBRE_SEC, 5)
141
142     for storey in range(1, p["n_storeys"] + 1):
143         for ix, iy in lines:
144             ops.element("forceBeamColumn", column_tag(ix, iy, storey),
145                     node_tag(ix, iy, storey - 1), node_tag(ix, iy, storey),
146                     COL_TRANSF, COL_INTEG)
147
148     # Elastic beams, both directions.
149     for storey in range(1, p["n_storeys"] + 1):
150         for iy in (0, 1):
151             ops.element("elasticBeamColumn", beam_tag(0, 0, iy, storey),
152                     node_tag(0, iy, storey), node_tag(1, iy, storey),
153                     p["A_beam"], p["E"], p["G"], p["J_col"],
154                     p["I_beam"], p["I_beam"], BEAM_X_TRANSF)
155         for ix in (0, 1):
156             ops.element("elasticBeamColumn", beam_tag(1, ix, 0, storey),
```

```

157         node_tag(ix, 0, storey), node_tag(ix, 1, storey),
158         p["A_beam"], p["E"], p["G"], p["J_col"],
159         p["I_beam"], p["I_beam"], BEAM_Y_TRANSF)

```

The columns are `forceBeamColumn` elements carrying week 8's fibre section; the beams are elastic. That is capacity design exactly backwards, and it is deliberate — [Example 11.2](#) is about what the model does as a result.

Definition 11.1: Beam integration

A `forceBeamColumn` samples its section response at a few points along its length. `beamIntegration('Lobatto', tag, sec, n)` places n points, two of them at the member ends where the moment peaks. The weight given to the end point acts as an implicit plastic hinge length.

That last sentence is the one week 8's final exercise promised. The number of integration points is not a numerical detail; it is a modelling decision about how far the plasticity is allowed to spread.

11.2 Staging the analysis

Listing 11.3: `w11/pushover.py` — Gravity first, then frozen.

```

164 def apply_gravity(p):
165     """Put the building's own weight on, then freeze it.
166
167     loadConst('-time', 0.0) is the important line. It tells OpenSees to keep
168     the gravity loads at their current value and reset pseudo-time to zero, so
169     the pushover that follows starts from a loaded, already-deformed structure
170     rather than from an unloaded one. Forget it and you push a weightless
171     frame.
172     """
173     ops.timeSeries("Constant", 1)
174     ops.pattern("Plain", 1, 1)
175     for storey in range(1, p["n_storeys"] + 1):
176         for ix, iy in ((0, 0), (1, 0), (0, 1), (1, 1)):
177             ops.load(node_tag(ix, iy, storey),
178                     0.0, 0.0, -p["mass_per_floor"], 0.0, 0.0, 0.0)
179
180     ops.constraints("Transformation")
181     ops.numberer("RCM")
182     ops.system("UmfPack")
183     ops.test("NormDispIncr", 1.0e-8, 20)
184     ops.algorithm("Newton")
185     ops.integrator("LoadControl", 0.1)
186     ops.analysis("Static")
187     if ops.analyze(10) != 0:
188         raise RuntimeError("the gravity analysis did not converge")
189     ops.loadConst("-time", 0.0)

```

`loadConst("-time", 0.0)` is the line that matters. It holds the gravity loads at their current value and resets pseudo-time, so the pushover starts from a structure that is already loaded and already slightly deformed. Omit it and you push a weightless building — which converges beautifully and answers a question nobody asked.

Listing 11.4: `w11/pushover.py` — The load pattern. Its shape is most of the question.

```

194 def apply_lateral_pattern(p):
195     """An inverted triangular load pattern, growing with height.
196
197     The shape is a stand-in for the first mode. Its absolute size does not
198     matter -- displacement control scales whatever pattern it is given -- but
199     its SHAPE decides which storey is pushed hardest, and therefore where the
200     mechanism forms. A pushover answers the question you asked it, and the

```

```

201 pattern is most of the question.
202 """
203 ops.timeSeries("Linear", 2)
204 ops.pattern("Plain", 2, 2)
205 total = sum(range(1, p["n_storeys"] + 1))
206 for storey in range(1, p["n_storeys"] + 1):
207     share = storey / total
208     for ix, iy in ((0, 0), (1, 0), (0, 1), (1, 1)):
209         ops.load(node_tag(ix, iy, storey),
210                 share / 4.0, 0.0, 0.0, 0.0, 0.0, 0.0)

```

Listing 11.5: w11/pushover.py — Week 10's robust_step, unchanged.

```

215 def pushover(p, target, n_steps=120):
216     """Push the roof to `target` in x, recording the capacity curve.
217
218     The step is week 10's robust_step, unchanged. A pushover is exactly where
219     it earns its keep: the interesting part of a capacity curve is the part
220     after the structure has started to fail, which is also the part a plain
221     Newton loop refuses to enter.
222     """
223     control = node_tag(0, 0, p["n_storeys"])
224     increment = target / n_steps
225     effort = Effort()
226
227     roof, shear, history = [0.0], [0.0], [[0.0] * p["n_storeys"]]
228     for _ in range(n_steps):
229         if not robust_step(control, 1, increment, effort):
230             break
231         ops.reactions()
232         base = sum(ops.nodeReaction(node_tag(ix, iy, 0), 1)
233                 for ix, iy in ((0, 0), (1, 0), (0, 1), (1, 1)))
234         roof.append(ops.nodeDisp(control, 1))
235         shear.append(-base)
236         # The drift at every step, not only at the end. A snapshot of the
237         # final state cannot show WHEN a mechanism formed, and that is most
238         # of what a pushover has to say.
239         history.append(drift_ratios(p))
240     return roof, shear, effort, history

```

Example 11.1 (Capacity curve of a three-storey frame). Push the frame to 250 mm of roof displacement and report the base shear.

Solution

Model. One bay each way, three storeys, fibre columns, elastic beams, gravity held constant, inverted triangular lateral pattern, 120 displacement-controlled steps.

roof pushed to	250.0 mm	
peak base shear	184.4 kN	at 66.7 mm
shear at the end	170.6 kN	
effort: 120 attempts, 0 failed, 0 subdivisions [Newton 120]		

$$V_{b,\max} = 184 \text{ kN at } \Delta_{\text{roof}} = 66.7 \text{ mm}$$

Check. Two things. The curve has a descending branch — it loses 7.5% of its strength after the peak — and that branch exists in the output only because the analysis is displacement-controlled. Under load control there is no load that produces those points at all. And the effort counter reports 120 attempts, 0 failures: plain Newton carried every step. The fallback

ladder cost nothing here. That is the argument for always carrying it, not evidence that it was unnecessary; it costs nothing on the runs where it is not needed, which is how you can afford it on the ones where it is.

11.3 The answer a capacity curve cannot give

Example 11.2 (Where did the deformation go?). Report the inter-storey drift at the end of the push.

Solution

Inter-storey drift at the end of the push

storey 1: 6.190 %
storey 2: 0.598 %
storey 3: 0.355 %

$$\theta_1 = 6.19\%, \text{ ten times } \theta_2$$

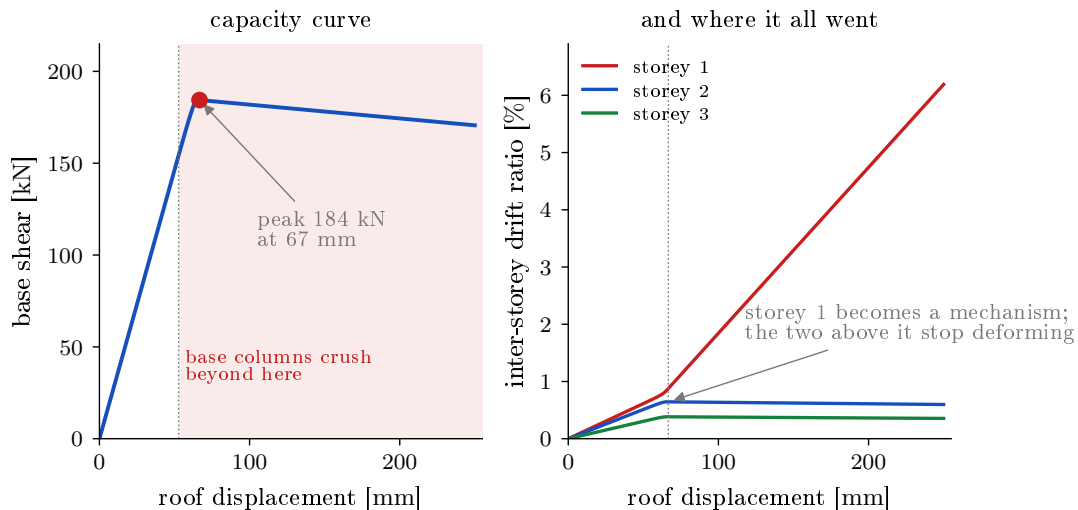


Figure 11.1: Left, the capacity curve. Right, the drift of each storey *at every step* — which the capacity curve gives no hint of.

What has happened. Storey 1 has taken more than ten times the drift of storey 2. That is a **soft-storey mechanism** (TR: *yumuşak kat mekanizması*): all the inelastic deformation has collected into one storey, and the rest of the building is riding on top of it essentially rigid.

It is here because the beams in this model were left elastic. Strong beams and weak columns is capacity design exactly reversed, and this is what it produces. [Example 11.3](#) of the exercises puts it right.

Is that ratio believable? Ten to one is a large number, and a table of three final values gives no way to judge it. The right-hand panel of [Fig. 11.1](#) shows how it was earned: all three storeys grow together while the frame is elastic, then at 66.7 mm the base columns hinge, storey 1 becomes a mechanism, and every millimetre after that goes into it. Storeys 2 and 3 stop deforming and very slowly *recover*, because the shear they carry falls as storey 1 softens.

Notice that 66.7 mm is exactly where the base shear peaks in the left-hand panel. That is not a coincidence. Storey 2 is elastic, so its drift is proportional to the shear it carries, and the base shear peaks at the moment storey 1 can no longer deliver any more.

And here is the check that settles it. If storeys 2 and 3 really do stay elastic, their drifts must remain in the ratio of the storey shears — and those are fixed by the load pattern alone, at $(3 + 2)/3 = 1.667$, whatever the analysis does. The model gives 1.68, at every one of the 120 steps.

Nothing in the model was told to do that. It is a free, independent confirmation that the surprising part of the answer is confined to storey 1 and that everything above it is behaving exactly as linear statics says it must.

Check. The mean drift over the three storeys is 2.38% — and the mean is a lie. It understates the worst storey by a factor of 2.6 and would pass a limit that storey 1 fails badly.

The capacity curve on the left of Fig. 11.1 is an average of exactly this kind. It is a smooth, well-behaved, entirely plausible curve, and nothing about it reveals that one storey is doing all the work. *A pushover's capacity curve tells you the strength; only the deformed shape tells you the mechanism*, and it is the mechanism that decides whether a building survives.

11.4 Looking at it

That last sentence deserves a picture rather than a bar chart, and it is time to meet the tool that draws one.

Definition 11.2: opsviz

opsviz plots an OpenSees model from the *live domain* — the nodes, elements and displacements currently in memory — rather than from anything the script tells it. A model must therefore exist, and be deformed, before the picture can be drawn.

Listing 11.6: w11/visualise.py — Two calls. Everything they draw comes out of the domain, as Definition 11.2 says — which is why the push has to run first.

```

37 def draw(sfac=6.0):
38     """Draw the model and the deformed shape side by side.
39
40     `sfac` is the displacement magnification. A real 250 mm sway on a 10.5 m
41     frame is visible but undramatic; six times makes the mechanism obvious.
42     Always say what the factor was -- an unlabelled exaggerated shape has
43     misled more than one review meeting.
44     """
45     fig = plt.figure(figsize=(11, 5))
46
47     ax1 = fig.add_subplot(121, projection="3d")
48     opsv.plot_model(node_labels=0, element_labels=0, ax=ax1,
49                     local_axes=False, gauss_points=False)
50     ax1.set_title("model")
51
52     ax2 = fig.add_subplot(122, projection="3d")
53     opsv.plot_defo(sfac=sfac, unDefoFlag=1, ax=ax2)
54     ax2.set_title(f"deformed, x{sfac:g}")
55
56     return fig

```

Figure 11.2 is what those two calls produce.

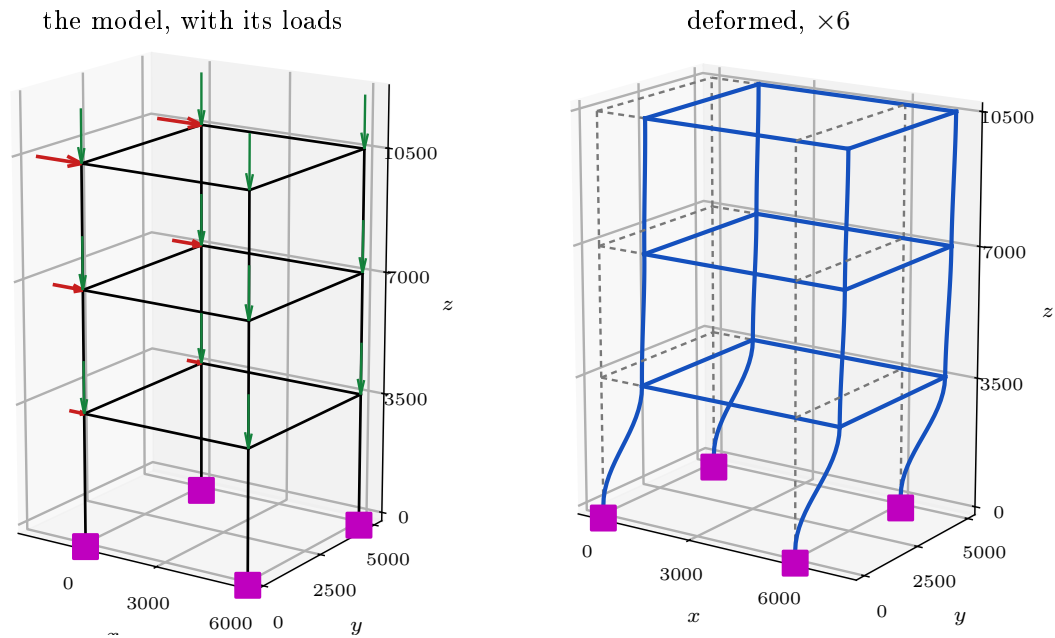


Figure 11.2: Left, the frame as OpenSees holds it, with gravity at every floor node and the lateral pattern growing with height. Right, the deformed shape at the end of the push, exaggerated six times, with the undeformed frame dashed behind it. Coordinates in mm.

Plot the model *before* analysing it, every time. It costs one line and it catches the errors that are hardest to find any other way — a node at the wrong coordinate, a member connected to the storey below, a support missing from one column line. None of those produce an error message; all of them are obvious in a picture.

Draw the *loads* as well as the geometry. A load pattern written into a loop is very easy to get upside down or back to front, and the inverted triangle in the left-hand panel is either obviously right or obviously wrong at a glance — which is more than can be said for reading the loop again. The arrows are drawn to be legible rather than to scale against one another: gravity is the same at every floor and the lateral force grows with height, and it is that *shape* the picture exists to convey.

And then look at the right-hand panel. [Example 11.2](#) measured the mechanism; this is the same fact in the form an engineer recognises on sight. The ground storey has gone over, and the two floors above it are riding on top still very nearly square. That is a soft-storey collapse, and no amount of staring at a capacity curve would have suggested it.

Note

plot the results → plot the model, then plot the results

The two pictures answer different questions. The left panel checks that the structure you built is the structure you meant; the right panel shows what it did. Skipping the first is how a beautifully presented analysis of the wrong building gets published.

11.5 How much of the curve is real

[Example 11.1](#) produced a peak base shear and [Example 11.2](#) produced a mechanism. Neither said anything about whether the materials were still behaving.

Example 11.3 (Week 10's question, asked of a building). What are the strains at the base of a corner column at the end of the push?

Solution

Listing 11.7: `w11/pushover.py` — The section deformation at integration point 1 — the column base.

```

259 def base_section_demand(p):
260     """Curvature and extreme-fibre strains at the base of a corner column.
261
262     Week 10's lesson, applied to a structure: the analysis finishing tells you
263     nothing about whether the answer is inside the range the materials were
264     calibrated for. A capacity curve is worth exactly as much as the strains
265     underneath it.
266
267     eleResponse(tag, 'section', 1, 'deformation') asks integration point 1 --
268     the base of the column -- for its axial strain and two curvatures.
269     """
270     deformation = ops.eleResponse(column_tag(0, 0, 1), "section", 1,
271                                   "deformation")
272     # For a 3D fibre section the vector is [axial, kappa_z, kappa_y, twist].
273     # Taking the RESULTANT of the two curvatures rather than one component is
274     # not fussiness: in three dimensions a column can bend about both axes at
275     # once, and reading only the one you pushed along understates the strain.
276     axial = deformation[0]
277     curvature = math.hypot(deformation[1], deformation[2])
278     y = COLUMN["h"] / 2.0
279     return {
280         "curvature": curvature,
281         "concrete": axial - curvature * y,
282         "steel": axial + curvature * (y - COLUMN["cover"]),
283     }

```

At the base of a corner column, at the end of the push

curvature	3.160e-04 1/mm
concrete strain	-0.0948 (crushes at -0.020)
steel strain	0.0821

$$\varepsilon_{\text{concrete}} = -9.5\% \text{ against a crushing strain of } -2\%$$

Where the curve stops meaning anything. The crushing curvature is arithmetic, exactly as in week 10: $\varphi_{cu} = \varepsilon_{ccu}/(h/2) = 0.020/300 = 6.67 \times 10^{-5}/\text{mm}$. Scaling back from the end of the run puts that at roughly 53 mm of roof displacement.

The peak base shear occurs at 66.7 mm.

Check. Read that again, because it is the most uncomfortable result in the chapter: the maximum base shear this analysis reports occurs *after* the base columns have passed their crushing strain. The headline number of the whole pushover — the capacity — already lies outside the range the concrete model describes, and the entire descending branch lies further outside still. Only about a fifth of the curve is inside it.

Nothing failed. The analysis converged at every step, the curve is smooth, and week 10's ladder was never even called upon. This is what that chapter meant by a robust strategy converting a visible failure into an invisible one.

Summary

Item	Command or expression	Note
3D model	<code>model('basic', '-ndm', 3, '-ndf', 6)</code>	six DOF per node
transformation	<code>geomTransf('PDelta', t, vx, vy, vz)</code>	vector not parallel to member
3D section	<code>section('Fiber', t, '-GJ', gj)</code>	compulsory; refused without
integration	<code>beamIntegration('Lobatto', t, sec, 5)</code>	Definition 11.1
column	<code>element('forceBeamColumn', t, i, j, tr, ig)</code>	
hold gravity	<code>loadConst('-time', 0.0)</code>	or you push a weightless frame
push	<code>DisplacementControl</code> + week 10's step	for the descending branch
section demand	<code>eleResponse(t, 'section', 1, 'deformation')</code>	[axial, kz, ky, twist]
capacity curve	V_b against Δ_{roof}	gives the strength
mechanism	drift, storey by storey	gives the failure mode
validity	$\varphi_{cu} = \varepsilon_{ccu}/(h/2)$	check it, every time

Exercises

Exercise 11.1. ●● The mean drift at the end of the push is 2.38 % and storey 1 is at 6.19 %. A drift limit of 3 % applies. What does each number say, and which is right?

Answer: the mean passes and storey 1 fails by more than double; the mean is simply the wrong statistic — a drift limit is a limit on each storey, because a mechanism forms in one storey and not in the average of them

Solution. Averaging a mechanism away is the error. The building does not deform uniformly and was never going to; the whole reason to run a nonlinear analysis rather than a linear one is to find out where the deformation collects.

The same error appears in week 5, where reading one node's displacement instead of a floor's mean reported sway that did not exist. It is worth seeing that the two mistakes are opposites — there the average was right and one node was wrong, here one storey is right and the average is wrong — and that no rule of thumb about averaging would have got both. What decides it is what the quantity *means*: sway is a property of a floor as a whole, drift is a property of each storey separately.

Exercise 11.2. ●● Change the lateral load pattern from inverted triangular to uniform, so every floor receives the same force. Predict what happens to the mechanism before running it.

Answer: a uniform pattern loads the lower storeys relatively harder, so the soft storey at the base becomes more pronounced and the peak base shear rises

Solution. The pattern decides how the storey shears are distributed, and the storey shear at level i is the sum of all the forces above it. A uniform pattern puts more of the total force low down relative to a triangular one, so the base storey shear for a given roof displacement is higher and the base columns reach their capacity sooner.

The general point is more important than the particular answer. A pushover answers the question you asked it, and the load pattern *is* most of the question. A structure has one capacity

curve per pattern, not one capacity curve — which is why assessment codes require more than one pattern to be run, and why quoting ‘the’ pushover result without saying which pattern produced it is meaningless.

Exercise 11.3. ●●● Replace the elastic beams with fibre-section beams weak enough to yield before the columns do. Describe what you expect to happen to Fig. 11.1’s two panels, and say which change matters more.

Answer: the capacity curve changes modestly — a slightly lower peak and a longer, flatter descending branch — while the drift profile changes completely, spreading the deformation over all three storeys instead of concentrating it in one; the drift profile is the change that matters

Solution. With weak beams the hinges form at the beam ends rather than in the columns, so the frame develops a beam-sway mechanism: every storey participates, every beam absorbs energy, and the structure keeps deforming without any single storey running away.

The capacity curve moves surprisingly little. The peak may even fall slightly. If you judged the two designs by base shear alone you would conclude they were nearly equivalent, which is the opposite of the truth — and it is precisely the reason strong-column weak-beam is a *detailing* rule rather than a strength requirement.

What changes is the drift profile and, behind it, the ductility. The column-sway frame concentrates all its inelastic demand into one storey’s columns, which are also carrying the gravity load of everything above them; when they fail, the building comes down. The beam-sway frame distributes the same total deformation across twelve beam ends, none of which carries gravity load to the ground. Same capacity curve, entirely different collapse.

Exercise 11.4. ●●● **Example 11.3** found the peak base shear occurring after the base columns had crushed. Does that make the peak value useless? Argue both ways, then decide.

Answer: not useless but not a capacity either: it is the strength of a model whose materials are being extrapolated, so it should be treated as an upper bound with unknown error rather than as a prediction; the defensible number is the shear at the last curvature all materials were still inside their calibrated range

Solution. The case for using it: the crushing strain is itself a modelling convention rather than a sharp physical event, confined concrete does retain strength past it, and every strength you have ever quoted rests on material models extrapolated somewhat. Refusing to report any number past ε_{ccu} would be over-scrupulous.

The case against: the error past that point is unbounded and one-directional. **Concrete01** holds its residual strength for ever and **Steel02** never fractures, so every extrapolation in this model runs the same way — towards more strength than the real column has. That is not scatter, it is bias, and it is bias in the unconservative direction.

Where I would come down: report the shear at 53mm as the defensible capacity, report the 184kN peak as what the model says with the strain state quoted beside it, and be explicit that the descending branch is illustrative. That costs one extra line in a table and it is the difference between a result and a plot.

The habit it points at is the one week 10 introduced and this chapter has now applied twice: decide what would make the answer invalid *before* you run, then check it. An analysis will never tell you.

Week 12: Introduction to Dynamic Analysis

Learning outcomes

After this week you should be able to:

1. assign mass in the course's unit system, and say what entering kilograms costs;
2. obtain natural periods with **eigen**, and choose a solver that works;
3. set Rayleigh damping from two periods, and say what it does to every other mode;
4. run a transient analysis with **Newmark**;
5. recover a period and a damping ratio from a response, as a measurement rather than a restatement of the input.

Notation. m mass k stiffness ω circular frequency T natural period ζ damping ratio δ logarithmic decrement $\mathbf{M}, \mathbf{C}, \mathbf{K}$ mass, damping and stiffness matrices

Everything so far has been static. Loads were applied through a pseudo-time that meant nothing, and the structure had no idea how fast anything happened. This week adds the two properties that only matter when it does: **mass** (*TR: kütü*), which resists acceleration, and **damping** (*TR: sönüm*), which removes energy.

The chapter is deliberately linear. There is quite enough new here without also asking what happens when a material yields, and — more importantly — everything in it can be checked against a closed form. That is exactly why it comes before week 13 rather than after: when the nonlinear time history of a laboratory specimen disagrees with the test, you will want to already know that the dynamics were right.

12.1 Mass, and the unit that has been waiting since week 0

The equation of motion is

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \vec{p}(t), \quad (12.1)$$

and the first of those three matrices in Eq. (12.1) is the one this course has warned about twice without ever using. The third has been doing all the work since week 2; the second arrives in §12.2.

Listing 12.1: `w12/dynamics.py` — `ops.mass` takes one value per degree of freedom, in the same order as `ops.fix`.

```

66 def build_sdof(p, mass_in_kg=False):
67     """A cantilever column with a mass at the top.
68
69     ops.mass() takes one value per degree of freedom, in the same order as
70     ops.fix(). Only the horizontal one is given a value here: the column is
71     axially very stiff, so vertical inertia contributes nothing at the periods
72     that matter, and leaving a rotational mass at zero is standard for a lumped
73     model.
74
75     Note that a zero mass on a FREE degree of freedom is legal. It gives that
76     direction an infinite frequency, and eigen() will happily return it.
77     """
78     ops.wipe()
79     ops.model("basic", "-ndm", 2, "-ndf", 3)
80     ops.node(1, 0.0, 0.0)
81     ops.node(2, 0.0, p["L"])
82     ops.fix(1, 1, 1, 1)
83
84     ops.geomTransf("Linear", 1)
85     ops.element("elasticBeamColumn", 1, 1, 2, p["A"], p["E"], p["I"], 1)
86
87     m = p["m"] * 1000.0 if mass_in_kg else p["m"]
88     ops.mass(2, m, 0.0, 0.0)
89     return m

```

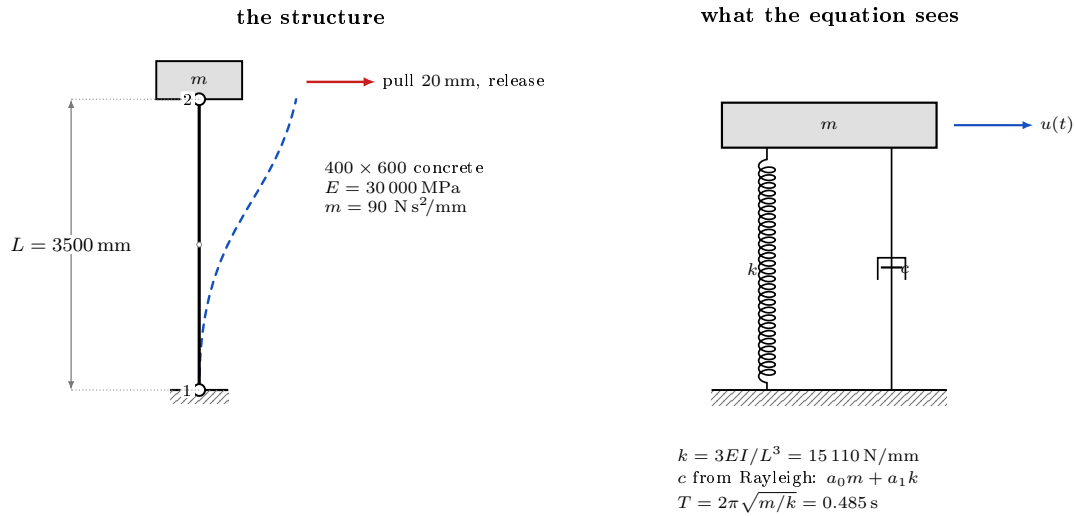
Note

mass in kilograms $\rightarrow$ mass in Ns^2/mm

$F = ma$ fixes the unit of mass once force and length are chosen. In newton and millimetre that unit is Ns^2/mm , which happens to equal 1000 kg — one tonne. A mass entered in kilograms is a thousand times too large, and since $T \propto \sqrt{m}$ every period comes out $\sqrt{1000} = 31.6$ times too long.

Week 0 predicted that number. [Example 12.2](#) collects it.

[Figure 12.1](#) is the system this whole chapter analyses, beside the idealisation the equations actually describe.



Three numbers stand between the two pictures. m is put in directly. k is a property of the column that OpenSees never sees as a single number — it assembles a stiffness matrix instead, and $3EI/L^3$ is what that matrix reduces to for this one degree of freedom. c exists only because **rayleigh** was called.

Knowing which of the three is measured, which is derived and which is chosen is most of knowing how far to trust the period.

Figure 12.1: Left, the column with its mass and the displacement that starts the free vibration. Right, the same thing as [Eq. \(12.1\)](#) sees it: one mass, one spring, one dashpot, one degree of freedom.

Three numbers stand between those two pictures, and they have three different standings. m is put in directly. k is a property of the column that OpenSees never holds as a single number — it assembles a stiffness matrix, and $3EI/L^3$ is what that matrix reduces to for one degree of freedom. And c exists only because **rayleigh** was called: nothing in the structure produced it.

Knowing which of the three is measured, which is derived and which is chosen is most of knowing how far to trust the period that comes out.

Example 12.1 (The period, twice). A 3.5 m concrete column, 400 mm by 600 mm, carries 90 t at its top. Find its natural period from the closed form and from **eigen**.

Solution

Model. One `elasticBeamColumn`, fixed at the base, with a lumped horizontal mass at the top. $E = 30\,000 \text{ MPa}$, $I = 7.2 \times 10^9 \text{ mm}^4$.

Governing relations.

$$k = \frac{3EI}{L^3}, \quad \omega = \sqrt{\frac{k}{m}}, \quad T = \frac{2\pi}{\omega}. \quad (12.2)$$

Numbers.

$$k = \frac{3(30000)(7.2 \times 10^9)}{3500^3} = 15\,110 \text{ N/mm}, \quad \omega = \sqrt{\frac{15110}{90.0}} = 12.96 \text{ rad/s},$$

$$T = \frac{2\pi}{12.96} = 0.4849 \text{ s}.$$

$$T = 0.485 \text{ s}$$

Listing 12.2: w12/dynamics.py — `eigen` returns ω^2 . The solver is not a detail.

```

94 def periods(n_modes=1, solver="-fullGenLapack"):
95     """Natural periods, in seconds.
96
97     eigen() returns the eigenvalues lambda = omega^2, so the period is
98     2*pi/sqrt(lambda). It uses the CURRENT tangent stiffness, which for a
99     linear model is the only stiffness there is -- but from week 13 it means
100    the period of a damaged structure differs from the period of an intact
101    one, and asking at the wrong moment gives the wrong answer.
102
103    The solver is not a detail. The default is Arpack, an ITERATIVE method
104    built for large systems, and on a lumped-mass model it has to work with a
105    singular mass matrix -- most degrees of freedom carry no mass at all. On a
106    model this small it simply gives up:
107
108        ArpackSolver::Error with _saupd info = -9999
109        Could not build an Arnoldi factorization.
110
111    '-fullGenLapack' solves the whole generalized problem directly. It is
112    correct on small models and, as OpenSees itself warns, very slow on large
113    ones. Massless degrees of freedom come back as eigenvalues of about 1e308,
114    which is the honest answer -- infinite frequency -- and the reason to ask
115    for only as many modes as the model has masses.
116    """
117    eigenvalues = ops.eigen(solver, n_modes)
118    return [2.0 * math.pi / math.sqrt(lam) for lam in eigenvalues]
```

stiffness	3EI/L ³	15113.7 N/mm
mass		90.0 N s ² /mm (= 90000 kg)
period, closed form		0.4849 s
period, eigen()		0.4849 s

Check. The two agree to machine precision, and as in week 1 that is the expected result rather than a pleasant surprise: a single elastic element with a lumped mass *is* the single-degree-of-freedom system Eq. (12.2) describes. There is no discretisation error to find.

On the solver. The default is Arpack, an *iterative* method built for large systems. A lumped-mass model gives it a singular mass matrix — most degrees of freedom carry no mass at all — and on a model this small it simply gives up:

```

ArpackSolver::Error with _saupd info = -9999
Could not build an Arnoldi factorization.
```

'-fullGenLapack' solves the whole generalised problem directly. It is right on small models and, as OpenSees itself warns, very slow on large ones.

Example 12.2 (The mass unit, as promised). Enter the same mass as 90 000 kg instead. What happens?

Solution

Listing 12.3: w12/dynamics.py — Two lines of arithmetic, and no analysis needed to predict it: the period of [Example 12.1](#) scaled by $\sqrt{1000}$.

```

45 def mass_note(p=SDOF):
46     """What the mass unit is, and what entering kilograms costs.
47
48     F = ma fixes the unit of mass once force and length are chosen. In N and
49     mm that unit is N*s^2/mm, which happens to equal 1000 kg -- one tonne.
50
51     A mass entered in kilograms is therefore 1000 times too large. Period goes
52     as sqrt(m), so every period comes out sqrt(1000) = 31.6 times too long: a
53     half-second building reports fifteen seconds. That is far too large to be
54     roundoff and far too orderly to be a modelling error, which is exactly why
55     it gets misdiagnosed as one.
56     """
57     k = 3.0 * p["E"] * p["I"] / p["L"] ** 3
58     right = 2.0 * math.pi * math.sqrt(p["m"] / k)
59     wrong = 2.0 * math.pi * math.sqrt(p["m"] * 1000.0 / k)
60     return {"k": k, "T_correct": right, "T_in_kg": wrong,
61            "factor": wrong / right}

```

What happens if the mass is entered in kilograms	
correct	0.4849 s
mass in kg	15.3326 s
ratio	31.6228 (sqrt(1000) = 31.6228)
eigen() agrees	15.3326 s

$$T = 15.3 \text{ s for a single-storey column}$$

Check. The ratio is $\sqrt{1000}$ to seven figures, which is the point: this is not a numerical accident but an exact consequence of $T \propto \sqrt{m}$.

Two things make this error dangerous rather than merely wrong. Nothing objects — OpenSees does not know what a kilogram is, so the analysis runs perfectly and returns an internally consistent answer. And the error is far too large to be roundoff and far too orderly to look like a modelling mistake, so it gets misdiagnosed. A 15 s period on a single-storey column is absurd on its face; a 1.5 s period on a ten-storey frame is not, and the same error would produce it.

12.2 Damping, and the modes you did not choose

Definition 12.1: Rayleigh damping

$\mathbf{C} = a_0 \mathbf{M} + a_1 \mathbf{K}$. Given a target ratio ζ at two frequencies,

$$a_0 = \zeta \frac{2\omega_1\omega_2}{\omega_1 + \omega_2}, \quad a_1 = \frac{2\zeta}{\omega_1 + \omega_2}, \quad (12.3)$$

and the ratio actually applied at any other frequency is $\zeta(\omega) = \frac{1}{2}(a_0/\omega + a_1\omega)$.

Listing 12.4: w12/dynamics.py — The two periods are a choice, and a consequential one.

```

138 def rayleigh_damping(p, T1, T2):
139     """Rayleigh damping matched to zeta at two periods.
140
141     C = a0*M + a1*K
142     a0 = zeta * 2*w1*w2/(w1+w2)    a1 = zeta * 2/(w1+w2)

```

```

143
144     The two periods are a CHOICE, and a consequential one. Damping is exactly
145     zeta at w1 and w2 and higher everywhere outside them, so a badly chosen
146     pair can put ten or twenty per cent damping on modes you meant to leave
147     alone -- quietly, and in the direction that makes a structure look safer.
148     """
149     w1, w2 = 2.0 * math.pi / T1, 2.0 * math.pi / T2
150     a0 = p["zeta"] * 2.0 * w1 * w2 / (w1 + w2)
151     a1 = p["zeta"] * 2.0 / (w1 + w2)
152     ops.rayleigh(a0, 0.0, a1, 0.0)
153     return a0, a1
154
155
156 def rayleigh_ratio(a0, a1, T):
157     """The damping ratio this Rayleigh pair actually applies at period T."""
158     w = 2.0 * math.pi / T
159     return 0.5 * (a0 / w + a1 * w)

```

Example 12.3 (What Rayleigh damping does to everything else). Anchor 5% damping at T_1 and $T_1/5$. What damping does the model apply at $T_1/20$?

Solution

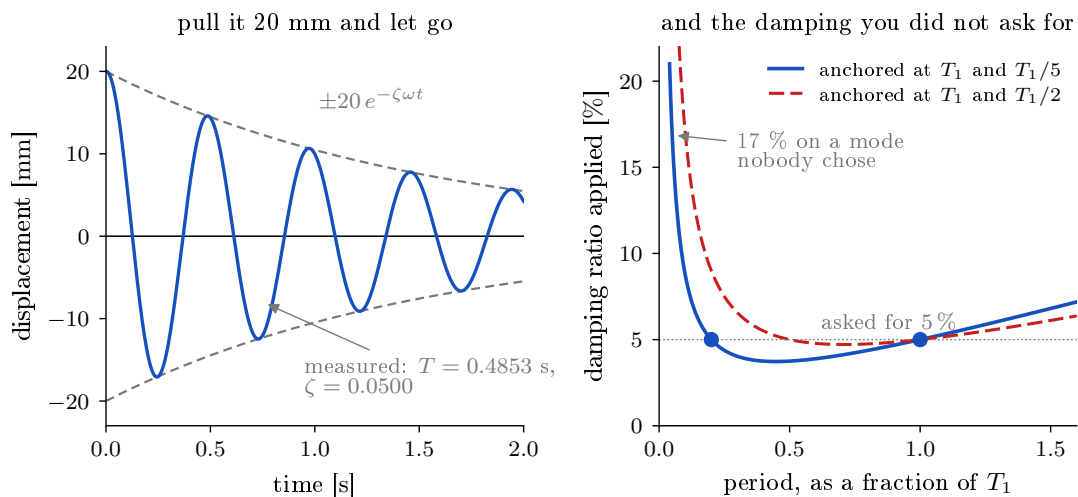


Figure 12.2: Left, the free-vibration ring-down of [Example 12.4](#). Right, the damping ratio Rayleigh actually applies — exactly 5% at the two anchors and more everywhere outside them.

Rayleigh a0, a1	1.07990 0.001286
-----------------	------------------

Substituting into [Definition 12.1](#)'s last expression gives 5.000% at T_1 and at $T_1/5$ — exactly, as it must — and

$$\zeta(T_1/20) = 16.9\%, \text{ more than three times what was asked}$$

Check. The shape in the right-hand panel is the thing to remember. Rayleigh damping has a minimum between its two anchors and rises without limit outside them, so *every* mode outside the chosen pair is damped harder than intended. On a multi-storey building the higher modes are exactly the ones this hits, and it hits them in the direction that makes the structure look better behaved than it is.

The dashed curve shows the second anchor moved to $T_1/2$. That looks like a tighter, more careful choice and damps the short periods far harder still. Neither choice is wrong; both are choices, and neither is visible anywhere in the output.

12.3 A response, and measuring it

With the damping of [Example 12.3](#) in place, the model can finally be asked to move.

Listing 12.5: `w12/dynamics.py` — Impose a displacement statically, remove the pattern, then integrate in real time.

```

164 def free_vibration(p, u0=20.0, duration=2.0, dt=0.002):
165     """Pull the mass sideways, let go, and watch it ring down.
166
167     The cleanest dynamic verification there is. The decay of successive peaks
168     gives the damping ratio back through the logarithmic decrement, and the
169     spacing of the peaks gives the period -- both independent of anything the
170     model was told.
171     """
172     ops.timeSeries("Constant", 1)
173     ops.pattern("Plain", 1, 1)
174     ops.sp(2, 1, u0)                                # impose the initial displacement
175
176     ops.constraints("Transformation")
177     ops.numberer("RCM")
178     ops.system("BandGeneral")
179     ops.test("NormDispIncr", 1.0e-10, 20)
180     ops.algorithm("Newton")
181     ops.integrator("LoadControl", 1.0)
182     ops.analysis("Static")
183     if ops.analyze(1) != 0:
184         raise RuntimeError("could not impose the initial displacement")
185     ops.loadConst("-time", 0.0)
186     ops.remove("loadPattern", 1)                    # release it: nothing holds it now
187
188     ops.integrator("Newmark", 0.5, 0.25)            # average acceleration, unconditionally stable
189     ops.analysis("Transient")
190
191     t, u = [0.0], [ops.nodeDisp(2, 1)]
192     for _ in range(int(duration / dt)):
193         if ops.analyze(1, dt) != 0:
194             break
195         t.append(ops.getTime())
196         u.append(ops.nodeDisp(2, 1))
197     return t, u

```

`ops.integrator('Newmark', 0.5, 0.25)` is the average-acceleration scheme: unconditionally stable and free of numerical damping, which matters here because the whole point is to measure the damping the model actually has.

Example 12.4 (Recovering the period and the damping from the motion). Pull the mass 20 mm sideways, release it, and measure what happens.

Solution

Listing 12.6: `w12/dynamics.py` — Nothing here knows what damping was applied.

```

202 def logarithmic_decrement(t, u):

```

```

203     """Recover the period and the damping ratio from the response alone.
204
205         delta = ln(u_i / u_{i+1})           zeta = delta / sqrt(4*pi^2 + delta^2)
206
207     Nothing here knows what damping was applied. That is the point: this is a
208     measurement of the model, not a restatement of its input.
209     """
210     peaks = [(t[i], u[i]) for i in range(1, len(u) - 1)
211              if u[i] > u[i - 1] and u[i] > u[i + 1] and u[i] > 0.0]
212     if len(peaks) < 3:
213         return None
214
215     period = (peaks[-1][0] - peaks[0][0]) / (len(peaks) - 1)
216     ratios = [math.log(peaks[i][1] / peaks[i + 1][1])
217              for i in range(len(peaks) - 1)]
218     delta = sum(ratios) / len(ratios)
219     zeta = delta / math.sqrt(4.0 * math.pi ** 2 + delta ** 2)
220     return {"period": period, "delta": delta, "zeta": zeta,
221            "n_peaks": len(peaks)}

```

Governing relation. For viscous damping, successive peaks fall by a constant ratio, and

$$\delta = \ln \frac{u_i}{u_{i+1}} = \frac{2\pi\zeta}{\sqrt{1-\zeta^2}}, \quad \zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}. \quad (12.4)$$

peaks found	4	
period, measured	0.4853 s	(eigen said 0.4849)
damping, measured	0.0500	(asked for 0.0500)

$$T = 0.4853 \text{ s}, \quad \zeta = 0.0500$$

Check. This is the strongest verification in the chapter, and it is worth being precise about why. The period was obtained by counting peaks in a displacement record; the damping by taking logarithms of their ratios. Neither calculation touches k , m , a_0 or a_1 . They are *measurements of how the model moved*, and they agree with what it was told to within 0.1 % and 0.1 % respectively.

From Eq. (12.4), $\delta = 2\pi(0.05)/\sqrt{1-0.05^2} = 0.3145$, so each peak should be $e^{0.3145} = 1.370$ times the next. It is.

This is exactly the procedure used on a real structure, where k and ζ are not known and the record is all you have. Week 13 applies it to a laboratory specimen.

Summary

Item	Command or expression	Note
mass	<code>mass(node, mx, my, mrz)</code>	N s ² /mm, i.e. tonnes
periods	<code>eigen('-fullGenLapack', n)</code>	returns ω^2
damping	<code>rayleigh(a0, 0, a1, 0)</code>	Eq. (12.3)
time integration	<code>integrator('Newmark', 0.5, 0.25)</code>	stable, no numerical damping
transient	<code>analysis('Transient'), analyze(1, dt)</code>	real time
impose then release	<code>sp → loadConst → remove</code>	free vibration
SDOF period	$T = 2\pi\sqrt{m/k}$, $k = 3EI/L^3$	Eq. (12.2)
Rayleigh ratio	$\zeta(\omega) = \frac{1}{2}(a_0/\omega + a_1\omega)$	minimum between the anchors
decrement	$\zeta = \delta/\sqrt{4\pi^2 + \delta^2}$	Eq. (12.4)
mass unit error	T out by $\sqrt{1000} = 31.6$	Example 12.2

Exercises

Exercise 12.1. ●●● A colleague reports a fundamental period of 4.7 s for a four-storey concrete frame. Without seeing their model, say what is most likely wrong and what period you would expect.

Answer: their mass is in kilograms; dividing by $\sqrt{1000} = 31.6$ gives 0.15 s, which is too short for four storeys, so the mass is probably in kilograms and something else is also wrong — but the kilogram error is the first thing to check

Solution. A four-storey frame should have a fundamental period of roughly 0.4 s by the usual rule of thumb of about 0.1 N seconds. 4.7 s is an order of magnitude out, and the first suspect for an order-of-magnitude error in a period is always the mass unit.

The interesting part is that the arithmetic does not quite work. Dividing by 31.6 gives 0.149 s, which is now too *short*. So either the mass is not simply in kilograms, or there is a second error — an over-stiff model, perhaps uncracked section properties.

The habit worth taking from this: when a suspected unit error does not resolve cleanly, do not force it. An error that is exactly a power of ten is a unit error; one that is nearly a power of ten is two errors, and stopping after finding the first is how the second survives.

Exercise 12.2. ●●● Anchor the Rayleigh damping at T_1 and $T_1/2$ instead of T_1 and $T_1/5$. What happens to the damping at $T_1/20$, and which choice would you defend?

Answer: it rises sharply — the narrower the anchor pair, the more severely everything outside it is damped; the wider pair is the safer default, and the honest answer is that the pair should bracket the modes that actually carry the response

Solution. Narrowing the anchors makes a_1 larger, and the stiffness-proportional term grows linearly with frequency, so short periods are punished hard. The dashed curve in [Fig. 12.2](#) shows it.

The defensible rule is to bracket the modes that matter: the fundamental at one end and the highest mode contributing meaningfully to the response at the other. What you must not do is choose the pair for numerical convenience and then report the result as though 5 % damping had been applied — because outside the anchors it was not.

There is a deeper point here, which is that Rayleigh damping is not a model of anything. Real energy dissipation in a structure comes from cracking, friction, and the surroundings, none of which is

proportional to mass or to stiffness. $\mathbf{C} = a_0\mathbf{M} + a_1\mathbf{K}$ is chosen because it can be diagonalised by the same modes as $\mathbf{M}$ and $\mathbf{K}$, which is a mathematical convenience and not a physical claim. Knowing that is what stops you defending the number too hard.

Exercise 12.3. ●●● Halve the mass and predict the new period before running it. Then halve the stiffness instead and predict again.

Answer: halving m gives $T = 0.343$ s, and halving k gives $T = 0.686$ s; both follow from

$$T \propto \sqrt{m/k}$$

Solution. $T = 2\pi\sqrt{m/k}$, so halving m divides T by $\sqrt{2}$ and halving k multiplies it by $\sqrt{2}$: $0.4849/1.4142 = 0.3429$ s and $0.4849 \times 1.4142 = 0.6858$ s.

Worth doing because the square root is the part people's intuition gets wrong. Stiffening a structure by 50 % does not shorten its period by 50 % — it shortens it by 18 %. Period is a stubborn quantity, and that is why a model whose period is out by a factor of two has something seriously wrong with it rather than slightly wrong.

Exercise 12.4. ●●● The measured period in [Example 12.4](#) was 0.4853 s and `eigen` said 0.4849 s. Explain the difference, and say which you would report.

Answer: the measured value is the damped period, which is longer by a factor $1/\sqrt{1-\zeta^2}$; at $\zeta = 0.05$ that is a factor of 1.00125, predicting 0.4855 s — essentially the whole difference; report the undamped period, because that is the property of the structure

Solution. A damped oscillator vibrates at $\omega_d = \omega\sqrt{1-\zeta^2}$, so its period is $T/\sqrt{1-\zeta^2}$. At 5 % damping that is a 0.125 % increase: $0.4849 \times 1.00125 = 0.4855$ s, against the 0.4853 s measured. The remaining difference is the finite time step and the peak-picking resolution.

That the discrepancy is explained rather than merely small is the point. A 0.1 % difference could easily have been waved away as roundoff; instead it turns out to be a real physical effect of exactly the predicted size, which is a much stronger confirmation that the analysis is doing what it should.

Report the undamped period. It is the property of the structure; the damped period also depends on ζ , which is the least reliable number in the whole model.

Exercise 12.5. ●●● [Example 12.4](#) recovered ζ from four peaks of a free-vibration record. Describe how you would apply the same procedure to a real building, and name two things that would make it harder.

Answer: excite the building — ambient vibration, a shaker, or an aftershock — record the response with accelerometers, and fit the decay of the free-vibration portion; it is harder because a real building has several modes contributing at once, and because its damping is not viscous, so the decay is not a clean exponential and ζ depends on the amplitude at which it was measured

Solution. The procedure is real and routinely done. What changes is that the record is not clean.

The first difficulty is modal superposition. A real record contains several modes at once, and the decrement assumes one. The standard answer is to band-pass filter around each mode first, which works when the modes are well separated and much less well when they are not.

The second is that real damping is not viscous. It comes from cracking, friction at connections, and energy radiated into the ground, and none of those is proportional to velocity. The practical consequence is that measured ζ rises with amplitude — a building shaken hard damps more than the same building shaken gently — so a value measured under ambient vibration will underestimate what happens in an earthquake, often by a factor of two or more.

This is worth carrying into week 13. When a nonlinear time-history analysis disagrees with a shake table test, the damping is among the first things to suspect, and 5 % is a convention rather than a measurement.

Week 13: Fully Nonlinear Dynamic Analysis

Learning outcomes

After this week you should be able to:

1. apply a ground motion with `UniformExcitation` and a `Path` time series;
2. write a transient step that retries, without losing track of elapsed time;
3. verify a nonlinear time history when no closed form exists;
4. locate a structure's elastic range by measurement rather than assumption;
5. report residual displacement and period lengthening, and say why each matters.

Notation. $\ddot{u}_g$ ground acceleration u displacement of the mass V_b base shear T period λ scale factor applied to the record

The whole course arrives here. The specimen carries week 8's fibre section, the step is week 10's ladder, the mass and damping are week 12's, and week 11's staging puts the gravity on first. What is new is one line — the base moves — and one absence: there is no closed form for the peak displacement of a yielding column under an earthquake.

That absence is the point. It is why the analysis is worth running, and it is why this chapter is mostly about how to believe the answer.

13.1 The specimen

Figure 13.1 is what is being shaken, beside a reminder of which week supplied each part of it.

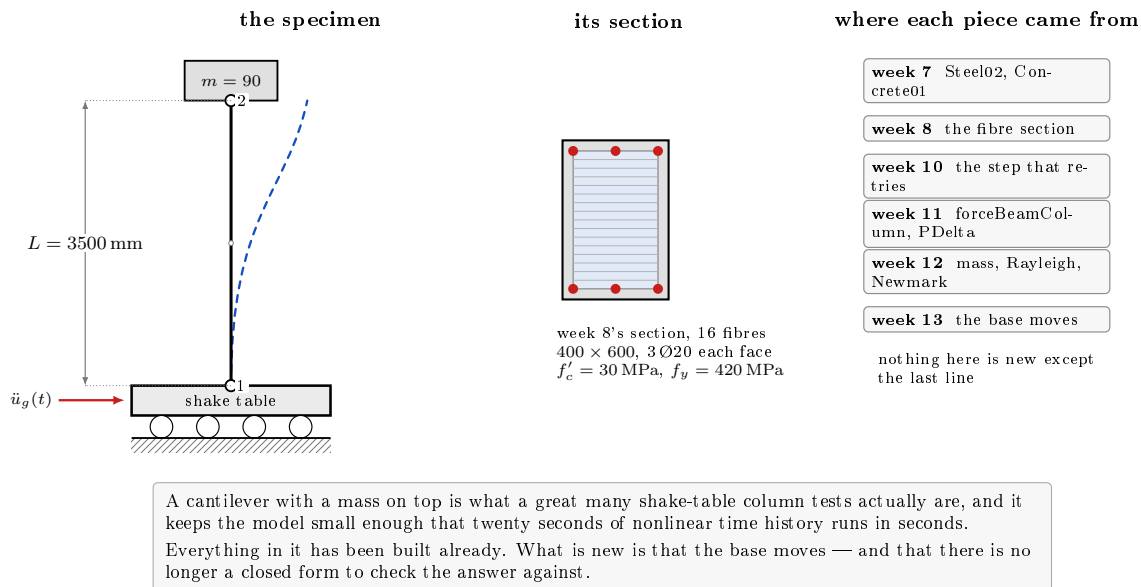


Figure 13.1: The specimen, its section, and where every piece of it came from. Nothing here is new except the last line.

Listing 13.1: `w13/timehistory.py` — A cantilever with a mass on top — which is what a great many shake-table column tests actually are.

```
91 def build_specimen(p=SPECIMEN):
92     """The specimen: one fibre-section column with a mass on top.
```

```

93
94     A cantilever with a lump of mass is what most shake-table column tests
95     actually are, and it keeps the model small enough that twenty seconds of
96     nonlinear time history runs in seconds rather than hours.
97     """
98     ops.wipe()
99     ops.model("basic", "-ndm", 2, "-ndf", 3)
100     ops.node(1, 0.0, 0.0)
101     ops.node(2, 0.0, p["L"])
102     ops.fix(1, 1, 1, 1)
103
104     define_section(COLUMN, p["n_fibres"])
105     ops.geomTransf("PDelta", 1)
106     ops.beamIntegration("Lobatto", COL_INTEG, 1, 5)
107     ops.element("forceBeamColumn", 1, 1, 2, 1, COL_INTEG)
108
109     # Horizontal mass only: the column is axially stiff and a rotational mass
110     # on a lumped model is a fiction nobody needs here.
111     ops.mass(2, p["m"], 0.0, 0.0)

```

Listing 13.2: w13/timehistory.py — The record is generated, not read from a file.

```

60 def accelerogram(p=GROUND, scale=1.0):
61     """A synthetic accelerogram, generated rather than read from a file.
62
63     Deterministic on purpose: fixed frequencies, fixed phases, no random seed
64     to remember. Anybody who runs this script gets the identical record, which
65     is what K19 asks of every number in these notes -- and a real record could
66     not be redistributed with them anyway.
67
68     The shape is the one every strong-motion record has: a short build-up, a
69     strong-motion plateau, and a long exponential decay. The frequency content
70     is deliberately placed around the specimen's own period, because a record
71     with no energy near the structure excites nothing and proves nothing.
72     """
73     n = int(p["duration"] / p["dt"])
74     freqs = (1.3, 2.1, 3.4, 5.5, 8.9)          # Hz -- spread around 1/T
75     phases = (0.0, 1.1, 2.3, 0.7, 1.9)        # rad, fixed
76
77     record = []
78     for i in range(n + 1):
79         t = i * p["dt"]
80         rise = min(t / 2.0, 1.0)
81         decay = math.exp(-max(0.0, t - 8.0) / 5.0)
82         envelope = rise * decay
83         a = sum(math.sin(2.0 * math.pi * f * t + ph) / (1.0 + k)
84                 for k, (f, ph) in enumerate(zip(freqs, phases)))
85         record.append(scale * p["pga"] * envelope * a / 1.6)
86     return record

```

Generating the record rather than shipping one is a deliberate choice and worth defending. It is deterministic, so everyone who runs these notes gets the identical input (K19); it needs no data file and no licence; and its frequency content can be placed deliberately around the specimen's own period. That last point matters more than it sounds — a record with no energy near the structure excites nothing and proves nothing.

Listing 13.3: w13/timehistory.py — UniformExcitation moves every restrained node together, which is what a shake table does.

```

141 def apply_ground_motion(record, dt, p=SPECIMEN):
142     """Shake the base.
143
144     A UniformExcitation pattern moves every restrained node together, which is

```

```

145     what a shake table does. It is applied as an ACCELERATION history, and
146     OpenSees turns it into the effective force  $-m \cdot a_g$  internally -- so the
147     record is in mm/s2 and the mass has already been given in tonnes.
148
149     Get either of those wrong and the specimen is shaken by something else
150     entirely, without complaint.
151     """
152     ops.timeSeries("Path", 2, "-dt", dt, "-values", *record)
153     ops.pattern("UniformExcitation", 2, 1, "-accel", 2)
154
155     w = periods_of(2)
156     rayleigh_from(p, w[0], w[0] / 5.0)
157     return w

```

Note

a ground motion is a load → a ground motion is an *acceleration*

UniformExcitation takes an acceleration history and forms the effective force $-m\ddot{u}_g$ itself. So the record must be in mm/s² and the mass in N s²/mm — and if either is wrong the specimen is shaken by something else entirely, without complaint. Week 12's factor of 31.6 is waiting here too.

13.2 A step that retries, in real time

Listing 13.4: w13/timehistory.py — Week 10's ladder, with one difference that matters.

```

176 def robust_transient(duration, dt, effort, min_frac=1.0 / 64.0):
177     """Week 10's ladder, in real time instead of pseudo-time.
178
179     One difference matters. In a static push, subdividing a step is free: the
180     analysis simply arrives at the same displacement in smaller pieces. Here
181     the step is TIME, so the sub-steps must add up to exactly the same elapsed
182     time or the record and the response drift out of step with each other --
183     silently, and worse the longer the analysis runs.
184     """
185     ops.integrator("Newmark", 0.5, 0.25)
186     ops.analysis("Transient")
187
188     t, u, v, shear = [0.0], [0.0], [0.0], [0.0]
189     remaining = duration
190     step = dt
191
192     while remaining > 1.0e-12:
193         this = min(step, remaining)
194         ok = False
195         for name, algorithm in ALGORITHMS:
196             ops.test("NormDispIncr", 1.0e-8, 20)
197             ops.algorithm(*algorithm)
198             ok = ops.analyze(1, this) == 0
199             effort.record(name, ok)
200             if ok:
201                 break
202
203         if not ok:
204             if step <= dt * min_frac:
205                 break
206             effort.subdivisions += 1
207             step /= 2.0
208             continue
209
210         remaining -= this
211         step = min(dt, step * 2.0)
212
213     t.append(ops.getTime())

```

```

214     u.append(ops.nodeDisp(2, 1))
215     v.append(ops.nodeVel(2, 1))
216     ops.reactions()
217     shear.append(-ops.nodeReaction(1, 1))
218
219     return {"t": t, "u": u, "v": v, "shear": shear, "effort": effort}

```

In a static push, subdividing a step is free: the analysis reaches the same displacement in smaller pieces. Here the step is *time*, and the sub-steps must add up to exactly the same elapsed time or the record and the response drift apart — silently, and worse the longer the analysis runs. That is what the **remaining** bookkeeping is for, and it is the one place where copying week 10's loop unchanged would have been wrong.

13.3 Verifying an analysis with no answer

Everything so far has been checked against something: a closed form, a hand calculation, a symmetry argument. None of those is available now. Three checks replace them — [Example 13.1](#) first, then the two in [Example 13.2](#) — and none needs anything from outside the model.

Example 13.1 (Check one: *nothing in, nothing out*). Run the analysis with the record scaled to zero.

Solution

```

Check 1 -- zero input: a stationary base leaves it stationary
peak displacement      7.843e-11 mm

```

$$7.8 \times 10^{-11} \text{ mm} \text{ — zero, to roundoff}$$

Why bother. It is the cheapest check in dynamics and it catches a whole family of errors that produce plausible output: an excitation attached to the wrong pattern tag, applied in the wrong direction, or a gravity load that was never held constant and is quietly still ramping. Any of those gives a response to no input. Run it once for every new model.

Example 13.2 (Checks two and three: *scale the record*). Run the same record at scale factors from 0.02 to 1.50 and compare the peaks.

Solution

The idea. Scaling a record scales it exactly — that is arithmetic. So if the structure were linear, every response quantity would scale with it exactly too. Divide the peak by the scale factor and a linear structure gives a constant.

Listing 13.5: `w13/timehistory.py` — The same record at seven amplitudes, and the period measured after each.

```

257 def scale_sweep(scales=SCALES, p=SPECIMEN, ground=GROUND):
258     """Run the same record at several amplitudes and compare.
259
260     The quantity to watch is peak/scale: what the peak WOULD have been at
261     scale 1 if the structure were linear. While it stays constant the response
262     is elastic; where it moves, the specimen has yielded. Nothing had to be
263     assumed about where that happens -- it was measured.
264
265     The damaged period is taken after the shaking stops. eigen() uses the
266     current tangent stiffness (week 12), so on a structure that has yielded it
267     reports the period of what is left, not of what was built.
268     """

```

```

269 rows = []
270 for s in scales:
271     out = run(s, p, ground)
272     got = summarise(out, p)
273     got["scale"] = s
274     got["normalised"] = got["peak_disp"] / s if s else 0.0
275     got["T_damaged"] = periods_of(1)[0]
276     rows.append(got)
277 return rows

```

scale	PGA [g]	peak	peak/scale	residual	shear kN	T after	
0.02	0.007	1.12	55.8	0.045	11.9	0.574	elastic
0.05	0.017	2.79	55.7	0.119	29.7	0.574	elastic
0.10	0.035	5.84	58.4	0.251	53.5	0.574	x1.05
0.25	0.087	33.61	134.4	1.567	114.8	0.685	x2.41
0.50	0.174	48.55	97.1	10.427	114.0	0.991	x1.74
1.00	0.347	59.83	59.8	10.118	114.0	1.050	x1.07
1.50	0.521	81.98	54.7	9.523	113.0	1.042	x0.98

elastic to 0.2% below $\lambda = 0.05$; $\times 2.41$ at $\lambda = 0.25$

Figure 13.2 plots both of the columns that matter.

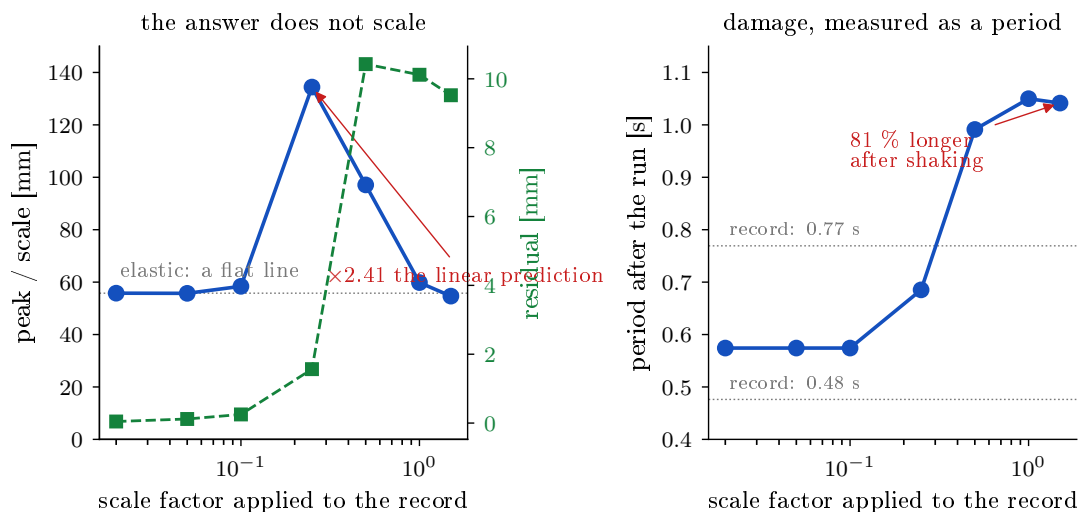


Figure 13.2: Left, the peak divided by the scale factor: a linear structure would give a flat line. Right, the period measured after each run.

Check two. At the two smallest amplitudes **peak/scale** is 55.8 and 55.7 — constant to 0.2% over a factor of 2.5 in amplitude. That is the elastic range, and note what has happened: it was *found*, not assumed. Nobody decided in advance where the specimen yields.

Check three, and the result of the chapter. It then leaves that range spectacularly. At $\lambda = 0.25$ the peak is 2.41 times what a linear structure would have given, and by $\lambda = 1.50$ it is back below the elastic value. *The response is not proportional to the input, and it is not even monotone.*

Two things confirm this is real rather than a numerical accident. Every one of the seven runs completed the full 20 s with no failed steps and no subdivisions. And the base shear saturates: 114.8, 114.0, 114.0, 113.0 kN across a factor of six in input. That plateau is the specimen's strength, measured four times over.

Example 13.3 (Damage, measured as a period). Measure the specimen's period after each run.

Solution

Week 12 noted that **eigen** uses the *current* tangent stiffness, and warned that on a damaged structure it therefore reports the period of what is left. Here that stops being a caveat and becomes an instrument.

$$T : 0.574 \rightarrow 1.042 \text{ s, a } 82\% \text{ lengthening}$$

Figure 13.3 shows the full-scale run in the three views worth having.

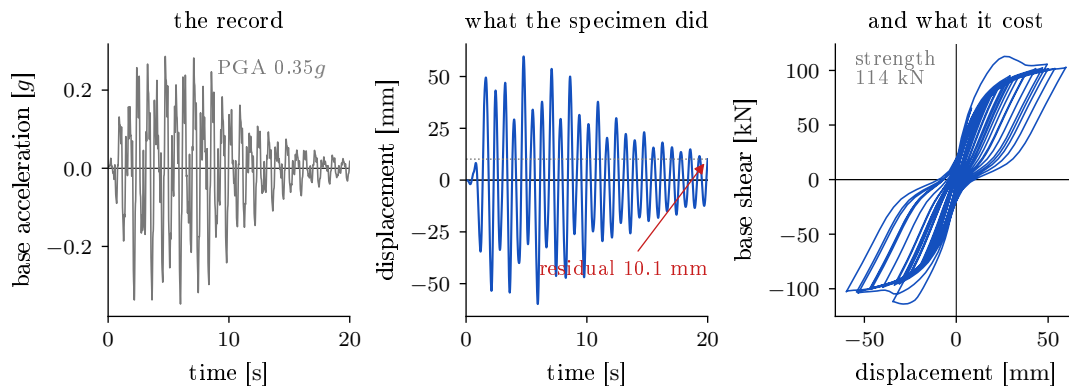


Figure 13.3: The record, the displacement of the mass, and the hysteresis loop at full scale. The specimen is left 10 mm out of plumb.

Check. The period lengthens *monotonically* with amplitude — 0.574, 0.574, 0.574, 0.685, 0.991, 1.050, 1.042 s — while the peak displacement does not. That makes it the better damage measure of the two, and it is also measurable on a real building with an accelerometer and no model at all.

It also explains the amplification. The record's strongest component sits at 0.77 s, and the specimen starts at 0.597 s and softens *upwards through it*. A structure that damages can walk into resonance rather than out of it, which is the opposite of the reassuring intuition that yielding protects a building by detuning it. That said, the figure does not prove the mechanism — exercise 13.4 tests it properly.

And the residual. At full scale the specimen ends 10.1 mm out of plumb. A linear analysis cannot produce a residual at all, and it is the quantity that decides whether a real building is repaired or demolished. It is also, notably, *not* monotone either: it peaks at 10.4 mm at $\lambda = 0.5$ and falls slightly thereafter.

Summary

Item	Command or expression	Note
the record	<code>timeSeries('Path', t, '-dt', dt, '-values', *a)</code>	mm/s ²
shake the base	<code>pattern('UniformExcitation', t, dir, '-accel', ts)</code>	acceleration, not force
transient step	<code>analyze(1, dt)</code>	sub-steps must sum to <code>dt</code>
velocity	<code>nodeVel(node, dof)</code>	for energy and for phase
damaged period	<code>eigen</code> after the run	current tangent stiffness
check 1	zero input $\Rightarrow$ zero response	catches a misdirected excitation
check 2	peak/ λ constant while elastic	locates the elastic range
check 3	where it stops being constant, it has yielded	a measurement, not an assumption
strength	the base-shear plateau	114 kN here
damage	period lengthening	monotone; the peak is not

Exercises

Exercise 13.1. ●●● The base shear saturates near 114 kN. Check that against the section: week 8 found a peak moment for this section, and the specimen is a 3.5 m cantilever.

Answer: $V = M/L$, so a base shear of 114 kN implies a moment of about 400 kN m — the right order for this section, though lower than week 8's 479 kN m because the axial load here is 883 kN rather than 1200 kN

Solution. $M = VL = 114 \times 3.5 = 399$ kN m.

Week 8 obtained 479 kN m at an axial load of 1200 kN. Here the axial load is the weight of the mass, $90 \times 9810 = 883$ kN, and exercise 8.1 showed that below the balance point moment capacity *increases* with compression. Less axial load, less moment capacity — and 399 kN m against 479 kN m is the right direction and a plausible size.

This is the closest thing to an external check the chapter has, and it is worth noticing what kind of check it is: not a closed form, but a result from an earlier chapter obtained by a different analysis on the same section. Consistency between two of your own results is weaker than agreement with theory and much better than nothing.

Exercise 13.2. ●●● Run the record at $\lambda = 0.03$ and $\lambda = 0.04$. Does **peak/scale** stay at 55.8, and what does the answer tell you about where the elastic limit is?

Answer: it does, to within a fraction of a percent; the elastic limit lies somewhere between $\lambda = 0.05$ and $\lambda = 0.10$, and the sweep locates it only to within that factor of two

Solution. The two new points fall on the same flat line, which confirms the elastic range rather than extending it.

The useful observation is about resolution. The sweep in [Example 13.2](#) used scale factors a factor of 2.5 apart, so it can only say that yielding begins somewhere between 0.05 and 0.10. Bisection would narrow it, at the cost of a run per bisection.

Whether that is worth doing depends on the question. For *verification* — confirming the model behaves linearly where it should — two points on a flat line are enough. For a *result*, such as the

ground motion intensity at which a column first yields, they are not, and you would bisect. Knowing which of the two you are doing decides how much computer time the answer deserves.

Exercise 13.3. ●●● **Example 13.3** claimed the amplification at $\lambda = 0.25$ happens because the softening specimen moves into resonance with the record's 0.77 s component. Design a test of that claim.

Answer: remove the 1.3 Hz component from the record, rescale to the same PGA, and re-run the sweep; if the explanation is right the amplification at $\lambda = 0.25$ should largely disappear, while the elastic range and the strength plateau should be unaffected

Solution. The claim is a causal one and the figure does not establish it — the period lengthening and the amplification could both be consequences of yielding without the resonance having anything to do with it.

The test is to remove the supposed cause and see whether the effect goes. Delete the 1.3 Hz term from **accelerogram**, rescale so the PGA is unchanged (otherwise you have changed two things at once, which week 10 warned about), and repeat the sweep. Two predictions follow: the amplification at $\lambda = 0.25$ should fall substantially, and the elastic peak/scale and the 114 kN plateau should not move at all, because neither depends on where the record's energy sits.

The second prediction is what makes this a good test rather than a fishing expedition. A test that can only confirm is not a test; one that predicts what should *not* change as well as what should is.

This is also the honest reason the chapter says the figure is 'consistent with' the mechanism rather than proving it. Writing that sentence carefully is not pedantry — it is the difference between a result and a story told about one.

Exercise 13.4. ●●● Neither the peak displacement nor the residual is monotone in the scale factor, but the period is. What does that suggest about which quantity to use when comparing an analysis with a shake-table test?

Answer: the period is the most robust comparison, because it depends on the accumulated damage rather than on the details of when peaks happened to coincide; peak and residual displacement are sensitive to the exact record and to small modelling differences, so disagreement in them is much weaker evidence that a model is wrong

Solution. Peak displacement is a single instant out of twenty seconds, and which instant it is depends on the phasing between a record and a structure whose period is changing as it damages. Small changes to either — a slightly different damping ratio, a slightly stiffer section — can move it a long way. Residual displacement is worse: it depends on the last few large cycles and on which direction the specimen happened to be leaning when the shaking stopped.

The period, and more generally the accumulated damage, integrates over the whole record. It is far less sensitive to phasing, and it is measurable on a real specimen with an accelerometer before and after.

The practical advice this yields is worth stating plainly. When a nonlinear time-history analysis disagrees with a test on peak displacement by 20 %, that is close to expected and is weak evidence of anything. When it disagrees on the period lengthening, or on the shape of the hysteresis loop, or on the strength plateau, something is genuinely wrong with the model. Choosing the comparison before running the analysis is what stops you from choosing it afterwards to suit the answer.

Week 14: Post-processing, Data Extraction and Visualization

Learning outcomes

After this week you should be able to:

1. reduce a time history to the scalars an engineer reports, and say what each one hides;
2. show when damage accumulated, not only how much;
3. explain why reconstructing a quantity the model already computed costs accuracy, and separate the two error sources involved;
4. write an intermediate data file that the next tool can safely difference;
5. choose between a plot and a 3D export.

Notation. u displacement $\dot{u}$ velocity V_b base shear W absorbed energy Δt time step

Week 13's analysis produced two thousand rows for every quantity, on every node, in every element, and then reported five numbers. This chapter is about doing that reduction deliberately rather than by habit.

There is no new mechanics here at all, and that is the point. Post-processing is where a correct analysis quietly becomes a wrong figure, and it earns a chapter rather than a paragraph at the end of one.

14.1 Five numbers, and what each hides

Listing 14.1: w14/postprocess.py — The scalars, and a comment on each saying what it does not tell you.

```

38 def reduce_history(out, p=SPECIMEN):
39     """The scalars a time history is usually reported as -- and what each hides.
40
41     peak          the largest excursion, in either direction. One instant out of
42                   two thousand, and the one most sensitive to phasing.
43     residual      where it finished. Decides repair or demolition, and cannot
44                   exist in a linear analysis.
45     peak_shear    what the foundation has to carry.
46     energy        the area swept in the base shear against displacement plane:
47                   everything the specimen absorbed instead of giving back.
48     cycles        how many times it crossed zero. A single large excursion and
49                   thirty small ones can share a peak and are not the same event.
50
51     """
52     u, v = out["u"], out["shear"]
53     crossings = sum(1 for i in range(1, len(u)) if u[i - 1] * u[i] < 0.0)
54     return {
55         "peak": max(abs(x) for x in u),
56         "peak_drift": max(abs(x) for x in u) / p["L"],
57         "residual": u[-1],
58         "peak_shear": max(abs(x) for x in v),
59         "energy": hysteretic_energy(u, v),
60         "half_cycles": crossings,
61     }
62
63 def hysteretic_energy(u, shear):
64     """Area swept in the force-displacement plane, by the trapezoid rule.
65
66     The same calculation as week 7's dissipated_energy, one level up: there it
67     integrated stress over strain for a material, here force over displacement
68     for a whole specimen. Units are N*mm = mJ.
69
70     """
71     total = 0.0
72     for i in range(1, len(u)):
73         total += 0.5 * (shear[i] + shear[i - 1]) * (u[i] - u[i - 1])
74     return abs(total)

```

Week 13's specimen, reduced		
peak displacement	59.83 mm	(1.71 % drift)
residual	10.12 mm	
peak base shear	114.0 kN	
energy absorbed	48.21 kN m	
zero crossings	51	

The last of those is the one usually left out. A peak of 59.8 mm could be one large excursion or it could be the largest of fifty; 51 zero crossings says which, and damage that accumulates cycle by cycle depends on the answer. The energy is the same trapezoid rule as week 7's material loop and week 11's capacity curve, one level up again — and a linear model would report it as zero, because an elastic loop encloses no area.

14.2 When, not only how much

Example 14.1 (The running maximum). When during the twenty seconds was the damage actually done?

Solution

Listing 14.2: `w14/postprocess.py` — A running maximum: three lines, and it answers a question the peak cannot.

```

78 def drift_envelope(out, n_bins=40):
79     """The running maximum: how the damage accumulated, not just its total.
80
81     A peak is a scalar and tells you nothing about when it happened. The
82     envelope shows whether the specimen was damaged early and then rode it out,
83     or crept up steadily -- two very different events with the same headline
84     number.
85     """
86     t, u = out["t"], out["u"]
87     step = max(1, len(t) // n_bins)
88     running, envelope = 0.0, []
89     for i in range(0, len(t), step):
90         running = max(running, max(abs(x) for x in u[max(0, i - step):i + 1]))
91         envelope.append((t[i], running))
92     return envelope

```

The left panel of [Fig. 14.1](#) is that envelope; its right panel belongs to [Example 14.2](#).

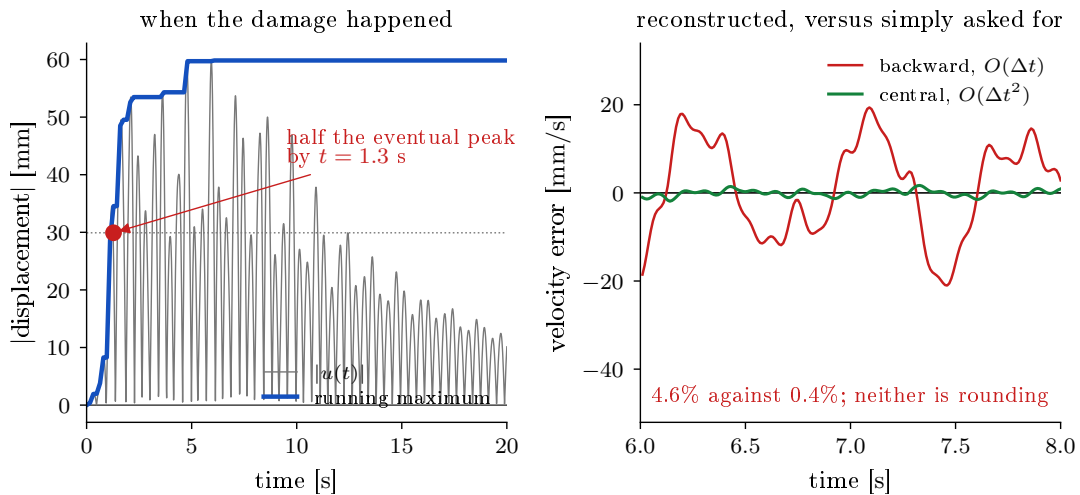


Figure 14.1: Left, when the damage happened. Right, what it costs to reconstruct a velocity instead of asking for it.

half the eventual peak was reached by $t = 1.3$ s of 20 s

Check. The envelope is a running maximum, so it can never decrease — which is worth asserting in the verification, because if it ever did the code would be wrong rather than the structure interesting. The result itself is the useful part. Half the eventual damage was done in the first 1.3 s, before the strong-motion phase had finished building. Two records with identical peak displacements can be entirely different events, and no scalar distinguishes them.

14.3 Reconstructing what you already had

Example 14.2 (The price of a difference). The velocity is available from `ops.nodeVel`. Suppose you kept only the displacement record and differenced it instead. What does that cost?

Solution

Listing 14.3: `w14/postprocess.py` — Two error sources, separated deliberately — because the obvious suspect is innocent.

```

97 def round_to(x, digits):
98     """Round to `digits` significant figures, as a recorder file would."""
99     if x == 0.0:
100         return 0.0
101     mag = math.floor(math.log10(abs(x)))
102     factor = 10.0 ** (digits - 1 - mag)
103     return round(x * factor) / factor
104
105
106 def velocity_error(out, digits=16):
107     """What it costs to RECONSTRUCT a quantity the model already computed.
108
109     The model carries velocity as a state variable; nodeVel returns it. It is
110     also tempting to take it from the displacement record by differencing,

```

```

111     especially when the record is all you kept -- and this function prices
112     that choice, three ways.
113
114     Two error sources are separated deliberately, because they behave quite
115     differently and the obvious suspect is not the culprit.
116
117     ROUNDING, from writing the record at finite precision: shrinks as `digits`
118     grows, exactly as you would expect.
119
120     TRUNCATION, from the difference formula itself: does NOT shrink with
121     precision at all. A backward difference is first-order accurate, so its
122     error is proportional to the time step, and at  $\Delta t = 0.01$  s that dominates
123     everything else. A central difference is second-order and roughly eleven
124     times better here for one extra line of code.
125
126     Neither is a reason to difference at all when nodeVel is available. This
127     exists to show what it costs when it is not.
128     """
129     t, u, v = out["t"], out["u"], out["v"]
130     u_file = [round_to(x, digits) for x in u]
131     peak = max(abs(x) for x in v)
132
133     backward = max(abs((u_file[i] - u_file[i - 1]) / (t[i] - t[i - 1]) - v[i])
134                    for i in range(1, len(t)) if t[i] > t[i - 1]))
135     central = max(abs((u_file[i + 1] - u_file[i - 1]) / (t[i + 1] - t[i - 1])
136                    - v[i])
137                    for i in range(1, len(t) - 1) if t[i + 1] > t[i - 1]))
138
139     return {
140         "worst_disp_error": max(abs(a - b) for a, b in zip(u, u_file)),
141         "backward": backward,
142         "central": central,
143         "backward_fraction": backward / peak,
144         "central_fraction": central / peak,
145         "peak_velocity": peak,
146     }

```

Reconstructing the velocity instead of asking for it

digits	disp err	backward	central
4	5.00e-03	4.88%	0.42%
6	4.99e-05	4.79%	0.42%
8	4.99e-07	4.79%	0.42%
12	4.99e-11	4.79%	0.42%

4.8 % error, and it is *not* the rounding

Read the columns against each other. The displacement error falls by eight orders of magnitude from four significant figures to twelve, exactly as it should. The differencing error does not move at all. So the rounding is innocent. What dominates is the difference *formula*: a backward difference is first-order accurate, so its error is proportional to Δt , and at $\Delta t = 0.01$ s that swamps everything else. A central difference is second-order and eleven times better here, for one extra line.

Note

write the file at higher precision → do not reconstruct what the model already has

Rounding is the obvious suspect, and week 3's -precision warning is what makes it obvious. The sweep above is what rules it out: the suspect does not respond when it is varied.

The lesson is not about precision at all. nodeVel returns the velocity Newmark carried as a state variable: exact, free, and requiring no formula. Reconstructing it is a choice, and 4.8 % is its price.

Check. That the error is unchanged from six digits to twelve, while the displacement error falls by 10^6 , is what makes this a controlled experiment rather than a guess. One thing was varied; the suspect responded and the symptom did not.

14.4 Writing a file the next tool can trust

Listing 14.4: w14/postprocess.py — repr, not a format string.

```

151 def write_csv(out, path=None):
152     """One tidy table, one row per step, ready for anything else.
153
154     Comma-separated, one header line, full precision. Not because anybody
155     reads sixteen digits, but because this file is an INTERMEDIATE -- whatever
156     reads it next may difference it, and week 3 explained what that costs.
157     """
158     path = path or (OUT / "response.csv")
159     path.parent.mkdir(parents=True, exist_ok=True)
160     with path.open("w", encoding="utf-8") as f:
161         f.write("time_s,disp_mm,vel_mm_s,base_shear_N\n")
162         for t, u, v, s in zip(out["t"], out["u"], out["v"], out["shear"]):
163             f.write(f"{t!r},{u!r},{v!r},{s!r}\n")
164     return path

```

Full precision in an intermediate file is not fussiness — it is [Example 14.2](#) taken seriously. Nobody reads sixteen digits, but the next program might difference them, and the digits you round away are exactly the ones a subtraction leaves behind.

Choosing how to present a result

1. A **scalar** when a single number decides something: a drift against a limit.
2. A **plot** when the shape carries the meaning: a hysteresis loop, a capacity curve, an envelope. Two dimensions, and label them.
3. **opsvis** when the answer is *where* rather than *how much* — a mechanism, a mode shape, a deformed shape. Week 11.
4. **ParaView**, through **opstool**'s VTK export, when the model is genuinely three-dimensional and the quantity varies over it: fibre-level stress in a 3D frame, or an animation of a collapse.
5. And a **table** when somebody has to reproduce your numbers.

The step from 3 to 4 is worth being deliberate about. A matplotlib figure is reproducible, goes in a report, and is read the same way by everyone. A ParaView session is none of those things, and it is the only way to see a field on a solid. Reach for the heavier tool when the lighter one has actually failed, not before.

Summary

Item	Command or expression	Note
peak, residual	<code>max(abs(u)), u[-1]</code>	residual needs nonlinearity
drift	peak / height	what a limit is written on
absorbed energy	$\oint V_b du$	zero in a linear model
cycle count	sign changes in u	one excursion or fifty?
envelope	running maximum of $ u $	when, not how much
velocity	<code>nodeVel(node, dof)</code>	exact; do not difference
backward diff.	$O(\Delta t)$, 4.8 % here	Example 14.2
central diff.	$O(\Delta t^2)$, 0.4 % here	one extra line
intermediate file	<code>repr()</code> , full precision	the next tool may difference it

Exercises

Exercise 14.1. ●●● The specimen absorbed 48.2 kN m. Check that against the peak base shear and the peak displacement, and say why the two do not simply multiply.

Answer: $114 \times 0.0598 = 6.8$ kN m would be the area of one triangular excursion to the peak; the actual figure is seven times that because the specimen went round some fifty half-cycles, each enclosing area

Solution. A single monotonic push to the peak would sweep roughly $\frac{1}{2}V_b u_{\max} = \frac{1}{2}(114)(0.0598) = 3.4$ kN m, or twice that for a full-height rectangle. The measured 48.2 kN m is an order of magnitude more.

The difference is the 51 zero crossings. Energy accumulates over every cycle, and a long moderate record can absorb far more than a single large pulse that reaches the same peak. This is the quantitative form of the point week 7 made with hysteresis loops, and it is why an energy-based damage measure and a displacement-based one can disagree about which of two records was worse.

Exercise 14.2. ●●● Halve the time step of the week 13 analysis and repeat [Example 14.2](#). Predict what happens to each of the three error columns before running it.

Answer: the displacement rounding error is unchanged, the backward-difference error roughly halves because it is $O(\Delta t)$, and the central-difference error falls by about four because it is $O(\Delta t^2)$

Solution. Rounding depends on how many digits are written, not on how often, so that column does not move. The two differencing errors do, and by different factors — which is the practical way to confirm the order of a scheme without any analysis at all.

Note what this exercise really tests: whether you can tell an error that depends on Δt from one that does not. That distinction is the whole content of [Example 14.2](#), and halving the step is the cheapest way to make it visible.

Exercise 14.3. ●●● You are given a recorder file of displacements from someone else's analysis, at $\Delta t = 0.02$ s and six significant figures, and asked for the peak velocity. What do you report, and what do you say alongside it?

Answer: report the central-difference estimate, and state the time step, the difference scheme and an estimate of the truncation error — because at 0.02 s the second-order error is roughly four times

the 0.4% found here, and the number is not reliable beyond about 2%

Solution. The central difference is the right choice: it costs nothing over the backward one and is an order of magnitude better. The rounding is irrelevant, as [Example 14.2](#) established.

What matters more is the sentence you attach. A peak velocity quoted bare implies a precision it does not have. Quoting it with the scheme and the step lets a reader judge it, and it invites the obvious better answer — ask whoever ran the analysis to record `nodeVel` and re-run, which costs them one line and removes the error entirely.

The general habit is the one this whole course has been circling: a number is not a result until you can say how it was obtained and how wrong it might be. That applies to a post-processed velocity exactly as much as to a capacity curve or a fibre count.

Exercise 14.4. ● ● ● [Example 14.1](#) showed that half the damage happened in the first 1.3s. Design a single figure for a client who is not an engineer, conveying what happened to this specimen. Say what you would leave out.

Answer: one panel: displacement against time with the residual marked and a drift limit drawn as a horizontal band, annotated in plain words; leave out the hysteresis loop, the base shear, the energy and the period, because each requires a paragraph of explanation to mean anything

Solution. The client's question is almost certainly *is the building usable?* — so the figure has to answer that, not display the analysis.

Displacement against time is the one plot that needs no training to read: it went this far, it wobbled, it stopped here. Marking the residual and the drift limit turns it into a verdict. Everything else in this chapter is diagnostic — essential for deciding whether *you* believe the analysis, and noise to somebody who is deciding whether to reoccupy a building.

The temptation to include the hysteresis loop should be resisted specifically. It is the most informative picture in the whole course to an engineer and completely opaque to anyone else, and a figure that impresses rather than informs has failed at the only job it had.

That the last exercise of the course is about audience rather than mechanics is deliberate. Every technique in these fourteen weeks exists to support a decision somebody else has to make.

S1 — Debugging OpenSees

Learning outcomes

After working through this chapter you should be able to:

1. sort a failure into one of three kinds — it refuses, it stops, it lies — and know which of the three costs the most;
2. read an `analyze()` return code and act on it, rather than ignoring it;
3. recognise the four failures that produce no message at all, and say what each one does to a result;
4. audit a domain before analysing it, and detect a mechanism in one line;
5. cut a broken model down to the smallest one that still fails.

Notation. λ_1 smallest eigenvalue T natural period Δu displacement increment in one iteration

Every previous chapter has shown you analyses that work. This one is about the other case, and it is placed in Part II not because it is peripheral but because it does not belong to any single week — you will need it in week 2 and still be using it in week 14.

Every failure catalogued here occurs on the versions pinned in `requirements.txt`, and every one of them can be reproduced by running `code/s1/diagnose.py`.

S1.1 Three kinds of failure

Definition S1.1: The three kinds

It refuses. An exception, or a **WARNING** line followed by nothing useful. The command that failed is named.

It stops. `analyze()` returns a negative number and the loop ends somewhere the structure has not. Week 10 is entirely about this one.

It lies. The script runs to the end, prints numbers, draws a figure, and every one of them is wrong. Nothing whatever is printed to say so.

They are listed in increasing order of danger, and the ordering is not close. A refusal costs minutes: the message names the command, and [Table S1.1](#) probably names the cause. The third kind has cost people papers.

The asymmetry is worth being explicit about, because it inverts the intuition students bring from other programming. In ordinary code an exception is bad news. In OpenSees an exception is the *good* outcome — it is the failure mode that tells you about itself. What you should fear is a script that finishes quietly.

Note

This is also why the habit from week 3 — check every number against something you already know — is not a nicety. It is the only defence that works against the third kind, because by construction no diagnostic will fire.

S1.2 When it stops: reading the return code

`analyze()` does not raise. It *returns* a number, and if you do not look at it, a failed step is indistinguishable from a converged one:

Listing S1.1: `s1/diagnose.py` — What the return values mean. Six of them, and only one is good news.

```
128 # What analyze() means when it does not return 0. It returns a code; it does
129 # not raise. `if ops.analyze(1) < 0` is therefore not optional -- without it a
130 # failed step is indistinguishable from a converged one, and the loop happily
131 # carries on integrating a structure that never came into equilibrium.
132 ANALYZE_CODES = {
133     0: "converged",
```

```

134     -1: "the algorithm failed -- residual never fell below the test tolerance",
135     -2: "the integrator failed -- often a step size the state cannot support",
136     -3: "the system of equations failed -- singular or ill-conditioned matrix",
137     -4: "the constraint handler failed",
138     -5: "the analysis could not be created or the domain changed under it",
139 }
140
141
142 def explain(code):
143     """Turn an analyze() return value into something actionable."""
144     return ANALYZE_CODES.get(code, f"undocumented return code {code}")

```

The consequence of ignoring this is worse than losing a step. A transient loop that carries on after a failure is integrating a structure that never came into equilibrium, and it will keep doing so for the remaining two thousand steps. So the guard is not optional:

Never call analyze bare

Write `if ops.analyze(1) < 0:` and decide what happens next — subdivide, change algorithm, or stop and report. Week 10's `robust_step` is that decision written down once. What you must not do is call it as a statement and move on.

S1.3 When it lies: four failures with no message

This is the substance of the chapter. Each of the four below is run twice in `code/s1/diagnose.py` — once wrong, once right — so that the cost is a number rather than a warning.

S1.3.1 The reactions are all exactly zero

Listing S1.2: `s1/diagnose.py` — Reactions are not computed until you ask for them.

```

171 def reactions_forbidden():
172     """The classic silent zero: reactions are not computed until you ask.
173
174     ops.reactions() runs the restoring-force recovery. Skip it and every
175     nodeReaction call returns exactly 0.0 -- not an error, not a warning, just
176     a beautifully formatted table of zeros that fails equilibrium by 100 %.
177     """
178     weight = _cantilever()
179     ops.timeSeries('Linear', 1)
180     ops.pattern('Plain', 1, 1)
181     ops.load(2, 0.0, -weight, 0.0)
182     ops.system('BandGeneral')
183     ops.numberer('RCM')
184     ops.constraints('Plain')
185     ops.integrator('LoadControl', 1.0)
186     ops.algorithm('Linear')
187     ops.analysis('Static')
188     ops.analyze(1)
189
190     without = ops.nodeReaction(1, 2)           # before reactions() -- zero
191     ops.reactions()
192     with_call = ops.nodeReaction(1, 2)         # after -- the real value
193     return without, with_call, weight

```

```

1. reactions() not called
   before the call      0.0 N
   after the call      50000.0 N   (applied -50000.0 N)

```

`nodeReaction` reads a value that `ops.reactions()` computes. Skip the call and every reaction is exactly 0.0 — not small, not noisy, exactly zero.

$$R_{\text{before}} = 0.0 \text{ exactly}, \quad R_{\text{after}} = 50.0 \text{ kN}$$

An exact zero is a missing call. A small non-zero number is a real out-of-balance force. The difference matters: the first is fixed with one line, the second means your model does not satisfy equilibrium and no line of code will help.

S1.3.2 Mass in kilograms

Week 0 declared N, mm, s, which makes the mass unit $\text{N s}^2/\text{mm}$ — a tonne. Dividing a weight in newtons by 9.81 instead of 9810 gives a mass a thousand times too large.

Listing S1.3: `s1/diagnose.py` — The single most expensive typo in the course.

```

198 def mass_in_kilograms():
199     """Mass in kg inside an N-mm-s model: every period wrong by the same factor.
200
201     In N, mm, s the mass unit is N*s^2/mm, which is a tonne. Dividing a weight
202     in newtons by 9.81 instead of 9810 gives a number a thousand times too
203     large, and since T goes as the square root of mass, every period comes out
204     sqrt(1000) = 31.6 times too long.
205
206     Nothing warns. The model runs, the mode shapes are perfect, and a building
207     that should have T = 0.5 s reports 15.8 s -- which, if you do not know the
208     answer already, looks like a very flexible building rather than a bug.
209     """
210     def period(unit):
211         _cantilever(mass_unit=unit)
212         w2 = ops.eigen('-fullGenLapack', 1)[0]
213         return 2.0 * math.pi / math.sqrt(w2)
214
215     right = period("tonne")
216     wrong = period("kg")
217     return right, wrong, wrong / right

```

```

2. mass entered in kg instead of tonnes
   correct T = 0.2379 s
   in kg    T = 7.5228 s    (31.62 times too long)

```

Since $T \propto \sqrt{m}$, every period is out by $\sqrt{1000} = 31.62$, and every period is out by the *same* factor — so the mode shapes are perfect, the ratios between modes are perfect, and nothing looks wrong internally.

Note

Note what makes this dangerous: 7.52s is a perfectly plausible period for *something*. It is only wrong because we happen to know this is a 3m cantilever. A number is only checkable against an expectation, which is why week 12 insists on estimating the period by hand before running the eigenvalue analysis.

S1.3.3 A second model built on the first

Listing S1.4: `s1/diagnose.py` — One global domain, and `model()` does not clear it.

```

222 def domain_contamination():
223     """No wipe(): the second model is built on top of the first one.
224
225     OpenSees keeps ONE global domain. Building a second model without wiping
226     does not replace the first -- it adds to it. In a notebook, where cells are
227     re-run out of order, this is the single most common source of results that
228     change between runs of identical code.
229
230     The tell is the node count. The symptom is a period that moves every time
231     you press run.

```

```

232     """
233     _cantilever()
234     first = len(ops.getNodeTags())
235
236     # A second cantilever, deliberately without wipe(), with fresh tags so
237     # nothing collides and nothing complains.
238     ops.node(3, 5000.0, 0.0)
239     ops.node(4, 5000.0, 3000.0)
240     ops.fix(3, 1, 1, 1)
241     ops.mass(4, 5.0, 5.0, 0.0)
242     ops.element('elasticBeamColumn', 2, 3, 4, 8.0e3, 200000.0, 1.6e8, 1)
243     after = len(ops.getNodeTags())
244
245     return first, after

```

```

3. a second model built without wipe()
   nodes after the first model  2
   nodes after the second      4   (not 2)

```

Week 2 introduced the global domain, and `ops.wipe()` at the top of every model function is K18 for exactly this reason. In a notebook, where cells are re-run out of order, this is the single most common source of results that change between runs of identical code. The tell is that running the same cell twice gives two different answers.

S1.3.4 The tolerance that stopped the complaints

The natural response to a model that will not converge is to loosen the tolerance until it stops complaining. Before reading on, predict what that does to the answer. Almost everyone predicts the same thing, and almost everyone is wrong.

Listing S1.5: `s1/diagnose.py` — A bar pulled past yield twice: once with $\text{tol} = 10^{-10}$, once with 200, which is larger than any increment in the run and so is accepted after a single iteration every time.

```

250 def _pull_truss(tol, steps=20):
251     """A bar pulled well past yield in `steps` load increments. Returns the path.
252
253     Steel02 rather than Steel01 so the tangent varies smoothly through yield;
254     with a bilinear material the transition occupies a single step and there is
255     much less for a bad tolerance to get wrong.
256     """
257     ops.wipe()
258     ops.model('basic', '-ndm', 1, '-ndf', 1)
259     ops.node(1, 0.0)
260     ops.node(2, 1000.0)
261     ops.fix(1, 1)
262     ops.uniaxialMaterial('Steel02', 1, 420.0, 200000.0, 0.01, 18.0, 0.925, 0.15)
263     ops.element('truss', 1, 1, 2, 500.0, 1)
264
265     ops.timeSeries('Linear', 1)
266     ops.pattern('Plain', 1, 1)
267     ops.load(2, 400.0e3) # well past the 210 kN yield load
268     ops.system('BandGeneral')
269     ops.numberer('RCM')
270     ops.constraints('Plain')
271     ops.test('NormDispIncr', tol, 20)
272     ops.algorithm('Newton')
273     ops.integrator('LoadControl', 1.0 / steps)
274     ops.analysis('Static')
275
276     path, failures = [], 0
277     for _ in range(steps):
278         if ops.analyze(1) < 0:
279             failures += 1

```

```

280     path.append(ops.nodeDisp(2, 1))
281     return path, failures
282
283
284 # --8<-- [start:tolerance]
285 def tolerance_too_loose(loose=200.0, tight=1.0e-10, steps=20):
286     """A tolerance so loose that "converged" stops meaning anything -- and what
287     that actually costs, which is not what you would guess.
288
289     test('NormDispIncr', tol, iters) accepts the step when the displacement
290     increment between two iterations falls below `tol`. Set `tol` large enough
291     and the first iteration always passes: every step reports success and the
292     answer is whatever one tangent step happened to give.
293
294     The obvious expectation is that the final answer is then wrong. It is not.
295     Pulled monotonically past yield, the loose run finishes on exactly the same
296     displacement as the tight one, to every printed digit.
297
298     The reason is worth understanding, because it is what makes this failure
299     hard to catch: the force left unbalanced by a lazy step is not discarded.
300     It stays in the residual and turns up in the NEXT step, so under a
301     monotonic load with a hardening material the errors are repaid and the
302     endpoint recovers.
303
304     What does not recover is the PATH. Through the two steps where the bar
305     actually yields, the loose run is out by more than eighty per cent -- it
306     simply has not noticed yet. So anything path-dependent is wrong: which step
307     yielded, the energy under the curve, the peak of a cycle, the state a
308     transient analysis carries into its inertia terms, where the unbalance has
309     nowhere to be repaid from.
310
311     Which is the real lesson. Loosening the tolerance until the complaints stop
312     does not fix the failure and does not always corrupt the headline number.
313     It relocates the damage to the part of the answer nobody checks.
314     """
315     tight_path, _ = _pull_truss(tight, steps)
316     loose_path, failures = _pull_truss(loose, steps)
317
318     err = [abs(a - b) / max(abs(a), 1.0e-9)
319           for a, b in zip(tight_path, loose_path)]
320     worst = max(range(len(err)), key=err.__getitem__)
321     return {
322         "tight": tight_path,
323         "loose": loose_path,
324         "final_error": err[-1],
325         "worst_error": err[worst],
326         "worst_step": worst + 1,
327         "reported_failures": failures,
328     }

```

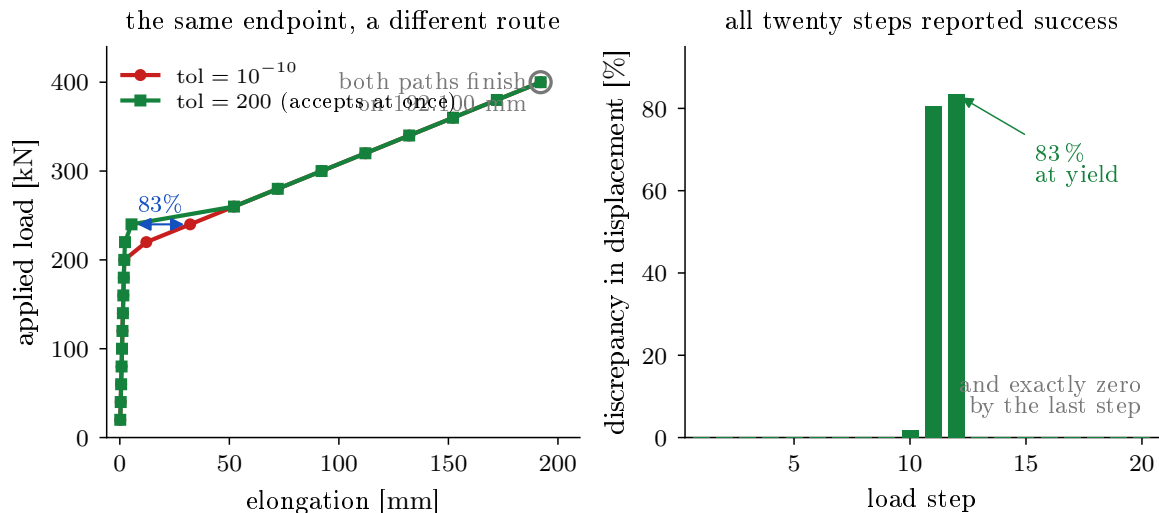


Figure S1.1: Both runs report success at all twenty steps and both finish on exactly 192.100 mm. The discrepancy is not spread thinly across the run: it is zero for ten steps, 83 % for two, and zero again.

```
4. a convergence tolerance chosen to stop the complaints
   steps reported as failed      0
   error in the final answer     0.00%
   worst error along the way     83.39% (step 12, where the bar yields)
```

Figure S1.1 is the whole finding in one picture, and it is not the picture this section was drawn to contain.

final answer 0.00 % out, worst point on the path 83.4 % out

A loose tolerance did *not* corrupt the final displacement. The unbalanced force a lazy step leaves behind is not discarded — it stays in the residual and reappears in the next step, so under a monotonic load with a hardening material the debt is repaid and the endpoint recovers exactly.

What does not recover is the *path*. Through the two steps where the bar yields, the loose run is out by 83%.

Check what you compare

The usual short account of convergence testing says that a loose tolerance gives a wrong answer. The experiment above does not support it:

a loose tolerance gives a wrong answer → a loose tolerance relocates the damage to the part of the answer nobody checks

Two variations are worth running before you accept that. Repeat it with a bilinear material and the two runs agree exactly, because on a piecewise-linear material Newton reaches the answer in one iteration whatever the tolerance is. Loosen the tolerance a hundredfold further and they still agree. The disagreement is only visible when the whole path is compared instead of the endpoint, which is the actual lesson: a check that looks only at the final number cannot see this class of error at all.

That relocation is the point. Ask what your result depends on:

If you report...	a loose tolerance...
the final displacement of a monotonic push	is probably repaid
the load at which the first hinge formed	is wrong, and quietly
the area under a hysteresis loop	is wrong at every reversal
anything from a transient analysis	is wrong and stays wrong

The last row is the one to remember. In a transient analysis the unbalanced force does not wait politely in the residual — it goes into the inertia term and becomes part of the state that the next step integrates from. There is nothing to repay it.

S1.4 Before you analyse: two cheap checks

Both of these run in less than a second and both catch things that would otherwise present as a solver failure with a misleading message.

S1.4.1 Auditing the domain

Listing S1.6: `s1/diagnose.py` — Four questions to ask a model before trusting it.

```

35 def audit(dynamic=False):
36     """A health check on whatever is in the domain RIGHT NOW.
37
38     Call it after building and before analysing. Every check here corresponds
39     to a mistake that OpenSees itself will not report -- it will either fail
40     with a message that names the wrong culprit, or succeed and be wrong.
41
42     Returns a list of complaints; an empty list is the good case.
43     """
44     problems = []
45     nodes = ops.getNodeTags()
46     eles = ops.getEleTags()
47
48     if not nodes:
49         return ["the domain is empty -- did the model get built at all?"]
50     if not eles:
51         problems.append("nodes exist but no elements: nothing connects them")
52
53     # A node no element refers to. Usually a typo in an element's node list,
54     # and the giveaway is that the model looks right in a plot.
55     used = set()
56     for e in eles:
57         used.update(ops.eleNodes(e))
58     orphans = [n for n in nodes if n not in used]
59     if orphans:
60         problems.append(f"nodes attached to no element: {orphans}")
61
62     # Duplicate coordinates: two nodes at the same point are not connected to
63     # each other, and the structure has a hidden hinge that no plot will show.
64     seen = {}
65     for n in nodes:
66         key = tuple(round(c, 6) for c in ops.nodeCoord(n))
67         seen.setdefault(key, []).append(n)
68     doubled = [v for v in seen.values() if len(v) > 1]
69     if doubled:
70         problems.append(f"nodes sharing a location: {doubled} -- intentional "
71                         "only if you meant a zeroLength element there")
72
73     # For a transient run, a free DOF with no mass makes the mass matrix

```

```

74 # singular. OpenSees will report a solver failure and blame the solver.
75 if dynamic:
76     massless = []
77     for n in nodes:
78         m = ops.nodeMass(n)
79         free = [i for i in range(len(m))
80                 if not _is_fixed(n, i + 1)]
81         if free and all(m[i] == 0.0 for i in free):
82             massless.append(n)
83     if massless:
84         problems.append(f"free but massless nodes: {massless} -- a "
85                         "transient analysis needs mass on every free DOF")
86
87     return problems
88
89
90 def _is_fixed(node, dof):
91     """True if `dof` (1-based) of `node` carries a single-point constraint."""
92     try:
93         return dof in ops.getFixedDOFs(node)
94     except Exception:
95         # older builds lack the query
96         return False

```

The two that earn their place most often are the orphan node and the coincident pair. A typo in an element's node list leaves a node attached to nothing; the model still *plots* correctly, because a plot draws the nodes you defined, not the ones the elements use. Two nodes at the same coordinate are likewise invisible in a plot and are not connected to each other — the structure has a hinge you did not put there.

S1.4.2 Is it a mechanism?

Listing S1.7: `s1/diagnose.py` — One line, and it names the failure the solver would only describe as a singular matrix.

```

100 def mechanism_check(n=3):
101     """Is the structure a mechanism? Ask for eigenvalues and read the smallest.
102
103     A properly restrained structure has a positive-definite stiffness matrix,
104     so every eigenvalue is comfortably positive. Leave one restraint off and
105     the structure can move without straining: that motion has zero stiffness,
106     and the eigenvalue collapses towards zero.
107
108     This is the cheapest possible test for the single most common modelling
109     error, and it costs one line. It is also far more informative than the
110     solver's own message, which typically says the matrix is singular without
111     saying which direction is free.
112     """
113     try:
114         vals = ops.eigen('-fullGenLapack', n)
115     except Exception as exc:
116         # a hard singularity throws
117         return None, f"eigen failed outright: {exc}"
118
119     smallest = min(vals)
120     if smallest < 1.0e-8:
121         return smallest, ("a zero eigenvalue: the structure is a mechanism. "
122                         "Something is unrestrained, or two elements meet at "
123                         "coincident but unconnected nodes")
124     return smallest, "restrained: the smallest eigenvalue is positive"

```

```

base released:  smallest eigenvalue 0.000e+00
base fixed:     smallest eigenvalue 6.976e+02

```

A properly restrained structure has a positive-definite stiffness matrix, so every eigenvalue is comfortably

positive. Leave a restraint off and the structure can move without straining: that motion costs no energy, and its eigenvalue collapses to zero.

Note

This is the same **eigen** call as week 12, used for an entirely different purpose. There it gave you periods; here the only thing that matters is whether the smallest value is zero. Note the **-fullGenLapack** flag — week 12 explains why the default Arpack solver fails on small models, which is exactly the size of model you will be debugging.

S1.5 When none of that helps: cut the model down

Bisecting a model

When you have no idea which part is wrong, stop reading the model and start shrinking it.

1. Replace every nonlinear material with **Elastic**. If it now runs, the problem is material, not model.
2. Replace **forceBeamColumn** with **elasticBeamColumn**. If it now runs, the problem is the section or the integration.
3. Delete half the structure and restrain the cut. Repeat until it runs.
4. Reduce the load until it runs, then double it until it does not. The last surviving load is where the state goes bad.

Each step either fixes it — in which case you have found the layer — or does not, in which case you have eliminated one. Four steps eliminate a lot.

The single most common outcome of this procedure is that the model runs fine and the fault is in the analysis object rather than the structure. That is worth knowing in advance, because it is not where anybody looks first.

S1.6 The message index

Table S1.1: Messages you are most likely to meet, and what each one actually meant.

What OpenSees says	What it usually means
WARNING torsion not specified for ↪ FiberSection	A fibre section used in a 3D model needs -GJ . Week 11.
Could not build an Arnoldi ↪ factorization	The default eigen solver on a model with too few mass degrees of freedom. Use -fullGenLapack . Week 12.
WARNING invalid int inputs on an element	A forceBeamColumn without a named beamIntegration object.
node already constrained	fix() called twice on the same node — often a loop that includes the support.
WARNING CTestNormDispIncr failed to ↪ converge	A real non-convergence. Week 10, not this chapter.
analyze returns -3	Singular matrix. Run the mechanism check before assuming it is the solver.
Nothing at all, and the reactions are zero	ops.reactions() was never called.
Nothing at all, and every period is 31.6× too long	Mass in kilograms.
Nothing at all, and the answer changes between identical runs	A missing ops.wipe() .

S1.7 Summary

Kind	How you find out	Typical cost
it refuses	a message naming the command	minutes
it stops	a negative <code>analyze()</code> code, if you look	hours
it lies	only by checking against something you know	the whole result

Exercise S1.1. `ops.reactions()` is called in `reactions_forgotten` after the analysis. Move it to before `ops.analyze(1)` and report the reaction. Explain the number you get.

Answer: Zero: `reactions()` computes from the current state, and before the analysis that state is the undeformed, unloaded one.

Solution. The call is not a switch that turns reactions on; it performs the restoring-force recovery using the state the domain is in at that moment. Called first, it dutifully recovers the reactions of an unloaded structure. This is a good illustration of the general rule that in OpenSees the order of commands is the program — there is no declaration phase separate from an execution phase.

Exercise S1.2. The audit in `diagnose.py` does not check for elements of zero length, or for an `elasticBeamColumn` with a zero or negative area. Add one of these checks and demonstrate it firing on a deliberately broken model.

Answer: Any check that returns a complaint on the broken model and none on a healthy one.

Solution. For zero length, compare `ops.nodeCoord` of the two ends returned by `ops.eleNodes(tag)`; a zero separation is legitimate only for a `zeroLength` element, so the complaint should say so rather than assert an error. Section properties are harder — they are not queryable after the fact — which is itself the lesson: validate the dictionary you build the model from, because the domain will not give it back to you.

Exercise S1.3 (find the bug). A student's script builds a 2D frame, runs a static analysis, and reports a base shear of exactly 0. The model plots correctly and `analyze()` returned 0. Give three candidate causes, in the order you would check them.

Answer: (i) `ops.reactions()` not called; (ii) the load applied in a pattern that was never activated, or applied to the wrong node; (iii) the base shear summed over the wrong nodes.

Solution. Order matters here and the ordering is by cost, not by likelihood: check (i) first because it takes one line, and it produces an *exact* zero, which is what was reported. A load on the wrong node would generally give a non-zero shear somewhere; summing the wrong nodes would give zero only if all of them were free. The exactness of the reported zero is the strongest clue available, and [Definition S1.1](#) is what tells you to notice it.

Exercise S1.4. Run `tolerance_too_loose` with `steps=200` instead of 20. Does the worst path error grow, shrink, or stay the same? Explain the result before you run it, then check.

Answer: It shrinks substantially: with a smaller load increment, one tangent step is a much better approximation over the increment.

Solution. The prediction most people make is that it stays the same, on the grounds that the tolerance has not changed. But the error of a single Newton step is governed by how far the tangent has to be extrapolated, and that is set by the increment size, not the tolerance. This is worth internalising because it explains a common and confusing observation: refining the step size makes a badly converged analysis look better without fixing anything about its convergence.

Exercise S1.5. Repeat the tolerance experiment under a *cyclic* load — push to 150 mm, back to −150 mm, and return to zero — using displacement control. Compare the dissipated energy (week 7’s trapezoid rule) between the tight and loose runs.

Answer: The energies differ; unlike the monotonic case, the endpoint no longer agrees either.

Solution. This is the experiment that settles [Section S1.3.4](#) in a single run. Under reversal, the material’s state at the point of unloading determines the whole subsequent branch, so an error made at a reversal is not repaid — it is inherited. The monotonic hardening case is the most forgiving one there is, and it is unfortunately also the one everybody tests on.

Exercise S1.6. Take any model from weeks 11–13, delete one `fix()` call, and run `mechanism_check`. Then run the full analysis and record what OpenSees says instead. Which message would have been quicker to act on?

Answer: The eigenvalue collapses to zero; the analysis reports a solver or convergence failure that does not mention restraints.

Solution. The point of the comparison is the distance between the symptom and the cause. The solver reports the property it observed — a matrix it cannot factor — which is true and nearly useless. The mechanism check reports the same fact one level up, where it can be acted on. Most of debugging is building that second layer for yourself, because the first one is written by people who cannot know what you were trying to model.

S2 — OpenSeesPy Command Reference

Learning outcomes

After working through this chapter you should be able to:

1. state the nine stages a model is built in, and place any command in one of them;
2. explain why the *order* of commands is the program, and what goes wrong when it is disturbed;
3. find the argument list your build expects, rather than the one the manual documents;
4. recognise the whole course in about fifty commands.

The OpenSeesPy interface has a few hundred functions. Listing all of them is what the online manual does, and it does it well. What a manual cannot give you is the much shorter list that is actually enough — and, more usefully, the *order* those commands have to be called in.

S2.1 Order is the program

Definition S2.1: No declaration phase

OpenSees has no separate declaration and execution phases. Every command acts on the domain the moment it is called. A material defined after the element that uses it does not exist as far as that element is concerned, and no later command will repair the situation.

That single property explains a large share of beginner errors, and it is why this reference is organised by *when* rather than by *what*. The nine stages below are a build order, and following them in sequence is the safest way to write a model.

The nine stages

1. **Start clean** — wipe, model
2. **Geometry** — node, fix, equalDOF, mass
3. **Materials and sections** — uniaxialMaterial, nDMaterial, section, patch, layer, beamIntegration, geomTransf
4. **Elements** — element
5. **Loads** — timeSeries, pattern, load, eleLoad, sp
6. **The analysis object** — system, numberer, constraints, test, algorithm, integrator, analysis
7. **Run it** — analyze, eigen, loadConst, rayleigh
8. **Ask for results** — reactions, nodeDisp, eleResponse, recorder
9. **Test a material alone** — testUniaxialMaterial, setStrain, getStress

Stages 1–4 build a structure; 5–7 ask it a question; 8 reads the answer. Stage 9 is a side entrance used only in weeks 6 and 7.

S2.2 Counting the reference instead of writing it

A reference typed out by hand goes stale the first time the code changes, and nothing announces it. So the tables here are not typed. `code/s2/commands.py` reads every script under `code/` and records every `ops.*` call it finds:

Listing S2.1: `s2/commands.py` — Parsed, not pattern-matched.

```

30 def scan(root=CODE):
31     """Every ops.<name>( call in every script, with the weeks it appears in.
32
33     Parsed with `ast` rather than matched with a regular expression. A regex
34     would also find ops.node inside a string or a comment, and this file's own

```

```

35 docstring would poison its own result -- which is a small illustration of
36 a general point: parse the language, do not pattern-match it.
37 """
38 found = {}
39 for path in sorted(root.rglob("*.py")):
40     if "__pycache__" in path.parts or path.name == "commands.py":
41         continue
42     where = path.parent.name
43     try:
44         tree = ast.parse(path.read_text(encoding="utf-8"))
45     except SyntaxError:
46         continue
47     for node in ast.walk(tree):
48         if not isinstance(node, ast.Call):
49             continue
50         fn = node.func
51         if (isinstance(fn, ast.Attribute)
52             and isinstance(fn.value, ast.Name)
53             and fn.value.id == "ops"):
54             found.setdefault(fn.attr, set()).add(where)
55 return found

```

Note

The parsing matters. A regular expression searching for `ops.node` would also match the text inside a string or a comment — including inside this very script's own docstring, which mentions several command names. Parsing the source with `ast` asks Python what is actually a function call. The general principle is worth carrying away: parse the language, do not pattern-match it.

Listing S2.2: `s2/commands.py` — The build order, and the check that keeps it honest.

```

60 # The commands, grouped by WHEN they are called rather than by what they are.
61 # This ordering is the actual content of the reference: OpenSees has no
62 # declaration phase, so the sequence below is the program, and getting it out
63 # of order is a large fraction of all beginner errors.
64 STAGES = [
65     ("1. start clean", [
66         "wipe", "model",
67     ]),
68     ("2. geometry", [
69         "node", "fix", "equalDOF", "rigidDiaphragm", "mass",
70     ]),
71     ("3. materials and sections", [
72         "uniaxialMaterial", "nDMaterial", "section", "patch", "layer",
73         "fiber", "beamIntegration", "geomTransf",
74     ]),
75     ("4. elements", [
76         "element",
77     ]),
78     ("5. loads", [
79         "timeSeries", "pattern", "load", "eleLoad", "sp", "remove",
80     ]),
81     ("6. the analysis object", [
82         "system", "numberer", "constraints", "test", "algorithm",
83         "integrator", "analysis",
84     ]),
85     ("7. run it", [
86         "analyze", "eigen", "loadConst", "wipeAnalysis", "setTime",
87         "rayleigh",
88     ]),
89     ("8. ask for results", [
90         "reactions", "nodeDisp", "nodeVel", "nodeAccel", "nodeReaction",
91         "nodeCoord", "nodeMass", "eleResponse", "eleNodes", "recorder",
92         "getNodeTags", "getEleTags", "getTime", "getFixedDOFs",

```

```

93     "printModel",
94 ],
95 ("9. test a material on its own", [
96     "testUniaxialMaterial", "setStrain", "getStress", "getTangent",
97     "getStrain",
98 ]),
99 ]
100
101
102 def coverage(found=None):
103     """Which staged commands the course uses, and which it uses unstaged.
104
105     The second list is the interesting one: anything in it is a command the
106     scripts rely on that this reference has not placed in the build order,
107     which means the reference is incomplete rather than the script wrong.
108     """
109     found = scan() if found is None else found
110     staged = {c for _, cs in STAGES for c in cs}
111     used = set(found)
112     return {
113         "used": sorted(used),
114         "staged_and_used": sorted(used & staged),
115         "staged_but_unused": sorted(staged - used),
116         "used_but_unstaged": sorted(used - staged),
117     }

```

48 distinct commands across all scripts.
48 of them are placed in the build order above.

The second line is the one that does the work. If a script ever uses a command this reference has not placed in a stage, the count stops matching and the build fails. The reference cannot drift away from the code, because the code is where it comes from.

S2.3 Where each command appears

Two things in [Table S2.1](#) are worth pausing on.

nDMaterial appears three times in the whole course: once in week 0's wall, which is there to show what a continuum model looks like, and twice in Part II. Every teaching week from 1 to 14 is built from line elements, which is not a simplification for teaching — it is what structural analysis mostly is. S4 is the exception and explains why it is one.

Week 14 uses no OpenSees command at all. It reads week 13's results and never touches the domain. That is exactly why post-processing is a week of its own rather than the last section of week 13: it is a different activity, with different failure modes.

S2.4 Finding the arguments your build expects

The manual documents an argument list. Your compiled interpreter has one too, and they are not always the same — materials and elements gain optional arguments over time. The compiled one is the one that will be enforced, and you can read it directly:

```

>>> ops.uniaxialMaterial('Elastic', 1)
Invalid #args, want: uniaxialMaterial Elastic tag? E? <eta?> <Eneg?> ...

```

Call any constructor with only its tag and OpenSees prints the signature it is actually expecting. S3 turns the same behaviour into a survey of which materials exist at all; here it is simply the fastest way to settle an argument-order question without leaving the interpreter.

Note

The message goes to the C-level error stream, so in a notebook it may appear in the terminal rather than under the cell. S3 shows how to capture it if you need it in a variable.

Table S2.1: The course subset, by stage, with the weeks each command is used in. The dagger marks commands listed here for completeness that no script in the course uses.

Stage	Command	Used in
1	<code>wipe, model</code>	every script
2	<code>node, fix</code>	every model
	<code>mass</code>	12, 13, S1
	<code>equalDOF</code>	11, S4
	<code>rigidDiaphragm[†]</code>	—
3	<code>uniaxialMaterial</code>	6, 7, 8, 10, 11, 12, 13, S1, S3
	<code>section, patch, layer</code>	8, 10, 11, 13
	<code>beamIntegration</code>	8, 10, 11, 13
	<code>geomTransf</code>	4, 5, 11, 12, 13, S1
	<code>nDMaterial</code>	0, S3, S4
4	<code>element</code>	every model
5	<code>timeSeries, pattern</code>	every analysis
	<code>load</code>	most
	<code>eleLoad</code>	4, 5
	<code>sp</code>	12, S4
	<code>remove</code>	11, 12
6	<code>system ... analysis</code>	every analysis
7	<code>analyze</code>	every analysis
	<code>eigen</code>	12, 13, S1
	<code>loadConst</code>	8, 11, 12, 13
	<code>rayleigh</code>	12, 13
8	<code>nodeDisp</code>	almost every script
	<code>reactions, nodeReaction</code>	3, 4, 5, 11, 13, S1, S4
	<code>eleResponse</code>	0, 2, 4, 8, 10, 11, S4
	<code>nodeVel</code>	13
	<code>recorder</code>	3
	<code>getNodeTags, getEleTags, eleNodes</code>	S1; <code>eleNodes</code> also 0
9	<code>testUniaxialMaterial, setStrain, getStress</code>	6, S3

S2.5 The smallest complete model

Every model in this course is this shape, with more in each stage:

```
ops.wipe() # 1
ops.model('basic', '-ndm', 2, '-ndf', 2)
ops.node(1, 0.0, 0.0); ops.node(2, 3000.0, 0.0) # 2
ops.fix(1, 1, 1)
ops.uniaxialMaterial('Elastic', 1, 200000.0) # 3
ops.element('truss', 1, 1, 2, 500.0, 1) # 4
ops.timeSeries('Linear', 1) # 5
ops.pattern('Plain', 1, 1)
ops.load(2, 10000.0, 0.0)
ops.system('BandGeneral'); ops.numberer('RCM') # 6
ops.constraints('Plain'); ops.test('NormDispIncr', 1e-8, 10)
ops.algorithm('Linear'); ops.integrator('LoadControl', 1.0)
ops.analysis('Static')
ops.analyze(1) # 7
ops.reactions() # 8
print(ops.nodeDisp(2, 1), ops.nodeReaction(1, 1))
```

Sixteen lines, nine stages, and every later chapter is a variation on it.

Exercise S2.1. Run `commands.py` and find a command used in exactly one week. Why is it used only there?

Answer: Several qualify — `nodeVel` (week 13), `recorder` (week 3), `getTangent` (week 6).

Solution. The interesting ones are the commands that *could* have been used more widely and were not. `recorder` appears only in week 3 because every later chapter keeps its results in Python lists instead, which is a deliberate choice: a recorder writes to a file, and week 3 shows what a number costs once it has been through one.

Exercise S2.2. Move the `uniaxialMaterial` line in the model above to after the `element` line and run it. Write down the message. Then move it after `analyze`. What is different?

Answer: The first fails when the element is created, naming the missing material. The second fails the same way, because the element is still created before the material exists.

Solution. Both fail identically, which is the point: it is not the *distance* that matters but the order relative to the command that consumes it. This is [Definition S2.1](#) demonstrated, and it is a useful thing to have seen once rather than read once.

Exercise S2.3 (read and explain). `scan()` skips `commands.py` itself. Remove that condition, re-run, and explain the change in the count.

Answer: The count is unchanged: `commands.py` makes no `ops.*` calls, only mentions names inside strings.

Solution. The exclusion is defensive rather than necessary, and re-running it demonstrates why the `ast` parse was worth the extra lines. Had the scan been a regular expression, this file's own tables of command names would have been counted as usage, and the reference would have reported perfect coverage of itself.

Exercise S2.4. Extend `STAGES` with a tenth stage for the commands you expect to need in a project of your own — for example the `Pinching4` family, shell elements, or parallel solvers. Which stage does each belong to?

Answer: Most new commands fall into stages 3 and 4; almost nothing new appears in stages 6–8.

Solution. The distribution is the useful observation. Stages 6 to 8 are essentially fixed: the analysis stack and the result queries are the same in a first-week truss and in a research model. What grows is the modelling vocabulary. That is a reassuring thing to know before reading somebody else's script — the unfamiliar parts are almost always materials and elements.

S3 — Choosing a Material Model

Learning outcomes

After working through this chapter you should be able to:

1. place any OpenSees material on a tree whose branches are *capabilities*, not substances;
2. find out what your own build contains, by asking it rather than reading a manual;
3. separate three different questions — is the name recognised, does it construct, does it behave — and know which of them a successful run answers;
4. choose the smallest material that does the job, and say what each extra parameter costs;
5. explain why one line on the tree, and not the `uniaxialMaterial`/`ndMaterial` split, decides how hard a model will be to run.

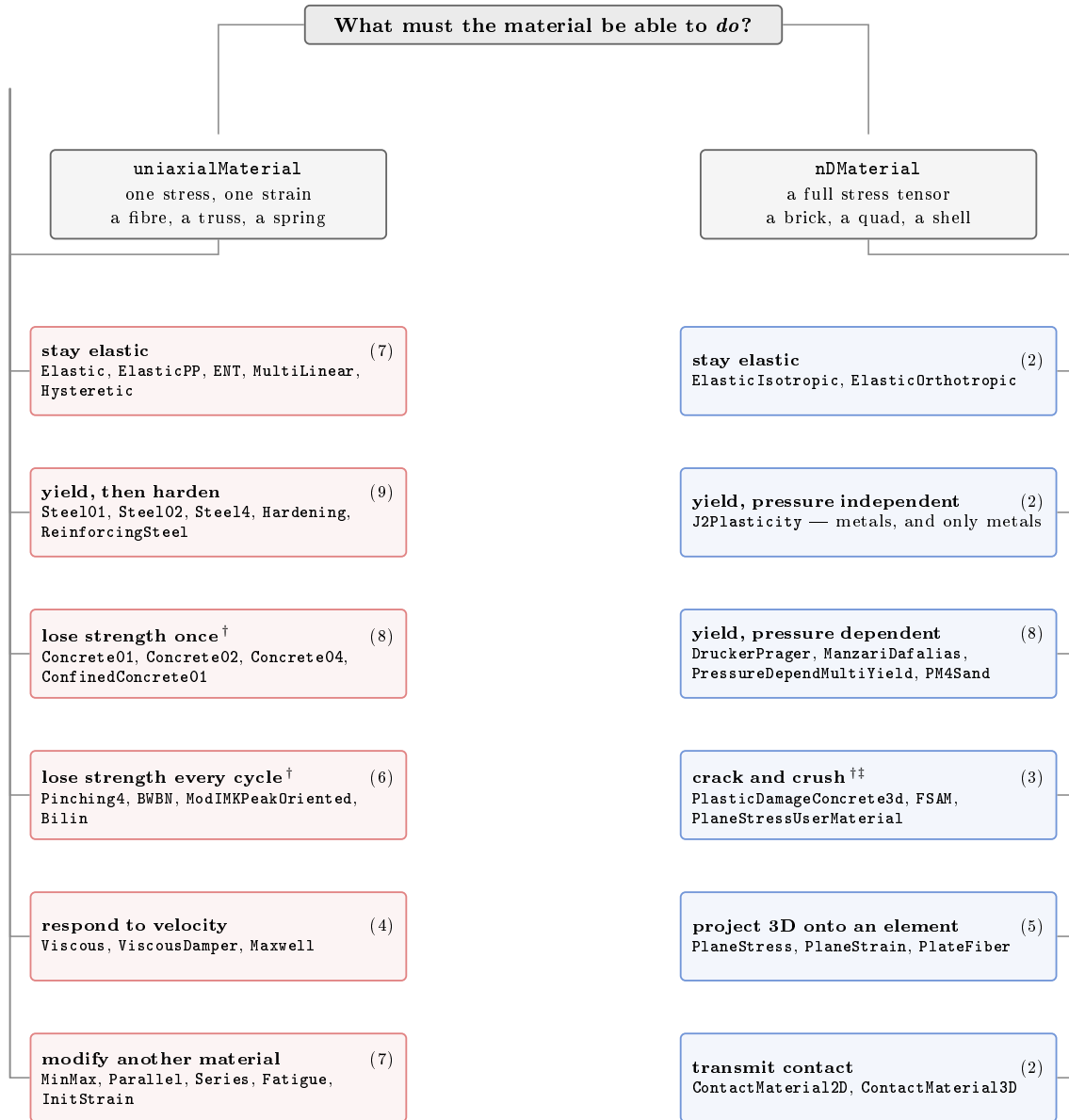
Notation. σ stress ε strain f'_c concrete compressive strength f_t tensile strength E_t tangent modulus

Weeks 6 and 7 taught you four materials in detail. The library has well over a hundred, listed online in roughly the order they were contributed over thirty years, with no organising principle at all. That list is not a reference you can choose from; it is a reference you can look things up in once you have already chosen.

This chapter supplies the missing half.

S3.1 Sort by capability, not by substance

The instinct is to sort materials by what they are made of — steel here, concrete there, soil somewhere else. It is the wrong axis, and following it will cost you.



[†] The tangent can go negative on these branches, so week 4's analysis recipe will not converge on them and week 10's is required. That line, not the uniaxial/nD split, is the one that decides how much work a model will be.

[‡] Softening in a continuum also makes the answer depend on the mesh unless the softening is regularised — which is why *S4* exists and why it is the last chapter rather than an early one.

Counts are what `code/s3/catalogue.py` found present in this build, by asking the interpreter rather than reading the manual. Four further documented names — *Damage2p*, *ConcreteMcftNonLinear5*, *ModIMKBilin* and *Clough* — are not compiled in.

Figure S3.1: The library sorted by what a material must be able to *do*. The counts are what `code/s3/catalogue.py` found in this build, not what the manual lists.

Figure S3.1 sorts by capability instead. A steel brace that has to buckle and a concrete fibre that has to crush sit on the same branch, because both need to lose strength, and nothing on the branch above can do that whatever it is named.

The expensive question is not what a material is made of. It is whether its tangent can turn over:

$$\frac{d\sigma}{d\varepsilon} < 0 \quad ?$$

That dagger line in Fig. S3.1 is the one that matters. Above it the tangent modulus stays positive, the stiffness matrix stays positive definite, and week 4's analysis recipe will converge. Below it E_t can go negative, and week 10's machinery — displacement control, algorithm fallback, step subdivision — stops being optional. Choosing a material from below the line commits you to a different analysis, and that is a much larger decision than the material itself.

Note

Notice which split is *not* the important one. `uniaxialMaterial` versus `nDMaterial` looks like the major division, and it is the one the manual leads with, but it only tells you which element types you can use. Two materials on opposite sides of that split can give you the same amount of trouble, and two on the same side can differ enormously.

S3.2 Ask the build, do not read the manual

The online documentation describes the union of every OpenSees that has ever been compiled. Your interpreter contains a subset, and no page tells you which subset.

You can find out in about a second, and the trick is worth knowing for its own sake:

Listing S3.1: `s3/catalogue.py` — Every name is constructed with *no* arguments on purpose.

```

39 def _stderr_of(fn):
40     """Run `fn` and return whatever it wrote to the real stderr stream.
41
42     OpenSees is a C++ library. Its warnings go to file descriptor 2 directly
43     and never pass through sys.stderr, so contextlib.redirect_stderr cannot
44     see them. Capturing them means swapping the descriptor itself.
45     """
46     fd = 2
47     saved = os.dup(fd)
48     with tempfile.TemporaryFile(mode="w+b") as tmp:
49         os.dup2(tmp.fileno(), fd)
50         try:
51             fn()
52         except Exception:
53             pass # the message is the result
54         finally:
55             sys.stderr.flush()
56             os.dup2(saved, fd)
57             os.close(saved)
58         tmp.seek(0)
59         return tmp.read().decode("utf-8", "replace")
60
61
62 def available(kind, name):
63     """Is `name` compiled into THIS build? Ask it, do not look it up.
64
65     Called with no constructor arguments on purpose. A material that exists
66     then complains about the argument count; one that does not exist complains
67     about the name. Those are different sentences, and telling them apart is
68     the whole trick.
69     """
70     add = ops.uniaxialMaterial if kind == "uniaxial" else ops.nDMaterial
71     msg = _stderr_of(lambda: add(name, 999999))
72     if "is unknown" in msg or "not recognized" in msg:
73         return False
74     return True

```

An existing material then complains about the argument count; a missing one complains about the name. Those are different sentences, and telling them apart is the whole method — [Section S3.2](#) is what happens to a probe that cannot.

Note

The awkward part is that OpenSees is a C++ library and writes its warnings to file descriptor 2 directly, so they never pass through `sys.stderr` and `contextlib.redirect_stderr` cannot see them. Capturing them means swapping the descriptor itself with `os.dup2`. This is worth having met once: any output you cannot seem to capture from a compiled library is likely to be this.

Running the survey over the tree gives:

```
67 of 71 probed names are in this build.
```

The four absentees are `Damage2p`, `ConcreteMcftNonLinear5`, `ModIMKBilin` and `Clough`. All four are documented. The first of them is the obvious first choice for a concrete continuum, which is precisely how this gets expensive: you design a model around a material, and find out it is missing after you have written the script.

Why no arguments

The obvious way to write this probe is to construct each material with realistic parameters and see whether it succeeds. That version reports `PlaneStressUserMaterial` as absent, and it is not — it is present, and needs an external subroutine it was not given.

it failed, so the material is missing → it failed, and the probe cannot say why

The fault is in the design of the test, not in the material. A probe that varies two things at once — the name and the arguments — cannot attribute a failure to either of them. Removing the arguments entirely leaves one variable, and one variable is what makes an answer attributable.

S3.3 Recognised, usable, correct

Definition S3.1: Three levels

Level 1 — recognised. The parser knows the name. This is what the survey above tests.

Level 2 — usable. It accepts a plausible parameter set and builds. A name can be recognised and still be unusable, because it needs an external subroutine, or because the argument list in the manual belongs to a different version.

Level 3 — correct. It behaves the way its name implies. *Nothing but running it answers this.*

Listing S3.2: `s3/catalogue.py` — Levels 2 and 3. The second is week 6's test rig, put to a different use.

```
163 def constructs(kind, name, args):
164     """Level 2: does it accept a plausible set of parameters and build?
165
166     A name can be recognised by the parser and still be unusable -- because it
167     needs an external subroutine, because the build lacks a dependency, or
168     because the argument list in the manual belongs to a different version.
169     """
170     add = ops.uniaxialMaterial if kind == "uniaxial" else ops.nDMaterial
171     msg = _stderr_of(lambda: add(name, 999998, *args))
172     return msg.strip() == ""
173
174
175 def tension_carried(tag, strains=(0.00005, 0.0001, 0.0002, 0.0005)):
176     """Level 3: does it BEHAVE the way the name implies? Only a rig can say.
177
178     This is week 6's material test rig, used here for a different purpose:
179     not to plot a curve, but to ask a yes/no question about a material whose
180     documentation does not make the answer obvious.
```

```

181
182 Concrete01 and Concrete02 both pass levels 1 and 2, are both named for the
183 same material, and differ on whether concrete can carry tension at all.
184 Nothing but running them tells you which is which.
185 """
186 ops.testUniaxialMaterial(tag)
187 out = []
188 for e in strains:
189     ops.setStrain(e)
190     out.append(ops.getStress())
191 return out

```

Here are two materials named for the same substance, both level 1, both level 2, pulled in tension:

strain	Concrete01	Concrete02
0.00005	0.000	1.500
0.00010	0.000	3.000
0.00020	0.000	2.700
0.00050	0.000	1.800

Concrete01 carries no tension at all — not a little, exactly zero — while **Concrete02** cracks at 3.0 MPa and then softens. That is a deliberate and entirely defensible modelling decision on **Concrete01**'s part, and the name does not disclose it. Neither does the fact that the model ran.

Note

A successful analysis certifies level 2. It says nothing whatever about level 3.

This is the same lesson week 3 taught about verification, arriving from a different direction, and it is the reason the material test rig is introduced in week 6 rather than being left as an advanced topic. A rig is the only instrument that reaches level 3.

S3.4 Choosing: a walk down the tree

Listing S3.3: s3/catalogue.py — The choice as a sequence of questions about the physics.

```

209 # The decision, written as questions rather than as a list of names. Each
210 # answer eliminates a branch, and the leaf names the smallest material that
211 # still does the job. "Smallest" matters: every extra parameter is another
212 # number you cannot justify, and week 7 showed what an unjustified parameter
213 # does to a result.
214 CHOOSE_UNIAXIAL = [
215     ("Does it ever leave the elastic range?",
216      "no", "Elastic -- and check afterwards that the strain stayed inside it"),
217     ("Does it need to carry compression only, or tension only?",
218      "yes", "ENT for no-tension, ElasticPPGap for a gap that closes"),
219     ("Is it steel that yields and hardens, without losing strength?",
220      "yes", "Steel01 if a corner is acceptable, Steel02 if the cyclic "
221           "transition matters -- see week 7"),
222     ("Is it concrete, so that it crushes and loses strength?",
223      "yes", "Concrete01 for a first model, Concrete02 to get tension, "
224           "Concrete04 for a smooth Popovics curve"),
225     ("Does it lose strength with repeated cycles at the same amplitude?",
226      "yes", "Pinching4 or one of the ModIMK family -- these are the only "
227           "branch that models cyclic deterioration"),
228     ("Does the force depend on how fast it moves?",
229      "yes", "Viscous or ViscousDamper -- and note this makes a static "
230           "analysis meaningless"),
231     ("Do you need an existing material to stop at a strain limit?",
232      "yes", "MinMax, which wraps another material rather than replacing it"),
233 ]

```

Two principles run through it.

Take the smallest material that does the job. Every additional parameter is a number you have to justify, and week 7 showed what an unjustified parameter does — it moves the answer without announcing that it did. **Concrete04** is a better model than **Concrete01** and needs more numbers you probably do not have.

Wrappers before replacements. If an otherwise suitable material lacks one property, wrap it rather than swapping it. **MinMax** adds a strain limit to anything; **Parallel** and **Series** combine; **InitStress** adds a preload; **Fatigue** adds low-cycle failure. Seven wrappers cover a large fraction of the cases people go looking for a new material for.

Before committing to a material

1. Ask what it must *do*, and find the branch. Do not start from the substance.
2. Check it is in your build — one line, and it costs a second.
3. Build it with your parameters and confirm it constructs.
4. **Put it in the test rig** and plot what it does. Check the peak, the initial slope, and the behaviour on unloading against something you know.
5. Only then put it in a structure.

Step 4 is the one that gets skipped, and it is the one that separates level 2 from level 3.

S3.5 The nD side, and why it is harder

Everything above applies to **nDMaterial** as well, with one addition that changes the character of the problem. When a uniaxial material softens, the consequence is a hard analysis. When a continuum material softens, the consequence is a hard analysis *and* an answer that depends on the mesh — deformation localises into one band of elements, and refining the mesh makes the band narrower and the structure weaker, without limit.

That is a property of the mathematics, not of OpenSees, and it does not go away by choosing a better material. S4 is where it gets confronted.

Note

Note also how thin the concrete-continuum branch is: three entries against eight for soil. Soil mechanics has invested far more in continuum constitutive modelling than concrete engineering has, and the library records that. If you need a cracking, crushing concrete solid, you will be choosing from a much shorter list than you expect — which is the finding S4 opens with.

S3.6 Summary

Question	Answered by	Cost of skipping it
what must it do?	the tree	the wrong branch entirely
is it in my build?	a one-line probe	a rewrite, late
does it construct?	building it	minutes
does it behave?	the test rig	the whole result
can the tangent go negative?	the branch you landed on	the wrong analysis

Exercise S3.1. Run `catalogue.py` on your own machine. Do you get 67 of 71? If not, which names differ, and what does that tell you about reproducing somebody else's script?

Answer: Build-dependent; the point is the comparison, not the number.

Solution. K19 pins the OpenSeesPy version precisely so that this number is reproducible, and it is a concrete illustration of why a pinned version is not bureaucracy. A script that runs on one machine

and not on yours, with no error other than a material name, is a long afternoon.

Exercise S3.2. Add `Steel14`, `ConcreteCM` and `BWBN` to the level-3 test: put each in the rig, apply a monotonic tensile strain to 0.02, and report the peak stress. Which of the three surprised you?

Answer: All three build; the surprises are in the parameters each demands and in what happens outside the range they were calibrated for.

Solution. The instructive part is not the peaks but the argument lists. `Steel14` has optional argument groups that silently change the model, and `BWBN` has seven parameters with no physical interpretation you can estimate from a material certificate. That is the real cost of descending the tree, and it is not visible until you try to fill the arguments in.

Exercise S3.3 (read and explain). `available()` returns `True` when the stderr message does *not* contain "is unknown". Why is it written as a negative test rather than checking for "Invalid #args"?

Answer: Because a material taking zero arguments would print nothing at all, and a check for the argument-count message would then wrongly report it absent.

Solution. This is a general principle for probes: test for the condition you are certain about. We know what a missing name says, because that message is produced in one place in the source. We do not know that every existing material produces an argument error — so the negative test has one failure mode and the positive test has an unbounded number.

Exercise S3.4. Take the frame from week 5 and replace its `Elastic` material with `ElasticPP` at $f_y = 420$ MPa. Run the same analysis, unchanged. Does it converge? Now move down one more branch to `Concrete01` and try again.

Answer: ElasticPP generally converges with the week 5 recipe; Concrete01 generally does not, once any fibre passes the peak.

Solution. This is the dagger line in [Fig. S3.1](#), demonstrated rather than asserted. `ElasticPP` has a zero tangent after yield, which is a badly conditioned matrix but not an indefinite one. `Concrete01` has a negative one. The step from zero to negative is the step that changes what analysis you need, and it is worth having felt it once.

Exercise S3.5. You need a steel material that yields, hardens, and then fails at a limiting strain of 0.05. Nothing in the library does all three. What do you do?

Answer: Wrap: MinMax around Steel02, with -min -0.05 -max 0.05.

Solution. The instinct is to go looking for a material that does everything, and the library will reward that search with several candidates carrying a dozen parameters each. The wrapper branch is the answer, and it exists precisely so the other branches do not have to multiply. Getting into the habit of asking “can I wrap this?” before “is there a better material?” will save you a great deal of parameter estimation.

Exercise S3.6. Extend `available()` to also report the argument *signature* OpenSees prints for a material that exists — the `want: uniaxialMaterial Elastic tag? E? <eta?>` line. Use it to compare what your build expects against the online manual for three materials.

Answer: The captured message already contains the signature; it needs parsing rather than new probing.

Solution. This turns the probe into something more useful than a checklist, because the signature comes from the compiled code and the manual does not. Where they disagree, the compiled code is right — and this is a common disagreement for materials that have gained optional arguments over

time.

S4 — Continuum Elements: Crushing a Cylinder

Learning outcomes

After working through this chapter you should be able to:

1. say when a line element is not enough, and accept the cost of leaving it;
2. build a structured hexahedral mesh of a circular section without a degenerate cell;
3. calibrate a plasticity model whose parameters are not the strength;
4. design a mesh-refinement study, and interpret one that does *not* show mesh dependence;
5. separate a rate effect from an inertia effect in a dynamic analysis;
6. export a result to ParaView, and say why a plot cannot replace it.

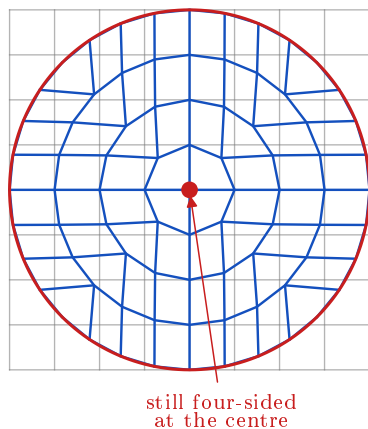
Notation. f'_c compressive strength σ_y yield-cone parameter H hardening modulus K, G bulk and shear moduli ε_q deviatoric strain

Every model in weeks 1 to 14 was made of lines. A truss is a line; a beam is a line; the 3D frame in week 11 was a set of lines with sections attached. That is not a limitation of OpenSees, and it is not a simplification made for teaching — it is what structural analysis mostly is, because a line element with a fibre section carries an enormous amount of physics for very little cost.

This chapter is about the case where it cannot be done: a standard 150 mm × 300 mm concrete test cylinder, crushed until it fails. There is no line to reduce it to, no section to integrate over, and — unlike every other chapter — no closed form to check against.

S4.1 Meshing a circle

square grid, concentric map, disc



441 nodes, 288 hexahedra

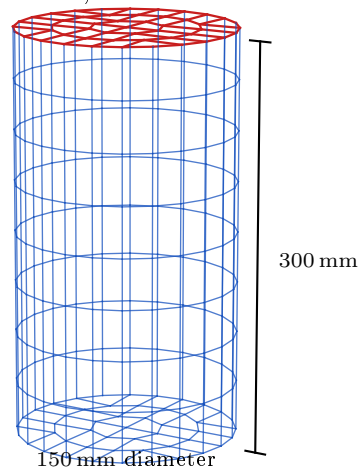


Figure S4.1: A square grid mapped onto a disc, and the disc extruded into the specimen. Every cell stays four-sided, including the one at the centre.

The left panel of Fig. S4.1 shows why polar coordinates, the obvious choice, are a trap: every cell in the innermost ring degenerates to a triangle at $r = 0$, and a hexahedron with two coincident nodes has a singular Jacobian there. The concentric map avoids it by keeping the larger of the two square coordinates as a radius and turning the smaller into an angle:

Listing S4.1: `s4/cylinder.py` — The map, and the extrusion into hexahedra. Note the node ordering — `stdBrick` wants the bottom face counter-clockwise, then the top face the same way round.

```

57 def concentric_disc(n):
58     """Map a square grid onto a disc without collapsing a cell at the centre.
59
60     A disc is the standard awkward case for a structured mesh. Polar
61     coordinates look obvious and are a trap: every cell in the innermost ring
62     degenerates to a triangle at  $r = 0$ , and a brick with two coincident nodes
63     has a singular Jacobian.
64
65     The concentric map (Shirley and Chiu) avoids that entirely. It sends the
66     square  $[-1,1]^2$  to the unit disc by keeping the larger coordinate as the
67     radius and turning the smaller one into an angle, so a square grid becomes
68     a round grid with the same topology -- and the centre stays a proper
69     four-sided cell.
70
71     Returns  $(2n+1)^2$  points in  $[-1,1]^2$  mapped into the unit disc, as a list of
72     (x, y) in row-major order.
73     """
74     pts = []
75     for j in range(2 * n + 1):
76         v = j / n - 1.0
77         for i in range(2 * n + 1):
78             u = i / n - 1.0
79             if u == 0.0 and v == 0.0:
80                 pts.append((0.0, 0.0))
81             elif abs(u) >= abs(v):
82                 r, a = u, (math.pi / 4.0) * (v / u)
83                 pts.append((r * math.cos(a), r * math.sin(a)))
84             else:
85                 r, a = v, (math.pi / 2.0) - (math.pi / 4.0) * (u / v)
86                 pts.append((r * math.cos(a), r * math.sin(a)))
87     return pts
88
89
90 def mesh(p=CYLINDER):
91     """Nodes and hexahedra for the cylinder. Returns (coords, cells, groups).
92
93     coords {tag: (x, y, z)}
94     cells {tag: (n1..n8)} in the node order stdBrick expects -- bottom face
95         counter-clockwise, then the top face the same way round
96     groups {'base': [...], 'top': [...]} the node tags on each end
97     """
98     n, nz = p["n_r"], p["n_z"]
99     R, L = 0.5 * p["D"], p["L"]
100     side = 2 * n + 1
101
102     disc = concentric_disc(n)
103     coords, k = {}, 1
104     for layer in range(nz + 1):
105         z = L * layer / nz
106         for (x, y) in disc:
107             coords[k] = (R * x, R * y, z)
108             k += 1
109
110     def tag(layer, i, j):
111         return layer * side * side + j * side + i + 1
112
113     cells, c = {}, 1
114     for layer in range(nz):
115         for j in range(2 * n):
116             for i in range(2 * n):
117                 cells[c] = (
118                     tag(layer, i, j), tag(layer, i + 1, j),
119                     tag(layer, i + 1, j + 1), tag(layer, i, j + 1),

```

```

120         tag(layer + 1, i, j), tag(layer + 1, i + 1, j),
121         tag(layer + 1, i + 1, j + 1), tag(layer + 1, i, j + 1),
122     )
123     c += 1
124
125     groups = {
126         "base": [tag(0, i, j) for j in range(side) for i in range(side)],
127         "top": [tag(nz, i, j) for j in range(side) for i in range(side)],
128     }
129     return coords, cells, groups

```

150 x 300 mm, 3 divisions per quadrant, 8 layers
441 nodes, 288 brick elements

Note

Compare that with week 11: a three-storey frame, a complete structure, 16 elements. Here 288 elements model one laboratory specimen, and the mesh is coarse. That ratio is the honest price of leaving line elements, and it is worth having a number for before deciding to pay it.

S4.2 A material that will lose strength

S3 established that this build has exactly three concrete continuum materials, and none of them is straightforwardly usable: `PlasticDamageConcrete3d` reaches a plausible peak and then holds it indefinitely, `FSAM` is for reinforced membranes, and `PlaneStressUserMaterial` wants an external subroutine.

So the material comes from the soil branch of S3's tree instead:

Listing S4.2: `s4/cylinder.py` — `DruckerPrager` with a *negative* hardening modulus.

```

134 def drucker_prager(tag, p, sigma_y, hardening):
135     """A pressure-dependent plastic material that can actually lose strength.
136
137     S3 found three concrete continuum materials in this build.
138     PlasticDamageConcrete3d reaches a plausible peak and then stays there
139     for ever -- it does not soften, and its four damage parameters change
140     nothing. FSAM is for membranes with reinforcement in them.
141     PlaneStressUserMaterial needs an external subroutine.
142
143     So this uses DruckerPrager, from the soil branch of S3's tree. It is not a
144     concrete model, and pretending otherwise would be dishonest: it has no
145     crack, no tension cut-off worth the name, and no distinction between
146     loading and reloading. What it does have is the one property the chapter
147     cannot proceed without -- a NEGATIVE hardening modulus, which makes the
148     yield surface shrink as plastic strain accumulates, and so makes the
149     specimen lose strength.
150
151     Note what the parameters are not. `sigma_y` is not the compressive
152     strength; it sets the size of the yield cone, and the uniaxial strength
153     that follows from it also depends on the friction parameter rho. The
154     strength is an OUTPUT of this material, which is why calibrate() exists.
155     """
156     K = p["E"] / (3.0 * (1.0 - 2.0 * p["nu"]))
157     G = p["E"] / (2.0 * (1.0 + p["nu"]))
158     ops.ndMaterial('DruckerPrager', tag,
159                   K, G, sigma_y,
160                   0.15,      # rho      -- friction, drives pressure sensitivity
161                   0.05,      # rhoBar   -- non-associated flow, limits dilation
162                   0.0, 0.0,   # Kinf, Ko -- no isotropic saturation hardening
163                   0.0, 0.0,   # delta1, delta2 -- no tension softening of its own
164                   hardening, # H        -- NEGATIVE: this is the whole point
165                   1.0,       # theta   -- fully isotropic hardening
166                   p["rho"], 101.0)

```

This is a substitution, not a concrete model, and it is worth being exact about what is given up: no crack, no tension cut-off worth the name, and no distinction between loading and reloading. What it provides is the one property the chapter cannot proceed without — $H < 0$ shrinks the yield surface as plastic strain accumulates, so the specimen loses strength.

S4.2.1 The parameters are not the strength

σ_y is not f'_c . It sets the size of the yield cone, and the uniaxial strength that follows from it depends on the friction parameter as well. The strength is an *output*:

Listing S4.3: `s4/cylinder.py` — One element, bisection, a dozen analyses.

```

214 def calibrate(p=CYLINDER, hardening=-500.0, tol=0.01, lo=10.0, hi=40.0):
215     """Find the sigma_y that gives the target compressive strength.
216
217     This function exists because of a fact worth stating plainly: for this
218     material the compressive strength is not something you enter. It emerges
219     from the yield-cone size and the friction parameter together, so the only
220     way to hit fc = 30 MPa is to run the one-element test, look at the peak,
221     and adjust.
222
223     That is not a defect of OpenSees. It is what using a soil plasticity model
224     for concrete costs, and being explicit about the cost is the point of
225     borrowing it. Bisection, because the relationship is monotonic and we need
226     perhaps a dozen evaluations.
227     """
228     target = p["fc"]
229     for _ in range(40):
230         mid = 0.5 * (lo + hi)
231         curve = uniaxial_curve(p, mid, hardening, steps=60)
232         peak = max(s for _, s in curve) if curve else 0.0
233         if abs(peak - target) / target < tol:
234             return mid, peak
235         if peak < target:
236             lo = mid
237         else:
238             hi = mid
239     return mid, peak

```

$\sigma_y = 24.53$ MPa gives $f_c = 30.00$ MPa (target 30)

$$\sigma_y = 24.53 \text{ MPa} \longrightarrow f'_c = 30.00 \text{ MPa}$$

Note

This is a general feature of plasticity models and not a quirk of this one. Yield-surface parameters are defined in stress-invariant space; a compressive strength is the result of a particular loading path through it. Whenever a material's inputs are K , G and a cone size rather than f'_c and ε_c , expect to calibrate.

S4.3 Crushing it

Listing S4.4: `s4/cylinder.py` — Two platens, and the modelling decision hiding in them.

```

244 def build(p, sigma_y, hardening, weakened=0.0):
245     """The whole specimen: mesh, material, elements, and the two platens.
246
247     The platens are the modelling decision that matters. A real cylinder is
248     crushed between steel plates, which restrain the ends laterally through
249     friction -- and that restraint is why a cylinder fails in a cone rather
250     than splitting cleanly. Model it as frictionless and you get a different

```

```

251 failure. Model it as fully bonded and you get a different one again.
252
253 What is done here is the frictionless idealisation, with the top nodes
254 tied to one another vertically so the platen stays flat. That choice is
255 revisited in the exercises, because it changes the answer.
256 """
257 ops.wipe()
258 ops.model('basic', '-ndm', 3, '-ndf', 3)
259
260 coords, cells, groups = mesh(p)
261 for tag, (x, y, z) in coords.items():
262     ops.node(tag, x, y, z)
263
264 drucker_prager(1, p, sigma_y, hardening)
265
266 # An imperfection, if asked for. A perfectly uniform specimen softens
267 # uniformly and never localises, so there is nothing for the mesh to
268 # disagree about; real specimens are not uniform. The weakened region is
269 # a FIXED PHYSICAL VOLUME -- a disc of radius 0.5R spanning the middle 30 % of the height -- so
270 # that refining the mesh does not also change the size of the flaw.
271 weak = set()
272 if weakened > 0.0:
273     drucker_prager(2, p, sigma_y * (1.0 - weakened), hardening)
274     R, L = 0.5 * p["D"], p["L"]
275     for tag, nodes in cells.items():
276         cx = sum(coords[n][0] for n in nodes) / 8.0
277         cy = sum(coords[n][1] for n in nodes) / 8.0
278         cz = sum(coords[n][2] for n in nodes) / 8.0
279         if (cx * cx + cy * cy) ** 0.5 < 0.35 * R and abs(cz - 0.5 * L) <
↪ 0.15 * L:
280             weak.add(tag)
281
282 for tag, nodes in cells.items():
283     ops.element('stdBrick', tag, *nodes, 2 if tag in weak else 1)
284
285 # Bottom platen: no vertical movement anywhere, and just enough in-plane
286 # restraint to stop the specimen sliding away. Restraining every base node
287 # in x and y would be a bonded platen, which is a different experiment.
288 for n in groups["base"]:
289     ops.fix(n, 0, 0, 1)
290 centre = groups["base"][len(groups["base"]) // 2]
291 ops.fix(centre, 1, 1, 0)
292 ops.fix(groups["base"][0], 0, 1, 0)
293
294 # Top platen: flat and rigid vertically, free to expand sideways.
295 master = groups["top"][len(groups["top"]) // 2]
296 for n in groups["top"]:
297     if n != master:
298         ops.equalDOF(master, n, 3)
299
300 return coords, cells, groups, master, weak

```

The platens matter more than they look. A real cylinder is crushed between steel plates that restrain its ends through friction — which is why a cylinder fails in a cone rather than splitting cleanly. The idealisation used here is frictionless, with the top nodes tied vertically so the platen stays flat. [Exercise S4.3](#) changes it, and the answer changes with it.

Listing S4.5: s4/cylinder.py — Displacement control, because the interest is entirely past the peak — week 10.

```

305 def crush(p=CYLINDER, sigma_y=24.53, hardening=-500.0,
306         steps=90, d_eps=1.2e-4, record=None, weakened=0.0):
307     """Squeeze the cylinder until it will not take any more.
308

```

```

309     Displacement control on the platen, because the whole interest is past the
310     peak and load control cannot go there -- week 10.
311
312     `record`, if given, is called as record(step, coords, cells) after every
313     converged step, which is how the VTK series gets written without this
314     function knowing anything about file formats.
315     """
316     coords, cells, groups, master, weak = build(p, sigma_y, hardening,
317                                                weakened)
318
319     L = p["L"]
320     area = math.pi * (0.5 * p["D"]) ** 2
321
322     ops.timeSeries('Linear', 1)
323     ops.pattern('Plain', 1, 1)
324     ops.load(master, 0.0, 0.0, -1.0)
325
326     ops.system('UmfPack')
327     ops.numberer('RCM')
328     ops.constraints('Transformation')
329     ops.test('NormDispIncr', 1.0e-6, 40)
330     ops.algorithm('Newton')
331     ops.integrator('DisplacementControl', master, 3, -d_eps * L)
332     ops.analysis('Static')
333
334     out = {"eps": [], "sig": [], "converged": True}
335     for step in range(steps):
336         if ops.analyze(1) < 0:
337             out["converged"] = False
338             break
339         ops.reactions()
340         # The platen pushes down, so the base pushes up: the reaction is
341         # positive and no sign flip belongs here. Get this backwards and the
342         # least-compressed early step is reported as the "peak", which looks
343         # like a specimen that failed immediately.
344         force = sum(ops.nodeReaction(n, 3) for n in groups["base"])
345         out["eps"].append(-ops.nodeDisp(master, 3) / L)
346         out["sig"].append(force / area)
347         if record is not None:
348             record(step, coords, cells)
349
350     out["peak"] = max(out["sig"]) if out["sig"] else 0.0
351     out["eps_at_peak"] = (out["eps"][out["sig"].index(out["peak"])])
352     if out["sig"] else 0.0)
353     out["final"] = out["sig"][-1] if out["sig"] else 0.0
354     out["nodes"] = len(coords)
355     out["elements"] = len(cells)
356     out["weakened"] = len(weak)
357     return out

```

```

peak 29.66 MPa at a strain of 0.0011
carried to a strain of 0.0108, by which point it
was down to 23.25 MPa (78% of the peak)

```

S4.4 Does the mesh change the answer?

The standard warning about softening continua is that they localise. Deformation collects into a band one element wide whatever the element size, so refining the mesh narrows the band, concentrates the strain and drops the post-peak curve — without converging to anything. Everything after the peak is then mesh dependent and no amount of refinement fixes it.

That is a claim about a model, and this chapter can test it. Before reading on, predict what the three meshes will do.

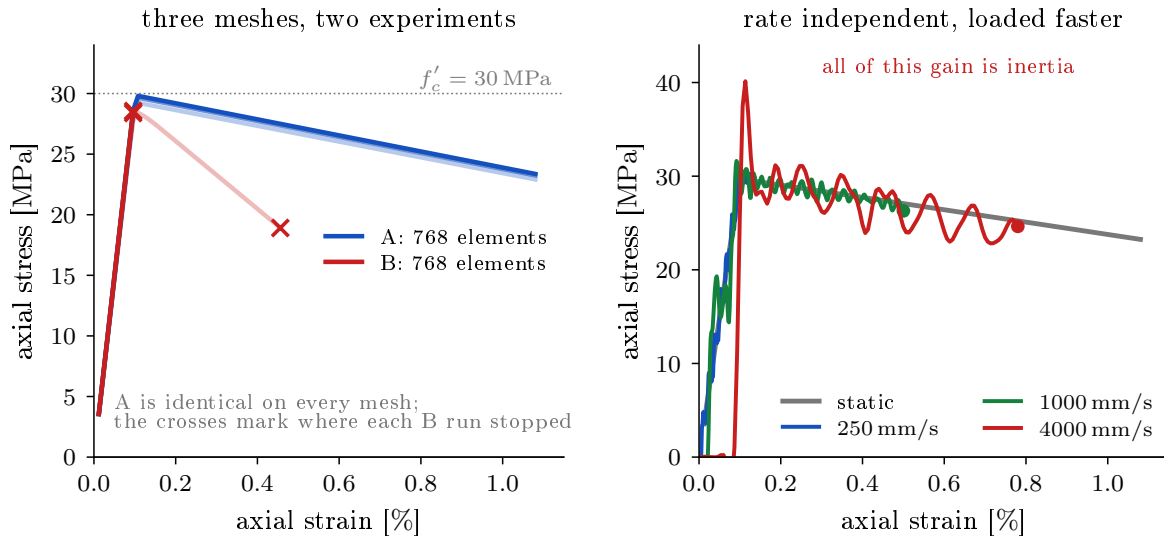


Figure S4.2: Left: two mesh studies. Right: the same specimen crushed at three platen speeds by a material that has no rate dependence at all.

Listing S4.6: s4/cylinder.py — The same specimen at three densities, twice.

```

57 def concentric_disc(n):
58     """Map a square grid onto a disc without collapsing a cell at the centre.
59
60     A disc is the standard awkward case for a structured mesh. Polar
61     coordinates look obvious and are a trap: every cell in the innermost ring
62     degenerates to a triangle at  $r = 0$ , and a brick with two coincident nodes
63     has a singular Jacobian.
64
65     The concentric map (Shirley and Chiu) avoids that entirely. It sends the
66     square  $[-1,1]^2$  to the unit disc by keeping the larger coordinate as the
67     radius and turning the smaller one into an angle, so a square grid becomes
68     a round grid with the same topology -- and the centre stays a proper
69     four-sided cell.
70
71     Returns  $(2n+1)^2$  points in  $[-1,1]^2$  mapped into the unit disc, as a list of
72      $(x, y)$  in row-major order.
73     """
74     pts = []
75     for j in range(2 * n + 1):
76         v = j / n - 1.0
77         for i in range(2 * n + 1):
78             u = i / n - 1.0
79             if u == 0.0 and v == 0.0:
80                 pts.append((0.0, 0.0))
81             elif abs(u) >= abs(v):
82                 r, a = u, (math.pi / 4.0) * (v / u)
83                 pts.append((r * math.cos(a), r * math.sin(a)))
84             else:
85                 r, a = v, (math.pi / 2.0) - (math.pi / 4.0) * (u / v)
86                 pts.append((r * math.cos(a), r * math.sin(a)))
87     return pts
88
89
90 def mesh(p=CYLINDER):
91     """Nodes and hexahedra for the cylinder. Returns (coords, cells, groups).
92

```

```

93     coords {tag: (x, y, z)}
94     cells {tag: (n1..n8)} in the node order stdBrick expects -- bottom face
95         counter-clockwise, then the top face the same way round
96     groups {'base': [...], 'top': [...]} the node tags on each end
97     """
98     n, nz = p["n_r"], p["n_z"]
99     R, L = 0.5 * p["D"], p["L"]
100     side = 2 * n + 1
101
102     disc = concentric_disc(n)
103     coords, k = {}, 1
104     for layer in range(nz + 1):
105         z = L * layer / nz
106         for (x, y) in disc:
107             coords[k] = (R * x, R * y, z)
108             k += 1
109
110     def tag(layer, i, j):
111         return layer * side * side + j * side + i + 1
112
113     cells, c = {}, 1
114     for layer in range(nz):
115         for j in range(2 * n):
116             for i in range(2 * n):
117                 cells[c] = (
118                     tag(layer, i, j), tag(layer, i + 1, j),
119                     tag(layer, i + 1, j + 1), tag(layer, i, j + 1),
120                     tag(layer + 1, i, j), tag(layer + 1, i + 1, j),
121                     tag(layer + 1, i + 1, j + 1), tag(layer + 1, i, j + 1),
122                 )
123                 c += 1
124
125     groups = {
126         "base": [tag(0, i, j) for j in range(side) for i in range(side)],
127         "top": [tag(nz, i, j) for j in range(side) for i in range(side)],
128     }
129     return coords, cells, groups
130 # --8<-- [end:mesh]
131
132 # --8<-- [start:material]
133 def drucker_prager(tag, p, sigma_y, hardening):
134     """A pressure-dependent plastic material that can actually lose strength.
135
136     S3 found three concrete continuum materials in this build.
137     PlasticDamageConcrete3d reaches a plausible peak and then stays there
138     for ever -- it does not soften, and its four damage parameters change
139     nothing. FSAM is for membranes with reinforcement in them.
140     PlaneStressUserMaterial needs an external subroutine.
141
142     So this uses DruckerPrager, from the soil branch of S3's tree. It is not a
143     concrete model, and pretending otherwise would be dishonest: it has no
144     crack, no tension cut-off worth the name, and no distinction between
145     loading and reloading. What it does have is the one property the chapter
146     cannot proceed without -- a NEGATIVE hardening modulus, which makes the
147     yield surface shrink as plastic strain accumulates, and so makes the
148     specimen lose strength.
149
150     Note what the parameters are not. `sigma_y` is not the compressive
151     strength; it sets the size of the yield cone, and the uniaxial strength
152     that follows from it also depends on the friction parameter rho. The
153     strength is an OUTPUT of this material, which is why calibrate() exists.
154     """
155     K = p["E"] / (3.0 * (1.0 - 2.0 * p["nu"]))
156     G = p["E"] / (2.0 * (1.0 + p["nu"]))
157

```

```

158 ops.ndMaterial('DruckerPrager', tag,
159               K, G, sigma_y,
160               0.15,      # rho    -- friction, drives pressure sensitivity
161               0.05,      # rhoBar  -- non-associated flow, limits dilation
162               0.0, 0.0,   # Kinf, Ko -- no isotropic saturation hardening
163               0.0, 0.0,   # delta1, delta2 -- no tension softening of its own
164               hardening, # H      -- NEGATIVE: this is the whole point
165               1.0,      # theta   -- fully isotropic hardening
166               p["rho"], 101.0)
167 # --8<-- [end:material]
168
169
170 def _single_brick(p, sigma_y, hardening, size=100.0):
171     """One element, for calibration. The cheapest possible test of a material."""
172     ops.wipe()
173     ops.model('basic', '-ndm', 3, '-ndf', 3)
174     corners = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0),
175               (0, 0, 1), (1, 0, 1), (1, 1, 1), (0, 1, 1)]
176     for i, (x, y, z) in enumerate(corners, start=1):
177         ops.node(i, x * size, y * size, z * size)
178     for i in (1, 2, 3, 4):
179         ops.fix(i, 0, 0, 1)
180     ops.fix(1, 1, 1, 0)
181     ops.fix(2, 0, 1, 0)
182     ops.fix(4, 1, 0, 0)
183     drucker_prager(1, p, sigma_y, hardening)
184     ops.element('stdBrick', 1, *range(1, 9), 1)
185     ops.timeSeries('Linear', 1)
186     ops.pattern('Plain', 1, 1)
187     for i in (5, 6, 7, 8):
188         ops.load(i, 0.0, 0.0, -1.0)
189     return size
190
191
192 def uniaxial_curve(p, sigma_y, hardening, steps=200, d_eps=2.0e-4):
193     """Crush one brick and return (strain, stress) with compression positive."""
194     size = _single_brick(p, sigma_y, hardening)
195     ops.system('BandGeneral')
196     ops.numberer('RCM')
197     ops.constraints('Transformation')
198     ops.test('NormDispIncr', 1.0e-7, 60)
199     ops.algorithm('Newton')
200     ops.integrator('DisplacementControl', 5, 3, -d_eps * size)
201     ops.analysis('Static')
202
203     curve = []
204     for _ in range(steps):
205         if ops.analyze(1) < 0:
206             break
207         s = ops.eleResponse(1, 'stresses')
208         szz = sum(s[g * 6 + 2] for g in range(8)) / 8.0
209         curve.append((-ops.nodeDisp(5, 3) / size, -szz))
210     return curve
211
212
213 # --8<-- [start:calibrate]
214 def calibrate(p=CYLINDER, hardening=-500.0, tol=0.01, lo=10.0, hi=40.0):
215     """Find the sigma_y that gives the target compressive strength.
216
217     This function exists because of a fact worth stating plainly: for this
218     material the compressive strength is not something you enter. It emerges
219     from the yield-cone size and the friction parameter together, so the only
220     way to hit fc = 30 MPa is to run the one-element test, look at the peak,
221     and adjust.
222

```

```

223     That is not a defect of OpenSees. It is what using a soil plasticity model
224     for concrete costs, and being explicit about the cost is the point of
225     borrowing it. Bisection, because the relationship is monotonic and we need
226     perhaps a dozen evaluations.
227     """
228     target = p["fc"]
229     for _ in range(40):
230         mid = 0.5 * (lo + hi)
231         curve = uniaxial_curve(p, mid, hardening, steps=60)
232         peak = max(s for _, s in curve) if curve else 0.0
233         if abs(peak - target) / target < tol:
234             return mid, peak
235         if peak < target:
236             lo = mid
237         else:
238             hi = mid
239     return mid, peak
240 # --8<-- [end:calibrate]
241
242
243 # --8<-- [start:build]
244 def build(p, sigma_y, hardening, weakened=0.0):
245     """The whole specimen: mesh, material, elements, and the two platens.
246
247     The platens are the modelling decision that matters. A real cylinder is
248     crushed between steel plates, which restrain the ends laterally through
249     friction -- and that restraint is why a cylinder fails in a cone rather
250     than splitting cleanly. Model it as frictionless and you get a different
251     failure. Model it as fully bonded and you get a different one again.
252
253     What is done here is the frictionless idealisation, with the top nodes
254     tied to one another vertically so the platen stays flat. That choice is
255     revisited in the exercises, because it changes the answer.
256     """
257     ops.wipe()
258     ops.model('basic', '-ndm', 3, '-ndf', 3)
259
260     coords, cells, groups = mesh(p)
261     for tag, (x, y, z) in coords.items():
262         ops.node(tag, x, y, z)
263
264     drucker_prager(1, p, sigma_y, hardening)
265
266     # An imperfection, if asked for. A perfectly uniform specimen softens
267     # uniformly and never localises, so there is nothing for the mesh to
268     # disagree about; real specimens are not uniform. The weakened region is
269     # a FIXED PHYSICAL VOLUME -- a disc of radius 0.5R spanning the middle 30 % of the height -- so
270     # that refining the mesh does not also change the size of the flaw.
271     weak = set()
272     if weakened > 0.0:
273         drucker_prager(2, p, sigma_y * (1.0 - weakened), hardening)
274         R, L = 0.5 * p["D"], p["L"]
275         for tag, nodes in cells.items():
276             cx = sum(coords[n][0] for n in nodes) / 8.0
277             cy = sum(coords[n][1] for n in nodes) / 8.0
278             cz = sum(coords[n][2] for n in nodes) / 8.0
279             if (cx * cx + cy * cy) ** 0.5 < 0.35 * R and abs(cz - 0.5 * L) <
280                 ↪ 0.15 * L:
281                 weak.add(tag)
282
283     for tag, nodes in cells.items():
284         ops.element('stdBrick', tag, *nodes, 2 if tag in weak else 1)
285
286     # Bottom platen: no vertical movement anywhere, and just enough in-plane
287     # restraint to stop the specimen sliding away. Restraining every base node

```

```

287     # in x and y would be a bonded platen, which is a different experiment.
288     for n in groups["base"]:
289         ops.fix(n, 0, 0, 1)
290     centre = groups["base"][len(groups["base"]) // 2]
291     ops.fix(centre, 1, 1, 0)
292     ops.fix(groups["base"][0], 0, 1, 0)
293
294     # Top platen: flat and rigid vertically, free to expand sideways.
295     master = groups["top"][len(groups["top"]) // 2]
296     for n in groups["top"]:
297         if n != master:
298             ops.equalDOF(master, n, 3)
299
300     return coords, cells, groups, master, weak
301 # --8<-- [end:build]
302
303
304 # --8<-- [start:crush]
305 def crush(p=CYLINDER, sigma_y=24.53, hardening=-500.0,
306         steps=90, d_eps=1.2e-4, record=None, weakened=0.0):
307     """Squeeze the cylinder until it will not take any more.
308
309     Displacement control on the platen, because the whole interest is past the
310     peak and load control cannot go there -- week 10.
311
312     `record`, if given, is called as record(step, coords, cells) after every
313     converged step, which is how the VTK series gets written without this
314     function knowing anything about file formats.
315     """
316     coords, cells, groups, master, weak = build(p, sigma_y, hardening,
317                                                weakened)
318
319     L = p["L"]
320     area = math.pi * (0.5 * p["D"]) ** 2
321
322     ops.timeSeries('Linear', 1)
323     ops.pattern('Plain', 1, 1)
324     ops.load(master, 0.0, 0.0, -1.0)
325
326     ops.system('UmfPack')
327     ops.numberer('RCM')
328     ops.constraints('Transformation')
329     ops.test('NormDispIncr', 1.0e-6, 40)
330     ops.algorithm('Newton')
331     ops.integrator('DisplacementControl', master, 3, -d_eps * L)
332     ops.analysis('Static')
333
334     out = {"eps": [], "sig": [], "converged": True}
335     for step in range(steps):
336         if ops.analyze(1) < 0:
337             out["converged"] = False
338             break
339         ops.reactions()
340         # The platen pushes down, so the base pushes up: the reaction is
341         # positive and no sign flip belongs here. Get this backwards and the
342         # least-compressed early step is reported as the "peak", which looks
343         # like a specimen that failed immediately.
344         force = sum(ops.nodeReaction(n, 3) for n in groups["base"])
345         out["eps"].append(-ops.nodeDisp(master, 3) / L)
346         out["sig"].append(force / area)
347         if record is not None:
348             record(step, coords, cells)
349
350     out["peak"] = max(out["sig"]) if out["sig"] else 0.0
351     out["eps_at_peak"] = (out["eps"][out["sig"].index(out["peak"])]
352                          if out["sig"] else 0.0)

```

```

352 out["final"] = out["sig"][-1] if out["sig"] else 0.0
353 out["nodes"] = len(coords)
354 out["elements"] = len(cells)
355 out["weakened"] = len(weak)
356 return out
357 # --8<-- [end:crush]
358
359
360 # --8<-- [start:vtk]
361 def export_vtk(coords, cells, path, point_fields=None,
362               cell_fields=None):
363     """Write one .vtu file that ParaView can open. No library required.
364
365     A displacement is a vector on a line and matplotlib draws it. A stress
366     field inside a solid is not: any single view of it hides most of it, and
367     what you actually want is to rotate the specimen, slice it, and threshold
368     on a value. That is what ParaView is, and getting data into it is the only
369     part that concerns us here.
370
371     The format is written out by hand rather than through a library, because
372     it is about forty lines and knowing what is in the file is worth more than
373     one import. VTK's unstructured grid is: a list of points, a list of cells
374     given as indices into those points, a type code per cell (12 is a linear
375     hexahedron), and any number of named arrays attached to points or cells.
376
377     Point and cell fields are passed separately and deliberately so. Working
378     out which is which by testing whether the keys are node tags or element
379     tags does not work: element tags 1..288 are a subset of node tags 1..441,
380     so every element field is silently written as point data instead. Tag
381     ranges overlap in OpenSees. Do not infer meaning from them.
382     """
383     path = Path(path)
384     path.parent.mkdir(parents=True, exist_ok=True)
385
386     order = sorted(coords)
387     index = {tag: i for i, tag in enumerate(order)}
388     cell_order = sorted(cells)
389
390     def arr(name, data, tags):
391         first = data[tags[0]]
392         ncomp = 1 if isinstance(first, (int, float)) else len(first)
393         vals = []
394         for t in tags:
395             v = data[t]
396             vals.extend([v] if ncomp == 1 else list(v))
397         body = " ".join(f"{x:.6g}" for x in vals)
398         return (f'<DataArray type="Float32" Name="{name}" '
399               f'NumberOfComponents="{ncomp}" format="ascii">\n'
400               f'    {body}\n    </DataArray>\n')
401
402     point_fields = point_fields or {}
403     cell_fields = cell_fields or {}
404
405     with path.open("w", encoding="utf-8") as f:
406         f.write('<?xml version="1.0"?>\n')
407         f.write('<VTKFile type="UnstructuredGrid" version="0.1" '
408               'byte_order="LittleEndian">\n  <UnstructuredGrid>\n')
409         f.write(f'    <Piece NumberOfPoints="{len(order)}" '
410               f'NumberOfCells="{len(cell_order)}">\n')
411
412         f.write('      <Points>\n        <DataArray type="Float32" '
413               'NumberOfComponents="3" format="ascii">\n          ')
414         f.write(" ".join(f"{c:.6g}" for t in order for c in coords[t]))
415         f.write('\n        </DataArray>\n      </Points>\n')
416

```

```

417         f.write('        <Cells>\n        <DataArray type="Int32" '
418                 'Name="connectivity" format="ascii">\n        ')
419         f.write(" ".join(str(index[n]) for c in cell_order
420                             for n in cells[c]))
421         f.write('\n        </DataArray>\n')
422         f.write('        <DataArray type="Int32" Name="offsets" '
423                 'format="ascii">\n        ')
424         f.write(" ".join(str(8 * (i + 1)) for i in range(len(cell_order))))
425         f.write('\n        </DataArray>\n')
426         f.write('        <DataArray type="UInt8" Name="types" '
427                 'format="ascii">\n        ')
428         f.write(" ".join("12" for _ in cell_order)) # 12 = linear hexahedron
429         f.write('\n        </DataArray>\n        </Cells>\n')
430
431         if point_fields:
432             f.write('        <PointData>\n')
433             for name, data in point_fields.items():
434                 f.write(arr(name, data, order))
435             f.write('        </PointData>\n')
436         if cell_fields:
437             f.write('        <CellData>\n')
438             for name, data in cell_fields.items():
439                 f.write(arr(name, data, cell_order))
440             f.write('        </CellData>\n')
441
442         f.write('    </Piece>\n    </UnstructuredGrid>\n</VTKFile>\n')
443         return path
444 # --8<-- [end:vtk]
445
446
447 # --8<-- [start:series]
448 def write_series(paths, times, path):
449     """A .pvd collection, which is what makes ParaView treat the files as time.
450
451     Without this, twenty .vtu files are twenty unrelated models. With it they
452     are one model at twenty instants, and the animation controls light up.
453     """
454     path = Path(path)
455     with path.open("w", encoding="utf-8") as f:
456         f.write('<?xml version="1.0"?>\n<VTKFile type="Collection" '
457                 'version="0.1" byte_order="LittleEndian">\n <Collection>\n')
458         for t, p in zip(times, paths):
459             f.write(f'    <DataSet timestep="{t:.6g}" part="0" '
460                     f'file="{Path(p).name}">\n')
461         f.write(' </Collection>\n</VTKFile>\n')
462     return path
463 # --8<-- [end:series]
464
465
466 # --8<-- [start:snapshot]
467 def snapshot(coords, cells, scale=1.0):
468     """Current geometry and the fields worth looking at, ready for export.
469
470     Three fields, and the choice is deliberate. Displacement so the shape can
471     be exaggerated; vertical stress because that is what is being applied; and
472     the equivalent plastic strain, which is the one that shows WHERE the
473     specimen is failing rather than how hard it is being pushed.
474     """
475     moved, disp = {}, {}
476     for tag, (x, y, z) in coords.items():
477         d = ops.nodeDisp(tag)
478         disp[tag] = tuple(d[:3])
479         moved[tag] = (x + scale * d[0], y + scale * d[1], z + scale * d[2])
480
481     szz, plastic = {}, {}

```

```

482     for tag in cells:
483         s = ops.eleResponse(tag, 'stresses')
484         szz[tag] = -sum(s[g * 6 + 2] for g in range(8)) / 8.0
485         e = ops.eleResponse(tag, 'strains')
486         # Deviatoric strain magnitude: a scalar that rises where the material
487         # is yielding and stays near zero where it is still elastic.
488         acc = 0.0
489         for g in range(8):
490             ex, ey, ez = e[g * 6], e[g * 6 + 1], e[g * 6 + 2]
491             m = (ex + ey + ez) / 3.0
492             acc += math.sqrt((2.0 / 3.0) * ((ex - m) ** 2 + (ey - m) ** 2
493                                     + (ez - m) ** 2))
494         plastic[tag] = acc / 8.0
495
496     return moved, {"displacement": disp}, {"stress_zz": szz,
497                                           "deviatoric_strain": plastic}
498 # --8<-- [end:snapshot]
499
500
501 # --8<-- [start:impact]
502 def impact(p=CYLINDER, sigma_y=24.53, hardening=-500.0,
503           rate=2000.0, duration=1.5e-3, dt=5.0e-6, weakened=0.0):
504     """The same specimen crushed dynamically, by a platen driven at `rate`.
505     The material used here has no rate dependence whatever -- DruckerPrager's
506     yield surface does not know how fast it is being loaded. So every
507     difference between this and the static crush is INERTIA, and nothing else.
508     That makes it a clean experiment: the specimen has to accelerate its own
509     mass before the far end learns that the near end has been hit, and at a
510     high enough rate the top crushes before the bottom has felt anything.
511
512     `rate` is the platen speed in mm/s. Prescribed motion goes in as an sp
513     constraint inside a pattern, which needs constraints('Transformation') --
514     'Plain' cannot impose a non-zero support displacement.
515     """
516     coords, cells, groups, master, weak = build(p, sigma_y, hardening,
517                                                  weakened)
518
519     L = p["L"]
520     area = math.pi * (0.5 * p["D"]) ** 2
521
522     steps = int(duration / dt)
523     # A Linear series returns factor * t, so the factor IS the platen speed
524     # and the prescribed displacement is rate * t. Writing rate * duration
525     # here instead -- the total travel -- multiplies by the duration twice
526     # and drives the platen a thousand times too slowly.
527     ops.timeSeries('Linear', 1, '-factor', rate)
528     ops.pattern('Plain', 1, 1)
529     ops.sp(master, 3, -1.0)
530
531     ops.system('UmfPack')
532     ops.numberer('RCM')
533     ops.constraints('Transformation')
534     ops.test('NormDispIncr', 1.0e-6, 40)
535     ops.algorithm('Newton')
536     ops.integrator('Newmark', 0.5, 0.25)
537     ops.analysis('Transient')
538
539     out = {"t": [], "eps": [], "sig": [], "top_sig": [], "converged": True}
540     for _ in range(steps):
541         if ops.analyze(1, dt) < 0:
542             out["converged"] = False
543             break
544         ops.reactions()
545         force = sum(ops.nodeReaction(n, 3) for n in groups["base"])
546         # The topmost layer of elements, which is what the platen is actually

```

```

547     # pushing on. In a static run this equals the base stress; here it
548     # does not, and the gap is the whole point.
549     top_cells = [c for c in cells
550                  if min(coords[n][2] for n in cells[c]) > 0.85 * L]
551     top = 0.0
552     for c in top_cells:
553         s = ops.eleResponse(c, 'stresses')
554         top += -sum(s[g * 6 + 2] for g in range(8)) / 8.0
555     out["t"].append(ops.getTime())
556     out["eps"].append(-ops.nodeDisp(master, 3) / L)
557     out["sig"].append(force / area)
558     out["top_sig"].append(top / max(1, len(top_cells)))
559
560     out["peak"] = max(out["sig"]) if out["sig"] else 0.0
561     out["peak_top"] = max(out["top_sig"]) if out["top_sig"] else 0.0
562     out["steps"] = len(out["sig"])
563     return out
564 # --8<-- [end:impact]
565
566
567 # --8<-- [start:mesh-study]
568 def mesh_study(p=CYLINDER, sigma_y=24.53, hardening=-500.0,
569               densities=((2, 6), (3, 8), (4, 12)), weakened=0.0):
570     """The same specimen at three mesh densities. The result is not the
571     expected one, and the difference is the most useful thing in this file.
572
573     The standard warning about softening continua is that they localise:
574     deformation collects into a band one element wide whatever the element
575     size, so refining the mesh narrows the band and drops the post-peak curve
576     without converging to anything. Everything after the peak is then mesh
577     dependent, and no amount of refinement fixes it.
578
579     The experiment below does not support that warning as stated. With mild
580     softening and a uniform specimen the residual strength is IDENTICAL on
581     every mesh -- because a uniform specimen softens uniformly, nothing
582     localises, and there is no band for the mesh to resolve differently.
583     Localisation is not a consequence of softening alone; it needs a trigger.
584
585     Give it one -- the `weakened` region -- and steepen the softening enough
586     for the tangent to turn indefinite, and what happens is not a mesh
587     dependent post-peak curve. The analysis simply stops, at the peak, and how
588     far it got depends on the mesh. Which is the practical form of the
589     warning: you will meet the convergence wall long before you get a chance
590     to be misled by a mesh dependent answer.
591     """
592     rows = []
593     for n_r, n_z in densities:
594         q = dict(p, n_r=n_r, n_z=n_z)
595         out = crush(q, sigma_y, hardening, weakened=weakened)
596         rows.append({
597             "n_r": n_r, "n_z": n_z,
598             "weakened": out["weakened"],
599             "elements": out["elements"],
600             "peak": out["peak"],
601             "eps_at_peak": out["eps_at_peak"],
602             "final": out["final"],
603             "residual_fraction": out["final"] / out["peak"] if out["peak"]
604             else 0.0,
605         })
606     return rows
607 # --8<-- [end:mesh-study]
608
609
610 if __name__ == "__main__":
611     p = CYLINDER

```

```

612 coords, cells, groups = mesh(p)
613 print("A standard cylinder, meshed")
614 print(f" {p['D']:.0f} x {p['L']:.0f} mm, "
615       f"{p['n_r']} divisions per quadrant, {p['n_z']} layers")
616 print(f" {len(coords)} nodes, {len(cells)} brick elements")
617 print(f" {len(groups['base'])} nodes on each end")
618
619 print("\nCalibrating: the parameters are not the strength")
620 sigma_y, peak = calibrate(p)
621 print(f" sigma_y = {sigma_y:6.2f} MPa gives fc = {peak:5.2f} MPa"
622       f" (target {p['fc']:.0f})")
623 print(" sigma_y is the size of the yield cone, not a strength. The")
624 print(" strength comes out of the analysis, so it has to be calibrated.")
625
626 print("\nCrushing it")
627 frames, times = [], []
628
629 def record(step, coords, cells):
630     if step % 6:
631         return
632     moved, pf, cf = snapshot(coords, cells, scale=8.0)
633     frames.append(export_vtk(moved, cells,
634                             OUT / f"cylinder-{len(frames):03d}.vtu",
635                             pf, cf))
636     times.append(step)
637
638 out = crush(p, sigma_y, record=record)
639 print(f" peak {out['peak']:5.2f} MPa at a strain of "
640       f"{out['eps_at_peak']:.4f}")
641 print(f" carried to a strain of {out['eps'][-1]:.4f}, by which point it")
642 print(f" was down to {out['final']:5.2f} MPa "
643       f"f({out['final'] / out['peak']:.0%} of the peak)")
644 if not out["converged"]:
645     print(" and then it stopped converging, which is what failure looks")
646     print(" like when the tangent has gone indefinite -- see week 10.")
647
648 series = write_series(frames, times, OUT / "cylinder.pvd")
649 print(f"\n {len(frames)} frames written; open {series.name} in ParaView")
650 print(" Colour by deviatoric_strain and the failure band is immediate.")
651
652 print("\nDoes the mesh change the answer? Two experiments, and the")
653 print("textbook warning turns out to be only half of the story.")
654 print(f" {'mesh':>8}{ 'elements':>10}{ 'weak':>6}{ 'peak':>9}"
655       f"{'residual':>10}{ 'ran':>6}")
656 print(" A -- mild softening (H = -500), no flaw:")
657 for r in mesh_study(p, sigma_y, -500.0):
658     print(f" {r['n_r']}x{r['n_z']:<6}{r['elements']:10d}"
659           f"{r['weakened']:6d}{r['peak']:9.2f}"
660           f"{r['residual_fraction']:9.1%}{ 'yes':>6}")
661 print(" B -- steep softening (H = -2000), 15 % flaw at mid-height:")
662 for r in mesh_study(p, sigma_y, -2000.0, weakened=0.15):
663     print(f" {r['n_r']}x{r['n_z']:<6}{r['elements']:10d}"
664           f"{r['weakened']:6d}{r['peak']:9.2f}"
665           f"{r['residual_fraction']:9.1%}{ 'no':>6}")
666 print(" A runs to the end and gives the SAME residual on every mesh.")
667 print(" Nothing localised, so there was nothing for the meshes to")
668 print(" disagree about. B never gets past the peak at all, and how far")
669 print(" it gets depends on the mesh. What stops you in practice is")
670 print(" convergence, not mesh dependence.")
671
672 print("\nCrushed dynamically instead, by a platen driven at a speed.")
673 print("The material has NO rate dependence, so this is inertia alone.")
674 print(f" {'rate mm/s':>11}{ 'steps':>8}{ 'peak':>9}{ 'top/base':>10}")
675 for rate in (250.0, 1000.0, 4000.0):
676     d = impact(p, sigma_y, rate=rate)

```

```

677     pk = d["peak"] or float("nan")
678     print(f" {rate:9.0f} {d['steps']:8d}{d['peak']:9.2f}"
679           f"{d['peak_top'] / pk:10.2f}")
680     print(f" Static, for comparison: {out['peak']:.2f} MPa.")
681     print(" A rate-independent material apparently gains a third of its")
682     print(" strength. It has not: the specimen is carrying its own inertia,")
683     print(" and at the highest rate the top runs 6 % ahead of the base")
684     print(" because the stress wave has not finished crossing it.")

```

```

A -- mild softening (H = -500), no flaw:
2x6      96      0      29.23      78.4%      yes
3x8      288     0      29.66      78.4%      yes
4x12     768     0      29.81      78.4%      yes
B -- steep softening (H = -2000), 15 % flaw at mid-height:
2x6      96      8      28.42      66.5%      no
3x8      288     8      28.39      100.0%     no
4x12     768    16      28.56      100.0%     no

```

Both experiments are drawn in the left panel of [Fig. S4.2](#), and neither behaves as advertised.

Two halves of the warning

Experiment A does not show mesh dependence at all. Three meshes spanning a factor of eight in element count give a residual strength identical to three significant figures. Softening alone does not produce localisation: a uniform specimen softens uniformly, there is no band, and so there is nothing for the meshes to resolve differently.

a softening material makes the post-peak answer mesh dependent → localisation needs a trigger, and without one the answer is perfectly well behaved

Experiment B, with a trigger and steeper softening, does not show mesh dependence either. It shows something worse. The tangent turns indefinite at the peak and every mesh stops there — the finer the mesh, the sooner. There is no post-peak curve to be mesh dependent about.

That is the practically useful form of the warning, and it is not the one usually given:

you meet the convergence wall long before you meet the mesh dependence

The peak, meanwhile, is reliable on every mesh in both experiments — 29.2 MPa–29.8 MPa across a factor of eight in element count. If what you need is a strength, this model gives it to you cheaply. If what you need is a post-peak curve, no mesh in this study delivers one you could defend, and the fix is not refinement but a regularised material with a fracture energy in it.

S4.5 Crushing it dynamically

The original question was a dynamic analysis to failure, and it is worth doing precisely because this material is *rate independent*. **DruckerPrager**'s yield surface has no idea how fast it is being loaded. Every difference between a slow crush and a fast one is therefore inertia and nothing else — which makes it a genuinely controlled experiment, of the kind week 14 recommends.

Listing S4.7: `s4/cylinder.py` — Prescribed platen motion, which needs `sp` inside a pattern and `constraints('Transformation')` — Plain cannot impose a non-zero support displacement.

```

502 def impact(p=CYLINDER, sigma_y=24.53, hardening=-500.0,
503           rate=2000.0, duration=1.5e-3, dt=5.0e-6, weakened=0.0):
504     """The same specimen crushed dynamically, by a platen driven at `rate`.
505
506     The material used here has no rate dependence whatever -- DruckerPrager's
507     yield surface does not know how fast it is being loaded. So every
508     difference between this and the static crush is INERTIA, and nothing else.
509     That makes it a clean experiment: the specimen has to accelerate its own
510     mass before the far end learns that the near end has been hit, and at a
511     high enough rate the top crushes before the bottom has felt anything.
512

```

```

513 `rate` is the platen speed in mm/s. Prescribed motion goes in as an sp
514 constraint inside a pattern, which needs constraints('Transformation') --
515 'Plain' cannot impose a non-zero support displacement.
516 """
517 coords, cells, groups, master, weak = build(p, sigma_y, hardening,
518                                             weakened)
519 L = p["L"]
520 area = math.pi * (0.5 * p["D"]) ** 2
521
522 steps = int(duration / dt)
523 # A Linear series returns factor * t, so the factor IS the platen speed
524 # and the prescribed displacement is rate * t. Writing rate * duration
525 # here instead -- the total travel -- multiplies by the duration twice
526 # and drives the platen a thousand times too slowly.
527 ops.timeSeries('Linear', 1, '-factor', rate)
528 ops.pattern('Plain', 1, 1)
529 ops.sp(master, 3, -1.0)
530
531 ops.system('UmfPack')
532 ops.numberer('RCM')
533 ops.constraints('Transformation')
534 ops.test('NormDispIncr', 1.0e-6, 40)
535 ops.algorithm('Newton')
536 ops.integrator('Newmark', 0.5, 0.25)
537 ops.analysis('Transient')
538
539 out = {"t": [], "eps": [], "sig": [], "top_sig": [], "converged": True}
540 for _ in range(steps):
541     if ops.analyze(1, dt) < 0:
542         out["converged"] = False
543         break
544     ops.reactions()
545     force = sum(ops.nodeReaction(n, 3) for n in groups["base"])
546     # The topmost layer of elements, which is what the platen is actually
547     # pushing on. In a static run this equals the base stress; here it
548     # does not, and the gap is the whole point.
549     top_cells = [c for c in cells
550                  if min(coords[n][2] for n in cells[c]) > 0.85 * L]
551     top = 0.0
552     for c in top_cells:
553         s = ops.eleResponse(c, 'stresses')
554         top += -sum(s[g * 6 + 2] for g in range(8)) / 8.0
555     out["t"].append(ops.getTime())
556     out["eps"].append(-ops.nodeDisp(master, 3) / L)
557     out["sig"].append(force / area)
558     out["top_sig"].append(top / max(1, len(top_cells)))
559
560 out["peak"] = max(out["sig"]) if out["sig"] else 0.0
561 out["peak_top"] = max(out["top_sig"]) if out["top_sig"] else 0.0
562 out["steps"] = len(out["sig"])
563 return out

```

rate mm/s	steps	peak	top/base
250	300	30.34	1.01
1000	300	31.63	0.98
4000	117	40.14	0.94

30.3 MPa $\longrightarrow$ 40.1 MPa with no rate dependence in the material

A third more apparent strength, and none of it is material. Two things produce it. The specimen has to accelerate its own mass, and that inertia force is carried by the platen and read as strength; and at the highest rate the stress wave has not finished crossing the specimen, so the top is 6% out of step with the base.

Note

Real concrete *does* gain strength with strain rate, by a comparable amount at comparable rates. That is exactly why this experiment is worth doing: if you fit a rate-dependent material to dynamic test data without separating the inertia first, you will fit the inertia into the material and then carry it into every model that uses it. The wave transit time here is $L/\sqrt{E/\rho} = 85\text{ }\mu\text{s}$, and any loading faster than a few multiples of that is not measuring material behaviour.

S4.6 Looking at it: the ParaView route

Every figure so far in this course has been a line on a plane, and matplotlib draws those better than anything else. A stress field inside a solid is a different object: any single view hides most of it, and what you actually want is to rotate the specimen, slice it, threshold on a value and watch it evolve. That is what ParaView is for, and this is the first model in the course that justifies it.

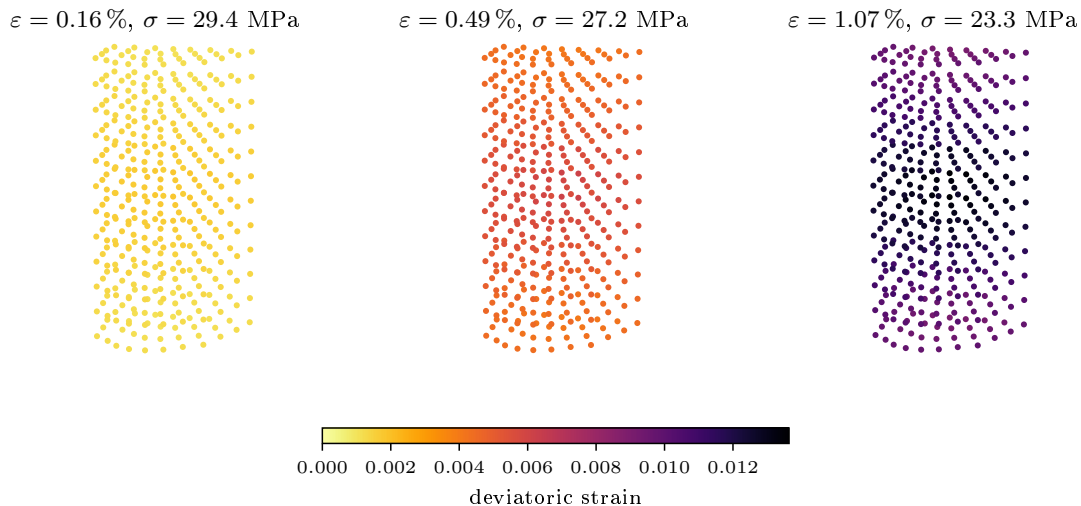


Figure S4.3: Half the specimen at three instants, coloured by deviatoric strain. Taking half is what ParaView's clip filter does in one click; doing it by hand is the argument for using the tool.

Figure S4.3 is the best a still can do, and it is not good enough: half the elements are behind the other half. Getting the data across is the only part that concerns us, and it needs no library:

Listing S4.8: `s4/cylinder.py` — A `.vtu` file, written by hand in about forty lines.

```

361 def export_vtk(coords, cells, path, point_fields=None,
362               cell_fields=None):
363     """Write one .vtu file that ParaView can open. No library required.
364
365     A displacement is a vector on a line and matplotlib draws it. A stress
366     field inside a solid is not: any single view of it hides most of it, and
367     what you actually want is to rotate the specimen, slice it, and threshold
368     on a value. That is what ParaView is, and getting data into it is the only
369     part that concerns us here.
370
371     The format is written out by hand rather than through a library, because
372     it is about forty lines and knowing what is in the file is worth more than
373     one import. VTK's unstructured grid is: a list of points, a list of cells
374     given as indices into those points, a type code per cell (12 is a linear
375     hexahedron), and any number of named arrays attached to points or cells.
376
377     Point and cell fields are passed separately and deliberately so. Working
378     out which is which by testing whether the keys are node tags or element

```

```

379 tags does not work: element tags 1..288 are a subset of node tags 1..441,
380 so every element field is silently written as point data instead. Tag
381 ranges overlap in OpenSees. Do not infer meaning from them.
382 """
383 path = Path(path)
384 path.parent.mkdir(parents=True, exist_ok=True)
385
386 order = sorted(coords)
387 index = {tag: i for i, tag in enumerate(order)}
388 cell_order = sorted(cells)
389
390 def arr(name, data, tags):
391     first = data[tags[0]]
392     ncomp = 1 if isinstance(first, (int, float)) else len(first)
393     vals = []
394     for t in tags:
395         v = data[t]
396         vals.extend([v] if ncomp == 1 else list(v))
397     body = " ".join(f"{x:.6g}" for x in vals)
398     return (f'      <DataArray type="Float32" Name="{name}" '
399           f'NumberOfComponents="{ncomp}" format="ascii">\n'
400           f'      {body}\n      </DataArray>\n')
401
402 point_fields = point_fields or {}
403 cell_fields = cell_fields or {}
404
405 with path.open("w", encoding="utf-8") as f:
406     f.write('<?xml version="1.0"?>\n'
407           '<VTKFile type="UnstructuredGrid" version="0.1" '
408           '<byte_order="LittleEndian">\n  <UnstructuredGrid>\n')
409     f.write(f'    <Piece NumberOfPoints="{len(order)}" '
410           f'NumberOfCells="{len(cell_order)}">\n')
411
412     f.write('      <Points>\n        <DataArray type="Float32" '
413           '<NumberOfComponents="3" format="ascii">\n          ')
414     f.write(" ".join(f"{c:.6g}" for t in order for c in coords[t]))
415     f.write('\n        </DataArray>\n      </Points>\n')
416
417     f.write('      <Cells>\n        <DataArray type="Int32" '
418           '<Name="connectivity" format="ascii">\n          ')
419     f.write(" ".join(str(index[n]) for c in cell_order
420                     for n in cells[c]))
421     f.write('\n        </DataArray>\n')
422     f.write('      <DataArray type="Int32" Name="offsets" '
423           '<format="ascii">\n          ')
424     f.write(" ".join(str(8 * (i + 1)) for i in range(len(cell_order))))
425     f.write('\n      </DataArray>\n')
426     f.write('      <DataArray type="UInt8" Name="types" '
427           '<format="ascii">\n          ')
428     f.write(" ".join("12" for _ in cell_order)) # 12 = linear hexahedron
429     f.write('\n      </DataArray>\n    </Cells>\n')
430
431     if point_fields:
432         f.write('    <PointData>\n')
433         for name, data in point_fields.items():
434             f.write(arr(name, data, order))
435         f.write('    </PointData>\n')
436     if cell_fields:
437         f.write('    <CellData>\n')
438         for name, data in cell_fields.items():
439             f.write(arr(name, data, cell_order))
440         f.write('    </CellData>\n')
441
442     f.write('  </Piece>\n </UnstructuredGrid>\n</VTKFile>\n')
443     return path

```

VTK's unstructured grid is a list of points, a list of cells given as indices into those points, a type code per cell (12 is a linear hexahedron), and any number of named arrays attached to points or to cells. Writing it out rather than importing a library is worth doing once: what you gain is knowing what is in the file, which is the difference between fixing an export and guessing at it.

Note

Note the one trap. Point data and cell data are passed as separate arguments here, not worked out from the keys — because element tags 1–288 are a *subset* of node tags 1–441, so a function that inferred the difference would silently write every element field as point data. Tag ranges overlap in OpenSees. Never infer meaning from them.

Listing S4.9: s4/cylinder.py — Three fields, and why these three.

```

467 def snapshot(coords, cells, scale=1.0):
468     """Current geometry and the fields worth looking at, ready for export.
469
470     Three fields, and the choice is deliberate. Displacement so the shape can
471     be exaggerated; vertical stress because that is what is being applied; and
472     the equivalent plastic strain, which is the one that shows WHERE the
473     specimen is failing rather than how hard it is being pushed.
474     """
475     moved, disp = {}, {}
476     for tag, (x, y, z) in coords.items():
477         d = ops.nodeDisp(tag)
478         disp[tag] = tuple(d[:3])
479         moved[tag] = (x + scale * d[0], y + scale * d[1], z + scale * d[2])
480
481     szz, plastic = {}, {}
482     for tag in cells:
483         s = ops.eleResponse(tag, 'stresses')
484         szz[tag] = -sum(s[g * 6 + 2] for g in range(8)) / 8.0
485         e = ops.eleResponse(tag, 'strains')
486         # Deviatoric strain magnitude: a scalar that rises where the material
487         # is yielding and stays near zero where it is still elastic.
488         acc = 0.0
489         for g in range(8):
490             ex, ey, ez = e[g * 6], e[g * 6 + 1], e[g * 6 + 2]
491             m = (ex + ey + ez) / 3.0
492             acc += math.sqrt((2.0 / 3.0) * ((ex - m) ** 2 + (ey - m) ** 2
493                                     + (ez - m) ** 2))
494         plastic[tag] = acc / 8.0
495
496     return moved, {"displacement": disp}, {"stress_zz": szz,
497                                         "deviatoric_strain": plastic}

```

Displacement so the shape can be exaggerated; vertical stress because that is what is applied; and deviatoric strain, which is the one that shows *where* the specimen is failing rather than how hard it is being pushed.

S4.6.1 Making it a time series

Twenty .vtu files are twenty unrelated models. A one-line collection file turns them into one model at twenty instants, and ParaView's animation controls appear:

Listing S4.10: s4/cylinder.py — The .pvd collection.

```

448 def write_series(paths, times, path):
449     """A .pvd collection, which is what makes ParaView treat the files as time.
450
451     Without this, twenty .vtu files are twenty unrelated models. With it they
452     are one model at twenty instants, and the animation controls light up.
453     """

```

```

454 path = Path(path)
455 with path.open("w", encoding="utf-8") as f:
456     f.write('<?xml version="1.0"?>\n<VTKFile type="Collection" '
457           'version="0.1" byte_order="LittleEndian">\n  <Collection>\n')
458     for t, p in zip(times, paths):
459         f.write(f'    <DataSet timestep="{t:.6g}" part="0" '
460               f'file="{Path(p).name}"/>\n')
461     f.write('  </Collection>\n</VTKFile>\n')
462 return path

```

Opening the result in ParaView

1. Run `python code/s4/cylinder.py`. It writes `code/s4/out/cylinder.pvd` and the frames beside it.
2. Open the `.pvd`, not the individual `.vtu` files, and press **Apply**.
3. Set **Representation** to *Surface With Edges* and colour by `deviatoric_strain`.
4. Add a **Clip** filter to see inside, and a **Threshold** on the same field to keep only the elements that have yielded.
5. Press play. The colour bar rescales per frame by default — turn that off, or every frame will look equally damaged.

The last point is the one that catches people, and it is week 14's lesson in a new place: a correct analysis becoming a misleading picture.

Note

`opsviz`, used in week 11, and this export are for different jobs rather than competing. Line models are lines, and matplotlib is the right tool for them; solids need a viewer that can cut into them. Reach for ParaView when the answer is a field inside a volume, and not before.

S4.7 Summary

Leaving line elements costs you here

elements, for one specimen	288 instead of 16
a material that fits the problem	a soil model, substituted
parameters you can look up	a calibration run for f'_c
a closed form to check against	none; only convergence studies
a figure that shows the answer	an export, and another program

Exercise S4.1. Replace `concentric_disc` with a polar mesh — rings at constant radius, including one at $r = 0$ — and build the specimen. What does OpenSees say?

Answer: The elements at the centre have coincident nodes; the analysis fails on a singular or near-singular Jacobian.

Solution. The message will name the solver rather than the mesh, which is S1's point about the distance between symptom and cause. Running `audit()` from S1 first reports the coincident nodes directly. It is worth doing this deliberately once, because a degenerate cell hidden in a large mesh presents in exactly the same way and is far harder to find.

Exercise S4.2. Run `calibrate` with the friction parameter `rho` changed from 0.15 to 0.05. What σ_y is needed for the same f'_c , and what does that tell you about the two parameters?

Answer: A different σ_y produces the same f'_c : the pair is not identifiable from a compressive strength alone.

Solution. This is the practically important consequence. One test gives one number and the model has two parameters, so infinitely many pairs reproduce it — and they differ enormously under any other loading, because **rho** is precisely the pressure sensitivity. Calibrating a pressure-dependent model against unconfined compression alone is not calibration; it is fitting one point.

Exercise S4.3. Change the bottom platen to fully bonded — restrain every base node in x and y — and rerun the crush. Compare the peak, and look at the deviatoric strain field in ParaView.

Answer: The peak rises: end restraint confines the specimen, and this material is pressure dependent.

Solution. The field is the more interesting output. Bonded ends produce the cone-shaped failure that a real cylinder test shows, because the restrained ends stay intact while the mid-height expands laterally. That the frictionless idealisation does *not* produce it is a good reminder that a boundary condition is a modelling assumption with visible consequences, not a formality.

Exercise S4.4. Experiment A used a uniform specimen. Add the imperfection but keep $H = -500$, and repeat the three-mesh study. Does mesh dependence appear now?

Answer: Only weakly: the flaw yields first, but with mild softening the surrounding material yields too and no narrow band forms.

Solution. This isolates the two ingredients. A trigger alone is not enough and steep softening alone is not enough; localisation needs both, and by the time you have both the analysis has usually stopped converging. The exercise is worth doing because it is the controlled version of [Section S4.4](#) — one variable at a time.

Exercise S4.5. Rerun **impact** at 4000 mm/s with the time step halved. Does the peak change? Now halve the specimen's density instead and rerun at the original step.

Answer: Halving Δt changes the peak very little; halving the density lowers it substantially.

Solution. Two candidate explanations for the 40 MPa peak — an under-resolved time step and inertia — and one experiment each. The time step is nearly innocent; the mass is the culprit, which is what the inertia explanation predicts. Note that halving the density is not a physically meaningful change to the specimen; it is an instrument for attributing an effect, and using models that way is most of what computational mechanics is for.

Exercise S4.6 (find the bug). A student exports the specimen and reports that the deviatoric strain field is uniform and equal to the element tag divided by a thousand. What went wrong?

Answer: Point and cell fields were swapped or auto-detected: the element tags were written as data.

Solution. This is the overlapping-tag trap made visible. The giveaway is the phrase “equal to the element tag” — a field that reproduces an index is not a field, it is a labelling accident. Whenever an exported quantity looks suspiciously structured, check that it is the quantity you meant before looking for physics in it.

Exercise S4.7. The chapter ends without a validated post-peak curve. Write half a page on what you would need to obtain one — material, mesh, and analysis — and say which part you think is hardest.

Answer: A regularised material with a fracture energy, an element size tied to that energy, and an arc length or dynamic relaxation solver.

Solution. The material is the hardest part and it is not close. Regularisation ties the softening branch to the element size so that the dissipated energy per unit crack area stays constant, which requires a material that accepts a characteristic length — and S3's survey shows none of the three concrete continuum materials in this build does. That is a real limitation of the tool, honestly arrived at, and knowing where a tool stops is a more useful outcome than a curve you could not defend.

Notes